

CHAPTER 1

Basic Concepts

1.1 OVERVIEW: SYSTEM LIFE CYCLE

We assume that our readers have a strong background in programming, typically attained through the completion of an elementary programming course. Such an initial course usually emphasizes mastering the syntax of a programming language (its grammar rules) and applying this language to the solution of several relatively small problems. In this text we want to move beyond these rudiments by providing the tools and techniques necessary to design and implement large-scale computer systems. We believe that a solid foundation in data abstraction and encapsulation, algorithm specification, and performance analysis and measurement provides the necessary methodology. In this chapter, we will discuss each of these areas in detail. We will also briefly discuss recursive programming because many of you probably have only a fleeting acquaintance with this important technique. However, before we begin, we want to place these tools in a context that views programming as more than writing code. Good programmers regard large-scale computer programs as systems that contain many complex interacting parts. As systems, these programs undergo a development

2 Basic Concepts

process called the system life cycle. This cycle consists of requirements, analysis, design, coding, and verification phases. Although we will consider them separately, these phases are highly interrelated and follow only a very crude sequential time frame. The References and Selected Readings section lists several sources on the system life cycle and its various phases that will provide you with additional information.

(1) Requirements. All large programming projects begin with a set of specifications that define the purpose of the project. These requirements describe the information that we, the programmers, are given (input) and the results that we must produce (output). Frequently the initial specifications are defined vaguely, and we must develop rigorous input and output descriptions that include all cases.

(2) Analysis. After we have delineated carefully the system's requirements, the analysis phase begins in earnest. In this phase, we begin to break the problem down into manageable pieces. There are two approaches to analysis: bottom-up and top-down. The bottom-up approach is an older, unstructured strategy that places an early emphasis on the coding fine points. Since the programmer does not have a master plan for the project, the resulting program frequently has many loosely connected, error-ridden segments. Bottom-up analysis is akin to constructing a building by first designing specific aspects of the building such as walls, a roof, plumbing, and heating, and then trying to put these together to construct the building. The specific purpose to which the building will be put is not considered in this approach. Although few of us would want to live in a home constructed using this technique, many programmers, particularly beginning ones, believe that they can create good, error-free programs without prior planning.

In contrast, the top-down approach begins by developing a high-level plan for dividing the program into manageable segments. This plan is subsequently refined to take into account low-level details. This technique generates diagrams that are used to design the system. Frequently, several alternate solutions to the programming problem are developed and compared during this phase. The top-down approach has been the preferred approach for developing complex software systems.

(3) Design. This phase continues the work done in the analysis phase. The designer approaches the system from the perspectives of the data objects that the program needs and the operations performed on them. The first perspective leads to the creation of abstract data types, whereas the second requires the specification of algorithms and a consideration of algorithm design strategies. For example, suppose that we are designing a scheduling system for a university.

Typical data objects might include students, courses, and professors. Typical operations might include inserting, removing, and searching within each object or between them. That is, we might want to add a course to the list of university courses or search for the courses taught by a specific professor.

Since the abstract data types and the algorithm specifications are language-independent, we postpone implementation decisions. Although we must specify the information required for each data object, we ignore coding details. For example, we might decide that the student data object should include name, social security number, major, and phone number. However, we would not yet pick a specific implementation for the list of students. As we will see in later chapters, there are several possibilities, including arrays, linked lists, or trees. By deferring implementation issues as long as possible, we not only create a system that could be written in several programming languages, but we also have time to pick the most efficient implementations within our chosen language.

(4) Refinement and coding. In this phase, we choose representations for our data objects and write algorithms for each operation on them. The order in which we do this is crucial because a data object's representation can determine the efficiency of the algorithms related to it. Typically this means that we should write the algorithms that are independent of the data objects first.

Frequently at this point we realize that we could have created a much better system. Perhaps we have spoken with a friend who has worked on a similar project, or we realize that one of our alternate designs is superior. If our original design is good, it can absorb changes easily. In fact, this is a reason for avoiding an early commitment to coding details. If we must scrap our work entirely, we can take comfort in the fact that we will be able to write the new system more quickly and with fewer errors.

(5) Verification. This phase consists of developing correctness proofs for the program, testing the program with a variety of input data, and removing errors. Each of these areas has been researched extensively, and a complete discussion is beyond the scope of this text. However, we summarize briefly the important aspects of each area.

Correctness proofs: Programs can be proven correct using the same techniques that abound in mathematics. Unfortunately, these proofs are very time-consuming and difficult to develop for large projects. Frequently, scheduling constraints prevent the development of a complete set of proofs for a large system. However, selecting algorithms that have been proven correct can reduce the number of errors. In this text, we will provide you with an arsenal of algorithms, some of which have been proven correct using formal techniques, that

4 Basic Concepts

you may apply to many programming problems.

Testing: Since our algorithms need not be written in a specific programming language, we can construct our correctness proofs before and during the coding phase. Testing, however, requires the working code and sets of test data. This data should be developed carefully so that it includes all possible scenarios. Frequently, beginning programmers assume that if their program ran without producing a syntax error, it must be correct. Little thought is given to the input data, and usually only one set of data is used. Good test data should verify that every piece of code runs correctly. For example, if our program contains a **switch** statement, our test data should be chosen so that we can check each **case** within the **switch** statement.

Initial system tests focus on verifying that a program runs correctly. Although this is a crucial concern, a program's running time is also important. An error-free program that runs slowly is of little value. Theoretical estimates of running time exist for many algorithms. We will derive these estimates as we introduce new algorithms. In addition, we may want to gather performance estimates for portions of our code. Constructing these timing tests is also a topic that we pursue later in this chapter.

Error removal: If done properly, the correctness proofs and system tests will indicate erroneous code. The ease with which we can remove these errors depends on the design and coding decisions made earlier. A large undocumented program written in "spaghetti" code is a programmer's nightmare. When debugging such programs, each corrected error possibly generates several new errors. On the other hand, debugging a well-documented program that is divided into autonomous units that interact through parameters is far easier, especially if each unit is tested separately and then integrated into the system.

1.2 OBJECT-ORIENTED DESIGN

Object-oriented design represents a fundamental change from the structured programming design method. The two approaches are similar in that both believe that the way to develop a complex system is by using the philosophy of divide-and-conquer; that is, break up a complex software design project into a number of simpler subprojects, and then tackle these subprojects individually. The two approaches disagree on *how* a project should be decomposed.

1.2.1 Algorithmic Decomposition Versus Object-Oriented Decomposition

Traditional programming techniques have used algorithmic decomposition. Algorithmic or functional decomposition views software as a *process*. It decomposes the software into modules that represent *steps* of the process. These modules are implemented by language constructs such as procedures in Pascal, subprograms in Fortran, and functions in C. The data structures required to implement the program are a secondary concern, which is addressed after the project has been decomposed into functional modules.

Object-oriented decomposition views software as a set of well-defined *objects* that model entities in the application domain. These objects interact with each other to form a software system. Functional decomposition is addressed after the system has been decomposed into objects.

The principal advantage of object-oriented decomposition is that it encourages the reuse of software. This results in flexible software systems that can evolve as system requirements change. It allows a programmer to use object-oriented programming languages effectively. Object-oriented decomposition is also more intuitive than algorithm-oriented decomposition because objects naturally model entities in the application domain.

1.2.2 Fundamental Definitions and Concepts of Object-Oriented Programming

Before proceeding, we provide definitions of basic terminology used in the object-oriented paradigm. Our definitions are adapted from *Object-Oriented Design with Applications*, by Grady Booch.

Definition: An *object* is an entity that performs computations and has a local state. It may therefore be viewed as a combination of data and procedural elements.

Definition: *Object-oriented programming* is a method of implementation in which

- (1) Objects are the fundamental building blocks.
- (2) Each object is an instance of some type (or class).
- (3) Classes are related to each other by inheritance relationships. (Programming methods that do not use inheritance are not considered to be object-oriented.)

6 Basic Concepts

Definition: A language is said to be an *object-oriented* language if

- (1) It supports objects.
- (2) It requires objects to belong to a class.
- (3) It supports inheritance.

A language that supports the first two features but does not support inheritance, is an *object-based language*. C++ is an object-oriented language while JavaScript is an object-based language.

1.2.3 Evolution of Programming Languages and History of C++

Higher order programming languages may be classified into four generations.

- (1) **First Generation Languages:** An example of this is FORTRAN. The salient feature of these languages is their ability to evaluate mathematical expressions.
- (2) **Second Generation Languages:** Examples of these include Pascal and C. The emphasis of these languages is on effectively expressing algorithms.
- (3) **Third Generation Languages:** An example of this is Modula, which introduced the concept of abstract data types.
- (4) **Fourth Generation Languages (Object-Oriented languages):** Examples include Smalltalk, Objective C, and C++. These languages emphasize the expression of the relationship between abstract data types through the use of inheritance.

C++ was designed by Bjarne Stroustrup of AT&T Bell Laboratories in the early 1980s. It was designed to incorporate the object-oriented paradigm into the C programming language. C is widely-used in industry because it is:

- (1) **Efficient:** it has a number of low-level features, which utilize hardware better than other languages.
 - (2) **Flexible:** it can be used to solve problems in most application areas.
 - (3) **Available:** compilers for C are readily available for most computers.
- Other influences on the design of C++ were Simula67 and Algol68.

There are a number of features of C++ that improve on C, but do not implement data abstraction or inheritance. These are discussed later in this chapter. The C++ class, which implements data abstraction, is discussed in

Chapter 2. C++ functions and classes can be made more powerful by augmenting them with *templates*. Templates are discussed in Chapter 3. Simple inheritance in C++ is also discussed in Chapter 3. Advanced concepts on inheritance and polymorphism are discussed in Chapter 4.

1.3 DATA ABSTRACTION AND ENCAPSULATION

The notions of abstraction and encapsulation are commonly used in human-machine interaction and are widely prevalent in the technology-oriented world we live in. Consider, for example, the digital video player (DVD player), which is an integral part of many households today. We make two observations about how DVD players are packaged and used.

(1) All our interactions with a DVD player are made through buttons on the control panel (or remote control) such as PLAY, STOP and PAUSE. The DVD player is packaged so that we cannot directly interact with the circuitry inside. So, the *internal representation* of the DVD player is *hidden* from the user. This is the principle of *encapsulation*.

(2) The instruction manual that accompanies the DVD player tells us *what* the DVD player is supposed to do if we press a particular button. It does not tell us *how* this function is implemented inside the DVD player; it does not, for example, explain the sequence of electronic and mechanical events that result when the PLAY button is pressed. So, the instruction manual makes a clear distinction between *what* the DVD player does and *how* it does it. This distinction is the principle of *abstraction*.

These same principles may be applied to the packaging or organizing of data in a computer program, resulting in the concepts of data encapsulation and data abstraction, which are defined below.

Definition: *Data Encapsulation* or *Information Hiding* is the concealing of the implementation details of a data object from the outside world.

Definition: *Data Abstraction* is the separation between the *specification* of a data object and its *implementation*.

We will see later that these concepts are of great importance in software development because they result in better quality programs and more efficient programming techniques. These, in turn, reduce the number of person-hours

8 Basic Concepts

required to develop a software system, and hence reduce the total cost of the final software product.

First, however, we shall explore the fundamental data types of C++. These include **char**, **int**, **float**, and **double**. Some of these data types may be modified by keywords **short**, **long**, **signed**, and **unsigned**. The modifiers **short** and **long** specify the amount of storage allocated to the fundamental type they modify. The modifiers **signed** and **unsigned** specify whether the most significant bit in the binary representation of an integer represents a sign bit or not. C++ also supports types derived from the fundamental data types listed above. These include pointer and reference types. Ultimately, the real world abstractions we wish to deal with must be represented in terms of these data types. In addition to these types, C++ helps us by providing three mechanisms for grouping data together. These are the array, the struct, and the class. *Arrays* are collections of elements of the same basic data type; *structs* and *classes* are collections of elements whose data types need not be the same.

All programming languages provide at least a minimal set of predefined data types, plus the ability to construct new, or *user-defined* types.

Definition: A *data type* is a collection of *objects* and a set of *operations* that act on those objects. □

Whether your program is dealing with predefined data types or user-defined data types, these two aspects must be considered: objects and operations. For example, the data type **int** consists of the objects {0, +1, -1, +2, -2, ..., MAXINT, MININT}, where MAXINT and MININT are the largest and smallest integers that can be represented by an **int** on your machine. The operations on integers are many and would certainly include the arithmetic operators +, -, *, and /. There is also testing for equality/inequality and the operation that assigns an integer to a variable.

Definition: An *abstract data type (ADT)* is a data type that is organized in such a way that the specification of the objects and the specification of the operations on the objects is separated from the representation of the objects and the implementation of the operations. □

Throughout this text, we will emphasize the distinction between specification and implementation. To help us do this, we will typically begin with an ADT definition of the object that we intend to study. This will permit the reader to grasp the essential elements of the object, without having the discussion complicated by the representation of the objects or by the actual implementation of the operations. Once the ADT definition is fully explained, we will move on to discussions of representation and implementation. These are quite

important in the study of data structures. To help us accomplish this goal, we introduce a notation for expressing an ADT. This notation is independent of the syntax of the C++ language. Later we shall see how the syntax of the C++ class can be used to express an ADT.

Example 1.1 [Abstract data type *NaturalNumber*]: As this is the first example of an ADT, we will spend some time explaining the notation. ADT 1.1 contains the ADT definition of *NaturalNumber*. The definition begins with the name of the abstract data type. There are two main sections in the definition: the objects and the functions. The objects are defined in terms of the integers, but we make no explicit reference to their representation. The function definitions are a bit more complicated. First, the definitions use the symbols x and y to denote two elements of the set *NaturalNumber*, whereas TRUE and FALSE are elements of the set of *Boolean* values. In addition, the definition makes use of functions that are defined on the set of integers, namely, plus, minus, equal, and less than. This is an indication that in order to define one data type, we may need to use operations from another data type. For each function, we place the result type to the left of the function name and a definition of the function to the right. The symbols "::<=" should be read as "is defined as."

The first function, *Zero*, has no arguments and returns the natural number zero. The function *Successor*(x) returns the next natural number in sequence. Notice that if there is no next number in sequence, that is, if the value of x is already MAXINT, then we define the action of *Successor* to return MAXINT. Some programmers might prefer that in such a case *Successor* return an error flag. This is also perfectly permissible. Other functions are *Add* and *Subtract*. They might also return an error condition, although here we decided to return an element of the set *NaturalNumber*. □

Next, we see how the principles of data abstraction and data encapsulation help us to efficiently develop well-designed programs.

(1) Simplification of software development: The chief advantage of data abstraction is that it facilitates the decomposition of the complex task of developing a software system into a number of simpler subtasks. Consider the following scenario: A problem has been assigned to a single programmer or to a team of programmers. Suppose that a top-down review of the problem results in a decision that three data types A , B , and C will be used along with some additional code (which we will call *glue*) to facilitate interactions among the three data types. Assume, also, that the specifications of each data type are provided at the outset; i.e., the operations that each data type must support are completely specified.

10 Basic Concepts

ADT *NaturalNumber* is

objects: An ordered subrange of the integers starting at zero and ending at the maximum integer (MAXINT) on the computer.

functions:

for all $x, y \in \text{NaturalNumber}$; $\text{TRUE}, \text{FALSE} \in \text{Boolean}$
and where $+$, $-$, $<$, $==$, and $=$ are the usual integer operations

<i>Zero()</i> : <i>NaturalNumber</i>	::=	0
<i>IsZero</i> (x) : <i>Boolean</i>	::=	if ($x == 0$) <i>IsZero</i> = true else <i>IsZero</i> = false
<i>Add</i> (x, y) : <i>NaturalNumber</i>	::=	if ($x + y \leq \text{MAXINT}$) <i>Add</i> = $x + y$ else <i>Add</i> = MAXINT
<i>Equal</i> (x, y) : <i>Boolean</i>	::=	if ($x == y$) <i>Equal</i> = TRUE else <i>Equal</i> = FALSE
<i>Successor</i> (x) : <i>NaturalNumber</i>	::=	if ($x == \text{MAXINT}$) <i>Successor</i> = x else <i>Successor</i> = $x + 1$
<i>Subtract</i> (x, y) : <i>NaturalNumber</i>	::=	if ($x < y$) <i>Subtract</i> = 0 else <i>Subtract</i> = $x - y$

end *NaturalNumber*

ADT 1.1: Abstract data type *NaturalNumber*

(a) Scenario 1 - A team of four programmers: the subtasks may be allotted to members of a team of four programmers as follows. A programmer is assigned to each of the three data types. Each programmer's job is to implement his/her data type according to the specifications agreed upon earlier. The fourth programmer implements the glue, assuming that the data types are, in fact, implemented according to the specifications. No programmer needs to know *how* the other programmers implement their portion of the code. Consequently, each programmer can focus exclusively on his/her portion of the code without worrying about what the other programmers are doing and how that might affect what he/she is doing.

(b) Scenario 2 - A single programmer: here data abstraction helps reduce the number of things the programmer has to keep in mind at any time. In the example, the programmer would implement each data type one by one according to the specifications. When the programmer implements data type *A*, he/she can do so without worrying about data types *B* and *C* and the glue. After the individual data types are implemented, the programmer can turn his/her attention to the glue. At this point, the programmer is no longer concerned with the

implementations of the three data types, but rather with how to use operations on them to achieve the desired goal.

(2) Testing and Debugging: Breaking down a complex task into a number of easier subtasks also simplifies testing and debugging. In the above example, each data type can be tested and debugged separately. After each data type is tested and debugged, the entire program can be tested. If a bug is detected at this stage, it is unlikely that there is a bug in the data structures *A*, *B*, and *C* (represented by the shaded boxes of Figure 1.1(a)) because they have already been tested. So, the bug will most likely be in the glue represented by the unshaded portions of Figure 1.1(a). Compare this with the amount of code to be searched if we had *not* used data abstraction (represented by the unshaded portions Figure 1.1(b)). It is easy to see that employing data abstraction in software development leads to more efficient testing and debugging.

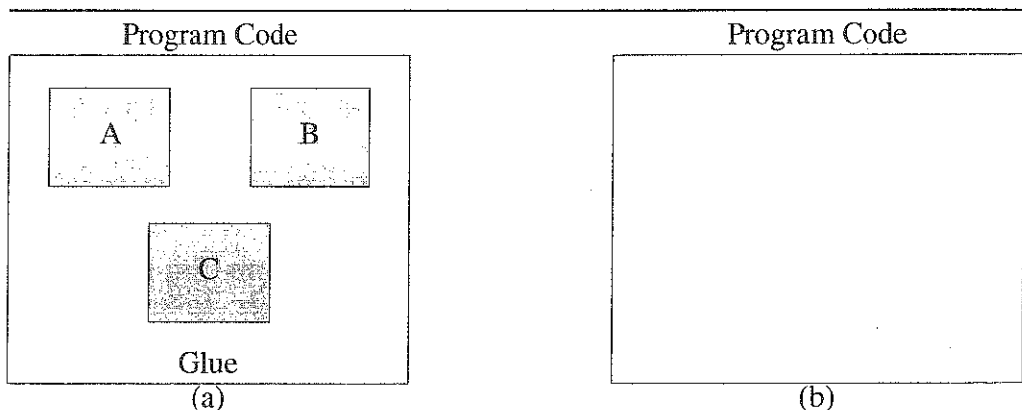


Figure 1.1: Unshaded areas represent code that has to be searched for bugs: (a) data abstraction is used (b) data abstraction is not used.

(3) Reusability: Data abstraction and encapsulation typically give rise to data structures that are implemented as distinct entities of a software system. This makes it easier to extract the code for a data structure and its operations from a software system and use it in another software system, rather than if it were inextricably integrated with the original software system. We will later discuss other techniques in C++ that enhance the reusability of software.

(4) Modifications to the representation of a data type: a significant

12 Basic Concepts

consequence of information-hiding is that the implementation of a data type is not visible to the rest of the program. Specifically, the rest of the program does not have direct access to the internal representation of the data type. It manipulates the data type solely through a suite of operations that the data type has made available to it. Thus, a change in the internal implementation of a data type will not affect the rest of the program as long as the data type continues to provide the same operations, and as long as these operations continue to do the same things. All that is required is a change in the implementation of the operations that directly access the internal implementation of the data type.

This is easier to appreciate if we examine the consequences of *not* employing data encapsulation. Consider a program that directly manipulates the internal representation of its data types. Suppose a change is made to the implementation of a data type. You, the programmer, are now confronted with the tasks of first examining the program code to locate instances where the program accesses the internal representation of the data type and then making the appropriate changes. This could be an extremely laborious task if the program accesses the internal representation of the data type many times.

EXERCISES

For each of these exercises, provide a definition of the abstract data type using the form illustrated in ADT 1.1.

1. Add the following operations to the *NaturalNumber* ADT: *Predecessor*, *IsGreater*, *Multiply*, *Divide*.
2. Create an ADT, *Set*. Use the standard mathematical definition and include the following operations: *Create*, *Insert*, *Remove*, *IsIn*, *Union*, *Intersection*, *Difference*.
3. Create an ADT, *Bag*. In mathematics a *bag* is similar to a *set* except that a bag may contain duplicate elements. The minimal operations should include *Create*, *Insert*, *Remove*, and *IsIn*.

1.4 BASICS OF C++

The expression "a better C" is often used to describe the C++ language, implying that:

- (1) C and C++ have a lot of features in common.
- (2) C++ has a number of features not associated with either data abstraction or inheritance that improve on C.

In this section we will review features of the C++ language and identify

similarities and differences with C. Our presentation assumes that readers are familiar with the C language.

1.4.1 Program Organization in C++

As in C, a single C++ program is usually spread out over a number of files. Two kinds of files are used in C++: header files and source files. Header files have a .h suffix and are used to store declarations. Some header files (such as <iostream>) are system-defined. (The .h suffix is omitted for system-defined header files.) Others are user-defined. Source files are used to store C++ source code. The suffix used in source-file names depends on the compiler. We will use a .C suffix for source files. Header files are included in the appropriate files through the use of the #include preprocessor directive.

Before a C++ program can run, its source files must be individually compiled, linked, and loaded. Because a C++ program typically consists of many source and header files, it is possible that a header file gets included multiple times, giving rise to compilation errors. To ensure that a header file is never included twice, the contents of a header file are enclosed inside the following preprocessor directives.

```
#ifndef FILENAME_H
#define FILENAME_H
// insert contents of the header file here
.
.
.
#endif
```

Another consequence of using multiple source files is that the process of typing compilation commands every time we modify and recompile a program becomes rather tedious. To avoid this, we suggest that you use the *make* facility in UNIX, or its equivalent, to maintain a C++ program. In addition to saving the effort of repeatedly typing compilation commands, *make* also saves time by not recompiling source files that are not affected by modifications made to other files.

1.4.2 Scope in C++

Another consequence of using many files in a single program is that questions arise about the visibility of a variable declared in one file and used in other files. In view of this, we take a brief look at program scope in C++. C++ program text can be classified into one of four scopes:

14 Basic Concepts

File scope: Declarations that are not contained in a function definition, a class definition, or a namespace belongs to a file scope.

Namespace scope: A namespace is a mechanism that permits logically related variables and functions to be grouped together. For example, the standard library is defined in a namespace called **std**. In order to access an entity defined in a namespace from outside that namespace, it is necessary to provide the scope information. For example, the **cout** operator in the standard library is accessed using **std::cout**. The repeated use of the scope information may be avoided through the use of the **using** declaration (as in **using std**).

Local scope: A name declared in a block belongs to a local scope consisting of that block. (A block is a section of code delimited by a { } pair.) It can also be used by sub-blocks contained within the block in which it was declared.

Class scope: Declarations associated with a class definition belongs to a class scope. Each class represents a distinct class scope. We discuss classes in greater detail later in Chapter 2.

Each variable has a scope or a context. A variable is uniquely identified by its scope and its name. A variable is visible to a program only from within its scope. For example, a variable defined in a block can only be accessed from within the block. A variable defined at file scope (a global variable), however, can be accessed anywhere in the program. Some questions that arise as a result are

(1) What do we do if a local variable reuses a global variable name in a block; but, we want to access the global variable?

Solution: use the scope operator **::** to access the global variable.

(2) A global variable is defined in `file1.C`, but used in `file2.C`. How do we declare the global variable in `file2.C`? If we declare it, as we normally declare variables, in `file2.C`, the compiler tells us that it is multiply defined. If we don't declare it, the compiler tells us that it hasn't been defined!

Solution: use **extern** to declare the variable in `file2.C`.

(3) In our program, both `file1.C` and `file2.C` define a global variable with the same name; but the two global variables are meant to be different entities. How do I get the program to compile without changing the global variable name in one of the files.

Solution: use **static** to declare the variables in both files.

For further details, the reader is referred to one of the introductory texts on C++

listed in the references at the end of the chapter.

1.4.3 C++ Statements and Operators

C++ statements have the same syntax and semantics as C statements. C++ statements are briefly reviewed later in this chapter in the context of performance analysis and evaluation.

C++ operators are identical to C operators with the exception of **new** and **delete**. We will study **new** and **delete** shortly when we discuss dynamic memory management in C++. Another difference is that C++ uses the shift left (<<) and shift right (>>) operators for input and output. We will also study these shortly when discussing input/output in C++. An important difference between C and C++ is that C++ allows *operator overloading*; that is, an operator is allowed to have different functions depending on the types of the operands that it is being applied to, and furthermore, these functions can be defined by a programmer. We will discuss operator overloading in detail in Chapter 2.

1.4.4 Data Declarations in C++

A data declaration associates a data type with a name. We have already listed the fundamental types of data provided by C++ in Section 1.3 on Data Abstraction and Encapsulation. Now, we list the various options available for declaring data in C++ below. All of these options are available in C with the exception of the reference type.

- (1) Constant values: these consist of literals such as 5, 'a', or 4.3.
- (2) Variables: these are instances of data types that can be modified during the course of a program.
- (3) Constant variables: these are variables that cannot be assigned a value during their lifetime. Because of this, they must be initialized; that is, their contents must be fixed at the time they are declared. A constant type is declared by adding the keyword **const** to its declaration (e.g., **const int MAX = 500 ;**).
- (4) Enumeration types: This is an alternate mechanism for declaring a series of integer constants. It can also be used to create a new data type by giving the enumeration type a name:

```
enum semester { SUMMER, FALL, SPRING };
```

16 Basic Concepts

SUMMER, FALL and SPRING are constants of type *semester*. Their values are 0, 1 and 2, respectively, by default.

- (5) Pointers: these hold memory addresses of objects. For example,

```
int i = 25 ;  
int *np ;  
np = &i ;
```

Here *np* is declared to be a pointer to an integer. Next, it is assigned the address of integer variable *i*. So, *np* is now a pointer to the integer stored in variable *i* (in this case 25).

- (6) Reference types: This is a feature of C++ that is not a feature of C. The reference type is a mechanism to provide an alternate name for an object. A reference to an object of type *T* is declared by appending an *&* to *T* (i.e., *T&*). For example,

```
int i = 5 ;  
int& j = i ;  
i = 7 ;  
printf ("i= %d, j= %d", i, j) ;
```

In the example *j* is a reference type. It represents an alternate name for *i*. When *i*'s value is changed, *j*'s value also changes correspondingly. When the `printf` statement is executed, the final value of *i* and *j* is 7.

1.4.5 Comments in C++

Comments in C++ are specified in two ways:

- (1) Multiline comments: these are identical to comments in C. All text enclosed within the `/* */` delimiters is ignored by the compiler.
- (2) Single line comments: these are unique to C++. All text after `//` on a line is ignored by the compiler.

The C++ single-line comment format overcomes the following disadvantage of the C multi-line comment: if one multi-line comment is nested inside another, the portion of the text between the terminating delimiters of the inner and outer comments is not ignored by the compiler. This could result in the inclusion of code

into the program that the programmer believes was commented out.

1.4.6 Input/Output in C++

To perform I/O in C++, it is necessary to include the system-defined header file *iostream*. The keyword **cout** is used for output to the standard output device. The **cout** keyword and each entity being printed are separated from each other by the **<<** operator. The entities being output are printed in left to right order on the standard output device.

```
#include <iostream>

main()
{
    int n = 50 ; float f = 20.3 ;
    cout << "n: " << n << endl ;
    cout << "f: " << f << endl ;
}
```

Program 1.1: Output in C++

Program 1.1 prints the following on the standard output device:

```
n: 50
f: 20.3
```

The **cin** keyword is used for input in C++. The operator **>>** is used to separate variables being input. Whitespace (i.e., the tab, newline, or blank characters) is used to separate items corresponding to different variables on the standard input device

When Program 1.2 is executed, both of the following inputs on the standard input device result in variable *a* being set to 5 and *b* to 10:

```
Input1:
5 10 <Enter>
```

```
Input2:
5 <Enter>
```

18 Basic Concepts

```
#include <iostream>
```

```
main()
{
    int a, b ;
    cin >> a >> b ;
}
```

Program 1.2: Input in C++

10 <Enter>

An advantage of I/O in C++ is that it is format-free; that is, the programmer is not required to use formatting symbols to specify the type and order of items being input/output. Another advantage is that, like other C++ operators, I/O operators can be overloaded.

File I/O in C++ is performed by including the header file *fstream* as shown in Program 1.3. A filestream variable (*outFile*, in our example) is initialized with a string which represents the name of the file (here *my.out*) and a second argument which specifies the mode in which the file will be used (e.g., *ios::out* specifies that the file is to be used for output). If the file is not opened, *outFile* = 0. If *outFile* is successfully opened, it is used instead of *cout* to direct output to file *my.out*.

1.4.7 Functions in C++

There are two kinds of functions in C++: regular functions and member functions. Member functions are functions that are associated with specific C++ classes. We postpone a detailed discussion of member functions to Chapter 2. The features of functions that we review here are common to both types of functions.

A function consists of a function name, a list of arguments or *signature* (input), a return type (output), and the body (code that implements a function). In Program 1.4, *Max* is the function name, *int a*, *int b* is the list of arguments, *int* is the return type, and the statements between { and } form the body of the function.

All functions in C++ return a value. If a function is not meant to return anything, we use **void** to denote its return type. A value is returned from a function by

```
#include <iostream>
#include <fstream>

main()
{
    ofstream outFile("my.out", ios::out);
    if (!outFile) {
        cerr << "cannot open my.out " << endl; // standard error device
        return;
    }
    int n = 50 ; float f = 20.3 ;
    outFile << "n: " << n << endl;
    outFile << "f: " << f << endl;
}
```

Program 1.3: File I/O in C++

```
int Max (int a, int b)
{
    if (a > b) return a;
    else return b;
}
```

Program 1.4: An example of a function

using the **return** statement. The **return** statement must return a value that is of the same type as the function's return type or can be converted to the desired type. The function terminates when a **return** statement is encountered. A function is invoked by supplying the actual arguments (e.g., *Max(x, 100)* returns the larger of *x* and 100).

1.4.8 Parameter Passing in C++

In C++, arguments are passed by *value*. This is the default parameter-passing mechanism. When an object is passed by value, it is *copied* into the function's local storage. The function accesses this local copy. Consequently, any change

20 Basic Concepts

made to a parameter that is passed by value inside the function only changes its local copy. In other words, passing by value does not affect the actual arguments.

An argument may also be passed by *reference*. To do this, one needs to explicitly declare it to be a reference type. This is done by appending an `&` to its type specifier. In Program 1.4, we can pass *a* by reference by declaring it as `int& a` in the function's argument list. When an object is passed by reference, only the address of its location is copied into the function's local store; the object itself is not copied into the function's local store. Thus, the function accesses the objects referred to by the address; i.e., the actual arguments.

The advantage of passing by value is that variables that were supplied as actual arguments to the function are not inadvertently modified by it. The advantage of passing by reference is that the function executes faster if the object being passed requires more memory than its address. This is because the overhead of copying the actual argument into the function's local store is replaced by the overhead of copying its address. Note that pass-by-reference would be slower for an argument type such as `char` that occupies less memory than its address. Another reason for using reference parameters is if you *want* the function to modify the actual argument. This technique is used if a function is to return two or more items to the calling function. One of these items is returned by using the **return** statement. The others are returned through the use of reference arguments.

One technique for retaining the advantages of both parameter-passing methods is to pass *constant references* such as `const T& a`, where *T* is the type of the argument. Any attempt to modify a **const** argument in the function body will result in a compile-time error. It is also helpful to use the **const** keyword to characterize an argument that is not modified in a function, because it conveys this information to a user of the function. For a more detailed discussion on parameter passing mechanisms and the circumstances in which they should be used, see the book *Effective C++*, by Scott Meyers, listed in the references at the end of the chapter.

There is one exception to the rule that the default parameter passing mechanism in C++ is pass-by-value: array types are passed by reference; that is, the array is *not* copied into the function's local store. Therefore, any changes made to the array inside the function *are* reflected in the actual array. When a function is invoked with an array argument *a* (e.g., *f(a)*), a *pointer to the first element of a* (i.e., `&a[0]`) is actually being passed. This is the reason that, in function definitions, arrays are usually denoted by pointers to the appropriate type (e.g., *f(int *a)*). Because arrays are passed in this manner, the function does not know the size of the array. This is typically rectified by explicitly passing the size of the array as a separate parameter of the function.

1.4.9 Function Name Overloading in C++

Finally, we discuss *function name overloading* in C++. Function overloading means that there can be more than one function with the same name *as long as they have different signatures*. For example, C++ would allow all the following to coexist even though they have the same name *Max*:

```
int Max(int, int);
int Max(int, int, int);
int Max(int*, int);
int Max(float, int);
int Max(int, float);
```

1.4.10 Inline Functions

An inline function is declared by adding the keyword **inline** to the function definition as follows:

```
inline int Sum( int a, int b)
{
    return a + b ;
}
```

The **inline** keyword tells the compiler that any calls to *Sum* must be replaced by the body of *Sum*. This eliminates the overhead of performing a function call and copying arguments when the program is executing. So, the statement *i = Sum (x, 12);* is replaced by *i = x + 12 ;*.

The objective of the **inline** and **const** keywords is to eliminate the use of preprocessor directives such as **#define**, which have traditionally been used to perform macro substitution. Excessive use of preprocessor directives makes it difficult to use programming tools such as debuggers and profilers effectively.

1.4.11 Dynamic Memory Allocation in C++

Objects may be allocated from and released to the free store during runtime through the use of the **new** and **delete** operators. The **new** operator creates an object of the desired type and returns a pointer to the data type that follows it. If it is unable to create the desired object, it throws an exception. An object created by **new** exists for the duration of the program unless it is explicitly deleted by applying the **delete** operator to a pointer to the object:

22 Basic Concepts

```
int *ip = new int;
```

```
delete ip ;
```

Operators **new** and **delete** can also be used to create and delete an array of objects as follows:

```
int *jp = new int[10] ; // jp is an array of integers
```

```
delete [ ] jp ; // delete the array jp
```

The operator ([]) is used to inform the compiler that the object being created or deleted is an array.

1.4.12 Exceptions

Throwing an Exception

Exceptions are used to signal the occurrence of errors and other special conditions. For example, the evaluation of the expression $a + b * c + b / c$ with $a = 2$, $b = 1$, and $c = 0$ requires us to divide by zero, which is an error. Although this error is not detected by the C++ compiler, your hardware will detect the error and throw an exception.

We can write C++ programs that check for exceptional conditions and throw an exception when such a condition is detected. For example, we may wish to write a function *DivZero* that computes the expression $a + b * c + b / c$ only when each of a , b , and c is greater than 0. Such a function would first check that the values of a , b , and c are actually > 0 . If one or more of these is ≤ 0 , we can signal an exceptional condition by throwing an exception as is done in Program 1.5. The exception thrown by this program is of type **char*** and the return expression is not evaluated.

We get more flexibility in processing exceptions when we define an exception class (or type) for each of the different kinds of exceptions (e.g., divide by zero, illegal parameter value, illegal input value, array index out of range) that our program may throw. For example, the C++ operator **new** that does dynamic memory allocation throws an exception of type *bad_alloc* when it is unable to make the requested memory allocation.

Handling Exceptions

Exceptions that might be thrown by a piece of code can be handled by enclosing this code within a **try** block. The **try** block is then followed by zero or more

```
int DivZero(int a, int b, int c)
{
    if (a <= 0 || b <= 0 || c <= 0)
        throw "All parameters should be > 0";
    return a + b * c + b / c;
}
```

Program 1.5: Throwing an exception of type `char*`

catch blocks. Each **catch** block has a parameter or argument whose type determines the type of exception that may be caught by that **catch** block. For example, the block

```
catch (char* e) {}
```

catches exceptions of type `char*` while the block

```
catch (bad_alloc e) {}
```

catches exceptions of type `bad_alloc`. The block

```
catch (...) {}
```

catches all exceptions regardless of their type.

A **catch** block typically contains code to recover from the exception that has occurred, or if recovery is not possible, the code in the **catch** block prints out an error message. Program 1.6 shows an example of the **try-catch** construct.

Although Program 1.6 has a single **catch** block following the **try** block, it is possible to follow a **try** block with several **catch** blocks. When the code within a **try** block terminates with no exception, we bypass the **catch** blocks. When an exception is thrown, normal execution of the **try** block terminates and we enter the first **catch** block that matches an exception of the type thrown. Following the execution of the code within this matching **catch** block, we bypass the remaining **catch** blocks. If no **catch** block matches the thrown exception type, then the exception propagates through the hierarchy of nested enclosing **try** blocks to the first **catch** block in this hierarchy that can handle the exception. If the exception is not caught by any **catch** block, the program terminates abnormally.

When Program 1.6 executes, *DivZero* throws an exception of type `char*`. This exception causes *DivZero* to terminate without the evaluation of the expression. Also, the **try** block terminates immediately (the **cout** in the **try** block

24 Basic Concepts

```
int main()
{
    try {cout << DivZero(2,0,4) << endl;}
    catch (char* e)
    {
        cout << "The parameters to DivZero were 2, 0, and 4" << endl;
        cout << "An exception has been thrown" << endl;
        cout << e << endl;
        return 1;
    }
    return 0;
}
```

Program 1.6: Catching an exception of type `char*`

doesn't complete). Since the type of the exception thrown by *DivZero* is the same as that of the **catch** block's parameter *e*, the exception is caught by this **catch** block; *e* is assigned the thrown exception; and the catch block is entered. Figure 1.2 gives the output generated by Program 1.6.

```
The parameters to DivZero were 2, 0, and 4
An exception has been thrown
All parameters should be > 0
```

Figure 1.2: Output from Program 1.6

EXERCISES

1. Modify Program 1.5 so that it throws an exception of type *int*. The value of the thrown exception should be 1 if *a*, *b*, and *c* are all less than 0; the value should be 2 if all three equal 0. When neither of these conditions is satisfied, no exception is thrown. Write a *main* function that uses your modified code, catches the exception if thrown, and outputs a message that depends on the value of the thrown exception. Test your code.
2. Write a C++ function to return the sum of the first *n* numbers in the integer array *a*. Your function should throw an exception in case *n* < 0. Test your code.

1.5 ALGORITHM SPECIFICATION

1.5.1 Introduction

The concept of an algorithm is fundamental to computer science. Algorithms exist for many common problems, and designing efficient algorithms plays a crucial role in developing large-scale computer systems. Therefore, before we proceed further, we discuss this concept more fully. We begin with a definition.

Definition: An *algorithm* is a finite set of instructions that, if followed, accomplishes a particular task. In addition, all algorithms must satisfy the following criteria:

- (1) **Input.** Zero or more quantities are externally supplied.
- (2) **Output.** At least one quantity is produced.
- (3) **Definiteness.** Each instruction is clear and unambiguous.
- (4) **Finiteness.** If we trace out the instructions of an algorithm, then for all cases, the algorithm terminates after a finite number of steps.
- (5) **Effectiveness.** Every instruction must be basic enough to be carried out, in principle, by a person using only pencil and paper. It is not enough that each operation be definite as in (3); it also must be feasible. $\square$

In computational theory, one distinguishes between an algorithm and a program, the latter of which does not have to satisfy the fourth condition. For example, we can think of an operating system that continues in a “wait” loop until more jobs are entered. Such a program does not terminate unless the system crashes. Since our programs will always terminate, we will use the terms algorithm and program interchangeably in this text.

We can describe an algorithm in many ways. We can use a natural language like English, although if we select this option, we must make sure that the resulting instructions are definite. Graphic representations called flowcharts are another possibility, but they work well only if the algorithm is small and simple. In this text, we will present most of our algorithms in C++, occasionally resorting to a combination of English and C++ for our specifications. Two examples should help to illustrate the process of translating a problem into an algorithm.

Example 1.2 [Selection sort]: Suppose we must devise a program that sorts a collection of $n \geq 1$ integers. A simple solution is given by the following:

26 Basic Concepts

From those integers that are currently unsorted, find the smallest and place it next in the sorted list.

Although this statement adequately describes the sorting problem, it is not an algorithm because it leaves several unanswered questions. For example, it does not tell us where and how the integers are initially stored or where we should place the result. We assume that the integers are stored in an array, a , such that the i th integer is stored in $a[i-1]$, $1 \leq i \leq n$. Program 1.7 is our first attempt at deriving a solution. Notice that it is written partially in C++ and partially in English.

```
for (int i = 0; i < n; i++) {  
    examine  $a[i]$  to  $a[n-1]$  and suppose the smallest integer is at  $a[j]$ ;  
    interchange  $a[i]$  and  $a[j]$  ;  
}
```

Program 1.7: Selection sort algorithm

To turn Program 1.7 into a real C++ program, two clearly defined subtasks remain: finding the smallest integer and interchanging it with $a[i]$. We can solve the latter problem by using the code:

$$temp = a[i]; a[i] = a[j]; a[j] = temp;$$

We implement this by calling the standard C++ function $swap(a[i], a[j])$, whose arguments are passed by reference. The first subtask can be solved by assuming the minimum is $a[i]$, checking $a[i]$ with $a[i+1]$, $a[i+2]$, $\dots$ and whenever a smaller element is found, regarding it as the new minimum. Eventually $a[n-1]$ is compared to the current minimum, and we are done. Putting all these observations together, we get the function *sort* (Program 1.8).

At this point, the obvious question to ask is: Does this function work correctly?

Theorem 1.1: *SelectionSort(a, n)* correctly sorts a set of $n \geq 1$ integers; the result remains in $a[0] \dots a[n-1]$ such that $a[0] \leq a[1] \leq \dots \leq a[n-1]$.

Proof: We first note that for any i , say $i = q$, following the execution of lines 5-9, it is the case that $a[q] \leq a[r]$, $q < r \leq n-1$. Also observe that when i becomes greater than q , $a[0] \dots a[q]$ is unchanged. Hence, following the last execution of these lines (i.e., $i = n-1$), we have $a[0] \leq a[1] \leq \dots \leq a[n-1]$. $\square$

We observe at this point that the upper limit of the **for** loop in line 3 can be

```

1 void SelectionSort (int *a, const int n)
2 { // Sort the  $n$  integers  $a[0]$  to  $a[n-1]$  into nondecreasing order.
3   for ( int  $i = 0$ ;  $i < n$  ;  $i++$  )
4   {
5       int  $j = i$ ;
6       // find smallest integer in  $a[i]$  to  $a[n-1]$ 
7       for ( int  $k = i + 1$  ;  $k < n$  ;  $k++$  )
8           if ( $a[k] < a[j]$ )  $j = k$  ;
9       swap ( $a[i]$ ,  $a[j]$ );
10  }
11 }
```

Program 1.8: Selection sort

changed to $n - 1$ without damaging the correctness of the algorithm.

Example 1.3 [Binary search]: Assume that we have $n \geq 1$ distinct integers that are already sorted and stored in the array $a[0] \cdots a[n-1]$. Our task is to determine if the integer x is present and if so to return j such that $x = a[j]$; otherwise return -1 . By making use of the fact that the set is sorted, we conceive of the following efficient method:

Let *left* and *right*, respectively, denote the left and right ends of the list to be searched. Initially, *left* = 0 and *right* = $n-1$. Let $middle = (left + right) / 2$ be the middle position in the list. If we compare $a[middle]$ with x , we obtain one of three results:

- (1) $x < a[middle]$. In this case, if x is present, it must be in the positions between 0 and $middle - 1$. Therefore, we set *right* to $middle - 1$.
- (2) $x == a[middle]$. In this case, we return *middle*.
- (3) $x > a[middle]$. In this case, if x is present, it must be in the positions between $middle + 1$ and $n-1$. So, we set *left* to $middle + 1$.

If x has not been found and there are still integers to check, we recalculate *middle* and continue the search. The algorithm contains two subtasks: (1) determining if there are any integers left to check and (2) comparing x to $a[middle]$.

At this point you might try the method out on some sample numbers. This method is referred to as *binary search*. Note how at each stage the number of elements in the remaining set is decreased by about one-half. Note also that at

28 Basic Concepts

each stage, x is compared with $a[middle]$ and depending on whether $x > a[middle]$, $x < a[middle]$, or $x == a[middle]$, we do a different thing. We can now refine the description of binary search to get a pseudo-C++ function. The result is given in Program 1.9.

```
int BinarySearch (int *a, const int x, const int n)
{ // Search the sorted array a[0], ..., a[n-1] for x
  Initialize left and right;
  while (there are more elements)
  {
    Let middle be the middle element;
    if ( $x < a[middle]$ ) set right to middle-1;
    else if ( $x > a[middle]$ ) set left to middle+1;
    else return middle;
  }
  Not found;
}
```

Program 1.9: Algorithm for binary search

Another refinement yields the C++ function of Program 1.10.

```
int BinarySearch (int *a, const int x, const int n)
{ // Search the sorted array a[0], ..., a[n-1] for x.
  int left = 0, right = n - 1;
  while (left <= right)
  { // there are more elements
    int middle = (left + right)/2;
    if ( $x < a[middle]$ ) right = middle-1;
    else if ( $x > a[middle]$ ) left = middle+1;
    else return middle;
  } // end of while
  return -1; // not found
}
```

Program 1.10: C++ function for binary search

To prove this program correct we make assertions about the relationship between variables before and after each iteration of the **while** loop. As we enter this loop

and if x is present in a , the following holds:

$$left \leq right \ \&\& \ a[left] \leq x \leq a[right] \ \&\& \ SORTED(a, n)$$

Now, if control passes out of the **while** loop, then we know the condition of **while** loop is false, so $left > right$. This, combined with the above assertion, implies that x is not present.

Unfortunately, a complete proof takes us beyond the scope of this text, but those who wish to pursue program-proving should consult the references at the end of this chapter. $\square$

1.5.2 Recursive Algorithms

We have emphasized the need to structure a program to make it easier to achieve the goals of readability and correctness. One of the most useful syntactical features for accomplishing this is the function. A set of instructions that perform a logical operation, perhaps a very complex and long operation, can be grouped together as a function. The function name and its parameters are viewed as a new instruction that can be used in other programs. Given the input-output specifications of a function, we do not even have to know how the task is accomplished, only that it is available. This view of the function implies that it is invoked, executed and returns control to the appropriate place in the calling function. What this fails to stress is that functions may call themselves (*direct recursion*) before they are done or they may call other functions that again invoke the calling function (*indirect recursion*). These recursive mechanisms are extremely powerful, but even more importantly, often they can express an otherwise complex process very clearly. For these reasons we introduce recursion here.

Recursion is similar to the method of induction which is often used to prove mathematical statements. In mathematical induction, a statement about integers (e.g., the sum of the first n positive integers is $n(n+1)/2$) is proved by showing that the statement can be proved for integer k if it is assumed to be true for integer $k-1$. Similarly, in recursion, we write a function to produce an output (say $n!$) for some input (here, n) by assuming that the same function will compute the correct output for input $n-1$. In mathematical induction, we need a basis which can be directly proved (that is, the proof for the basis does not make any assumptions). Similarly, a recursive function requires a terminating condition. When the input to the function satisfies this terminating condition, the function directly computes the output without calling itself.

What kinds of problems are best solved by recursion? Typically, beginning programmers view recursion as a somewhat mystical technique that is useful only for some very special class of problems (such as computing factorials or

30 Basic Concepts

Ackermann's function). This is unfortunate because any program that can be written using assignment, the **if-else** statement, and the **while** statement can also be written using assignment, **if-else**, and recursion. Of course, this does not mean that the resulting program will necessarily be easier to understand. However, there are many instances when this will be the case. When is recursion an appropriate mechanism for algorithm exposition? One instance is when the problem itself is recursively defined. Factorial fits this category, as well as binomial coefficients where

$$\binom{n}{m} = \frac{n!}{m!(n-m)!}$$

can be recursively computed by the formula

$$\binom{n}{m} = \binom{n-1}{m} + \binom{n-1}{m-1}$$

We use two examples to show you how to develop a recursive algorithm. In the first example, we take the binary search function that we created in Example 1.3 and transform it into a recursive function. In the second example, we generate all possible permutations of a list of characters. To understand a recursive function, you must

- (1) Formulate in your mind a statement of what it is that the function is supposed to do, for a given input.
- (2) Verify that the function does achieve its goal if the recursive invocations to itself do what they are supposed to.
- (3) Ensure that a finite number of recursive invocations of the function eventually lead to an invocation which satisfies the terminating condition (otherwise, the function will keep calling itself and not terminate!).
- (4) The function should perform the correct computations if the terminating condition is encountered.

Example 1.4 [Recursive binary search]: Program 1.10 gave the iterative version of binary search. In the recursive version we pass *left* and *right* as parameters (Program 1.11). The **for** loop of Program 1.10 has been replaced by recursive calls in Program 1.11. To invoke the recursive function, we use the statement

BinarySearch(a, x, 0, n-1);

You should verify that *BinarySearch* satisfies the four conditions stated above for recursive functions. Notice that both the iterative (Program 1.10) and recursive

(Program 1.11) functions perform the same computation. $\square$

```

int BinarySearch (int *a, const int x, const int left, const int right)
// Search the sorted array a[left], ..., a[right] for x
    if (left <= right) {
        int middle = (left + right)/2;
        if (x < a[middle]) return BinarySearch(a, x, left, middle - 1);
        else if (x > a[middle]) return BinarySearch(a, x, middle + 1, right);
        return middle;
    } // end of if
    return -1; // not found
}

```

Program 1.11: Recursive implementation of binary search

Example 1.5 [Permutation generator]: Given a set of $n \geq 1$ elements, the problem is to print all possible permutations of this set. For example if the set is $\{a, b, c\}$, then the set of permutations is $\{(a, b, c), (a, c, b), (b, a, c), (b, c, a), (c, a, b), (c, b, a)\}$. It is easy to see that given n elements, there are $n!$ different permutations. A simple algorithm can be obtained by looking at the case of four elements (a, b, c, d) . The answer can be constructed by writing

- (1) a followed by all permutations of (b, c, d)
- (2) b followed by all permutations of (a, c, d)
- (3) c followed by all permutations of (a, b, d)
- (4) d followed by all permutations of (a, b, c)

The expression “followed by all permutations” is the clue to recursion. It implies that we can solve the problem for a set with n elements if we have an algorithm that works on $n - 1$ elements. These observations lead to Program 1.12, which is invoked by *Permutations*($a, 0, n - 1$).

Try this algorithm out on sets of length one, two, and three to ensure that you understand how it works. $\square$

Another time when recursion is useful is when the data structure that the algorithm is to operate on is recursively defined. We shall see several important examples of such structures in this book.

32 Basic Concepts

```
void Permutations (char *a, const int k, const int m)
{ // Generate all the permutations of a[k], ..., a[m].
    if (k == m) { // output permutation
        for (int i = 0; i <= m; i++) cout << a[i] << " ";
        cout << endl;
    }
    else // a[k:m] has more than one permutation. Generate these recursively.
        for (i = k; i <= m; i++) {
            swap(a[k], a[i]);
            Permutations(a, k + 1, m);
            swap(a[k], a[i]);
        }
}
```

Program 1.12: Recursive permutation generator

EXERCISES

1. Horner's rule is a means for evaluating a polynomial $A(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$ at a point x_0 using a minimum number of multiplications. This rule is:

$$A(x) = (\cdots (a_n x_0 + a_{n-1}) x_0 + \cdots + a_1) x_0 + a_0$$

Write a C++ program to evaluate a polynomial using Horner's rule. Determine how many times each statement is executed.

2. Given n Boolean variables $x_1, \cdots, x_n$ we wish to print all possible combinations of truth values they can assume. For instance, if $n = 2$, there are four possibilities: true, true; true, false; false, true; false, false. Write a C++ program to accomplish this and do a frequency count.
3. Trace the action of the code

```
    i = 0; j = n - 1 ;
    do {
        k = (i + j)/2;
        if (a[k] <= x) i = k + 1;
        else j = k - 1; }
    while (i <= j);
```

on the elements 2, 4, 6, 8, 10, 12, 14, 16, 18, and 20 searching for $x = 1, 3, 13$, or 21.

4. Write a C++ program that prints out the integer values of x , y , and z in non-decreasing order. What is the computing time of your method?
5. Write a C++ function that searches an unsorted array $a[0:n-1]$ for the element x . If x occurs, then return the leftmost position of x in the array, else return -1 .
6. The factorial function $n!$ has value 1 when $n \leq 1$ and value $n*(n-1)!$ when $n > 1$. Write both a recursive and an iterative C++ function to compute $n!$.
7. The Fibonacci numbers are defined as: $f_0 = 0$, $f_1 = 1$, and $f_i = f_{i-1} + f_{i-2}$ for $i > 1$. Write both a recursive and an iterative C++ function to compute f_i .
8. Write a recursive function for computing the binomial coefficient $\binom{n}{m}$ as defined in Section 1.5.2, where $\binom{n}{0} = \binom{n}{n} = 1$. Analyze the time and space requirements of your algorithm.
9. Write an iterative function to compute a binomial coefficient; then transform it into an equivalent recursive function.
10. Ackermann's function $A(m, n)$ is defined as follows:

$$A(m, n) = \begin{cases} n + 1 & , \text{ if } m = 0 \\ A(m - 1, 1) & , \text{ if } n = 0 \\ A(m - 1, A(m, n - 1)) & , \text{ otherwise} \end{cases}$$

This function is studied because it grows very fast for small values of m and n . Write a recursive function for computing this function. Then write a nonrecursive algorithm for computing Ackermann's function.

11. The *pigeonhole principle* states that if a function f has n distinct inputs but less than n distinct outputs, then there exist two inputs a and b such that $a \neq b$ and $f(a) = f(b)$. Write a program to find the values a and b for which the range values are equal. Assume that the inputs are $1, 2, \dots, n$.
12. Given n , a positive integer, determine if n is the sum of all of its divisors — i.e., if n is the sum of all t such that $1 \leq t < n$, and t divides n .
13. Consider the function $F(x)$ defined by

if (x is even) $F = x / 2$;
 else $F = F(F(3x + 1))$;

Prove that $F(x)$ terminates for all integers x . (Hint: Consider integers of the form $(2i + 1)2^k - 1$ and use induction.)

14. If S is a set of n elements, the *powerset* of S is the set of all possible subsets of S . For example, if $S = (a, b, c)$, then $\text{powerset}(S) = \{(), (a), (b), (c), (a, b), (a, c), (b, c), (a, b, c)\}$. Write a recursive function to compute

34 Basic Concepts

powerset (S).

15. [**Towers of Hanoi**] There are three towers and sixty-four disks of different diameters placed on the first tower. The disks are in order of decreasing diameter as one scans up the tower. Monks were supposed to move the disks from tower 1 to tower 3 obeying the following rules: (a) only one disk can be moved at any time and (b) no disk can be placed on top of a disk with smaller diameter. Write a recursive function that prints the sequence of moves that accomplish this task.

1.6 THE STANDARD TEMPLATE LIBRARY

The C++ standard templates library (STL) is a collection of containers, adaptors, iterators, function objects (also known as functors) and algorithms. Through the judicious use of elements of the STL, the task of writing application codes is greatly simplified. In this section, we introduce a few of these elements. To use these STL elements in your programs, you should add the statement

```
#include <algorithm>
```

to your programs.

Example 1.6 [*The STL algorithm accumulate*]: The STL has an algorithm *accumulate* that may be used to sum the elements in a sequence. The syntax is

accumulate (start, end, initialValue)

where *start* points to the first element to be accumulated and *end* points to one position after the last element to be accumulated. So, elements in the range [*start*, *end*) are accumulated. The invocation

accumulate (a, a + n, initialValue)

where *a* is a one-dimensional array, for example, returns the value

$$initialValue + \sum_{i=0}^{n-1} a[i]$$

The STL algorithm *accumulate* accesses successive elements of the sequence to be summed by performing the ++ operator on *start* and terminating when the pointer value becomes *end*. So, this algorithm may be used to sum the values of any sequence whose elements may be obtained by repeated application

of the `++` operator. One-dimensional arrays and the STL container *vector* are two examples of sequences whose elements may be accessed in this way. We shall see other examples later in this book.

The STL has a more general form of the *accumulate* algorithm, which has the following syntax

accumulate (*start*, *end*, *initValue*, *operator*)

where *operator* is a function that defines the operation to be used during the accumulation process. Using the STL functor *multiplies*<int>, which multiplies two integers, we can find the product of an array of integers using the code of Program 1.13. □

```
int Product(int *a , int n)
{ // Return sum of the numbers a [0:n-1].
    int initVal = 1;
    return accumulate (a, a + n, initVal, multiplies<int>());
}
```

Program 1.13: Compute the product of the elements *a* [0:*n*-1]

Example 1.7: [The STL algorithms *copy* and *next_permutation*] The *copy* algorithm copies a sequence of elements from one location to another. The syntax is

copy (*start*, *end*, *to*)

where *to* gives the location to which the first element is to be copied. So, elements are copied from locations *start*, *start* + 1, ..., *end* - 1 to the locations *to*, *to* + 1, ..., *to* + *end* - *start*.

The algorithm *next_permutation*, which has the syntax

next_permutation (*start*, *end*)

creates the next lexicographically larger permutation of the elements in the range [*start*, *end*); it returns the value *true* if-and-only-if (iff) such a next permutation exists. By starting with the smallest lexicographic permutation of a sequence of distinct elements and making successive calls to *next_permutation*, we can obtain all permutations. Program 1.14 does just this. The invocation of *copy* in this program copies the elements *a* [0:*m*] to the output stream *cout*; each copied element is followed by a space (" "). Notice that Program 1.14 outputs no

36 Basic Concepts

permutation that is lexically smaller than the initial sequence whereas Program 1.15 outputs all permutations regardless of the initial sequence. The exercises examine ways to modify Program 1.14 so as to obtain all permutations.

```
void Permutations(char *a, const int m)
{ // Generate all permutations of a [0:m].
  // Output the permutations one by one
  do {
    copy(a, a + m + 1, ostream_iterator<char>(cout, " "));
    cout << endl;
  } while (next_permutation(a, a + m + 1));
}
```

Program 1.14: Permutations using the STL algorithm *next_permutation*

A more general form of the *next_permutation* algorithm takes a third parameter *compare* as in

next_permutation(start, end, compare)

When this form is used, the binary function *compare* is used to determine whether one element is smaller than another. In the two-parameter version, this comparison is done using the operator *<*. □

The STL contains many algorithms in addition to the ones used in the preceding two examples. The exercises explore STL algorithms further.

EXERCISES

1. Modify Program 1.14 so that it outputs all permutations of distinct elements. Do this by sorting the element list into ascending order prior to generating the permutations. To sort, use the STL algorithm

sort(start, end)

which sorts elements in the range [*start*, *end*) into ascending order. Test your code.

2. Modify Program 1.14 so that it outputs all permutations of distinct elements. Do this by first using *next_permutation* to generate permutations that are lexically larger than the initial permutation and then using the STL

algorithm *prev_permutation* to generate permutations that are lexicographically smaller than the initial permutation. Test your code.

3. Modify Program 1.14 so that it outputs all permutations of distinct elements. Do this by using the fact that when *next_permutation* returns the value *false* the sequence $[start, end)$ is the lexicographically smallest sequence. Hence, subsequent invocations of *next_permutation* will get you the remaining (if any) permutations you need. Test your code.
4. The STL algorithm *count*, which has the syntax

count(*start*, *end*, *value*)

returns the number of occurrences of *value* in the range $[start, end)$. Write a program that uses this algorithm to determine the number of occurrences of *a*[0] in the integer array *a* $[0:n-1]$. Test your code.

5. The STL algorithm *fill* which has the syntax

fill(*start*, *end*, *value*)

sets all positions in the range $[start, end)$ to *value*. Write a program that creates an integer array of a specified size and initializes all positions of this array to 0. Test your code.

1.7 PERFORMANCE ANALYSIS AND MEASUREMENT

One goal of this book is to develop skills for making evaluative judgments about programs. There are many criteria upon which we can judge a program, for instance:

- (1) Does it do what we want it to do?
- (2) Does it work correctly according to the original specifications of the task?
- (3) Is there documentation that describes how to use it and how it works?
- (4) Are functions created in such a way that they perform logical subfunctions?
- (5) Is the code readable?

The above criteria are all vitally important when it comes to writing software, most especially for large systems. Though we will not be discussing how to reach these goals, we will try to achieve them throughout this book with the programs we write. Hopefully this more subtle approach will gradually infect your

38 Basic Concepts

own program-writing habits so that you will automatically strive to achieve these goals.

There are other criteria for judging programs that have a more direct relationship to performance. These have to do with their computing time and storage requirements.

Definition: The *space complexity* of a program is the amount of memory it needs to run to completion. The *time complexity* of a program is the amount of computer time it needs to run to completion. $\square$

Performance evaluation can be loosely divided into two major phases: (1) a priori estimates and (2) a posteriori testing. We refer to these as *performance analysis* and *performance measurement* respectively.

1.7.1 Performance Analysis

1.7.1.1 Space Complexity

Function *Abc* (Program 1.16) computes the expression $a + b + b * c + (a + b - c) / (a + b) + 4.0$; function *Sum* (Program 1.17) computes the sum $\sum_{i=0}^{n-1} a[i]$ iteratively, where the $a[i]$'s are of type **float**; and function *Rsum* (Program 1.18) is a recursive program that computes $\sum_{i=0}^{n-1} a[i]$.

```
float Abc(float a, float b, float c)
{
    return a + b + b * c + (a + b - c) / (a + b) + 4.0;
}
```

Program 1.16: Function to compute $a + b + b * c + (a + b - c) / (a + b) + 4.0$

The space needed by each of these programs is seen to be the sum of the following components:

- (1) A fixed part that is independent of the characteristics (e.g., number, size) of the inputs and outputs. This part typically includes the instruction space (i.e., space for the code), space for simple variables and fixed-size component variables (also called *aggregate*), space for constants, etc.

```

line float Sum (float *a, const int n)
1 {
2   float s = 0;
3   for (int i = 0; i < n; i++)
4     s += a[i];
5   return s;
6 }

```

Program 1.17: Iterative function for sum

```

line   float Rsum (float *a, const int n)
1     {
2       if (n <= 0) return 0;
3       else return (Rsum (a, n-1) + a [n-1]);
4     }

```

Program 1.18: Recursive function for sum

- (2) A variable part that consists of the space needed by component variables whose size is dependent on the particular problem instance being solved, the space needed by referenced variables (to the extent that this depends on instance characteristics), and the recursion stack space (insofar as this space depends on the instance characteristics).

The space requirement $S(P)$ of any program P may therefore be written as $S(P) = c + S_P(\text{instance characteristics})$ where c is a constant.

When analyzing the space complexity of a program, we shall concentrate solely on estimating S_P (instance characteristics). For any given problem, we shall need to first determine which instance characteristics to use to measure the space requirements. This is very problem-specific, and we shall resort to examples to illustrate the various possibilities. Generally speaking, our choices are limited to quantities related to the number and magnitude of the inputs to and outputs from the program. At times, more complex measures of the interrelationships among the data items are used.

Example 1.8: For Program 1.16, the problem instance is characterized by the specific values of a , b , and c . Making the assumption that one word is adequate

40 Basic Concepts

to store the values of each of a , b , c , and the value returned by Abc , we see that the space needed by function Abc is independent of the instance characteristics. Consequently, $S_P(\text{instance characteristics}) = 0$. $\square$

Example 1.9: The problem instances for Program 1.17 are characterized by n , the number of elements to be summed. Since n is passed by value, one word must be allocated for it. Since a is actually the address of the first element in $a[]$ (i.e. $a[0]$), the space needed by it is also one word. So, the space needed by the function is independent of n and $S_{Sum}(n) = 0$. $\square$

Example 1.10: Let us consider the function $Rsum$. As in the case of Sum , the instances are characterized by n . The recursion stack space includes space for the formal parameters, the local variables, and the return address. Since a is the address of $a[0]$, it requires only one word of memory on the stack. Assume that the return address requires only one word of memory. Each call to $Rsum$ requires at least 4 words (including space for the values of n , a , the returned value, and the return address). Since the depth of recursion is $n+1$, the recursion stack space needed is $4(n+1)$. For $n = 1000$, the recursion stack space is 4004. $\square$

1.7.1.2 Time Complexity

The time, $T(P)$, taken by a program P is the sum of the compile time and the run (or execution) time. The compile time does not depend on the instance characteristics. Also, we may assume that a compiled program will be run several times without recompilation. Consequently, we shall concern ourselves with just the run time of a program. This run time is denoted by t_P (instance characteristics).

Because many of the factors t_P depends on are not known at the time a program is conceived, it is reasonable to attempt only to estimate t_P . If we knew the characteristics of the compiler to be used, we could proceed to determine the number of additions, subtractions, multiplications, divisions, compares, loads, stores, and so on that would be made by the code for P . So, we could obtain an expression for $t_P(n)$ of the form

$$t_P(n) = c_a ADD(n) + c_s SUB(n) + c_m MUL(n) + c_d DIV(n) + \dots$$

where n denotes the instance characteristics, and c_a , c_s , c_m , c_d , etc., respectively, denote the time needed for an addition, subtraction, multiplication, division, etc., and ADD , SUB , MUL , DIV , etc., are functions whose value is the number of additions, subtractions, multiplications, divisions, etc., that will be performed when the code for P is used on an instance with characteristic n .

Obtaining such an exact formula is in itself an impossible task, since the

time needed for an addition, subtraction, multiplication, etc., often depends on the actual numbers being added, subtracted, multiplied, etc. In reality then, the true value of $t_p(n)$ for any given n can be obtained only experimentally. The program is typed, compiled, and run on a particular machine. The execution time is physically clocked and $t_p(n)$ obtained. Even with this experimental approach, one could face difficulties. In a multiuser system, the execution time will depend on such factors as system load, the number of other programs running on the computer at the time program P is run, the characteristics of these other programs, and so on.

Given the minimal utility of determining the exact number of additions, subtractions, etc., that are needed to solve a problem instance with characteristics given by n , we might as well lump all the operations together (provided that the time required by each is relatively independent of the instance characteristics) and obtain a count for the total number of operations. We can go one step further and count only the number of program steps.

A *program step* is loosely defined as a syntactically or semantically meaningful segment of a program that has an execution time that is independent of the instance characteristics. For example, the entire statement

```
return  $a + b + b * c + (a + b - c) / (a + b) + 4.0;$ 
```

of Program 1.16 could be regarded as a step since its execution time is independent of the instance characteristics (this statement is not strictly true, since the time for a multiply and divide will generally depend on the actual numbers involved in the operation).

The number of steps any program statement is to be assigned depends on the nature of that statement. The following discussion considers the various statement types that can appear in a C++ program and states the complexity of each in terms of the number of steps:

- (1) *Comments.* Comments are not executable statements and have a step count of zero.
- (2) *Declarative statements.* This category includes all statements that define or characterize variables and constants (**int**, **long**, **short**, **char**, **float**, **double**, **const**, **enum**, **signed**, **unsigned**, **static**, **extern**), all statements that enable users to define their own data types (**class**, **struct**, **union**, **template**), all statements that determine access (**private**, **public**, **protected**, **friend**), and all statements that characterize functions (**void**, **virtual**). These count as zero steps, since these are either not executable or their cost may be lumped into the cost of invoking the function they are associated with.
- (3) *Expressions and assignment statements.* Most expressions have a step count of one. The exceptions are expressions that contain function calls.

42 Basic Concepts

In this case, we need to determine the cost of invoking the functions. This cost can be large if the functions employ many-element pass-by-value parameters because the values of all actual parameters need to be assigned to the formal parameters. This is discussed further under function and function invocation. When the expression contains functions, the step count is the sum of the step counts assignable to each function invocation.

The assignment statement $\langle \text{variable} \rangle = \langle \text{expr} \rangle$ has a step count equal to that of $\langle \text{expr} \rangle$ unless the size of $\langle \text{variable} \rangle$ is a function of the instance characteristics. In this latter case, the step count is the size of $\langle \text{variable} \rangle$ plus the step count of $\langle \text{expr} \rangle$. For example, the assignment $a = b$, where a and b are of type *ElementList*, has a step count equal to the size of *ElementList*.

- (4) *Iteration statements.* This class of statements includes the **for**, **while**, and **do** statements. We shall consider the step counts only for the control part of these statements. These have the following form:

```
for (<init-stmt>; <expr1>; <expr2>)
while <expr> do
do ...while <expr>
```

Each execution of the control part of a **while** and **do** statement will be given a step count equal to the number of step counts assignable to $\langle \text{expr} \rangle$. The step count for each execution of the control part of a **for** statement is one, unless the counts attributable to $\langle \text{init-stmt} \rangle$, $\langle \text{expr1} \rangle$, or $\langle \text{expr2} \rangle$ are a function of the instance characteristics. In this latter case, the first execution of the control part of the **for** has a step count equal to the sum of the counts for $\langle \text{init-stmt} \rangle$ and $\langle \text{expr1} \rangle$; subsequent executions of the **for** statement have a step count equal to the sum of the step counts for $\langle \text{expr1} \rangle$ and $\langle \text{expr2} \rangle$.

- (5) *Switch statement.* This statement consists of a header followed by one or more sets of condition-statement pairs. We shall once again consider the costs for the control part of the statement.

```
switch (<expr>) {
    case cond1: <statement1>
    case cond2: <statement2>
    .
    .
    default: <statement>
}
```

The header **switch** (<expr>) is given a cost equal to that assignable to <expr>. The cost of each condition is its cost plus that of all preceding conditions.

- (6) *If-else statement.* The **if-else** statement consists of three parts:

```
if (<expr>) <statements1>;
else <statements2>;
```

Each part is assigned the number of steps corresponding to <expr>, <statements1>, and <statements2> respectively. Note that if the **else** clause is absent, then no cost is assigned to it. Step counts are computed for the **arithmetic-if** operation in a similar manner.

- (7) *Function invocation.* All invocations of functions count as one step unless the invocation involves pass-by-value parameters whose size depends on the instance characteristics. In this latter case, the count is the sum of the sizes of these value parameters. If the function being invoked is recursive, then we must also consider the local variables in the function being invoked. The sizes of local variables that are characteristic-dependent need to be added to the step count.
- (8) *Memory management statements.* These include **new object**, **delete object**, and **sizeof(object)**. The step count associated with each is 1. The statements **new** and **delete** can potentially invoke the constructor and destructor for *object*, respectively. In this case, the step counts are computed in a similar manner to a function invocation.
- (9) *Function statements.* These count as zero steps because their cost has already been assigned to the invoking statements.
- (10) *Jump statements.* These include **continue**, **break**, **goto**, **return**, and **return <expr>**. Each has a step count of one, with the possible exception of **return <expr>**. This has a step count of 1 unless the step count attributable to <expr> is a function of instance characteristics. In this case, the step count is the cost of <expr>.

With the above assignment of step counts to statements, we can proceed to determine the number of steps needed by a program to solve a particular problem instance. We can go about this in one of two ways. In the first method, we introduce a new variable, *count*, into the program. This is a global variable with initial value 0. Statements to increment *count* by the appropriate amount are introduced into the program. This is done so that each time a statement in the original program is executed, *count* is incremented by the step count of that statement.

Example 1.11: When the statements to increment *count* are introduced into Program 1.17, the result is Program 1.19. The change in the value of *count* by the

44 Basic Concepts

time this program terminates is the number of steps executed by Program 1.17.

Since we are interested in determining only the change in the value of *count*, Program 1.19 may be simplified to Program 1.20. It should be easy to see that for every initial value of *count*, Program 1.19 and Program 1.20 compute the same final value for *count*. It is easy to see that in the **for** loop, the value of *count* will increase by a total of $2n$. If *count* is zero to start with, then it will be $2n+3$ on termination. So, each invocation of *Sum* (Program 1.17) executes a total of $2n+3$ steps. $\square$

```
float Sum (float *a, const int n)
{
    float s = 0;
    count++; // count is global
    for (int i = 0; i < n; i++) {
        count++; // for for
        s += a[i];
        count++; // for assignment
    }
    count++; // for last time of for
    count++; // for return
    return s;
}
```

Program 1.19: Program 1.17 with count statements added

```
void Sum (float *a, const int n)
{
    for (int i = 0; i < n; i++)
        count += 2;
    count += 3;
}
```

Program 1.20: Simplified version of Program 1.19

Example 1.12: When the statements to increment *count* are introduced into Program 1.18, Program 1.21 is obtained. Let $t_{Rsum}(n)$ be the increase in the value of *count* when Program 1.21 terminates. We see that $t_{Rsum}(0) = 2$. When $n > 0$, *count* increases by 2 plus whatever increase results from the invocation of *Rsum*

from within the **else** clause. From the definition of t_{Rsum} , it follows that this additional increase is $t_{Rsum}(n-1)$. So, if the value of *count* is zero initially, its value at the time of termination is $2+t_{Rsum}(n-1)$, $n > 0$.

```
float Rsum (float *a, const int n)
{
    count++; // for if conditional
    if (n <= 0) {
        count++; // for return
        return 0;
    }
    else {
        count++ ; // for return
        return (Rsum(a, n - 1) + a[n - 1]);
    }
}
```

Program 1.21: Program 1.18 with count statements added

When analyzing a recursive program for its step count, we often obtain a recursive formula for the step count (i.e., say $t_{Rsum}(n) = 2+t_{Rsum}(n-1)$, $n > 0$ and $t_{Rsum}(0) = 2$). These recursive formulas are referred to as *recurrence relations*. This recurrence may be solved by repeatedly substituting for t_{Rsum} as below:

$$\begin{aligned}
 t_{Rsum}(n) &= 2 + t_{Rsum}(n-1) \\
 &= 2 + 2 + t_{Rsum}(n-2) \\
 &= 2 \times 2 + t_{Rsum}(n-2) \\
 &\vdots \\
 &= 2n + t_{Rsum}(0) \\
 &= 2n + 2
 \end{aligned}$$

So, the step count for function *Rsum* (Program 1.18) is $2n + 2$. $\square$

Comparing the step count of Program 1.17 to that of Program 1.18, we see that the count for Program 1.18 is less than that for Program 1.17. From this, we cannot conclude that Program 1.17 is slower than Program 1.18. This is so because a step does not correspond to a definite time unit. Each step of *Rsum* may take more time than every step of *Sum*. So, it might well be (and we expect)

46 Basic Concepts

that *Rsum* is slower than *Sum*.

The step count is useful in that it tells us how the run time for a program changes with changes in the instance characteristics. From the step count for *Sum*, we see that if n is doubled, the run time will also double (approximately); if n increases by a factor of 10, we expect the run time to increase by a factor of 10; and so on. So, we expect the run time to grow *linearly* in n .

Example 1.13 [Matrix addition]: Program 1.22 is a program to add two $m \times n$ matrices a and b together. Note that the argument `**a` refers to a two-dimensional array $a[][]$. Introducing the *count*-incrementing statements leads to Program 1.23. Program 1.24 is a simplified version of Program 1.23 that computes the same value for *count*. Examining Program 1.24, we see that line 5 is executed n times for each value of i or a total of mn times; line 6 is executed m times; and line 8 is executed once. If *count* is zero to begin with, it will be $2mn + 2m + 1$ when Program 1.24 terminates.

From this analysis we see that if $m > n$, then it is better to interchange the two **for** statements in Program 1.22. If this is done, the step count becomes $2mn + 2n + 1$. Note that in this example the instance characteristics are given by m and n . $\square$

```
line void Add (int **a, int **b, int **c, int m, int n)
```

```
1 {
2   for (int i = 0; i < m; i++)
3     for (int j = 0; j < n; j++)
4       c[i][j] = a[i][j] + b[i][j];
5 }
```

Program 1.22: Matrix addition

The second method to determine the step count of a program is to build a table in which we list the total number of steps contributed by each statement. This figure is often arrived at by first determining the number of steps per execution of the statement and the total number of times (i.e., frequency) each statement is executed. By combining these two quantities, the total contribution of each statement is obtained. By adding up the contributions of all statements, the step count for the entire program is obtained.

There is an important difference between the step count of a statement and its steps per execution (s/e). The step count does not necessarily reflect the complexity of the statement. For example, the statement

$$x = \text{Sum}(a, m);$$

```

line void Add (int **a, int **b, int **c, int m, int n)
{
    for (int i = 0; i < m ; i++)
    {
        count++; // for for i
        for (int j = 0; j < n; j++)
        {
            count++; // for for j
            c[i][j] = a[i][j] + b[i][j];
            count++; // for assignment
        }
        count++; // for last time of for j
    }
    count++; // for last time of for i
}

```

Program 1.23: Matrix addition with counting statements

```

line void Add (int **a, int **b, int **c, int m, int n)
1      {
2          for (int i = 0; i < m; i++)
3          {
4              for (int j = 0; j < n ; j++)
5                  count += 2;
6              count += 2;
7          }
8          count++;
9      }

```

Program 1.24: Simplified program with counting only

has a step count of 1, while the total change in *count* resulting from the execution of this statement is actually 1 plus the change resulting from the invocation of *Sum* (i.e., $2m+3$). The steps per execution of the above statement is $1+2m+3 = 2m+4$. The *s/e* of a statement is the amount by which *count* changes as a result of the execution of that statement.

In Table 1.1, the number of steps per execution and the frequency of each of the statements in function *Sum* (Program 1.17) have been listed. The total

48 Basic Concepts

number of steps required by the program is determined to be $2n+3$. It is important to note that the frequency of line 3 is $n+1$ and not n . This is so because i has to be incremented to n before the **for** loop can terminate.

line	s/e	frequency	total steps
1	0	1	0
2	1	1	1
3	1	$n+1$	$n+1$
4	1	n	n
5	1	1	1
6	0	1	0
Total number of steps			$2n+3$

Table 1.1: Step table for Program 1.17

Table 1.2 gives the step count for function *Rsum* (Program 1.18). Line 2(a) refers to the **if** conditional of line 2, and line 2(b) refers to the statement in the **if** clause. Notice that under the s/e (steps per execution) column, line 3 has been given a count of $1+t_{Rsum}(n-1)$. This is the total cost of line 3 each time it is executed. It includes all the steps that get executed as a result of the invocation of *Rsum* from line 3. The frequency and total steps columns have been split into two parts: one for the case $n=0$ and the other for the case $n>0$. This is necessary because the frequency (and hence total steps) for some statements is different for each of these cases.

line	s/e	frequency		total steps	
		$n=0$	$n>0$	$n=0$	$n>0$
1	0	1	1	0	0
2(a)	1	1	1	1	1
2(b)	1	1	0	1	0
3	$1+t_{Rsum}(n-1)$	0	1	0	$1+t_{Rsum}(n-1)$
4	0	1	1	0	0
Total number of steps				2	$2+t_{Rsum}(n-1)$

Table 1.2: Step table for Program 1.18

Table 1.3 corresponds to function *Add* (Program 1.22). Once again, note

that the frequency of line 2 is $m+1$ and not m . This is so as i needs to be incremented up to m before the loop can terminate. Similarly, the frequency for line 3 is $m(n+1)$. When you have obtained sufficient experience in computing step counts, you may avoid constructing the frequency table and obtain the step count as in the following example.

line	s/e	frequency	total steps
1	0	1	0
2	1	$m+1$	$m+1$
3	1	$m(n+1)$	$mn+m$
4	1	mn	mn
5	0	1	0
Total number of steps			$2mn+2m+1$

Table 1.3: Step table for Program 1.22

Example 1.14 [Fibonacci numbers]: The Fibonacci sequence of numbers starts as

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \dots$$

Each new term is obtained by taking the sum of the two previous terms. If we call the first term of the sequence F_0 then $F_0 = 0$, $F_1 = 1$, and in general

$$F_n = F_{n-1} + F_{n-2}, n \geq 2.$$

The program *Fibonacci* (Program 1.25) inputs any nonnegative integer n and prints the value F_n .

To analyze the time complexity of this program, we need to consider the two cases: (1) $n = 0$ or 1 and (2) $n > 1$. Line 3 will be regarded as two lines: 3(a), the conditional part, and 3(b), the *if* clause. When $n = 0$ or 1, lines 3(a) and 3(b), get executed once each. Since each line has an s/e of 1, the total step count for this case is 2. When $n > 1$, lines 3(a), 5, and 12 are each executed once. Line 6 gets executed n times, and lines 7-11 get executed $n-1$ times each (note that the last time line 6 is executed, i is incremented to $n+1$ and the loop exited). Line 5 has an s/e of 2; the remaining lines that get executed have an s/e of 1. The total steps for the case $n > 1$ is therefore $4n+1$. $\square$

50 Basic Concepts

```
1 void Fibonacci (int n)
2 // compute the Fibonacci number  $F_n$ 
3 if (n <= 1) cout << n << endl; //  $F_0 = 0$  and  $F_1 = 1$ 
4 else { // compute  $F_n$ 
5     int fn; int fnm 2 = 0; int fnm 1 = 1;
6     for (int i = 2; i <= n; i++)
7     {
8         fn = fnm 1 + fnm 2 ;
9         fnm 2 = fnm 1 ;
10        fnm 1 = fn ;
11    } // end of for
12    cout << fn << endl;
13 } // end of else
14 }
```

Program 1.25: Fibonacci numbers

Summary

The time complexity of a program is given by the number of steps taken by the program to compute the function it was written for. The number of steps is itself a function of the instance characteristics. Although any specific instance may have several characteristics (e.g., the number of inputs, the number of outputs, the magnitudes of the inputs and outputs), the number of steps is computed as a function of some subset of these. Usually, we choose those characteristics that are of importance to us. For example, we might wish to know how the computing (or run) time (i.e., time complexity) increases as the number of inputs increase. In this case the number of steps will be computed as a function of the number of inputs alone. For a different program, we might be interested in determining how the computing time increases as the magnitude of one of the inputs increases. In this case the number of steps will be computed as a function of the magnitude of this input alone. Thus, before the step count of a program can be determined, we need to know exactly which characteristics of the problem instance are to be used. These define the variables in the expression for the step count. In the case of *Sum*, we chose to measure the time complexity as a function of the number, n , of elements being added. For function *Add*, the choice of characteristics was the number, m , of rows and the number, n , of columns in the matrices being added.

Once the relevant characteristics ($n, m, p, q, r, \dots$) have been selected, we can define what a step is. A step is any computation unit that is independent of the characteristics ($n, m, p, q, r, \dots$). Thus, 10 additions can be one step;

100 multiplications can also be one step; but n additions cannot. Nor can $m/2$ additions, $p+q$ subtractions, and so on be counted as one step.

A systematic way to assign step counts was also discussed. Once this has been done, the time complexity (i.e., the total step count) of a program can be obtained using either of the two methods discussed.

The examples we have looked at so far were sufficiently simple that the time complexities were nice functions of fairly simple characteristics like the number of elements, and the number of rows and columns. For many programs, the time complexity is not dependent solely on the number of inputs or outputs or some other easily specified characteristic. Consider the function *BinarySearch* (Program 1.10). This function searches $a[0], \dots, a[n-1]$ for x . A natural parameter with respect to which you might wish to determine the step count is the number, n , of elements to be searched. That is, we would like to know how the computing time changes as we change the number of elements n . The parameter n is inadequate. For the same n , the step count varies with the position of x in a . We can extricate ourselves from the difficulties resulting from situations wherein the chosen parameters are not adequate to determine the step count uniquely by defining three kinds of step counts: best-case, worst-case, and average.

The *best-case step count* is the minimum number of steps that can be executed for the given parameters. The *worst-case step count* is the maximum number of steps that can be executed for the given parameters. The *average step count* is the average number of steps executed on instances with the given parameters.

1.7.1.3 Asymptotic Notation (O , Ω , Θ)

Our motivation to determine step counts is to be able to compare the time complexities of two programs that compute the same function and also to predict the growth in run time as the instance characteristics change.

Determining the exact step count (either worst-case or average) of a program can prove to be an exceedingly difficult task. Expending immense effort to determine the step count exactly is not a very worthwhile endeavor, since the notion of a step is itself inexact. (Both the instructions $x = y$ and $x = y + z + (x/y) + (x*y*z - x/z)$ count as one step.) Because of the inexactness of what a step stands for, the exact step count is not very useful for comparative purposes. An exception to this is when the difference in the step counts of two programs is very large, as in $3n + 3$ versus $100n + 10$. We might feel quite safe in predicting that the program with step count $3n + 3$ will run in less time than the one with step count $100n + 10$. But even in this case, it is not necessary to know that the exact step count is $100n + 10$. Something like, "it's about $80n$, or $85n$, or $75n$," is adequate to arrive at the same conclusion.

52 Basic Concepts

For most situations, it is adequate to be able to make a statement like $c_1n^2 \leq t_P(n) \leq c_2n^2$ or $t_Q(n, m) = c_1n + c_2m$ where c_1 and c_2 are nonnegative constants. This is so because if we have two programs with a complexity of $c_1n^2 + c_2n$ and c_3n respectively, then we know that the one with complexity c_3n will be faster than the one with complexity $c_1n^2 + c_2n$ for sufficiently large values of n . For small values of n , either program could be faster (depending on c_1 , c_2 , and c_3). If $c_1 = 1$, $c_2 = 2$, and $c_3 = 100$ then $c_1n^2 + c_2n \leq c_3n$ for $n \leq 98$, and $c_1n^2 + c_2n > c_3n$ for $n > 98$. If $c_1 = 1$, $c_2 = 2$, and $c_3 = 1000$, then $c_1n^2 + c_2n \leq c_3n$ for $n \leq 998$.

No matter what the values of c_1 , c_2 , and c_3 , there will be an n beyond which the program with complexity c_3n will be faster than the one with complexity $c_1n^2 + c_2n$. This value of n will be called the *break-even point*. If the break-even point is zero, then the program with complexity c_3n is always faster (or at least as fast). The exact break-even point cannot be determined analytically. The programs have to be run on a computer in order to determine the break-even point. To know that there is a break-even point, it is adequate to know that one program has complexity $c_1n^2 + c_2n$ and the other c_3n for some constants c_1 , c_2 , and c_3 . There is little advantage in determining the exact values of c_1 , c_2 , and c_3 .

With the previous discussion as motivation, we introduce some terminology that will enable us to make meaningful (but inexact) statements about the time and space complexities of a program. In the remainder of this chapter, the functions f and g are nonnegative functions.

Definition [Big "oh"]: $f(n) = O(g(n))$ (read as " f of n is big oh of g of n ") iff (if and only if) there exist positive constants c and n_0 such that $f(n) \leq cg(n)$ for all $n, n \geq n_0$. $\square$

Example 1.15: $3n + 2 = O(n)$ as $3n + 2 \leq 4n$ for all $n \geq 2$. $3n + 3 = O(n)$ as $3n + 3 \leq 4n$ for all $n \geq 3$. $100n + 6 = O(n)$ as $100n + 6 \leq 101n$ for $n \geq 10$. $10n^2 + 4n + 2 = O(n^2)$ as $10n^2 + 4n + 2 \leq 11n^2$ for $n \geq 5$. $1000n^2 + 100n - 6 = O(n^2)$ as $1000n^2 + 100n - 6 \leq 1001n^2$ for $n \geq 100$. $6 \cdot 2^n + n^2 = O(2^n)$ as $6 \cdot 2^n + n^2 \leq 7 \cdot 2^n$ for $n \geq 4$. $3n + 3 = O(n^2)$ as $3n + 3 \leq 3n^2$ for $n \geq 2$. $10n^2 + 4n + 2 = O(n^4)$ as $10n^2 + 4n + 2 \leq 10n^4$ for $n \geq 2$. $3n + 2 \neq O(1)$ as $3n + 2$ is not less than or equal to c for any constant c and all $n, n \geq n_0$. $10n^2 + 4n + 2 \neq O(n)$. $\square$

We write $O(1)$ to mean a computing time that is a constant. $O(n)$ is called linear, $O(n^2)$ is called quadratic, $O(n^3)$ is called cubic, and $O(2^n)$ is called exponential. If an algorithm takes time $O(\log n)$, it is faster, for sufficiently large n , than if it had taken $O(n)$. Similarly, $O(n \log n)$ is better than $O(n^2)$ but not as good as $O(n)$. These seven computing times, $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$, and $O(2^n)$ are the ones we will see most often in this book.

As illustrated by the previous example, the statement $f(n) = O(g(n))$ states only that $g(n)$ is an upper bound on the value of $f(n)$ for all $n, n \geq n_0$. It does not say anything about how good this bound is. Notice that $n = O(n^2)$, $n = O(n^{2.5})$, $n = O(n^3)$, $n = O(2^n)$, and so on. For the statement $f(n) = O(g(n))$ to be informative, $g(n)$ should be as small a function of n as one can come up with for which $f(n) = O(g(n))$. So, while we shall often say $3n + 3 = O(n)$, we shall almost never say $3n + 3 = O(n^2)$, even though this latter statement is correct.

From the definition of O , it should be clear that $f(n) = O(g(n))$ is not the same as $O(g(n)) = f(n)$. In fact, it is meaningless to say that $O(g(n)) = f(n)$. The use of the symbol “=” is unfortunate because this symbol commonly denotes the “equals” relation. Some of the confusion that results from the use of this symbol (which is standard terminology) can be avoided by reading the symbol “=” as “is” and not as “equals.”

Theorem 1.2 obtains a very useful result concerning the order of $f(n)$ (i.e., the $g(n)$ in $f(n) = O(g(n))$) when $f(n)$ is a polynomial in n .

Theorem 1.2: If $f(n) = a_m n^m + \cdots + a_1 n + a_0$, then $f(n) = O(n^m)$.

$$\begin{aligned} \text{Proof: } f(n) &\leq \sum_{i=0}^m |a_i| n^i \\ &\leq n^m \sum_{i=0}^m |a_i| n^{i-m} \\ &\leq n^m \sum_{i=0}^m |a_i|, \text{ for } n \geq 1 \end{aligned}$$

So, $f(n) = O(n^m)$. $\square$

Definition: [Omega] $f(n) = \Omega(g(n))$ (read as “ f of n is omega of g of n ”) iff there exist positive constants c and n_0 such that $f(n) \geq c g(n)$ for all $n, n \geq n_0$. $\square$

Example 1.16: $3n + 2 = \Omega(n)$ as $3n + 2 \geq 3n$ for $n \geq 1$ (actually the inequality holds for $n \geq 0$, but the definition of Ω requires an $n_0 > 0$). $3n + 3 = \Omega(n)$ as $3n + 3 \geq 3n$ for $n \geq 1$. $100n + 6 = \Omega(n)$ as $100n + 6 \geq 100n$ for $n \geq 1$. $10n^2 + 4n + 2 = \Omega(n^2)$ as $10n^2 + 4n + 2 \geq n^2$ for $n \geq 1$. $6 \cdot 2^n + n^2 = \Omega(2^n)$ as $6 \cdot 2^n + n^2 \geq 2^n$ for $n \geq 1$. Observe also that $3n + 3 = \Omega(1)$; $10n^2 + 4n + 2 = \Omega(n)$; $10n^2 + 4n + 2 = \Omega(1)$; $6 \cdot 2^n + n^2 = \Omega(n^{100})$; $6 \cdot 2^n + n^2 = \Omega(n^{50.2})$; $6 \cdot 2^n + n^2 = \Omega(n^2)$; $6 \cdot 2^n + n^2 = \Omega(n)$; and $6 \cdot 2^n + n^2 = \Omega(1)$. $\square$

As in the case of the “big oh” notation, there are several functions $g(n)$ for which $f(n) = \Omega(g(n))$. The function $g(n)$ is only a lower bound on $f(n)$. For the

54 Basic Concepts

statement $f(n) = \Omega(g(n))$ to be informative, $g(n)$ should be as large a function of n as possible for which the statement $f(n) = \Omega(g(n))$ is true. So, while we shall say that $3n + 3 = \Omega(n)$ and $6 \cdot 2^n + n^2 = \Omega(2^n)$, we shall almost never say that $3n + 3 = \Omega(1)$ or $6 \cdot 2^n + n^2 = \Omega(1)$, even though both of these statements are correct.

Theorem 1.3 is the analogue of Theorem 1.2 for the omega notation.

Theorem 1.3: If $f(n) = a_m n^m + \cdots + a_1 n + a_0$ and $a_m > 0$, then $f(n) = \Omega(n^m)$.

Proof: Left as an exercise. $\square$

Definition: [Theta] $f(n) = \Theta(g(n))$ (read as “ f of n is theta of g of n ”) iff there exist positive constants c_1 , c_2 , and n_0 such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all n , $n \geq n_0$. $\square$

Example 1.17: $3n + 2 = \Theta(n)$ as $3n + 2 \geq 3n$ for all $n \geq 2$, and $3n + 2 \leq 4n$ for all $n \geq 2$, so $c_1 = 3$, $c_2 = 4$, and $n_0 = 2$. $3n + 3 = \Theta(n)$; $10n^2 + 4n + 2 = \Theta(n^2)$; $6 \cdot 2^n + n^2 = \Theta(2^n)$; and $10 \cdot \log n + 4 = \Theta(\log n)$. $3n + 2 \neq \Theta(1)$; $3n + 3 \neq \Theta(n^2)$; $10n^2 + 4n + 2 \neq \Theta(n)$; $10n^2 + 4n + 2 \neq \Theta(1)$; $6 \cdot 2^n + n^2 \neq \Theta(n^2)$; $6 \cdot 2^n + n^2 \neq \Theta(n^{100})$; and $6 \cdot 2^n + n^2 \neq \Theta(1)$. $\square$

The theta notation is more precise than both the “big oh” and omega notations. $f(n) = \Theta(g(n))$ iff $g(n)$ is both an upper and lower bound on $f(n)$.

Notice that the coefficients in all of the $g(n)$ ’s used in the preceding three examples have been 1. This is in accordance with practice. We shall almost never find ourselves saying that $3n + 3 = O(3n)$, or that $10 = O(100)$, or that $10n^2 + 4n + 2 = \Omega(4n^2)$, or that $6 \cdot 2^n + n^2 = \Omega(6 \cdot 2^n)$, or that $6 \cdot 2^n + n^2 = \Theta(4 \cdot 2^n)$, even though each of these statements is true.

Theorem 1.4: If $f(n) = a_m n^m + \cdots + a_1 n + a_0$ and $a_m > 0$, then $f(n) = \Theta(n^m)$.

Proof: Left as an exercise. $\square$

Let us reexamine the time complexity analyses of the previous section. For function *Sum* (Program 1.17) we had determined that $t_{\text{Sum}}(n) = 2n + 3$. So, $t_{\text{Sum}}(n) = \Theta(n)$. $t_{\text{Rsum}}(n) = 2n + 2 = \Theta(n)$ and $t_{\text{Add}}(m, n) = 2mn + 2n + 1 = \Theta(mn)$.

Although we might all see that the O , Ω , and Θ notations have been used correctly in the preceding paragraphs, we are still left with the question, Of what use are these notations if one has to first determine the step count exactly? The answer to this question is that the asymptotic complexity (i.e., the complexity in terms of O , Ω , and Θ) can be determined quite easily without determining the exact step count. This is usually done by first determining the asymptotic

complexity of each statement (or group of statements) in the program and then adding up these complexities. Tables 1.4 through 1.6 do just this for *Sum*, *Rsum*, and *Add* (Programs 1.17, 1.18, and 1.22).

Note that in the table for *Add* (Table 1.6), lines 3 and 4 have been lumped together even though they have different frequencies. This lumping together of these two lines is possible because their frequencies are of the same order.

line(s)	s/e	frequency	total steps
1	0	—	$\Theta(0)$
2	1	1	$\Theta(1)$
3	1	$n+1$	$\Theta(n)$
4	1	n	$\Theta(n)$
5	1	1	$\Theta(1)$
6	0	—	$\Theta(0)$
$t_{Sum}(n) = \Theta(\max_{1 \leq i \leq 6} \{g_i(n)\}) = \Theta(n)$			

Table 1.4: Asymptotic complexity of *Sum* (Program 1.17)

line	s/e	frequency		total steps	
		$n = 0$	$n > 0$	$n = 0$	$n > 0$
1	0	—	—	0	$\Theta(0)$
2(a)	1	1	1	1	$\Theta(1)$
2(b)	1	1	0	1	$\Theta(0)$
3	$2 + t_{Rsum}(n-1)$	0	1	0	$\Theta(2 + t_{Rsum}(n-1))$
4	0	—	—	0	$\Theta(0)$
$t_{Rsum}(n) =$				2	$\Theta(2 + t_{Rsum}(n-1))$

Table 1.5: Asymptotic complexity of *Rsum* (Program 1.18)

Although the analyses of Tables 1.4 through 1.6 are actually carried out in terms of step counts, it is correct to interpret $t_P(n) = \Theta(g(n))$, or $t_P(n) = O(g(n))$, or $t_P(n) = \Omega(g(n))$ as a statement about the computing time of program *P*. This is so because each step takes only $\Theta(1)$ time to execute.

After you have had some experience using the table method, you will be in a position to arrive at the asymptotic complexity of a program by taking a more

56 Basic Concepts

line(s)	s/e	frequency	total steps
1	0	—	$\Theta(0)$
2	1	$\Theta(m)$	$\Theta(m)$
3,4	1	$\Theta(mn)$	$\Theta(mn)$
5	0	—	$\Theta(0)$
$t_{Add}(m,n) =$			$\Theta(mn)$

Table 1.6: Asymptotic complexity of *Add* (Program 1.22)

global approach. We elaborate on this method in the following examples.

Example 1.18 [*Permutation generator*]: Consider function *Permutations* (Program 1.12). Assume that a is of size n . When $k = n-1$, we see that the time taken is $\Theta(n)$. When $k < n-1$, the **else** clause is entered. At this time, the second **for** loop is entered $n-k$ times. Each iteration of this loop takes $\Theta(t_{Permutations}(k+1, n-1))$ time. So, $t_{Permutations}(k, n-1) = \Theta((n-k)(t_{Permutations}(k+1, n-1)))$ when $k < n-1$. Using the substitution method, we obtain $t_{Permutations}(0, n-1) = \Theta(n(n!))$, $n \geq 1$. $\square$

Example 1.19 [*Binary search*]: Let us obtain the time complexity of function *BinarySearch* (Program 1.10). The instance characteristic that we shall use is the number n of elements in a . Each iteration of the **for** loop takes $\Theta(1)$ time. We can show that the **for** loop is iterated at most $\lceil \log_2(n+1) \rceil$ times. Since an asymptotic analysis is being performed, we do not need such an accurate count of the worst-case number of iterations. Each iteration except for the last results in a decrease in the size of the segment of a that has to be searched by a factor of about 2. So, this loop is iterated $\Theta(\log n)$ times in the worst-case. As each iteration takes $\Theta(1)$ time, the overall worst-case complexity of *BinarySearch* is $\Theta(\log n)$. Note that, if a was passed by value, the complexity of using *BinarySearch* would be more than this because it would take $\Omega(n)$ time just to invoke the function. $\square$

Example 1.20 [*Magic square*]: Our final example is a problem from recreational mathematics. A magic square is an $n \times n$ matrix of the integers 1 to n^2 such that the sum of every row, column, and diagonal is the same. Figure 1.3 gives an example magic square for the case $n = 5$. In this example, the common sum is 65.

H. Coxeter has given the following simple rule for generating a magic

15	8	1	24	17
16	14	7	5	23
22	20	13	6	4
3	21	19	12	10
9	2	25	18	11

Figure 1.3: Example magic square

square when n is odd:

Start with 1 in the middle of the top row; then go up and left, assigning numbers in increasing order to empty squares; if you fall off the square imagine the same square as tiling the plane and continue; if a square is occupied, move down instead and continue.

The magic square of Figure 1.3 was formed using this rule. Program 1.26 is the C++ program for creating an $n \times n$ magic square for the case when n is odd. This results from Coxeter's rule.

The magic square is represented using a two-dimensional array having n rows and n columns. For this application it is convenient to number the rows (and columns) from zero to $n - 1$ rather than from one to n . Thus, when the program "falls off the square," i and/or j are set back to zero or $n - 1$.

The **while** loop is governed by the variable *key*, which is an integer variable initialized to 2 and increased by one each time through the loop. Thus, each statement within the **while** loop will be executed no more than $n^2 - 1$ times, and the computing time for *Magic* is $O(n^2)$. Since there are n^2 positions in which the algorithm must place a number, we see that $O(n^2)$ is the best bound an algorithm for the magic square problem can have. $\square$

58 Basic Concepts

```
void Magic (const int n)
{
    // Create a magic square of size n, n is odd.
    const int MaxSize = 51; // maximum square size
    int square[MaxSize][MaxSize], k, l;

    // check correctness of n
    if ((n > MaxSize) || (n < 1))
        throw "Error!..n out of range";
    else if (!(n%2)) throw "Error!..n is even";

    // n is odd. Coxeter's rule can be used
    for (int i = 0; i < n; i++) // initialize square to 0
        fill(square [i], square [i] + n, 0); // STL algorithm
    square[0][(n-1)/2] = 1; // middle of first row

    // i and j are current position
    int key = 2; int i = 0; int j = (n-1)/2;
    while (key <= n * n) {
        // move up and left
        if (i - 1 < 0) k = n - 1; else k = i - 1;
        if (j - 1 < 0) l = n - 1; else l = j - 1;
        if (square[k][l] i = (i+1)%n; // square occupied, move down
        else { // square [k][l] is unoccupied
            i = k; j = l;
        }
        square[i][j] = key;
        key++;
    } // end of while

    // output the magic square
    cout << "magic square of size " << n << endl;
    for (i = 0; i < n; i++) {
        copy(square [i], square [i] + n, ostream_iterator<int>(cout, " "));
        cout << endl;
    }
}
```

Program 1.26: Magic square

1.7.1.4 Practical Complexities

We have seen that the time complexity of a program is generally some function of the instance characteristics. This function is very useful in determining how the time requirements vary as the instance characteristics change. The complexity function may also be used to compare two programs P and Q that perform the same task. Assume that program P has complexity $\Theta(n)$ and program Q is of complexity $\Theta(n^2)$. We can assert that program P is faster than program Q for sufficiently large n . To see the validity of this assertion, observe that the actual computing time of P is bounded from above by cn for some constant c and for all $n, n \geq n_1$, whereas that of Q is bounded from below by dn^2 for some constant d and all $n, n \geq n_2$. Since $cn \leq dn^2$ for $n \geq c/d$, program P is faster than program Q whenever $n \geq \max\{n_1, n_2, c/d\}$.

You should always be cautiously aware of the presence of the phrase “sufficiently large” in the assertion of the preceding discussion. When deciding which of the two programs to use, we must know whether the n we are dealing with is, in fact, sufficiently large. If program P actually runs in $10^6 n$ milliseconds, whereas program Q runs in n^2 milliseconds, and if we always have $n \leq 10^6$, then, other factors being equal, program Q is the one to use.

To get a feel for how the various functions grow with n , you are advised to study Table 1.7 and Figure 1.4 very closely. It is evident from the table and the figure that the function 2^n grows very rapidly with n . In fact, if a program needs 2^n steps for execution, then when $n = 40$, the number of steps needed is approximately $1.1 \cdot 10^{12}$. On a computer performing 1 billion steps per second, this would require about 18.3 minutes. If $n = 50$, the same program would run for about 13 days on this computer. When $n = 60$, about 310.56 years will be required to execute the program and when $n = 100$, about $4 \cdot 10^{13}$ years will be needed. So, we may conclude that the utility of programs with exponential complexity is limited to small n (typically $n \leq 40$).

$\log n$	n	$n \log n$	n^2	n^3	2^n
0	1	0	1	1	2
1	2	2	4	8	4
2	4	8	16	64	16
3	8	24	64	512	256
4	16	64	256	4096	65,536
5	32	160	1024	32,768	4,294,967,296

Table 1.7: Function values

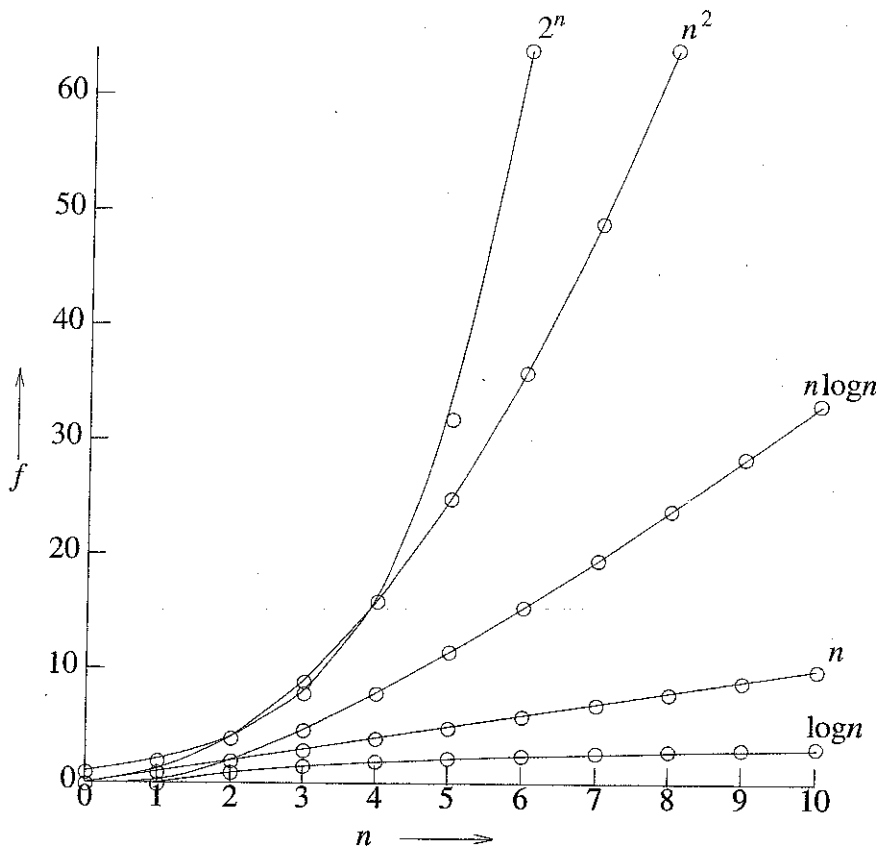


Figure 1.4: Plot of function values

Programs that have a complexity that is a polynomial of high degree are also of limited utility. For example, if a program needs n^{10} steps, then using our 1-billion-steps-per-second computer, we will need 10 seconds when $n = 10$; 3,171 years when $n = 100$; and $3.17 \cdot 10^{13}$ years when $n = 1000$. If the program's complexity had been n^3 steps instead, then we would need 1 second when $n = 1000$; 110.67 minutes when $n = 10,000$; and 11.57 days when $n = 100,000$.

Table 1.8 gives the time needed by a 1-billion-steps-per-second computer to execute a program of complexity $f(n)$ instructions. From a practical standpoint, it is evident that for reasonably large n (say $n > 100$), only programs of small complexity (such as n , $n \log n$, n^2 , n^3) are feasible. Further, this is the case even if one could build a computer capable of executing 10^{12} instructions per second. In this case, the computing times of Table 1.8 would decrease by a

n	$f(n)$						
	n	$n \log_2 n$	n^2	n^3	n^4	n^{10}	2^n
10	.01 μ s	.03 μ s	.1 μ s	1 μ s	10 μ s	10 s	1 μ s
20	.02 μ s	.09 μ s	.4 μ s	8 μ s	160 μ s	2.84 h	1 ms
30	.03 μ s	.15 μ s	.9 μ s	27 μ s	810 μ s	6.83 d	1 s
40	.04 μ s	.21 μ s	1.6 μ s	64 μ s	2.56 ms	121 d	18 m
50	.05 μ s	.28 μ s	2.5 μ s	125 μ s	6.25 ms	3.1 y	13 d
100	.10 μ s	.66 μ s	10 μ s	1 ms	100 ms	3171 y	$4 \cdot 10^{13}$ y
10^3	1 μ s	9.96 μ s	1 ms	1 s	16.67 m	$3.17 \cdot 10^{13}$ y	$32 \cdot 10^{283}$ y
10^4	10 μ s	130 μ s	100 ms	16.67 m	115.7 d	$3.17 \cdot 10^{23}$ y	
10^5	100 μ s	1.66 ms	10 s	11.57 d	3171 y	$3.17 \cdot 10^{33}$ y	
10^6	1 ms	19.92 ms	16.67 m	31.71 y	$3.17 \cdot 10^7$ y	$3.17 \cdot 10^{43}$ y	

μ s = microsecond = 10^{-6} seconds; ms = milliseconds = 10^{-3} seconds
s = seconds; m = minutes; h = hours; d = days; y = years

Table 1.8: Times on a 1-billion-steps-per-second computer

factor of 1000. Now, when $n = 100$ it would take 3.17 years to execute n^{10} instructions and $4 \cdot 10^{10}$ years to execute 2^n instructions.

1.7.2 Performance Measurement

Performance measurement is concerned with obtaining the actual space and time requirements of a program. These quantities are dependent on the particular compiler and options used as well as on the specific computer on which the program is run. So, when you repeat the performance experiments reported in this book, the run times you will observe will be quite different. In fact, as computers are continuously getting faster, your times will, most likely, be much smaller than those reported in this book.

In keeping with the discussion of the preceding section, we shall not concern ourselves with the space and time needed for compilation. We justify this by the assumption that each program (after it has been fully debugged) will be compiled once and then executed several times. Certainly, the space and time needed for compilation are important during program testing, when more time is spent on this task than in actually running the compiled code.

We shall not explicitly consider measuring the run-time space requirements of a program. Rather, we shall focus on measuring the computing time of a program. To obtain the computing (or run) time of a program, we need a clocking

62 Basic Concepts

function. We assume the existence of a function *time(hsec)* that returns in the variable *hsec* the current time in hundredths of a second.

Suppose we wish to measure the worst-case performance of the sequential search function (Program 1.27). Before we can do this, we need to: (1) decide on the values of *n* for which the times are to be obtained and (2) determine, for each of the above values of *n*, the data that exhibits the worst-case behavior.

```
int SequentialSearch (int *a, const int n, const int x)
{ // Search a[0:n-1].
  int i;
  for (i = 0; i < n && a[i] != x; i++);
  if (i == n) return -1;
  else return i;
}
```

Program 1.27: Sequential search

The decision on which values of *n* to use is to be based on the amount of timing we wish to perform and also on what we expect to do with the times once they are obtained. Assume that for Program 1.27, our intent is simply to predict how long it will take, in the worst-case, to search for *x* given the size *n* of *a*. An asymptotic analysis reveals that this time is $\Theta(n)$. So, we expect a plot of the times to be a straight line. Theoretically, if we know the times for any two values of *n*, the straight line is determined, and we can obtain the time for all other values of *n* from this line. In practice, we need the times for more than two values of *n*. This is so for the following reasons:

- (1) Asymptotic analysis tells us the behavior only for “sufficiently large” values of *n*. For smaller values of *n* the run time may not follow the asymptotic curve. To determine the point beyond which the asymptotic curve is followed, we need to examine the times for several values of *n*.
- (2) Even in the region where the asymptotic behavior is exhibited, the times may not lie exactly on the predicted curve (straight line in the case of Program 1.27) because of the effects of low-order terms that are discarded in the asymptotic analysis. For instance, a program with asymptotic complexity $\Theta(n)$ can have an actual complexity that is $c_1n + c_2\log n + c_3$, or for that matter any other function of *n* in which the highest-order term is c_1n for some constant, $c_1, c_1 > 0$.

It is reasonable to expect that the asymptotic behavior of Program 1.27 will begin for some *n* that is smaller than 100. So, for $n > 100$ we shall obtain the run

time for just a few values. A reasonable choice is $n = 200, 300, 400, \dots, 1000$. There is nothing magical about this choice of values. We can just as well use $n = 500, 1000, 1500, \dots, 10,000$ or $n = 512, 1024, 2048, \dots, 2^{15}$. It will cost us more in terms of computer time to use the latter choices, and we will probably not get any better information about the run time of Program 1.27 using these choices.

For n in the range $[0, 100]$ we shall carry out a more refined measurement, since we are not quite sure where the asymptotic behavior begins. Of course, if our measurements show that the straight-line behavior does not begin in this range, we shall have to perform a more detailed measurement in the range $[100, 200]$ and so on, until the onset of this behavior is detected. Times in the range $[0, 100]$ will be obtained in steps of 10 beginning at $n = 0$.

It is easy to see that Program 1.27 exhibits its worst-case behavior when x is chosen such that it is not one of the $a[i]$'s. For definiteness, we shall set $a[i] = i$, $1 \leq i \leq n$ and $x = 0$.

At this time, we envision using a program such as Program 1.28 to obtain the worst-case times.

```

void TimeSearch() {
    int a[1001], n[20];
    for (int j = 1; j <= 1000; j++) // initialize a
        a[j] = j;
    for (j = 0; j < 10; j++) { // values of n
        n[j] = 10 * j; n[j+10] = 100 * (j + 1);
    }
    cout << "  n  time" << endl;
    for(j = 0; j < 20; j++) { // obtain computing times
        long start, stop;
        time(start); // start timer
        int k = SequentialSearch(a, n[j], 0); // unsuccessful search
        time(stop); // stop timer
        long runTime = stop - start;
        cout << "    " << n[j] << "    " << runTime << endl;
    }
    cout << "Times are in hundredths of a second." << endl;
}

```

Program 1.28: Program to time Program 1.27

The output obtained from this program is summarized in Figure 1.5. All the times are zero, indicating that the precision of our clock is inadequate.

64 Basic Concepts

<i>n</i>	time	<i>n</i>	time
0	0	100	0
10	0	200	0
20	0	300	0
30	0	400	0
40	0	500	0
50	0	600	0
60	0	700	0
70	0	800	0
80	0	900	0
90	0	1000	0

Times in hundredths of a second

Figure 1.5: Output from Program 1.28

To time a short event, it is necessary to repeat it several times and divide the total time for the event by the number of repetitions.

Since our clock has an accuracy of about one-hundredth of a second, we should not attempt to time any single event that takes less than about 1 second. With an event time of at least 1 second, we can expect our observed times to be accurate to 1 percent.

The body of Program 1.28 needs to be changed to that of Program 1.29. In this program, $r[j]$ is the number of times the search is to be repeated when the number of elements in the array is $n[j]$. Notice that rearranging the timing statements as in Programs Program 1.30 or Program 1.31 does not produce the desired results. For instance, from the data of Figure 1.5, we expect that with the structure of Program 1.30, the value output will still be 0 because in each iteration of the **for** loop, 0 gets added to *total*. With the structure of Program 1.31, we expect the program never to exit the **for** loop. Yet another alternative is to move the first call to *time* out of the **for** loop of Program 1.31 and change the assignment to *total* within the **for** loop to

long *total* = *stop* - *start*;

This approach can be expected to yield satisfactory times. This approach cannot be used when the timing function available gives us only the time since the last invocation of *time*. Another difficulty is that the measured time includes the time needed to read the clock. For small n , this time may be larger than the time to run *SequentialSearch*. This difficulty can be overcome by determining the time

taken by the timing function and subtracting this time later. In further discussion, we shall use the explicit repetition factor technique.

```

void TimeSearch() {
    int a[1001], n[20];
    const long r[20] = {300000, 300000, 200000, 200000, 100000, 100000,
        100000, 80000, 80000, 50000, 50000, 25000, 15000, 15000, 10000, 7500, 7000,
        6000, 5000, 5000};

    for (int j = 1; j <= 1000; j++) // initialize a
        a[j] = j;

    for (j = 0; j < 10; j++) { // values of n
        n[j] = 10 * j; n[j+10] = 100 * (j+1);
    }

    cout << " n totalTime runTime" << endl;

    for(j = 0; j < 20; j++) { // obtain computing times
        long start, stop;
        time(start); // start timer
        for (long b = 1; b <= r[j]; b++)
            int k = SequentialSearch(a, n[j], 0); // unsuccessful search
        time(stop); // stop timer
        long totalTime = stop - start;
        float runTime = (float)(totalTime)/(float)(r[j]);
        cout << " " << n[j] << " " << totalTime << " " << runTime << endl;
    }
    cout << "Times are in hundredths of a second." << endl;
}

```

Program 1.29: Timing program

The output from the timing program, Program 1.29, is given in Figure 1.6. The times for n in the range $[0, 100]$ are plotted in Figure 1.7. The remaining values have not been plotted because this would lead to severe compression of the range $[0, 100]$. The linear dependence of the worst-case time on n is apparent from this graph.

The graph of Figure 1.7 can be used to predict the run time for other values of n . For example, we expect that when $n = 24$, the worst-case search time will be 0.0031 hundredths of a second. We can go one step further and get the

66 Basic Concepts

```
long total = 0;
for (long b = 1; b <= r[j]; b++)
{
    time(start);
    k = SequentialSearch(a, n[j], 0);
    time(stop);
    total += stop - start;
}
float runTime = (float) (total) / (float) (r[j]);
```

Program 1.30: Improper timing construct

```
for(long total = 0, i = 0; total < DesiredTime; i++)
{
    time(start);
    k = SequentialSearch(a, n[j], 0);
    time(stop);
    total += stop - start;
}
float runTime = (float) (total) / (float) (i);
```

Program 1.31: Another improper timing construct

equation of the straight line. The equation of this line is $t = c + mn$, where m is the slope and c the value for $n = 0$. From the graph, we see that $c = 0.0008$. Using the point $n = 60$ and $t = 0.0066$, we obtain $m = (t - c)/n = 0.0058/60 = 0.000096$. So, the line of Figure 1.7 has the equation $t = 0.0008 + 0.000096n$, where t is the time in hundredths of a second. From this, we expect that when $n = 1000$, the worst-case search time will be 0.0975 hsec, and when $n = 500$, it will be 0.0491 hsec. Compared with the actual observed times of Figure 1.6, we see that these figures are very accurate!

An alternate approach to obtain a good straight line for the data of Figure 1.6 is to obtain the straight line that is the least-squares approximation to the data. The result is $t = 0.00085454 + 0.00009564n$. When $n = 1000$ and 500, this equation yields $t = 0.0966$ and 0.0487.

Now, we are probably ready to pat ourselves on the back for a job well done. However, this action is somewhat premature, since our experiment is

<i>n</i>	<i>total</i>	<i>runTime</i>	<i>n</i>	<i>total</i>	<i>runTime</i>
0	241	0.0008	100	527	0.0105
10	533	0.0018	200	505	0.0202
20	582	0.0029	300	451	0.0301
30	736	0.0037	400	593	0.0395
40	467	0.0047	500	494	0.0494
50	565	0.0056	600	439	0.0585
60	659	0.0066	700	484	0.0691
70	604	0.0075	800	467	0.0778
80	681	0.0085	900	434	0.0868
90	472	0.0094	1000	484	0.0968

Times in hundredths of a second

Figure 1.6: Worst-case run times for Program 1.27

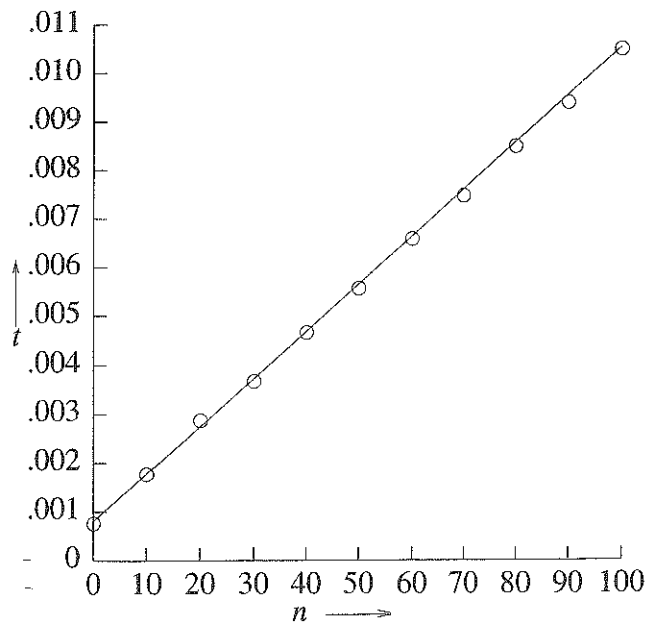


Figure 1.7: Plot of the data in Figure 1.6

68 Basic Concepts

flawed. First, the measured time includes the time taken by the repetition **for** loop. So, the times of Figure 1.6 are excessive. This can be corrected by determining the time for each iteration of this statement. A quick test run indicates that 300,000 executions take only 50 hundredths of a second. So, subtracting the time for the **for** ($b = 1$; $b \leq r[j]$; $b++$) statement reduces the reported times by only 0.000016. We can ignore this difference, since the use of a higher repetition factor could well result in measured times that are lower by about 0.000016 hsec per search.

Summary

To obtain the run time of a program, we need to plan the experiment. The following issues need to be addressed during the planning stage:

- (1) What is the accuracy of the clock? How accurate do our results have to be? Once the desired accuracy is known, we can determine the length of the shortest event that should be timed.
- (2) For each instance size, a repetition factor needs to be determined. This is to be chosen such that the event time is at least the minimum time that can be clocked with the desired accuracy.
- (3) Are we measuring worst-case or average performance? Suitable test data need to be generated.
- (4) What is the purpose of the experiment? Are the times being obtained for comparative purposes, or are they to be used to predict actual run times? If the latter is the case, then contributions to the run time from such sources as the repetition loop and data generation need to be subtracted (in case they are included in the measured time). If the former is the case, then these times need not be subtracted (provided they are the same for all programs being compared).
- (5) In case the times are to be used to predict actual run times, then we need to fit a curve through the points. For this, the asymptotic complexity should be known. If the asymptotic complexity is linear, then a least-squares straight line can be fit; if it is quadratic, then a parabola is to be used (i.e., $t = a_0 + a_1n + a_2n^2$). If the complexity is $\Theta(n \log n)$, then a least-squares curve of the form $t = a_0 + a_1n + a_2n \log_2 n$ can be fit. When obtaining the least-squares approximation, one should discard data corresponding to "small" values of n , since the program does not exhibit its asymptotic behavior for these n .

1.7.3 Generating Test Data

Generating a data set that results in the worst-case performance of a program is not always easy. In some cases, it is necessary to use a computer program to generate the worst-case data. In other cases, even this is very difficult. In these cases, another approach to estimating worst-case performance is taken. For each set of values of the instance characteristics of interest, we generate a suitably large number of random test data. The run times for these test data are obtained. The maximum of these times is used as an estimate of the worst-case time for this set of values of the instance characteristics.

To measure average-case times, it is usually not possible to average over all possible instances of a given characteristic. Although it is possible to do this for sequential and binary search, it is not possible for a sort program. If we assume that all keys are distinct, then for any given n , $n!$ different permutations need to be used to obtain the average time. Obtaining average-case data is usually much harder than obtaining worst-case data. So, we often adopt the strategy outlined above and simply obtain an estimate of the average time.

Whether we are estimating worst-case or average time using random data, the number of instances that we can try is generally much smaller than the total number of such instances. Hence, it is desirable to analyze the algorithm being tested to determine classes of data that should be generated for the experiment. This is a very algorithm-specific task, and we shall not discuss it here.

EXERCISES

1. Compare the two functions n^2 and $2^n/4$ for various values of n . Determine when the second becomes larger than the first.
2. Prove by induction:
 - (a) $\sum_{1 \leq i \leq n} i = n(n+1)/2, n \geq 1$
 - (b) $\sum_{1 \leq i \leq n} i^2 = n(n+1)(2n+1)/6, n \geq 1$
 - (c) $\sum_{0 \leq i \leq n} x^i = (x^{n+1} - 1)/(x - 1), x \neq 1, n \geq 0$
3. Determine the frequency counts for all statements in the following two program segments:

70 Basic Concepts

```
1 for (i = 1; i <= n; i++)
2   for (j = 1; j <= i; j++)
3     for (k = 1; k <= j; k++)
4       x++;
```

(a)

```
1 i = 1;
2 while (i <= n)
3 {
4   x++;
5   i++;
6 }
```

(b)

4. (a) Introduce statements to increment *count* at all appropriate points in Program 1.32.

```
void D(int *x, int n)
{
    int i = 1;
    do {
        x[i] += 2;
        i += 2;
    }
    while (i <= n);
    i = 1;
    while (i <= (n / 2))
    {
        x[i] += x[i + 1];
        i++;
    }
}
```

Program 1.32: Example program

- (b) Simplify the resulting program by eliminating statements. The simplified program should compute the same value for *count* as computed by the program of (a).
- (c) What is the exact value of *count* when the program terminates? You may assume that the initial value of *count* is 0.
- (d) Obtain the step count for Program 1.32 using the frequency method. Clearly show the step count table.
5. Do Exercise 4 for function *Transpose* (Program 1.33).
6. Do Exercise 4 for Program 1.34. This program multiplies two $n \times n$ matrices *a* and *b*.

```

void Transpose(int **a; int n)
{
    for (int i = 0; i < n-1; i++)
        for (int j = i+1; j < n; j++)
            swap(a[i][j], a[j][i]);
}

```

Program 1.33: Matrix transpose

```

void Multiply( int **a, int **b, int **c, int n)
{
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
        {
            c[i][j] = 0;
            for (int k = 0; k < n; k++)
                c[i][j] += a[i][k] * b[k][j];
        }
}

```

Program 1.34: Square matrix multiplication

7. (a) Do Exercise 4 for Program 1.35. This program multiplies two matrices a and b where a is an $m \times n$ matrix and b is an $n \times p$ matrix.
 (b) Under what conditions will it be profitable to interchange the two outermost for loops?
8. Show that the following equalities are correct:
 - (a) $5n^2 - 6n = \Theta(n^2)$
 - (b) $n! = O(n^n)$
 - (c) $2n^2 2^n + n \log n = \Theta(n^2 2^n)$
 - (d) $\sum_{i=0}^n i^2 = \Theta(n^3)$
 - (e) $\sum_{i=0}^n i^3 = \Theta(n^4)$
 - (f) $n^{2^n} + 6 \cdot 2^n = \Theta(2^{2^n})$

72 Basic Concepts

```
void Multiply(int **a, int **b, int **c, int m, int n, int p)
{
    for (int i = 0; i < m; i++)
        for (int j = 0; j < p; j++)
            {
                c[i][j] = 0;
                for (int k = 0; k < n; k++)
                    c[i][j] += a[i][k] * b[k][j];
            }
}
```

Program 1.35: Matrix multiplication

- (g) $n^3 + 10^6 n^2 = \Theta(n^3)$
 - (h) $6n^3 / (\log n + 1) = O(n^3)$
 - (i) $n^{1.001} + n \log n = \Theta(n^{1.001})$
 - (j) $n^{k+\epsilon} + n^k \log n = \Theta(n^{k+\epsilon})$ for all k and ϵ , $k \geq 0$, and $\epsilon > 0$
 - (k) $10n^3 + 15n^4 + 100n^2 2^n = O(100n^2 2^n)$
 - (l) $33n^3 + 4n^2 = \Omega(n^2)$
 - (m) $33n^3 + 4n^2 = \Omega(n^3)$
9. Show that the following equalities are incorrect:
- (a) $10n^2 + 9 = O(n)$
 - (b) $n^2 \log n = \Theta(n^2)$
 - (c) $n^2 / \log n = \Theta(n^2)$
 - (d) $n^3 2^n + 6n^2 3^n = O(n^3 2^n)$
10. Obtain the average run time of function *BinarySearch* (Program 1.10). Do this for suitable values of n in the range $[0, 100]$. Your report must include a plan for the experiment as well as the measured times. These times are to be provided both in a table and as a graph.
11. Analyze the computing time of function *SelectionSort* (Program 1.8).
12. Obtain worst-case run times for function *SelectionSort* (Program 1.8). Do this for suitable values of n in the range $[0, 3000]$. Your report must include a plan for the experiment as well as the measured times. These times are to be provided both in a table and as a graph.
-

13. Consider function *Add* (Program 1.22).
 - (a) Obtain run times for $n = 100, 200, \dots, 3000$.
 - (b) Plot the times obtained in part (a).
14. Do the previous exercise for matrix multiplication (Program 1.35).
15. A complex-valued matrix X is represented by a pair of matrices (A, B) where A and B contain real values. Write a program that computes the product of two complex-valued matrices (A, B) and (C, D) , where $(A, B) * (C, D) = (A + iB) * (C + iD) = (AC - BD) + i(AD + BC)$. Determine the number of additions and multiplications if the matrices are all $n \times n$.
16. Function *Magic* (Program 1.26) uses a 51×51 array *square* independent of the value of n . When $n < 51$, excess space is used and when $n > 51$, the function throws an exception. We can eliminate these shortcomings by using a dynamically allocated $n \times n$ 2-dimensional array. Modify function *Magic* to do this. Test your code.

1.8 REFERENCES AND SELECTED READINGS

A good introduction to programming in C++ can be found in the texts *C++ Program Design: An Introduction to Programming and Object-Oriented Design* by J. Cohoon and J. Davidson, 3rd Edition, McGraw Hill, NY, 2002 and *C++ How to Program*, 4th Edition, by H. Deitel and P. Deitel, Prentice Hall, Englewood Cliffs, NJ, 2002. For advanced C++ programming techniques, see *Effective C++*, Third Edition, by Scott Meyers, Addison-Wesley, Reading, MA, 2005. You can find a description of all components of the STL at the Web site <http://codeguru.earthweb.com/spp/stlguide> or *The C++ Standard Library: A Tutorial and Reference*, by N. Josuttis, Addison-Wesley, New York, 1999. For a discussion on the object-oriented paradigm, see *Object Oriented Analysis and Design with Applications*, Second Edition, by Grady Booch, Addison-Wesley, 1993.

The Art of Software Testing by G. Myers, John Wiley, New York, NY, 1979 and *Software Testing Techniques* by B. Beizer, Second Edition, Van Nostrand Reinhold, New York, NY, 1990 have more thorough treatments of software testing and debugging techniques.

The following books provide asymptotic analyses for several programs: *Fundamentals of Computer Algorithms* by E. Horowitz, S. Sahni, and S. Rajasekaran, W. H. Freeman and Co., New York, NY, 1998; *Introduction to Algorithms*, Second Edition, by T. Cormen, C. Leiserson, and R. Rivest, McGraw-Hill, New York, NY, 2002; and *Compared to What: An Introduction to the Analysis of Algorithms* by G. Rawlins, W. H. Freeman and Co., NY, 1992.

CHAPTER 2

Arrays

2.1 ABSTRACT DATA TYPES AND THE C++ CLASS

2.1.1 An Introduction to the C++ Class

C++ provides an explicit mechanism, the *class*, to support the distinction between specification and implementation and to hide the implementation of an ADT from its users. However, it is the programmer's responsibility to use the class mechanism judiciously so that it does, in fact, represent an ADT. The C++ class consists of four components (Program 2.1):

(1) A class name: (e.g., *Rectangle*).

(2) Data members: the data that makes up the class (e.g., *xLow*, *yLow*, *height*, and *width*).

(3) Member functions: the set of operations that may be applied to the objects

of a class (e.g., *GetHeight()*, *GetWidth()*).

(4) **Levels of program access:** these control the level of access to data members and member functions from program code that is outside the class. There are three levels of access to class members: **public**, **protected**, and **private**.

Any **public** data member (member function) can be accessed (invoked) from anywhere in the program. A **private** data member (member function) can only be accessed (invoked) from within its class or by a function or a class that is declared to be a **friend**. A **protected** data member (member function) can only be accessed (invoked) from within its class or from its subclasses or by a **friend**. We will discuss subclasses when we study inheritance in Chapter 3.

```

#ifndef RECTANGLE_H
#define RECTANGLE_H
// In the header file Rectangle.h
class Rectangle {
public:    // the following members are public
    // the next four members are member functions
    Rectangle();    // constructor
    ~Rectangle();    // destructor
    int GetHeight();    // returns the height of the rectangle
    int GetWidth();    // returns the width of the rectangle
private: // the following members are private
    // the following members are data members
    int xLow, yLow, height, width;
    // (xLow, yLow) are the coordinates of the bottom left corner of the rectangle
};
#endif

```

Program 2.1: Definition of the C++ class *Rectangle*

2.1.2 Data Abstraction and Encapsulation in C++

Data encapsulation is enforced in C++ by declaring all data members of a class to be **private** (or **protected**). External access to data members, if required, can be achieved by defining member functions that get and set data members. In Program 2.1, *GetHeight()* and *GetWidth()* are used to access the private data members *height* and *width*. Member functions that will be invoked externally are declared **public**; all others are declared **private** or **protected**.

76 Arrays

Next, we discuss how the specification of the operations of a class is separated from their implementation in C++. The specification, which must be contained inside the public portion of the class definition of the ADT, consists of the names of every public member function, the type of its arguments, and the type of its result (This information about a function is known as its *function prototype*). There should also be a description of what the function does, which does not refer to the internal representation or implementation details. This requirement is quite important because it implies that an abstract data type is *implementation-independent*. This description may be achieved in C++ by using comments to describe what each member function does (like the DVD instruction manual mentioned in Chapter 1). Finally, the specification of an operation is physically separated from its implementation by placing it in an appropriately named header file (e.g., the contents of Program 2.1 are placed in *Rectangle.h*). The implementations of the functions are typically placed in a source file of the same name (e.g., the contents of Program 2.2 are placed in *Rectangle.cpp*). Note that C++ syntax does allow you to include the implementation of a member function inside its class definition. In this case, the function is treated as an *inline* function.

```
// In the source file Rectangle.cpp
#include "Rectangle.h"

// The prefix "Rectangle::" identifies GetHeight() and GetWidth() as member
// functions belonging to class Rectangle. It is required because the member
// functions are implemented outside their class definition

int Rectangle::GetHeight() { return height; }
int Rectangle::GetWidth() { return width; }
```

Program 2.2: Implementation of operations on *Rectangle*

2.1.3 Declaring Class Objects and Invoking Member Functions

Program 2.3 shows a fragment of code that illustrates how class objects are declared and how member functions that operate on these class objects are invoked. Class objects are declared and created in the same way that variables are declared or created. Members of an object are accessed or invoked by using the component selection operators, a dot (.) for direct component selection and an arrow (->), which we shall write as →, for indirect component selection

through a pointer.

```
// In a source file main.cpp
#include <iostream>
#include "Rectangle.h"

main() {
    Rectangle r, s;           // r and s are objects of class Rectangle
    Rectangle *t = &s;        // t is a pointer to class object s

    // use . to access members of class objects.
    // use → to access members of class objects through pointers.
    if (r.GetHeight() * r.GetWidth() > t → GetHeight() * t → GetWidth())
        cout << " r ";
    else cout << " s ";
    cout << "has the greater area" << endl;
}
```

Program 2.3: A C++ code fragment demonstrating how *Rectangle* objects are declared and member functions invoked

2.1.4 Special Class Operations

Constructors and Destructors: The *constructor* and *destructor* are special member functions of a class. A constructor is a member function which initializes data members of an object. If a constructor is provided for a class, it is automatically executed when an object of that class is created. If a constructor is not defined for a class, memory is allocated for the data members of a class object, when it is created, but the data members are not initialized. The advantage of defining constructors for a class is that all class objects are well-defined as soon as they are created. This eliminates errors that result from accessing an undefined object. A destructor is a member function which deletes data members immediately before the object disappears. A constructor and destructor for class *Rectangle* are declared in Program 2.1.

A constructor must be declared as a public member function of its class; The name of a constructor must be identical to the name of the class to which it belongs; and a constructor must not specify a return type or return a value. Program 2.4 shows a constructor definition for class *Rectangle*.

78 Arrays

```
Rectangle::Rectangle( int x, int y, int h, int w)
{
    xLow = x; yLow = y;
    height = h; width = w;
}
```

Program 2.4: Definition of a constructor for *Rectangle*

Constructors may be used to initialize *Rectangle* objects as follows:

```
Rectangle r (1, 3, 6, 6);
Rectangle *s = new Rectangle (0, 0, 3, 4);
```

These create *r*, a square of side 6 whose bottom left corner is at (1, 3); and *s*, a pointer to a *Rectangle* object of height 3 and width 4 whose bottom left corner is at the origin. Note that the declaration

```
Rectangle t;
```

will result in a compile time error. The reason is that the compiler requires a *default constructor* (a constructor with no arguments) to initialize *t*. Had we not defined the constructor of Program 2.4, the compiler would have generated its own default constructor. However, since we have defined a constructor, it is our responsibility to provide a default constructor if we wish to use the above declaration. Program 2.5 shows a definition of the *Rectangle* constructor that also serves as a default constructor. This is achieved by providing a default value for each argument in the argument list. In this case, the default for each argument is the integer 0. If this constructor is used, the above declaration for object *t* will result in the creation of a *Rectangle* object, all of whose data members are initialized to 0.

```
Rectangle::Rectangle( int x = 0, int y = 0, int h = 0, int w = 0)
: xLow (x), yLow (y), height (h), width (w)
{ }
```

Program 2.5: Sophisticated definition of a constructor for *Rectangle*

The constructor of Program 2.5 also differs from that of Program 2.4 in another respect: its body is empty; the data members are initialized by using a member initialization list (consisting of a colon followed by a list of data members and the arguments to which they are to be initialized in parentheses). Program 2.4 first initializes the data members and then assigns arguments to them in two separate steps, while Program 2.5 directly initializes the data members to the corresponding arguments in a single step. Thus, the latter approach results in a more efficient constructor.

Destructors are automatically invoked when a class object goes out of scope or when a class object is deleted. Like a constructor, a destructor must be declared as a public member of its class; Its name must be identical to the name of its class prefixed with a tilde, ~; a destructor must not specify a return type or return a value, and a destructor may not take arguments. If a destructor is not defined for a class, the deletion of an object of that class results in the freeing of memory associated with data members of the class. If a data member is a pointer to some other object, the space allocated to the pointer is returned, but the object that it was pointing to is not deleted. If we also wish to delete this object, we must define a destructor that explicitly does so.

Operator Overloading: Consider the operator `==` which is used to check for equality between two data items. The operator `==` may be used to check for equality between two `float` data items; it can also be used to check for equality between two `int` items. The hardware algorithms implementing operator `==` depend on the type of the operands being compared; that is, the algorithm for comparing two `floats` is different from the one used to compare two `ints`. This is an example of *operator overloading*. However, if we were to try to use operator `==` to check for equality between two *Rectangle* objects, the compiler would complain that operator `==` is not defined for *Rectangle* objects. C++ allows the programmer to overload operators for user-defined data types. This is done by providing a definition that implements the operator for the particular data type. This definition takes the form of a class member function or an ordinary function, depending on the operator. The function prototype used must adhere to the specifications for the particular operator. For details about the specifications for various operators, see one of the introductory texts listed at the end of this chapter.

Program 2.6 overloads operator `==` for class *Rectangle*. Our program uses the `this` pointer which we describe briefly: The C++ keyword `this`, when used inside a member function of a class, represents a pointer to the particular class object that invoked it. The class object, itself, is therefore represented by `*this`.

We can now use the operator `==` to determine whether two rectangles are identical. Our program first evaluates the expression `"this == &c"`. This expression

80 Arrays

```
bool Rectangle::operator==(const Rectangle& s)
{
    if (this == &s) return true;
    if ((xLow == s.xLow) && (yLow == s.yLow)
        && (height == s.height) && (width == s.width)) return true;
    else return false;
}
```

Program 2.6: Overloading operator== for class *Rectangle*

checks to see if the two rectangles being compared are the same object. This would happen in the following two cases:

```
if (r == r) ...
```

```
Rectangle& u = r;
```

```
if (r == u) ...
```

In both cases, the expression "**this** == &s" evaluates to **true** because both operands represent the same object, and there is no need to compare the individual data members of the two rectangles. This is especially efficient if the class contains a large number of data members. If the expression evaluates to **false**, then the two rectangles are not the same object, and the individual data members must be compared. Program 2.7 overloads operator << so that *Rectangle* objects can be output by using **cout**.

```
ostream& operator<< (ostream& os, Rectangle& r)
{
    os << "Position is: " << r.xLow << " ";
    os << r.yLow << endl;
    os << "Height is: " << r.height << endl;
    os << "Width is: " << r.width << endl;
    return os;
}
```

Program 2.7: Overloading operator<< for class *Rectangle*

Notice that `operator<<` accesses private data members of class *Rectangle*. Therefore, it must be made a **friend** of *Rectangle*. In general, we want to minimize the number of **friend** declarations in a class because a **friend** represents an exception to the data encapsulation principle. However, there are instances, such as this, when they are necessary. The statement:

```
cout << r;
```

where *r* is defined as a square of side 6 with bottom left corner at (1, 3), will print the following:

Position is: 1 3

Height is: 6

Width is: 6

2.1.5 Miscellaneous Topics

In C++, a **struct** is identical to a **class**, except that the default level of access is public; that is, if the **struct** definition of a data type does not specify whether a given member (data or function) has **public**, **private**, or **protected** access, then the member has **public** access. In a class, the default is **private** access. Thus, the C++ **struct** is a generalization of the C **struct**.

A **union** is a structure that reserves storage for the largest of its data members so that only one of its data members can be stored, at any time. This is useful in applications where it is known that only one of many possible data items, each of a different type, needs to be stored in a structure; but there is no way to know what that data type is until runtime. The **struct** or **class** structures reserve memory for all their data members. Thus, using a **union** results in a more memory-efficient program, in these cases. We will use **union** in Chapter 4 on linked lists; we will also study a technique for improving on **union** by using inheritance.

A **static** class data member may be thought of as a global variable for its class. From the perspective of a class member function, a static data member is like any other data member. One difference is that each class object does not have its own exclusive copy. There is only one copy of a static data member and all class objects must share it. A second difference is that the declaration of a static data member in its class does not constitute a definition. Consequently, a definition of the data member is required somewhere else in the program. We

82 Arrays

will see an example of a static class member later in this chapter, when we implement the *Polynomial* data type.

2.1.6 ADTs and C++ classes

Example 2.1 [Abstract data type *NaturalNumber*]: ADT 2.1 contains the class definition of *NaturalNumber*. You will notice that the class definition of *NaturalNumber* in ADT 2.1 is very similar to the ADT definition of ADT 1.1. As a result, we will henceforth use the C++ class to define an ADT instead of the notation of ADT 1.1. Note that there is one significant aspect in which the format of ADT 1.1 differs from the C++ class: some operators in C++ such as **operator<<**, when overloaded for user-defined ADTs, do not exist as member functions of the corresponding class. Rather, these operators exist as ordinary C++ functions. Thus, these operations are declared outside the C++ class definition of the ADT even though they are actually part of the ADT.

```
class NaturalNumber {
// An ordered subrange of the integers starting at zero and ending at
// the maximum integer (MAXINT) on the computer.
public:
    NaturalNumber Zero();
    // Returns 0.

    bool IsZero();
    // If *this is 0, return true; otherwise, return false.

    NaturalNumber Add(NaturalNumber y);
    // Return the smaller of *this + y and MAXINT.

    bool Equal(NaturalNumber y);
    // Return true if *this == y; otherwise return false.

    NaturalNumber Successor();
    // If *this is MAXINT return MAXINT; otherwise return *this + 1.

    NaturalNumber Subtract(NaturalNumber y);
    // If *this < y, return 0; otherwise return *this - y.
};
```

ADT 2.1: Abstract data type *NaturalNumber*

EXERCISES

1. Overload **operator<** for class *Rectangle* such that $r < s$ if and only if the area of r is less than that of s .
2. Write and test C++ code for the class *MyRectangle*, which is an enhanced version of the class *Rectangle* (Program 2.1). In addition to the data members listed in Program 2.1, *MyRectangle* has the data member *color*. You must include functions to change as well as to return the value of each data member and functions to return the area and perimeter of a rectangle. The operators **<<** and **>>** should be overloaded to work with rectangles.
3. Write and test code for the C++ class *Currency*, which represents currency objects. Each currency object has two data members *\$* and *cents*, where *cents* is an integer between 0 and 99. You must include functions to set and return the data members, add and subtract currency objects, to multiply a currency object by numbers such as 2, 2.5 and so on. The operators **<<** and **>>** should be overloaded to work with currency objects.
4. Implement a class *Complex*, which represents the Complex Number data type. Implement the following operations:
 - (a) A constructor (including a default constructor which creates the complex number $0 + 0i$).
 - (b) Overload **operator+** to add two complex numbers.
 - (c) Overload **operator*** to multiply two complex numbers.
 - (d) Overload **<<** and **>>** to print and read complex numbers. To do this, you will need to decide what you want your input and output format to look like.

Write a program according to the following specifications: use the constructor to define two complex numbers: $3 + 2i$ and $0 + 0i$. Input two complex numbers $5 + 3i$ and $0 + 0i$ using **cin**. Obtain the sum and product of all four complex numbers using operators **+** and *****, respectively. Output the results using **cout**.

5. Implement a class *Quadratic* that represents 2-degree polynomials i.e., polynomials of type $ax^2 + bx + c$. Your class will require three data members corresponding to a , b , and c . Implement the following operations:
 - (a) A constructor (including a default constructor which creates the 0 polynomial).
 - (b) Overload **operator+** to add two polynomials of degree 2.
 - (c) Overload **<<** and **>>** to print and read polynomials. To do this, you will need to decide what you want your input and output format to look like.

84 Arrays

- (d) A function *Eval* that computes the value of a polynomial for a given value of x .
- (e) A function that computes the two solutions of the equation $ax^2+bx+c=0$. [Hint: use `class Complex` from the previous exercise.]

2.2 THE ARRAY AS AN ABSTRACT DATA TYPE

We begin our discussion by considering an array as an ADT. This is not the usual perspective, since many programmers view an array only as a consecutive set of memory locations. This is unfortunate because it clearly shows an emphasis on implementation issues. Although an array is usually implemented as a consecutive set of memory locations, this is not always the case. Intuitively, an array is a set of pairs, $\langle \text{index}, \text{value} \rangle$, such that each index that is defined has a value associated with it. In mathematical terms, we call this a *correspondence* or a *mapping*. However, when considering an ADT, we are more concerned with the operations that can be performed on an array. Aside from creating a new array, most languages provide only two standard operations for arrays, one that retrieves a value and one that stores a value. ADT 2.2 shows a class definition of the array ADT.

The constructor `GeneralArray(int j, RangeList list, float initValue = defaultValue)` produces a new array of the appropriate size and type. All of the items are initially set to the floating point variable *initValue*. *Retrieve* accepts an *index* and returns the value associated with the index if the index is valid or an error if the index is invalid. *Store* accepts an *index*, and a value of type float, and replaces the $\langle \text{index}, \text{oldvalue} \rangle$ pair with the $\langle \text{index}, \text{newvalue} \rangle$ pair.

GeneralArray is more general than the C++ array as it is more flexible about the composition of the index set. The C++ array requires the index set to be a set of consecutive integers starting at 0. Also, C++ does not check an array index to ensure that it belongs to the range for which the array is defined. Because of these disadvantages, it is often beneficial to define a more general and robust array class. Some of the member functions required to implement such a class are examined in the exercises. A general array class would typically be internally implemented by using the C++ array. To do this, it is necessary to be able to map a position in the general array onto a position in the C++ array. This is discussed in Section 2.5.

To simplify the discussion, we will hereafter use the C++ array. Therefore, before continuing, let us briefly review the C++ array. A C++ array *floatArray* of *floats* with index set ranging from 0 to $n-1$ is defined by:

```
float floatArray[n];
```

The *i*th element of *floatArray* may be accessed in two ways: *floatArray[i]* and **(floatArray + i)*. In the latter mechanism, the variable *floatArray* is actually

```

class GeneralArray {
// A set of pairs <index, value> where for each value of index in IndexSet
// there is a value of type float. IndexSet is a finite ordered set of one or more
// dimensions, for example, {0, ..., n - 1} for one dimension, {(0, 0),
// (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)} for two dimensions, etc.
public:
    GeneralArray(int j, RangeList list, float initValue = defaultValue);
    // This constructor creates a j dimensional array of floats; the
    // range of the kth dimension is given by the kth element of list. For each
    // index i in the index set, insert <i, initValue> into the array.

    float Retrieve(index i);
    // If i is in the index set of the array, return the float associated with i
    // in the array; otherwise throw an exception.

    void Store(index i, float x);
    // If i is in the index set of the array, replace the old value associated with i
    // by x; otherwise throw an exception.
}; //

```

ADT 2.2: Abstract data type *GeneralArray*

a pointer to the zeroth element of the array. The expression *floatArray* + *i* is a pointer to the *i*th element of array *floatArray*; it follows that **(floatArray + i)* is the *i*th element of array *floatArray*.

EXERCISES

1. Implement a class *CppArray* which is identical to a one-dimensional C++ array (i.e., the index set is a set of consecutive integers starting at 0) except for the following:
 - (i) It performs range checking.
 - (ii) It allows one array to be assigned to another array through the use of the assignment operator (e.g., *cp1* = *cp2*).
 - (iii) It supports a function that returns the size of the array.
 - (iv) It allows the reading or printing of arrays through the use of **cin** and **cout**.

To do this, you will have to define the following member functions on *CppArray*:

86 Arrays

- (a) A constructor *CppArray* (**int** *size* = defaultSize, **float** *initvalue* = defaultValue). This creates an array of size *size* all of whose elements are initialized to *initvalue*.
- (b) A copy constructor *CppArray* (**const** *CppArray*& *cp2*). This creates an array identical to *cp2*. Copy constructors are used to initialize a class object with another class object as in the following:
CppArray a = b ;
- (c) An assignment operator *CppArray*& **operator=** (**const** *CppArray*& *cp2*). This replaces the original array with *cp2*.
- (d) A destructor *~CppArray*() .
- (e) The subscript operator **float& operator**[] (**int** *i*). This should be implemented so that it performs range-checking.
- (f) A member function **int** *GetSize*() . This returns the size of the array.
- (g) Implement functions to read and print the elements of a *CppArray* by overloading operators << and >>. These functions will have to be made **friends** of *CppArray*.

2.3 THE POLYNOMIAL ABSTRACT DATA TYPE

Arrays are not only data structures in their own right; we can also use them to implement other abstract data types. For instance, let us consider one of the simplest and most common data structures: the *ordered*, or *linear*, *list*. We can find many examples of this data structure, including:

- Days of the week: (Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday)
- Values in a deck of cards: (Ace, 2, 3, 4, 5, 6, 7, 8, 9, 10, Jack, Queen, King)
- Floors of a building: (basement, lobby, mezzanine, first, second)
- Years the United States fought in World War II: (1941, 1942, 1943, 1944, 1945)
- Years Switzerland fought in World War II: ()

Notice that “Years Switzerland fought in World War II” is different because it contains no items. It is an example of an empty list, which we denote as (). The other lists all contain items that are written in the form ($a_0, a_1, \dots, a_{n-1}$).

We can perform many operations on lists, including:

- (1) Find the length, n , of the list.

- (2) Read the list from left to right (or right to left).
- (3) Retrieve the i th element, $0 \leq i < n$.
- (4) Store a new value into the i th position, $0 \leq i < n$.
- (5) Insert a new element at the position i , $0 \leq i < n$, causing elements numbered $i, i + 1, \dots, n - 1$ to become numbered $i + 1, i + 2, \dots, n$.
- (6) Delete the element at position i , $0 \leq i < n$, causing elements numbered $i + 1, \dots, n - 1$ to become numbered $i, i + 1, \dots, n - 2$.

It is not always necessary to be able to perform all of these operations; often a subset will suffice. In the study of data structures we are interested in ways of representing ordered lists so that these operations can be carried out efficiently.

Rather than state the formal specification of the ADT *ordered list*, we want to explore briefly its implementation. Perhaps the most common way to represent an ordered list is by an array where we associate the list element a_i with the array index i . We will refer to this as *sequential mapping* because, using the conventional array representation, we are storing a_i and a_{i+1} into consecutive locations i and $i + 1$ of the array. This gives us the ability to retrieve or modify the values of random elements in the list in a constant amount of time, essentially because a computer has random access to any word in its memory. We can access the list element values in either direction by changing the subscript values in a controlled way. Only operations (5) and (6) require real effort. Insertion and deletion using sequential allocation force us to move some of the remaining elements so that the sequential mapping is preserved in its proper form. It is precisely this overhead that leads us to consider nonsequential mappings of ordered lists in Chapter 4.

Let us jump right into a problem requiring ordered lists, which we shall solve by using one-dimensional arrays. The problem calls for building an ADT for the representation and manipulation of polynomials in a single variable (say x). Two such polynomials are

$$a(x) = 3x^2 + 2x - 4 \text{ and } b(x) = x^8 - 10x^5 - 3x^3 + 1$$

The polynomial $a(x)$ has 3 terms $3x^2$, $2x$, and -4 . The coefficients of these terms are 3, 2, and -4 , respectively; their exponents are 2, 1, and 0. A term of a polynomial may be represented as a (coefficient, exponent) pair. For example, (3, 2) represents the term $3x^2$. A term whose coefficient is nonzero is called a *nonzero term*. Normally, terms with zero coefficients are not displayed. The term with exponent equal to zero (e.g., $(-4, 0)$ in $a(x)$) does not show the variable, since x raised to a power of zero is 1. The *degree* of a polynomial is the largest exponent from among the nonzero terms. There are standard mathematical definitions for the sum and product of polynomials. Assume that we have two polynomials,

88 Arrays

$a(x) = \sum a_i x^i$ and $b(x) = \sum b_i x^i$; then

$$a(x) + b(x) = \sum (a_i + b_i) x^i$$

$$a(x) \cdot b(x) = \sum (a_i x^i \cdot \sum (b_j x^j))$$

Similarly, we can define subtraction and division on polynomials, as well as many other operations. We begin with an ADT definition (ADT 2.3) of a polynomial. ADT 2.3 shows only a subset of the functions we may want to perform on polynomials. The ADT is easily extended to include functions for the subtract, divide, input and output operations.

```
class Polynomial {  
    //  $p(x) = a_0 x^{e_0} + \dots + a_n x^{e_n}$ ; a set of ordered pairs of  $\langle e_i, a_i \rangle$ ,  
    // where  $a_i$  is a nonzero float coefficient and  $e_i$  is a non-negative integer exponent.  
    public:  
        Polynomial();  
        // Construct the polynomial  $p(x) = 0$ .  
        Polynomial Add(Polynomial poly);  
        // Return the sum of the polynomials *this and poly.  
        Polynomial Mult(Polynomial poly);  
        // Return the product of the polynomials *this and poly.  
        float Eval(float f);  
        // Evaluate the polynomial *this at f and return the result.  
};
```

ADT 2.3: Abstract data type *Polynomial*

2.3.1 Polynomial Representation

We are now ready to make some representation decisions. A very reasonable first decision would be to arrange the terms in decreasing order of exponent. This considerably simplifies many of the operations. We discuss three representations that are based on this principle:

Representation 1: One way to represent polynomials in C++ is to define the private data members of *Polynomial* as follows:

```
private:
    int degree;           // degree  $\leq$  MaxDegree
    float coef [MaxDegree + 1]; // coefficient array
```

where *MaxDegree* is a constant that represents the largest-degree polynomial that is to be represented. Now, if *a* is a *Polynomial* class object and $n \leq \text{MaxDegree}$, then the polynomial $a(x)$ above would be represented as:

```
a.degree = n
a.coef[i] =  $a_{n-i}$ ,  $0 \leq i \leq n$ 
```

Note that $a.coef[i]$ is the coefficient of x^{n-i} and the coefficients are stored in order of decreasing exponents. This representation leads to very simple algorithms for many of the operations on polynomials (addition, subtraction, evaluation, multiplication, etc.).

Representation 2: Representation 1 requires us to know the maximum degree of the polynomials we expect to work with and also is quite wasteful in its use of computer memory. For instance, if *a.degree* is much less than *MaxDegree*, then most of the positions in the array *a.coef* [] are unused. We can overcome both of these deficiencies by defining *coef* so that its size is *a.degree* + 1. This can be done by declaring the following private data members

```
private:
    int degree;
    float *coef;
```

and adding the following constructor to *Polynomial*:

```
Polynomial::Polynomial(int d)
{
    degree = d;
    coef = new float [degree+1];
}
```

Representation 3: Although Representation 2 solves the problems mentioned earlier, it does not yield a desirable representation. To see this, let us consider polynomials that have many zero terms. Such polynomials are called *sparse*. For instance, the polynomial $x^{1000} + 1$ has two nonzero terms and 999 zero terms. Consequently, 999 of the entries in *coef* will be zero if Representation 2 is used. To overcome this problem, we store only the nonzero terms. For this purpose, we define the class *term* as below.

90 Arrays

```
class Polynomial ; // forward declaration
```

```
class Term {  
friend Polynomial;  
private:  
    float coef; // coefficient  
    int exp;    // exponent  
};
```

The private data members of *Polynomial* are defined as follows:

```
private:  
    Term *termArray; // array of nonzero terms  
    int capacity;    // size of termArray  
    int terms;       // number of nonzero terms
```

Before proceeding, we should compare our current representation with Representation 2. Representation 3 is definitely superior when there are many zero terms. For example, $c(x) = 2x^{1000} + 1$ uses only 6 units of space (one for *c.capacity*, one for *c.terms*, two for the coefficients, and two for the exponents) when we use Representation 3 but when Representation 2 is used, 1002 units of space are required. However, when all terms are nonzero, as in $a(x)$ above, Representation 3 uses about twice as much space as does Representation 2. Unless we know beforehand that each of our polynomials has very few zero terms, Representation 3 is preferable. The exercises explore an alternative representation that uses the STL class *vector* instead of an array.

2.3.2 Polynomial Addition

Let us now write a C++ function to add two polynomials, a and b , to obtain the sum $c = a + b$. It is assumed that Representation 3, above, is used to store a and b . Function *Add* (Program 2.8) adds $a(x)$ (**this*) and $b(x)$ term by term to produce $c(x)$. This function assumes that the default constructor for *Polynomial* initializes *capacity* and *terms* to 1 and 0, respectively, and initializes *termArray* to an array with 1 position (i.e., the size or capacity of *termArray* is 1).

The basic loop of this algorithm consists of merging the terms of the two polynomials, depending upon the result of comparing the exponents. The **if** statement determines how the exponents are related and performs the proper action. Since the tests within the **while** statement require two terms, if one polynomial runs out of terms, we must exit the loop. The remaining terms of the other polynomial can be copied directly into the result. The terms of c are entered into its array *termArray* by calling function *NewTerm* (Program 2.9). In

```

1  Polynomial Polynomial::Add(Polynomial b)
2  { // Return the sum of of the polynomials *this and b.
3      Polynomial c;
4      int aPos = 0, bPos = 0;
5      while ((aPos < terms) && (bPos < b.terms))
6          if ((termArray[aPos].exp == b.termArray[bPos].exp) {
7              float t = termArray[aPos].coef + b.termArray[bPos].coef;
8              if (t) c.NewTerm(t, termArray[aPos].exp);
9              aPos++; bPos++;
10         }
11         else if ((termArray[aPos].exp < b.termArray[bPos].exp) {
12             c.NewTerm(b.termArray[bPos].coef, b.termArray[bPos].exp);
13             bPos++;
14         }
15         else {
16             c.NewTerm(termArray[aPos].coef, termArray[aPos].exp);
17             aPos++;
18         }
19     // add in remaining terms of *this
20     for (; aPos < terms ; aPos++)
21         c.NewTerm(termArray[aPos].coef, termArray[aPos].exp);
22     // add in remaining terms of b(x)
23     for (; bPos < b.terms; bPos++)
24         c.NewTerm(b.termArray[bPos].coef, b.termArray[bPos].exp);
25     return c;
26 }

```

Program 2.8: Adding two polynomials

case there is not enough space in *termArray* to accommodate the new term, its capacity is doubled. If we don't have enough memory to create the array *temp*, *new* will throw an exception. Since neither *NewTerm* nor *Add* catch this exception, control is passed to the function that invoked *Add* whenever *new* fails. If the thrown exception is not caught by any part of the program, the program terminates.

Analysis of *Add*: It is natural to carry out this analysis in terms of the number of nonzero terms in *a* (**this*) and *b*. Let *m* and *n* be the number of nonzero terms in *a* and *b* respectively. Lines 3 and 4 contribute $O(1)$ to the overall computing time. In each iteration of the **while** loop, *aPos* or *bPos* or both increase by 1.

```

void Polynomial::NewTerm(const float theCoeff, const int theExp)
{
    // Add a new term to the end of termArray.
    if (terms == capacity)
    {
        // double capacity of termArray
        capacity *= 2;
        term *temp = new term [capacity]; // new array
        copy(termArray, termArray + terms, temp);
        delete [] termArray; // deallocate old memory
        termArray = temp;
    }
    termArray [terms].coef = theCoeff;
    termArray [terms++].exp = theExp;
}

```

Program 2.9: Adding a new term, doubling array size when necessary

Since the **while** loop terminates when either $aPos$ equals $a.terms$ or $bPos$ equals $b.terms$, the number of iterations of this loop is bounded by $m + n - 1$. This worst-case is achieved, for instance, when $a(x) = \sum_{i=0}^n x^{2i}$ and $b(x) = \sum_{i=0}^n x^{2i+1}$. Since none of the exponents are the same in $a(x)$ and $b(x)$, $termArray[aPos].exp \neq termArray[bPos].exp$. Consequently, on each iteration the value of only one of $aPos$ or $bPos$ increases by 1. For the moment, let's ignore the time spent doubling the capacity of $c.termArray$ in *NewTerm*. Exclusive of this time, each iteration of the **while** loop takes $O(1)$ time. So, the total time contributed by this loop is $O(m + n)$. The **for** loops of lines 20 and 23 also contribute $O(m + n)$ time to the overall complexity. Adding together the time contributions of all components of *Add*, we obtain $O(m + n + \text{time spent in array doubling})$ as the asymptotic computing time of this algorithm.

Although it may appear that a lot of time is spent doubling the size of *termArray*, this is actually not the case. To create an array of type *Term* and size s , we must allocate sufficient space for s terms and also invoke the constructor for *Term* for each position in the array. Following the creation of the new array, we copy elements from the old array into the new one. In all our analyses of array doubling, we assume that it takes $O(1)$ time to allocate memory and that each invocation of the constructor and each element copy takes $O(1)$ time. Hence the time needed to double an array is linear in the size of the new array. Initially, $c.capacity$ is 1. Suppose that when *Add* terminates, $c.capacity$ is 2^k for some k , $k > 0$. The total time spent over all array doublings is $O(\sum_{i=1}^k 2^i) = O(2^{k+1}) = O(2^k)$.

Since $c.terms > 2^{k-1}$ (otherwise the array size would not have been doubled from 2^{k-1} to 2^k) and $m + n \geq c.terms$, the total time spent in array doubling is $O(c.terms) = O(m + n)$. Hence even with the time spent on array doubling added in, the total run time of *Add* is $O(m + n)$.

Array doubling contributes at most a constant multiplicative factor to the run time of *Add*! This latter statement is true even if we resize using any constant multiplicative factor greater than 1 (rather than by a factor of 2 as in array doubling) but is not true when we resize by an additive constant (i.e., increase the size by c , $c \geq 1$, whenever additional space is needed). Experiments indicate that array doubling is responsible for a very small fraction of the total run time of *Add*. $\square$

Notice that when *Add* terminates, $capacity \geq terms$ for the result polynomial. We can recover any excess space in the array *termArray* by reducing its size to equal the number of terms in the polynomial. The code to reduce the size of an array is very similar to that used in Program 2.9 to increase array size. This reduction in size doesn't change the asymptotic complexity of *Add*.

The exercises examine an alternative array representation of polynomials and, in Chapter 4, we develop a linked representation for them.

EXERCISES

1. Use the six operations defined in this section for an ordered list to arrive at an ADT specification for such a list.
2. If $a = (a_0, \dots, a_{n-1})$ and $b = (b_0, \dots, b_{m-1})$ are ordered lists, then $a < b$ if $a_i = b_i$ for $0 \leq i < j$ and $a_j < b_j$, or, if $a_i = b_i$ for $0 \leq i < n$ and $n < m$. Write a function that returns -1 , 0 , or $+1$, depending upon whether $a < b$, $a = b$, or $a > b$. Assume that the a_i s and b_j s are integer.
3. Modify function *Add* so that it reduces the size of *c.termArray* to *c.terms* prior to termination. With this modification, can we dispense with the data member *capacity*?
4. Write C++ functions to input and output polynomials represented as in this section. Your functions should overload the $<<$ and $>>$ operators.
5. Write a C++ function that multiplies two polynomials represented as in this section. What is the computing time of your function?
6. Write a C++ function that evaluates a polynomial at a value x_0 using the representation of this section. Try to minimize the number of operations.
7. The polynomials $a(x) = x^{2n} + x^{2n-2} + \dots + x^2 + x^0$ and $b(x) = x^{2n+1} + x^{2n-1} + \dots + x^3 + x$ cause *Add* to work very hard. For these polynomials, determine the exact number of times each statement will be executed.

94 Arrays

8. Develop a C++ class *Polynomial* that uses an STL *vector* instead of an array to hold the nonzero terms. You must include functions to add, subtract and multiply polynomials. Also, you must overload the input and output operators << and >> so that these work with objects of type *Polynomial*. What is the complexity of each function in your class? Compare the merits of using a *vector* over using an array.
9. In an alternative representation of polynomials, we store the nonzero terms of all polynomials in a single array called *termArray*. Each element in *termArray* is of type *Term*, which we defined in the section. Since the single array *termArray* is to be shared by all *Polynomial* objects, we declare it as a static class data member of *Polynomial*. The private data members of *Polynomial* are defined as follows:

private:

```
static Term *termArray;  
static int capacity;  
static int free ;  
int start, finish;
```

where *capacity* is the size of the array *termArray*. The required definitions of the static class members outside the class definition are:

```
int Polynomial::capacity = 100;  
Term Polynomial::termArray = new Term[100];  
int Polynomial::free = 0; // location of next free location in termArray
```

Consider the two polynomials $a(x) = 2x^{1000} + 1$ and $b(x) = x^4 + 10x^3 + 3x^2 + 1$. These could be stored in the array *termArray* as shown in Figure 2.1. Note that *a.start* and *b.start* give the location of the first term of *a* and *b* respectively, whereas *a.finish* and *b.finish* give the location of the last term of *a* and *b*. The static class member *free* gives the location of the next free location in the array *termArray*. For our example, *a.start* = 0, *a.finish* = 1, *b.start* = 2, *b.finish* = 5, and *free* = 6.

In general, any polynomial *a* that has *n* nonzero terms has *a.start* and *a.finish* such that *a.finish* = *a.start* + *n* - 1. If *a* has no nonzero terms (i.e., *a* is the zero polynomial), then *a.finish* = *a.start* - 1.

Write and test C++ functions to input, output, add, multiply, and evaluate polynomials represented in this way. Use array doubling to increase the size of *termArray* whenever needed. Is the representation of this exercise better or worse than the representation used in the text?

	<i>a.start</i>	<i>a.finish</i>	<i>b.start</i>			<i>b.finish</i>	<i>free</i>
	↓	↓	↓			↓	↓
<i>coef</i>	2	1	1	10	3	1	
<i>exp</i>	1000	0	4	3	2	0	
	0	1	2	3	4	5	6

Figure 2.1: Array representation of two polynomials

10. Show that if we start with an array whose size is 10 and increase its size by a constant multiplicative factor c , $c > 1$, whenever we need additional space, the total time spent in all array resizings is linear in the final size of the array. (Assume that the size of the new array is obtained by rounding *up* the product of c and the initial array size, if this quantity is not a whole number.)
11. Show that if we start with an array whose size is 1 and increase its size by a constant additive factor c , $c \geq 1$, whenever we need additional space, the total time spent in all array resizings is quadratic in the final size of the array.

2.4 SPARSE MATRICES

2.4.1 Introduction

A matrix is a mathematical object that arises in many physical problems. As computer scientists, we are interested in studying ways to represent matrices so that the operations to be performed on them can be carried out efficiently. A general matrix consists of m rows and n columns of numbers, as in Figure 2.2. The first matrix has five rows and three columns, the second six rows and six columns. In general, we write $m \times n$ (read “ m by n ”) to designate a matrix with m rows and n columns. Such a matrix has mn elements. When m is equal to n , we call the matrix *square*.

It is very natural to store a matrix in a two-dimensional array, say $a[m][n]$. Then we can work with any element by writing $a[i][j]$, and this element can be

	col 0	col 1	col 2		col 0	col 1	col 2	col 3	col 4	col 5
row 0	-27	3	4	row 0	15	0	0	22	0	-15
row 1	6	82	-2	row 1	0	11	3	0	0	0
row 2	109	-64	11	row 2	0	0	0	-6	0	0
row 3	12	8	9	row 3	0	0	0	0	0	0
row 4	48	27	47	row 4	91	0	0	0	0	0
				row 5	0	0	28	0	0	0

(a) (b)

Figure 2.2: Two matrices

found very quickly, as we will see in the next section. Now if we look at the second matrix of Figure 2.2, we see that it has many zero entries. Such a matrix is called *sparse*. There is no precise definition of when a matrix is sparse and when it is not, but it is a concept that we can all recognize intuitively. In the matrix of Figure 2.2(b), only eight out of 36 possible elements are nonzero, and that is sparse! A sparse matrix requires us to consider an alternative form of representation. This comes about because in practice many of the matrices we want to deal with are large, e.g., 5000×5000 , but at the same time they are sparse: say only 5000 out of 25 million possible elements are nonzero. When a 5000×5000 array is used to store this matrix, we need 25 million units of space. Also, 25 million units of time are needed for operations such as addition and transposition. Using an alternative representation, which stores only the nonzero elements, we can reduce both the space and time requirements considerably.

Before developing a particular representation for sparse matrices, we first must consider the operations that we want to perform on these matrices. A minimal set of operations includes matrix creation, transposition, addition, and multiplication. ADT 2.4 contains our specification of the matrix ADT.

```

class SparseMatrix
{
    // A set of triples, <row, column, value>, where row and column are non-negative
    // integers and form a unique combination; value is also an integer.
public:
    SparseMatrix(int r, int c, int t);
    // The constructor function creates a SparseMatrix with
    // r rows, c columns, and a capacity of t nonzero terms.

    SparseMatrix Transpose();
    // Returns the SparseMatrix obtained by interchanging the row and column
    // value of every triple in *this.

    SparseMatrix Add(SparseMatrix b);
    // If the dimensions of *this and b are the same, then the matrix produced by
    // adding corresponding items, namely those with identical row and column
    // values is returned; otherwise, an exception is thrown.

    SparseMatrix Multiply(SparseMatrix b);
    // If the number of columns in *this equals the number of rows in b then the
    // matrix d produced by multiplying *this and b according to the formula
    //  $d[i][j] = \sum (a[i][k] \cdot b[k][j])$ , where  $d[i][j]$  is the  $(i, j)$ th element, is returned.
    // k ranges from 0 to one less than the number of columns in *this;
    // otherwise, an exception is thrown.
};

```

ADT 2.4: Abstract data type *SparseMatrix*

2.4.2 Sparse Matrix Representation

Before implementing any of these operations, we must establish the representation of the sparse matrix. By examining Figure 2.2, we know that we can characterize uniquely any element within a matrix by using the triple $\langle \text{row}, \text{col}, \text{value} \rangle$. This means that we can use an array of triples to represent a sparse matrix. We require that these triples be stored by rows with the triples for the first row first, followed by those of the second row, and so on. We also require that all the triples for any row be stored so that the column indices are in ascending order. In addition, to ensure that the operations terminate, we must know the number of rows and columns and the number of nonzero elements in the matrix. Putting all this information together suggests that we define:

98 Arrays

```
class SparseMatrix ; // forward declaration
```

```
class MatrixTerm {
friend class SparseMatrix
private:
    int row, col, value;
};
```

and in class *SparseMatrix*:

```
private:
    int rows, cols, terms, capacity;
    MatrixTerm *smArray;
```

where *rows* is the number of rows in the matrix; *cols* is the number of columns; *terms* is the total number of nonzero entries; and *capacity* is the size of *smArray*. Figure 2.3(a) shows the representation of the matrix of Figure 2.2(b) using *smArray*. Positions 0 through 7 of *smArray* store the triples representing the nonzero entries. The triples are ordered by row and within rows by columns.

	<i>row</i>	<i>col</i>	<i>value</i>		<i>row</i>	<i>col</i>	<i>value</i>
<i>smArray</i> [0]	0	0	15	<i>smArray</i> [0]	0	0	15
[1]	0	3	22	[1]	0	4	91
[2]	0	5	-15	[2]	1	1	11
[3]	1	1	11	[3]	2	1	3
[4]	1	2	3	[4]	2	5	28
[5]	2	3	-6	[5]	3	0	22
[6]	4	0	91	[6]	3	2	-6
[7]	5	2	28	[7]	5	0	-15
(a)				(b)			

Figure 2.3: Sparse matrix and its transpose stored as triples

2.4.3 Transposing a Matrix

Figure 2.3(b) shows the transpose of the matrix of Figure 2.3(a). To transpose a matrix, we must interchange the rows and columns. This means that if an element is at position $[i][j]$ in the original matrix, then it is at position $[j][i]$ in the transposed matrix. When $i = j$, the elements on the diagonal will remain unchanged. Since the original matrix is organized by rows, our first idea for a transpose algorithm might be the following:

```
for (each row  $i$ )
    store ( $i, j, value$ ) of the original matrix as ( $j, i, value$ ) of the transpose;
```

The difficulty is in not knowing where to put the element $(j, i, value)$ until all other elements that precede it have been processed. In Figure 2.3(a), for instance, we encounter

(0, 0, 15)	which becomes	(0, 0, 15)
(0, 3, 22)	which becomes	(3, 0, 22)
(0, 5, -15)	which becomes	(5, 0, -15)
(1, 1, 11)	which becomes	(1, 1, 11)

We can avoid this difficulty of not knowing where to place an element by changing the order in which we place elements into the transpose. Consider the following strategy.

```
for (all elements in column  $j$ )
    store ( $i, j, value$ ) of the original matrix as ( $j, i, value$ ) of the transpose;
```

This says “find all elements in column 0 and store them in row 0, find all elements in column 1 and store them in row 1, etc.” Since the rows are originally in order, this means that we will locate elements in the correct column order as well. The function *Transpose* (Program 2.10) computes and returns the transpose of **this*.

It is not too difficult to see that the function is correct. The variable *currentB* always gives us the position in *b* where the next term in the transpose is to be inserted. The terms in *b.smArray* are generated by rows. Since the rows of *b* are the columns of **this*, row *c* of *b* is obtained by collecting all the nonzero terms in column *c* of **this*. This is precisely what is being done in lines 7 through 15. In the first iteration of the **for** loop of lines 7 through 15, all terms from column 0 of **this* are moved to *b*; in the next iteration, all terms from column 1 are moved; and so on.

Analysis of Transpose: Let *cols*, *rows*, and *terms* denote the number of

```

1  SparseMatrix SparseMatrix::Transpose( )
2  { // Return the transpose of *this.
3    SparseMatrix b(cols, rows, terms); // capacity of b.smArray is terms
4    if (terms > 0)
5        { // nonzero matrix
6            int currentB = 0 ;
7            for (int c = 0; c < cols ; c++) // transpose by columns
8                for (int i = 0 ; i < terms ; i++)
9                    // find and move terms in column c
10                   if (smArray[i].col == c)
11                       {
12                           b.smArray[currentB].row = c;
13                           b.smArray[currentB].col = smArray[i].row;
14                           b.smArray[currentB++].value = smArray[i].value;
15                       }
16        } // end of if (terms > 0)
17    return b;
18 }

```

Program 2.10: Transposing a matrix

columns, rows, and terms, respectively, in **this*. For each iteration of the loop of lines 7 through 15, the condition of line 10 is tested *terms* times. Since the number of iterations of this loop is *cols*, the total time for line 10 is *terms* · *cols*. The assignments of lines 12-14 take place exactly *terms* times, since there are only this many nonzero terms in the sparse matrix being generated. Lines 3-6 take a constant amount of time. The total time for *Transpose* is therefore $O(\text{terms} \cdot \text{cols})$. In addition to the space needed for **this* and *b*, the function requires only a constant amount of space, i.e., space for the variables *c*, *i*, and *currentB*. □

We now have a matrix transpose algorithm that we believe is correct and that has a computing time of $O(\text{terms} \cdot \text{cols})$. This computing time is a little disturbing, since we know that in case the matrices had been represented as two-dimensional arrays, we could have obtained the transpose of a *rows* × *cols* matrix in time $O(\text{rows} \cdot \text{cols})$. The algorithm for this has the simple form

```

for (int j = 0 ; j < cols ; j++)
    for (int i = 0 ; i < rows ; i++)
        b[j][i] = a[i][j];

```

The $O(\text{terms} \cdot \text{cols})$ time for function *transpose* becomes $O(\text{rows} \cdot \text{cols}^2)$ when *terms* is of the order of $\text{rows} \cdot \text{cols}$. This is worse than the $O(\text{rows} \cdot \text{cols})$ time using the simple form. Perhaps, in an effort to conserve space, we have traded away too much time. Actually, we can transpose a matrix represented as a sequence of triples in time $O(\text{terms} + \text{cols})$ by using a little more space. The new algorithm, *FastTranspose* (Program 2.11), proceeds by first determining the number of elements in each column of **this*. This gives us the number of elements in each row of *b*. From this information, the starting point in *b* of each of its rows is easily obtained. We can now move the elements of **this* one by one into their correct position in *b*.

The correctness of function *FastTranspose* follows from the preceding discussion and the observation that the starting point, *rowStart* [*i*], of row *i*, $i > 0$, of *b* is *rowStart* [*i* - 1] + *rowSize* [*i* - 1], where *rowSize* [*i* - 1] is the number of elements in row *i* - 1 of *b*. The computation of *rowSize* and *rowStart* is carried out in lines 9-13. In lines 14-21 the elements of **this* are placed one by one into the correct place in *b*. *rowStart* [*j*] is maintained so that it is always the position in *b* where the next element in row *j* of *b* is to be inserted. If we try the algorithm on the sparse matrix of Figure 2.3(a), then after the execution of line 13, the values of *rowSize* and *rowStart* are:

	[0]	[1]	[2]	[3]	[4]	[5]
<i>rowSize</i> =	3	2	1	0	1	1
<i>rowStart</i> =	0	3	5	6	6	7

There are three loops in *FastTranspose*, which are executed *terms*, *cols* - 1, and *terms* times respectively. Each iteration of each loop takes a constant amount of time, so the time for all 3 loops is $O(\text{cols} + \text{terms})$. Line 9 takes $O(\text{cols})$ time and the remaining lines take $O(1)$ time. So, the overall complexity is $O(\text{cols} + \text{terms})$.

The computing time of $O(\text{cols} + \text{terms})$ becomes $O(\text{rows} \cdot \text{cols})$ when *terms* is of the order of $\text{rows} \cdot \text{cols}$. This is the same as when two-dimensional arrays were in use. However, the constant factor associated with *FastTranspose* is larger than that for the array algorithm. When *terms* is sufficiently small compared to its maximum of $\text{rows} \cdot \text{cols}$, *FastTranspose* will be faster. Hence in this representation, we save both space and time! This was not true of *Transpose*, since *terms* will almost always be greater than $\max\{\text{rows}, \text{cols}\}$ and $\text{cols} \cdot \text{terms}$ will therefore always be at least $\text{rows} \cdot \text{cols}$. The constant factor associated with *Transpose* is also larger than the one in the array algorithm. Finally, you should note that *FastTranspose* requires more space than does *Transpose*. The space required by *FastTranspose* can be reduced by utilizing the same space to represent the two arrays *rowSize* and *rowStart*.

```

1 SparseMatrix SparseMatrix::FastTranspose()
2 { // Return the transpose of *this in O(terms + cols) time.
3   SparseMatrix b(cols, rows, terms);
4   if (terms > 0)
5     { // nonzero matrix
6       int *rowSize = new int[cols];
7       int *rowStart = new int[cols];
8       // compute rowSize[i] = number of terms in row i of b
9       fill(rowSize, rowSize + cols, 0); // initialize
10      for (i = 0 ; i < terms ; i++) rowSize[smArray[i].col]++;

11      // rowStart[i] = starting position of row i in b
12      rowStart[0] = 0 ;
13      for (i = 1 ; i < cols ; i++) rowStart[i] = rowStart[i - 1] + rowSize[i - 1] ;

14      for (i = 0 ; i < terms ; i++)
15        { // copy from *this to b
16          int j = rowStart[smArray[i].col];
17          b.smArray[j].row = smArray[i].col;
18          b.smArray[j].col = smArray[i].row ;
19          b.smArray[j].value = smArray[i].value ;
20          rowStart[smArray[i].col]++ ;
21        } // end of for
22      delete [] rowSize ;
23      delete [] rowStart ;
24    } // end of if
25    return b ;
26 }

```

Program 2.11: Transposing a matrix faster

2.4.4 Matrix Multiplication

Definition: Given a and b , where a is $m \times n$ and b is $n \times p$, the product matrix d has dimension $m \times p$. Its $[i] [j]$ element is

$$d_{ij} = \sum_{k=0}^{n-1} a_{ik} b_{kj}$$

for $0 \leq i < m$ and $0 \leq j < p$. $\square$

The product of two sparse matrices may no longer be sparse, as Figure 2.4

shows.

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Figure 2.4: Multiplication of two sparse matrices

We would like to multiply two sparse matrices a and b represented as ordered lists as in Figure 2.3. We need to compute the elements of d by rows so that we can store them in their proper place without moving previously computed elements. To do this we pick a row of a and find all elements in column j of b for $j = 0, 1, \dots, b.cols-1$, where $b.cols$ is the number of columns in b . Normally, we would have to scan all of b to find all the elements in column j . However, we can avoid this by first computing the transpose of b . This puts all column elements in consecutive order. Once we have located the elements of row i of A and column j of b , we just do a merge operation similar to that used for polynomial addition in Section 2.3. An alternative approach is explored in the exercises.

Before we write a matrix multiplication function, it will be useful to define the function of Program 2.12, which stores a matrix term. This function uses another function *ChangeSize 1D* (Program 2.13), which changes the size of a 1-dimensional array.

```
void SparseMatrix::StoreSum (const int sum, const int r, const int c)
// If sum != 0, then it along with its row and column position are stored as the
// last term in *this.
    if (sum != 0) {
        if (terms == capacity)
            ChangeSize 1D (2*capacity); // double size
        smArray[terms].row = r;
        smArray[terms].col = c;
        smArray[terms++].value = sum;
    }
}
```

Program 2.12: Storing a matrix term

The function *Multiply* (Program 2.14) multiplies the sparse matrices a ($*this$) and b to obtain the product matrix d using the strategy outlined above.

```

void ChangeSize1D(const int newSize)
{
    // Change the size of smArray to newSize.
    if (newSize < terms) throw "New size must be >= number of terms";
    MatrixTerm *temp = new MatrixTerm[newSize]; // new array
    copy(smArray, smArray + terms, temp);
    delete [] smArray; // deallocate old memory
    smArray = temp;
    capacity = newSize;
}

```

Program 2.13: Change the size of a 1-dimensional array

Function *Multiply* (which is invoked as *a.Multiply(b)*) makes use of variables *currRowIndex*, *currColIndex*, *currRowA*, *currColB*, and *currRowBegin*. The variable *currRowA* is the row of *a* that is currently being multiplied with the columns of *b*. *currRowBegin* is the position in *a* of the first element of row *currRowA*; and *currColB* is the column of *b* that is currently being multiplied with row *currRowA* of *a*. The variables *currRowIndex* and *currColIndex* are used to examine successive elements of row *currRowA* and column *currColB* of *a* and *b*, respectively. If matrices *a* and *b* are incompatible, then *Multiply* throws an exception in Line 3. Lines 10-14 of the function introduce a dummy term into each of *a* and *bXpose*. This enables us to handle end conditions (i.e., computations involving the last row of *a* or last column of *b*) in an elegant way. Notice that because of the use of array doubling in function *StoreSum*, the capacity of *d.smArray* may exceed the number of terms in the matrix. Also, because of the addition of the dummy term to **this* its capacity will be more than the number of terms in **this*. We can free the unused space in these two matrices by using the function *ChangeSize1D*.

```

1 SparseMatrix SparseMatrix::Multiply(SparseMatrix b)
2 { // Return the product of the sparse matrices *this and b.
3   if (cols != b.rows) throw "Incompatible matrices";
4   SparseMatrix bXpose = b.FastTranspose();
5   SparseMatrix d(rows, b.cols, 0);
6   int currRowIndex = 0,
7       currRowBegin = 0,
8       currRowA = smArray[0].row;
9   // set boundary conditions
10  if (terms == capacity) ChangeSize1D(terms + 1);

```

```

11  bXpose.ChangeSize1D(bXpose.terms + 1);
12  smArray[terms].row = rows;
13  bXpose.smArray[b.terms].row = b.cols;
14  bXpose.smArray[b.terms].col = -1 ;
15  int sum = 0;
16  while (currRowIndex < terms)
17  { // generate row currRowA of d
18      int currColB = bXpose.smArray[0].row;
19      int currColIndex = 0;
20      while (currColIndex <= b.terms)
21      { // multiply row currRowA of *this by column currColB of b
22          if (smArray[currRowIndex].row != currRowA)
23              // end of row currRowA
24              d.StoreSum(sum, currRowA, currColB);
25              sum = 0 ; // reset sum
26              currRowIndex = currRowBegin;
27              // advance to next column
28              while (bXpose.smArray[currColIndex].row == currColB)
29                  currColIndex++;
30              currColB = bXpose.smArray[currColIndex].row;
31          }
32          else if (bXpose.smArray[currColIndex].row != currColB)
33              // end of column currColB of b
34              d.StoreSum(sum, currRowA, currColB);
35              sum = 0 ; // reset sum
36              // set to multiply row currRowA with next column
37              currRowIndex = currRowBegin;
38              currColB = bXpose.smArray[currColIndex].row;
39          }
40      else
41          if (smArray[currRowIndex].col <
42              bXpose.smArray[currColIndex].col)
43              currRowIndex++ ; // advance to next term in row
44          else if (smArray[currRowIndex].col ==
45              bXpose.smArray[currColIndex].col)
46              { // add to sum
47                  sum += smArray[currRowIndex].value *
48                      bXpose.smArray[currColIndex].value;
49                  currRowIndex++; currColIndex++;
50              }
51          else currColIndex++; // next term in currColB
52      } // end of while (currColIndex <= b.terms)

```

```

53   while (smArray[currRowIndex].row == currRowA) // advance to next row
54       currRowIndex++;
55   currRowBegin = currRowIndex;
56   currRowA = smArray[currRowIndex].row;
57 } // end of while (currRowIndex < terms)
58 return d;
59 }

```

Program 2.14: Multiplying sparse matrices

Analysis of *Multiply*: We leave the correctness proof of this algorithm as an exercise. Let us examine its computing time. In addition to the space needed for a (**this*), b , d , and some simple variables, space is needed for the transpose matrix $bXpose$. Algorithm *FastTranspose* also needs some additional space. The exercises explore a strategy for *Multiply* that does not explicitly compute $bXpose$, and the only additional space needed is the same as that required by *FastTranspose*. Turning our attention to the computing time of *Multiply*, we see that lines 3-15 require only $O(b.cols + b.terms)$ time. The **while** loop of lines 16-57 is executed at most $a.rows$ times (once for each row of a). In each iteration of the **while** loop of lines 20-52, the value of *currRowIndex* or *currColIndex* or both increases by 1, or *currRowIndex* and *currColB* are reset. The maximum total increment in *currColIndex* over the whole loop is $b.terms$. If t_r is the number of terms in row r of a , then the value of *currRowIndex* can increase at most t_r times before *currRowIndex* moves to the next row of a . When this happens, *currRowIndex* is reset to *currRowBegin* in line 26. At the same time *currColB* is advanced to the next column. Hence, this resetting can take place at most $b.cols$ times. The total maximum increment in *currRowIndex* is therefore $b.cols \cdot t_r$. The maximum number of iterations of the **while** loop of lines 20-52 is therefore $b.cols + b.cols \cdot t_r + b.terms$. The time for this loop when multiplying with row r of a is $O(b.cols \cdot t_r + b.terms)$. Lines 53-54 take only $O(t_r)$ time. Hence, the time for the outer **while** loop, lines 16-57, for the iteration with row *currRowA* of a , is $O(b.cols \cdot t_r + b.terms)$. The overall time for this loop is then $O(\sum_r (b.cols \cdot t_r + b.terms)) = O(b.cols \cdot a.terms + a.rows \cdot b.terms)$.

Once again, we may compare the computing time with the time to multiply matrices when arrays are used. The classical multiplication algorithm is:

```

for (int i = 0; i < a.rows; i++)
    for (int j = 0; j < b.cols; j++)
    {
        sum = 0;
        for (int k = 0; k < a.cols; k++)
            sum += a[i][k] * b[k][j];
        c[i][j] = sum;
    }

```

The time for this is $O(a.rows \cdot a.cols \cdot b.cols)$. Since $a.terms \leq a.rows \cdot a.cols$ and $b.terms \leq a.cols \cdot b.rows$, the time for *Multiply* is at most $O(a.rows \cdot a.cols \cdot b.cols)$. However, its constant factor is greater than that for matrix multiplication using arrays. In the worst case, when $a.terms = a.rows \cdot a.cols$ or $b.terms = a.cols \cdot b.rows$, *Multiply* will be slower by a constant factor. However, when $a.terms$ and $b.terms$ are sufficiently smaller than their maximum values, i.e., a and b are sparse, *Multiply* will outperform the above multiplication algorithm for arrays.

The above analysis for *Multiply* is nontrivial. It introduces some new concepts in algorithm analysis and you should make sure you understand the analysis. $\square$

EXERCISES

1. How much time does it take to locate an arbitrary element $A[i][j]$ in the representation of this section and to change its value?
2. Analyze carefully the computing time and storage requirements of function *FastTranspose* (Program 2.11). What can you say about the existence of an even faster algorithm?
3. Write C++ functions to input and output a sparse matrix. These should be implemented by overloading the $\gg$ and $\ll$ operators. You should design the input and output formats. However, the internal representation should be a one dimensional array of nonzero terms as used in this section. Analyze the computing time of your functions.
4. Rewrite function *FastTranspose* (Program 2.11) so that it uses only one array rather than the two arrays required to hold *RowSize* and *RowStart*.
5. Develop a correctness proof for function *Multiply* (Program 2.14).
6. Use the row pointers idea used in *FastTranspose* and rewrite *Multiply* (Program 2.14) to multiply two sparse matrices represented as in Section 2.4 without explicitly transposing either. What is the computing time of your algorithm?

108 Arrays

7. A variation of the scheme discussed in this section for sparse matrix representation involves representing only the nonzero terms in a one-dimensional array v in the order described. In addition, a strip of $n \times m$ bits, $bits[n][m]$ is also kept. $bits[i][j] = 0$ if $a[i][j] = 0$, and $bits[i][j] = 1$ if $a[i][j] \neq 0$. The figure below illustrates the representation for the sparse matrix of Figure 2.2(b).

$$\begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 15 \\ 22 \\ -15 \\ 11 \\ 3 \\ -6 \\ 91 \\ 28 \end{bmatrix}$$

- On a computer with w bits per word, how much storage is needed to represent an $n \times m$ sparse matrix that has t nonzero terms?
- Write an algorithm to add two sparse matrices represented as above. How much time does your algorithm take?
- Discuss the merits of this representation versus the representation of Section 2.4. Consider space and time requirements for such operations as random access, add, multiply, and transpose. Note that the random access time can be improved somewhat by keeping another array ra such that $ra[i] =$ number of nonzero terms in rows 0 through $i - 1$.

2.5 REPRESENTATION OF ARRAYS

Multidimensional arrays are usually implemented by storing the elements in a one-dimensional array. In this section, we develop a representation in which an arbitrary array element, say $a[i_1][i_2], \dots, [i_n]$, gets mapped onto a position in a one-dimensional C++ array so that it can be retrieved efficiently. This is necessary since programs using arrays may, in general, use array elements in a random order. In addition to being able to retrieve array elements easily, it is also necessary to be able to determine the amount of memory to be reserved for a particular array. Assuming that each array element requires only one word of memory, the number of words needed is the number of elements in the array. If an array is declared $a[u_1][u_2], \dots, [u_n]$, where 0 through $u_i - 1$ is the range of index values in dimension i , then it is easy to see that the number of elements is

$$\prod_{i=1}^n u_i$$

One of the common ways to represent an array is in *row major order* (see Exercise 3 for column major order). If we have the declaration

$$a[2][3][2][2]$$

then we have a total of $2 \times 3 \times 2 \times 2 = 24$ elements. Using row major order, these elements will be stored as

$$a[0][0][0][0], a[0][0][0][1], a[0][0][1][0], a[0][0][1][1]$$

and continuing

$$a[0][1][0][0], a[0][1][0][1], a[0][1][1][0], a[0][1][1][1]$$

for three more sets of four until we get

$$a[1][2][0][0], a[1][2][0][1], a[1][2][1][0], a[1][2][1][1]$$

We see that the index at the right moves the fastest. In fact, if we view the indices as numbers, we see that they are, in some sense, increasing:

$$0000, 0001, \dots, 1210, 1211$$

A synonym for row major order is *lexicographic order*.

The problem is how to translate from the name $a[i_1][i_2], \dots, [i_n]$ to the correct location in the one-dimensional array. Suppose $a[0][0][0][0]$ is stored at position 0. Then $a[0][0][0][1]$ will be at position 1 and $a[1][2][1][1]$ at position 23. These two addresses are easy to guess. In general, we can derive a formula for the address of any element. This formula makes use of only the starting address of the array plus the declared dimensions.

Before obtaining a formula for the case of an n -dimensional array, let us look at the row major representation of one-, two-, and three-dimensional arrays. To begin with, if a is declared $a[u_1]$, then assuming one word per element, it may be represented in sequential memory as in Figure 2.5. If α is the address of $a[0]$, then the address of an arbitrary element $a[i]$ is just $\alpha + i$.

The two-dimensional array $a[u_1][u_2]$ may be interpreted as u_1 rows, $row_0, row_1, \dots, row_{u_1-1}$, each row consisting of u_2 elements. In a row major representation, these rows would be represented in memory as in Figure 2.6.

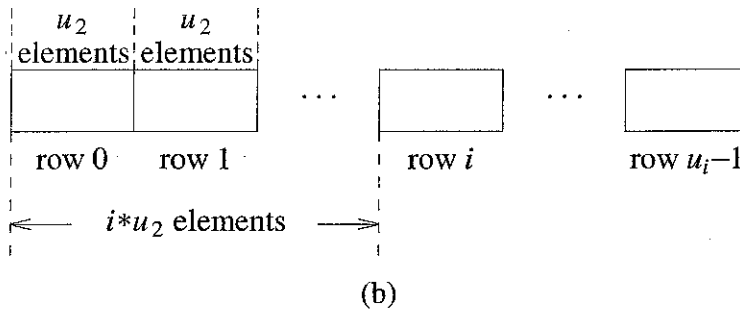
Again, if α is the address of $a[0][0]$, then the address of $a[i][0]$ is

array element:	$a[0]$	$a[1]$	$a[2]$	$\cdots$	$a[i]$	$\cdots$	$a[u_1-1]$
address:	α	$\alpha+1$	$\alpha+2$	$\cdots$	$\alpha+i$	$\cdots$	$\alpha+u_1-1$

Figure 2.5: Sequential representation of $a[u_1]$

	col 0	col 1	$\cdots$	col u_2-1
row 0	X	X	$\cdots$	X
row 1	X	X	$\cdots$	X
row 2	X	X	$\cdots$	X
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
row u_1-1	X	X	$\cdots$	X

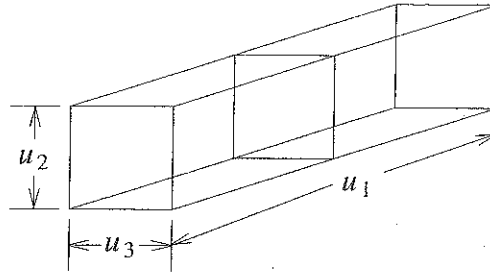
(a)

**Figure 2.6:** Sequential representation of $a[u_1][u_2]$

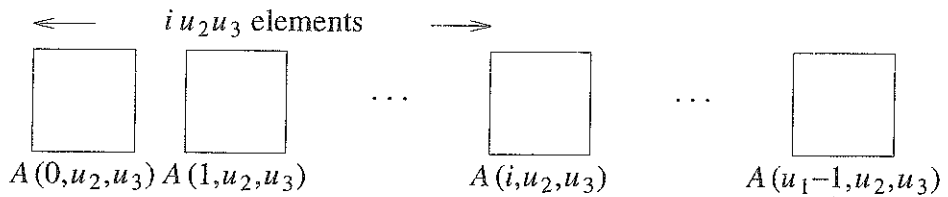
$\alpha + i * u_2$, as there are i rows, each of size u_2 , preceding the first element in the i th row. Knowing the address of $a[i][0]$, we can say that the address of $a[i][j]$ is then simply $\alpha + i * u_2 + j$.

Figure 2.7 shows the representation of the three-dimensional array $a[u_1][u_2][u_3]$. This array is interpreted as u_1 two-dimensional arrays of dimension $u_2 \times u_3$. To locate $a[i][j][k]$, we first obtain $\alpha + iu_2u_3$ as the address for $a[i][0][0]$ since there are i two-dimensional arrays of size $u_2 \times u_3$ preceding this element. From this and the formula for addressing a two-dimensional array, we obtain $\alpha + iu_2u_3 + ju_3 + k$ as the address of $a[i][j][k]$.

Generalizing on the preceding discussion, the addressing formula for any



(a) 3-dimensional array $A[u_1][u_2][u_3]$ regarded as u_1 2-dimensional arrays



(b) Sequential row major representation of a 3-dimensional array. Each 2-dimensional array is represented as in Figure 2.6

Figure 2.7: Sequential representation of $a[u_1][u_2][u_3]$

element $a[i_1][i_2], \dots, [i_n]$ in an n -dimensional array declared as $a[u_1][u_2], \dots, [u_n]$ may be easily obtained. If α is the address for $a[0][0] \dots, [0]$ then $\alpha + i_1 u_2 u_3 \dots u_n$ is the address for $a[i_1][0], \dots, [0]$. The address for $a[i_1][i_2][0] \dots, [0]$ is then $\alpha + i_1 u_2 u_3 \dots u_n + i_2 u_3 u_4 \dots u_n$.

Repeating in this way, the address for $a[i_1][i_2], \dots, [i_n]$ is

112 Arrays

$$\begin{aligned}
 & \alpha + i_1 u_2 u_3 \cdots u_n \\
 & \quad + i_2 u_3 u_4 \cdots u_n \\
 & \quad + i_3 u_4 u_5 \cdots u_n \\
 & \quad \vdots \\
 & \quad \vdots \\
 & \quad \vdots \\
 & \quad + i_{n-1} u_n \\
 & \quad + i_n
 \end{aligned}$$

$$= \alpha + \sum_{j=1}^n i_j a_j \text{ where } \begin{cases} a_j = \prod_{k=j+1}^n u_k & 1 \leq j < n \\ a_n = 1 \end{cases}$$

Note that a_j may be computed from a_{j+1} , $1 \leq j < n$, using only one multiplication as $a_j = u_{j+1} a_{j+1}$. Thus, a compiler will initially take the declared bounds $u_1, \dots, u_n$ and use them to compute the constants $a_1, \dots, a_{n-1}$ using $n-2$ multiplications. The address of $a[i_1], \dots, [i_n]$ can then be found using the formula, requiring $n-1$ more multiplications and n additions and n subtractions.

EXERCISES

1. **[Programming Project]** Even though the multidimensional array is provided as a standard data object in C++, it is often useful to define your own class for multidimensional arrays. This gives a more robust class that:
 - (a) Performs range checking.
 - (b) Does not require the index set in each dimension to consist of consecutive integers starting at 0.
 - (c) Allows array assignment.
 - (d) Allows initialization of an array with another array.
 - (e) Selects the range in each dimension of the array during runtime.
 - (f) Allows dynamic modification of the range and size of the array.
 - (g) Provides a mechanism to determine the size of an array.

Implement a class *mdArray* that stores floating point elements and provides the functionality specified above. You will need to define two pointer data members corresponding to one-dimensional arrays, which will be dynamically created: the array of elements, and an array that stores the upper and lower bounds of each dimension. Additional data members may be

required. The data members of your class must be initialized by a constructor that takes as input the number of dimensions and the range of each dimension. Also, provide operations to read/write array elements from/to a file.

2. How many values can be held by each of the arrays $a[n]$, $b[n][m]$, $c[n][2][3]$?
3. Obtain an addressing formula for the element $a[i_1][i_2], \dots, [i_n]$ in an array declared as $a[u_1][u_2], \dots, [u_n]$. Assume a column-major representation of the array with one word per element and α the address of $a[0][0], \dots, [0]$. In a column-major representation, a two-dimensional array is stored sequentially by columns rather than by rows.

2.6 THE STRING ABSTRACT DATA TYPE

In this section, we turn our attention to a data type, the string, whose component elements are characters. As an ADT, we define a string to have the form $S = s_0, \dots, s_{n-1}$, where s_i are characters taken from the character set of the programming language, and n is the length of the string. If $n = 0$, then S is an empty or null string.

There are several useful operations we could specify for strings. Some of these operations are similar to those required for other ADTs: creating a new empty string, reading a string or printing it out, appending two strings together (called *concatenation*), or copying a string. However, there are other operations that are unique to our new ADT, including comparing strings, inserting a substring into a string, removing a substring from a string, or finding a pattern in a string. We have listed some of the essential operations in ADT 2.5, which contains our specification of the string ADT.

C++ includes a *string* class that provides many more functions than are specified in our ADT. We do not go into the details of this C++ class here. Instead, we focus on the problem of string pattern matching (i.e., the function *Find* specified in ADT 2.5). In the remainder of this section, we assume that, in our string class, strings are represented by the private data member *str* of type **char***. We may access the *i*th character of *str* using either of the notations $str[i]$ and $*(str + i)$ and we may assign a string to *str* using a statement such as $str = \text{"abc"}$.

```

class String
{
public:
    String(char *init, int m);
    // Constructor that initializes *this to string init of length m.

    bool operator==(String t);
    // If the string represented by *this equals t, return true;
    // else return false.

    bool operator!();
    // If *this is empty then return true; else return false.

    int Length();
    // Return the number of characters in *this.

    String Concat(String t);
    // Return a string whose elements are those of *this followed by those of t.

    String Substr(int i, int j);
    // Return a string containing the j characters of *this at positions i, i+1, ...,
    // i + j - 1 if these are valid positions of *this; otherwise, throw an exception.

    int Find(String pat);
    // Return an index i such that pat matches the substring of *this that begins
    // at position i. Return -1 if pat is either empty or not a substring of *this.
};

```

ADT 2.5: Abstract data type *String*

2.6.1 String Pattern Matching: A Simple Algorithm

Assume that we have two strings, s and pat , where pat is a pattern to be searched for in s . We will determine if pat is in s by using the function *Find*. The invocation $s.Find(pat)$ returns an index i such that pat matches the substring of s that begins at position i . It returns -1 if and only if pat is either empty or is not a substring of s . Let us examine how a function *Find* may be implemented.

The easiest but least efficient method to determine whether pat is in s is to serially consider each position of s and determine if this position is the starting point of a match. Let $lengthP$ and $lengthS$ denote the lengths of the pattern pat and the string s , respectively. Positions of s to the right of position $lengthS - lengthP$ need not be considered, as there are not enough characters to their right to complete a match with pat . Function *Find* (Program 2.15)

implements this strategy. The complexity of this function is $O(\text{length}P \cdot \text{length}S)$.

```

int String::Find(String pat)
{ // Return -1 if pat does not occur in *this;
  // otherwise return the first position in *this, where pat begins.
    for (int start = 0; start <= Length() - pat.Length(); start++)
        { // check for match beginning at str[start]
            int j;
            for (j = 0; j < pat.Length() && str[start + j] == pat.str[j]; j++)
                if (j == pat.Length()) return start; // match found
            // no match at position start
        }
    return -1 ; // pat is empty or does not occur in s
}

```

Program 2.15: Exhaustive pattern matching

We can introduce heuristics into function *Find* that improve its performance on certain pairs of *s* and *pat*. For example, for each position *start* of *s* considered in function *Find*, we may check for a match of the last character of *pat* with the character at position *start* + *lengthP* - 1 of *s* before examining characters 0 through *lengthP* - 2 of *pat* for a match. The asymptotic complexity of the resulting pattern matching function is still $O(\text{length}P \cdot \text{length}S)$.

2.6.2 String Pattern Matching: The Knuth-Morris-Pratt Algorithm

Ideally, we would like an algorithm that works in $O(\text{length}P + \text{length}S)$ time. This is optimal for this problem, as in the worst case it is necessary to look at all characters in the pattern and string at least once. We want to search the string for the pattern without moving backwards in the string. That is, if a mismatch occurs we want to use our knowledge of the characters in the pattern and the position in the pattern, where the mismatch occurred to determine where we should continue the search. Knuth, Morris, and Pratt have developed a pattern-matching algorithm that works in this way and has linear complexity. Using their example, suppose

$$pat = a b c a b c a c a b$$

Let $s = s_0 s_1 \cdots s_{m-1}$ be the string and assume that we are currently

116 Arrays

determining whether or not there is a match beginning at s_i . If $s_i \neq a$, then clearly we may proceed by comparing s_{i+1} and a . Similarly, if $s_i = a$ and $s_{i+1} \neq b$, then we may proceed by comparing s_{i+1} and a . If $s_i s_{i+1} = ab$ and $s_{i+2} \neq c$ then we have the situation

$s =$	-	a	b	?	?	?	.	.	.	.	?
$pat =$		a	b	c	a	b	c	a	c	a	b

The ? implies that we do not know what the character in s is. The first ? in s represents s_{i+2} , where $s_{i+2} \neq c$. At this point we know that we may continue the search for a match by comparing the first character in pat with s_{i+2} . There is no need to compare this character of pat with s_{i+1} , since we already know that s_{i+1} is the same as the second character of pat , b , and so $s_{i+1} \neq a$. Let us try this again assuming a match of the first four characters in pat followed by a non-match, i.e., $s_{i+4} \neq b$. We now have the situation

$s =$	-	a	b	c	a	?	?	.	.	.	?
$pat =$		a	b	c	a	b	c	a	c	a	b

We observe that the search for a match can proceed by comparing s_{i+4} and the second character in pat , b . This is the first place a partial match can occur by sliding the pattern pat towards the right. Thus, by knowing the characters in the pattern and the position in the pattern where a mismatch occurs with a character in s , we can determine where in the pattern to continue the search for a match without moving backwards in s . To formalize this, we define a failure function for a pattern.

Definition: If $p = p_0 p_2 \cdots p_{n-1}$ is a pattern, then its *failure function*, f , is defined as

$$f(j) = \begin{cases} \text{largest } k < j \text{ such that } p_0 \cdots p_k = p_{j-k} \cdots p_j & \text{if such a } k \geq 0 \text{ exists} \\ -1 & \text{otherwise.} \end{cases} \quad \square$$

For the example pattern above, $pat = abcabcacab$, we have

j	0	1	2	3	4	5	6	7	8	9
pat	a	b	c	a	b	c	a	c	a	b
f	-1	-1	-1	0	1	2	3	-1	0	1

From the definition of the failure function, we arrive at the following rule for pattern matching: If a partial match is found such that $s_{i-j} \cdots s_{i-1} = p_0 \cdots p_{j-1}$ and $s_i \neq p_j$ then matching may be resumed by

comparing s_i and $p_{f(j-1)+1}$ if $j \neq 0$. If $j = 0$, then we may continue by comparing s_{i+1} and p_0 . This pattern-matching rule translates to function *FastFind* (Program 2.16). *FastFind* uses an array of ints, f , to represent the failure function. f is a private data member of *String*.

```

1 int String::FastFind(String pat)
2 { // Determine if pat is a substring of s.
3   int posP = 0, posS = 0 ;
4   int lengthP = pat.Length(), lengthS = Length();
5   while ((posP < lengthP) && (posS < lengthS))
6     if (pat.str[posP] == str[posS]) { // character match
7       posP++ ; posS++ ;
8     }
9     else
10      if (posP == 0)
11        posS++ ;
12      else posP = pat.f[posP - 1] + 1 ;
13  if (posP < lengthP) return -1 ;
14  else return posS - lengthP ;
15 }
```

Program 2.16: Pattern-matching with a failure function

Analysis of *FastFind*: The correctness of *FastFind* follows from the definition of the failure function. To determine the computing time, we observe that lines 7 and 11 can be executed for a total of at most $lengthS$ times, since in each iteration $posS$ is incremented by 1 but $posS$ is never decremented in the algorithm. As a result, $posP$ can move right on pat at most $lengthS$ times (line 7). Since each execution of line 12 moves $posP$ left on pat , it follows that this clause can be executed at most $lengthS$ times, since otherwise $posP$ becomes less than 0. Consequently, the maximum number of iterations of the **while** loop is $lengthS$, and the computing time of *FastFind* is $O(lengthS)$. $\square$

From the analysis of *FastFind*, it follows that if we can compute the failure function in $O(lengthP)$ time, then the entire pattern-matching process will have a computing time proportional to the sum of the lengths of the string and pattern. Fortunately, there is a fast way to compute the failure function. This is based upon the following restatement of the failure function:

$$f(j) = \begin{cases} -1 & \text{if } j = 0 \\ f^m(j-1) + 1 & \text{where } m \text{ is the least integer } k \text{ for which } p_{f^k(j-1)+1} = p_j \\ -1 & \text{if there is no } k \text{ satisfying the above} \end{cases}$$

118 Arrays

(Note that $f^1(j) = f(j)$ and $f^m(j) = f(f^{m-1}(j))$). This directly yields the function of Program 2.17 to compute f .

```
1 void String::FailureFunction()
2 // Compute the failure function for the pattern *this.
3 int lengthP = Length();
4 f[0] = -1;
5 for (int j = 1; j < lengthP; j++) // compute f[j]
6 {
7     int i = f[j-1];
8     while ((*str + j) != *(str+i+1)) && (i >= 0)) i = f[i];
9     if ((*str + j) == *(str + i + 1))
10         f[j] = i + 1;
11     else f[j] = -1;
12 }
13 }
```

Program 2.17: Computing the failure function

Analysis of FailureFunction: In each iteration of the **while** loop the value of i decreases (by the definition of f). The variable i is reset at the beginning of each iteration of the **for** loop. However, it is either reset to -1 (when $j = 1$ or when the previous iteration went through line 11), or it is reset to a value 1 greater than its terminal value on the previous iteration (i.e., when the previous iteration went through line 10). Since only $\text{lengthP} - 1$ executions of line 7 are made, the value of i therefore has a total increment of at most $\text{lengthP} - 1$. Hence it cannot be decremented more than $\text{lengthP} - 1$ times. Consequently, the **while** loop is iterated at most $\text{lengthP} - 1$ times over the whole algorithm, and the computing time of *FailureFunction* is $O(\text{lengthP})$. $\square$

Note that when the failure function is not known in advance, pattern matching can be carried out in time $O(\text{lengthP} + \text{lengthS})$ by first computing *FailureFunction* and then performing a pattern match using function *FastFind*.

EXERCISES

1. Write a function *String::Frequency* that determines the frequency of occurrence of each of the distinct characters in the string. Test your function using suitable data.

2. Write a function, *String::Delete*, that accepts two integers, *start* and *length*. The function computes a new string that is equivalent to the original string, except that *length* characters beginning at *start* have been removed.
3. Write a function, *String::CharDelete*, that accepts a character *c*. The function returns the string with all occurrences of *c* removed.
4. Write a function to make an in-place replacement of a substring *w* of a string by the string *x*. Note that *w* may not be of the same size as *x*. What is the complexity of your function?
5. If $x = (x_0, \dots, x_{m-1})$ and $y = (y_0, \dots, y_{n-1})$ are strings, where x_i and y_i are letters of the alphabet, then *x* is less than *y* if $x_i = y_i$ for $0 \leq i < j$ and $x_j < y_j$ or if $x_i = y_i$ for $0 \leq i \leq m$ and $m < n$. Write an algorithm that takes two strings *x, y* and returns either -1 , 0 , or $+1$ if $x < y$, $x = y$, or $x > y$ respectively.
6. (a) Find a string and a pattern for which function *Find* (Program 2.15) takes time proportional to $\text{length}P \cdot \text{length}S$.
(b) Do part (a) under the assumption that the function *Find* has been modified to check for a match with the last character of the pattern first (see text for an explanation of this heuristic).
7. Compute the failure function for each of the following patterns:
 - (a) *a a a a b*
 - (b) *a b a b a a*
 - (c) *a b a a b a a b b*
8. Let $p_0 p_1 \dots p_{n-1}$ be a pattern of length *n*. Let *f* be its failure function. Define $f^1(j) = f(j)$ and $f^m(j) = f(f^{m-1}(j))$, $0 \leq j < n$ and $m > 1$. Show, using the definition of *f*, that

$$f(j) = \begin{cases} -1 & \text{if } j = 0 \\ f^m(j-1)+1 & \text{where } m \text{ is the least integer } k \text{ for which } p_{f^k(j-1)+1} = p_j \\ -1 & \text{if there is no } k \text{ satisfying the above} \end{cases}$$

9. The definition of the failure function may be strengthened to

$$f(j) = \begin{cases} \text{largest } i < j \text{ such that } p_0 \dots p_i = p_{j-i} \dots p_j \text{ and } p_{i+1} \neq p_{j+1} \\ -1 & \text{if there is no } i \geq 0 \text{ satisfying above} \end{cases}$$

- (a) Obtain the new failure function for the pattern *pat* of the text.
- (b) Show that if this definition for *f* is used, then algorithm *FastFind*

(Program 2.16) still works correctly.

- (c) Modify algorithm *FailureFunction* (Program 2.17) to compute f under this definition. Show that the computing time is still $O(m)$.
- (d) Are there any patterns for which the observed computing time of *FastFind* is more with the new definition of f than with the old one? Are there any for which it is less? Give examples.

2.7 REFERENCES AND SELECTED READINGS

The Knuth, Morris, Pratt pattern-matching algorithm can be found in "Fast pattern matching in strings," *SIAM Journal on Computing*, 6:2, 1977, pp. 323-350. A discussion of the Knuth Morris Pratt algorithm, along with other string matching algorithms, may be found in *Introduction to Algorithms* Second Edition, by T. Cormen, C. Leiserson, R. Rivest and C. Stein, McGraw Hill, New York, 2002.

2.8 ADDITIONAL EXERCISES

1. Write a C++ function to make an in-place reversal of the order of elements in the array *list*. That is, the function should transform *list* such that following the transformation, *list*[i] contains the element originally in *list*[$n - i - 1$]th, where n is the total number of elements in *list*. The only additional space available to your function is that for simple variables. How much time does your function take to accomplish the reversal?
2. An $m \times n$ matrix is said to have a saddle point if some entry $a[i][j]$ is the smallest value in row i and the largest value in column j . Write a C++ function that determines the location of a saddle point if one exists. What is the computing time of your method?
3. When all the elements either above or below the main diagonal of a square matrix are zero, then the matrix is said to be triangular. Figure 2.8 shows a lower and an upper triangular matrix. In a lower triangular matrix, a , with n rows, the maximum number of nonzero terms in row i is $i + 1$. Hence, the total number of nonzero terms is $\sum_{i=0}^{n-1} (i + 1) = n(n + 1)/2$. For large n it would be worthwhile to save the space taken by the zero entries in the upper triangle. Obtain an addressing formula for elements a_{ij} in the lower triangle if this lower triangle is stored by rows in an array $b[n(n + 1)/2]$ with $a[0][0]$ being stored in $b[0]$. What is the relationship between i and j for elements in the zero part of A ?

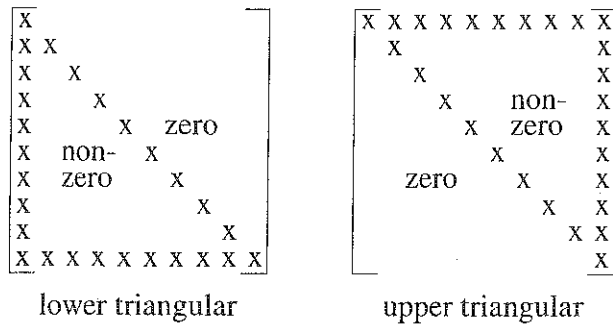


Figure 2.8: Lower and upper triangular matrices

4. Let a and b be two lower triangular matrices, each with n rows. The total number of elements in the lower triangles is $n(n + 1)$. Devise a scheme to represent both the triangles in an array $c[n][n+1]$. [Hint: Represent the triangle of a as the lower triangle of c and the transpose of b as the upper triangle of c .] Write algorithms to determine the values of $a[i][j]$ and $b[i][j]$, $0 \leq i, j < n$, from the array c .
5. Another kind of sparse matrix that arises often in practice is the tridiagonal matrix. In this square matrix, all elements other than those on the major diagonal and on the diagonals immediately above and below this one are zero (Figure 2.9).

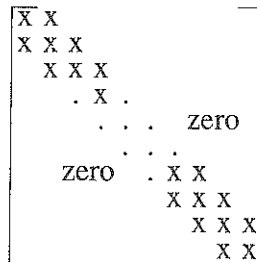


Figure 2.9: Tridiagonal matrix

If the elements in the band formed by these three diagonals are represented

by rows in an array, b , with $a[0][0]$ being stored in $b[0]$, obtain an algorithm to determine the value of $a[i][j]$, $0 \leq i, j < n$ from the array b .

6. A square band matrix $a_{n,a}$ is an $n \times n$ matrix in which all the nonzero terms lie in a band centered around the main diagonal. The band includes $a - 1$ diagonals below and above the main diagonal and also the main diagonal (Figure 2.10).

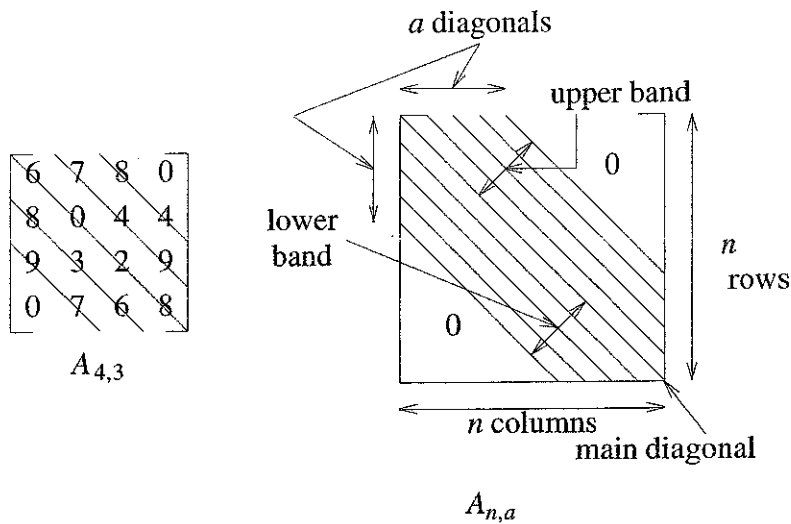


Figure 2.10: Square band matrix

- How many elements are there in the band of $a_{n,a}$?
- What is the relationship between i and j for elements a_{ij} in the band of $a_{n,a}$?
- Assume that the band of $a_{n,a}$ is stored sequentially in an array b by diagonals starting with the lowermost diagonal. Thus, $a_{4,3}$ above would have the following representation:

$b[0]$	$b[1]$	$b[2]$	$b[3]$	$b[4]$	$b[5]$	$b[6]$	$b[7]$	$b[8]$	$b[9]$	$b[10]$	$b[11]$	$b[12]$	$b[13]$
9	7	8	3	6	6	0	2	8	7	4	9	8	4
a_{20}	a_{31}	a_{10}	a_{21}	a_{32}	a_{00}	a_{11}	a_{22}	a_{33}	a_{01}	a_{12}	a_{23}	a_{02}	a_{13}

Obtain an addressing formula for the location of an element a_{ij} in the lower band of $a_{n,a}$, e.g., $LOC(a_{20}) = 0$, $LOC(a_{31}) = 1$ in the example above.

7. A generalized band matrix $a_{n,a,b}$ is an $n \times n$ matrix a in which all the nonzero terms lie in a band made up of $a - 1$ diagonals below the main diagonal, the main diagonal, and $b - 1$ diagonals above the main diagonal (Figure 2.11).
- How many elements are there in the band of $a_{n,a,b}$?
 - What is the relationship between i and j for elements a_{ij} in the band of $a_{n,a,b}$?
 - Obtain a sequential representation of the band of $a_{n,a,b}$ in the one-dimensional array c . For this representation, write a C++ function *value* (n,a,b,i,j,c) that determines the value of element a_{ij} in the matrix $a_{n,a,b}$. The band of $a_{n,a,b}$ is represented in the array c .

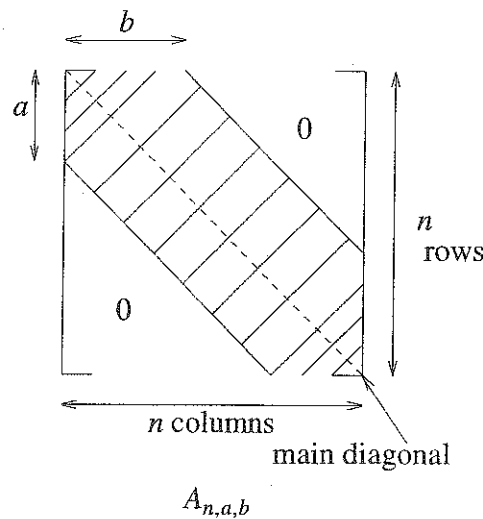


Figure 2.11: Generalized band matrix

8. [Programming Project] There are a number of problems, known collectively as “random walk” problems, that have been of longstanding interest to the mathematical community. All but the most simple of these are extremely difficult to solve and for the most part they remain largely unsolved. One such problem may be stated as

A (drunken) cockroach is placed on a given square in the middle of a

124 Arrays

tile floor in a rectangular room of size $n \times m$ tiles. The bug wanders (possibly in search of an aspirin) randomly from tile to tile throughout the room. Assuming that he may move from his present tile to any of the eight tiles surrounding him (unless he is against a wall) *with equal probability*, how long will it take him to touch every tile on the floor at least once?

Hard as this problem may be to solve by probability theory techniques, it is easy to solve using the computer. The technique for doing so is called "simulation." This technique is widely used in industry to predict traffic flow, inventory control, and so forth. The problem may be simulated using the following method:

An $n \times m$ array *count* is used to represent the number of times our cockroach has reached each tile on the floor. All the cells of this array are initialized to zero. The position of the bug on the floor is represented by the coordinates (*ibug*, *jbug*). The eight possible moves of the bug are represented by the tiles located at (*ibug* + *imove*[*k*], *jbug* + *jmove*[*k*]), where $0 \leq k \leq 7$ and

<i>imove</i> [0] =	-1	<i>jmove</i> [0] =	1
<i>imove</i> [1] =	0	<i>jmove</i> [1] =	1
<i>imove</i> [2] =	1	<i>jmove</i> [2] =	1
<i>imove</i> [3] =	1	<i>jmove</i> [3] =	0
<i>imove</i> [4] =	1	<i>jmove</i> [4] =	-1
<i>imove</i> [5] =	0	<i>jmove</i> [5] =	-1
<i>imove</i> [6] =	-1	<i>jmove</i> [6] =	-1
<i>imove</i> [7] =	-1	<i>jmove</i> [7] =	0

A *random walk* to one of the eight given squares is simulated by generating a random value for *k* lying between 0 and 7. Of course, the bug cannot move outside the room, so coordinates that lead up a wall must be ignored and a new random combination formed. Each time a square is entered, the count for that square is incremented so that a nonzero entry shows the number of times the bug has landed on that square so far. When every square has been entered at least once, the experiment is complete.

Write a program to perform the specified simulation experiment. Your program *MUST*:

- handle all values of *n* and *m*, $2 < n \leq 40$, $2 \leq m \leq 20$
 - perform the experiment for (1) $n = 15$, $m = 15$, starting point: (9,9) and (2) $n = 39$, $m = 19$, starting point: (0,0)
-

- (c) have an iteration limit, that is, a maximum number of squares the bug may enter during the experiment (this avoids getting hung in an infinite loop); a maximum of 50,000 is appropriate for this exercise
- (d) for each experiment, print (1) the total number of legal moves that the cockroach makes and (2) the final *count* array (this will show the density of the walk, that is, the number of times each tile on the floor was touched during the experiment).

(Have an aspirin.) This exercise was contributed by Steve Olson.

9. [**Programming Project**] Chess provides the setting for many fascinating diversions that are quite independent of the game itself. Many of these are based on the strange L-shaped move of the knight. A classical example is the problem of the knight's tour, which has captured the attention of mathematicians and puzzle enthusiasts since the beginning of the eighteenth century. Briefly stated, the problem is to move the knight, beginning from any given square on the chessboard, in such a manner that it travels successively to all 64 squares, touching each square once and only once. It is convenient to represent a solution by placing the numbers 1, 2, ..., 64 in the squares of the chessboard indicating the order in which the squares are reached. Note that it is not required that the knight be able to reach the initial position by one more move; if this is possible, the knight's tour is called re-entrant. One of the more ingenious methods for solving the problem of the knight's tour was that given by J. C. Warnsdorff in 1823. His rule was that the knight must always be moved to one of the squares from which there are the fewest exits to squares not already traversed. Use Warnsdorff's rule to construct a particular solution to the problem by hand before reading any further.

The most important decisions to be made in solving a problem of this type are those concerning how the data is to be represented in the computer. Perhaps the most natural way to represent the chessboard is by an 8×8 array *board*, as shown in Figure 2.12. The eight possible moves of a knight on square (4,2) are also shown in this figure. In general a knight at (i,j) may move to one of the squares $(i-2,j+1)$, $(i-1,j+2)$, $(i+1,j+2)$, $(i+2,j+1)$, $(i+2,j-1)$, $(i+1,j-2)$, $(i-1,j-2)$, $(i-2,j-1)$. Notice, however, that if (i,j) is located near one of the edges of the board, some of these possibilities could move the knight off the board, and, of course, this is not permitted. The eight possible knight moves may conveniently be represented by two arrays *ktmov1* and *ktmov2*:

	0	1	2	3	4	5	6	7
0								
1								
2		8		1				
3	7				2			
4			K					
5	6				3			
6		5		4				
7								

Figure 2.12: Legal moves for a knight

<i>ktmov1</i>	<i>ktmov2</i>
-2	1
-1	2
1	2
2	1
2	-1
1	-2
-1	-2
-2	-1

Then a knight at (i, j) may move to $(i + \text{ktmov1}[k], j + \text{ktmov2}[k])$, where k is some value between 0 and 7, provided that the new square lies on the chessboard.

An algorithm to solve the knight's tour problem using Warnsdoff's rule is described below.

- (a) [**Initialize chessboard**] For $0 \leq i, j \leq 7$, set $\text{board}[i][j]$ to 0.
 - (b) [**Set starting position**] Read and print i, j and then set $\text{board}[i][j]$ to
-

1.

- (c) **[Loop]** For $2 \leq m \leq 64$, do steps (d) through (g).
- (d) **[Form set of possible next squares]** Test each of the eight squares one knight's move away from (i, j) and form a list of the possibilities for the next square $(nexti[l], nextj[l])$. Let $npos$ be the number of possibilities. (That is, after performing this step we will have $nexti[l] = i + ktmov1[k]$ and $nextj[l] = j + ktmov2[k]$, for certain values of k between 0 and 7. Some of the squares $(i + ktmov1[k], j + ktmov2[k])$ may be impossible for the next move either because they lie off the chessboard or because they have been previously occupied by the knight (i.e., they contain a nonzero number). In every case we will have $0 \leq npos \leq 8$.)
- (e) **[Test special cases]** If $npos = 0$, the knight's tour has come to a premature end; report failure and then go to step (h). If $npos = 1$, there is only one possibility for the next move; set $min = 0$ and go right to step (g).
- (f) **[Find next square with minimum number of exits]** For $1 \leq l \leq npos$ set $exits[l]$ to the number of exits from square $(nexti[l], nextj[l])$. That is, for each of the values of l , examine each of the next squares $(nexti[l] + ktmov1[k], nextj[l] + ktmov2[k])$ to see if it is an exit from $(nexti[l], nextj[l])$, and count the number of such exits in $exits[l]$. (Recall that a square is an exit if it lies on the chessboard and has not been previously occupied by the knight.) Finally, set min to the location of the minimum value of $exits$. (There may be more than one occurrence of the minimum value of $exits$. If this happens, it is convenient to let min denote the first such occurrence, although it is important to realize that by so doing we are not actually guaranteed finding a solution. Nevertheless, the chances of finding a complete knight's tour in this way are remarkably good, and that is sufficient for the purposes of this exercise.)
- (g) **[Move knight]** Set $i = nexti[min]$, $j = nextj[min]$, and $board[i][j] = m$. Thus, (i, j) denotes the new position of the knight, and $board[i][j]$ records the move in proper sequence.
- (h) **[Output]** Output $board$ showing the solution to the knight's tour and then terminate the algorithm.

The problem is to write a C++ program that corresponds to this algorithm. This exercise was contributed by Legenhausen and Rebman.

CHAPTER 3

Stacks and Queues

3.1 Templates in C++

In this section, we introduce the concept of templates. This is a mechanism provided by C++ to make classes and functions more reusable. We shall discuss both template functions and template classes.

3.1.1 Template Functions

Consider function *SelectionSort* of Program 1.6, which sorts an array of integers using the selection sort method. Suppose, we now wish to use selection sort to sort an array of floating point numbers. The code of Program 1.6 can be used to do this if we change `int *a` to `float *a` in the function header. From a practical standpoint, this can be achieved by using a text editor to replicate the code for function *SelectionSort* and then making the stated change. This process would have to be repeated if we next wished to sort an array of, say, characters.

Considerable software development time and money can be saved if we

use *templates* or *parameterized types*. A template may be viewed as a variable that can be instantiated to any data type, irrespective of whether this data type is a fundamental C++ type or a user-defined type. The template function *SelectionSort* (Program 3.1) is defined using the parameterized type *T*.

```

1 template <class T>
2 void SelectionSort (T*a, const int n)
3 { // Sort a[0] to a[n-1] into nondecreasing order.
4     for ( int i = 0; i < n ; i++ )
5     {
6         int j = i;
7         // find smallest integer in a[i] to a[n - 1]
8         for ( int k = i + 1 ; k < n ; k++ )
9             if (a[k] < a[j]) j = k ;
10        swap (a[i], a[j]);
11    }
12 }
```

Program 3.1: Selection sort using templates

Function *SelectionSort* can now be used quite easily to sort an array of **ints** or **floats** as shown in Program 3.2. Function *SelectionSort* is instantiated to the type of the array argument that is supplied to it. For example, the first call to *SelectionSort* knows that it is to sort an array of floating point numbers because array *farray* is an array of floating point numbers.

```

float farray[100];
int intarray[250];

.

.

// assume that the arrays are initialized at this point
SelectionSort(farray, 100);
SelectionSort(intarray, 250);
```

Program 3.2: Code fragment illustrating template instantiation

Observe that *SelectionSort* uses the operator "<" to compare two objects of type *T* in line 9. So long as this operator (as well as others used by *swap*) is defined for the data type *T*, Program 3.1 may be used to sort an array of type *T*.

130 Stacks and Queues

Suppose we want to use Program 3.1 to sort an array of *Rectangles* in non-decreasing order of their areas. We cannot instantiate *T* to *Rectangle* as we did with **int** and **float** in Program 3.2 because operator "<" of line 9 is undefined for *Rectangle*. This can be remedied by overloading **operator<** so that it returns **true** if the first operand has less area than the second and **false**, otherwise. Of course, additional operators used by *swap* also must be defined for *Rectangle* unless the C++ default (if any) for these operators will suffice.

A template function that we shall find quite useful in this text is one to change the size of a 1-dimensional array. We have already seen two applications for array resizing (polynomial and matrix addition (Programs 2.6 and 2.13)). Program 3.3 gives a template function that changes the size of a 1-dimensional array of type *T* from *oldSize* to *newSize*.

```
template <class T>
void ChangeSize1D(T*& a, const int oldSize, const int newSize)
{
    if (newSize < 0) throw "New length must be >= 0";

    T* temp = new T[newSize];           // new array
    int number = min(oldSize, newSize); // number to copy
    copy(a, a + number, temp);
    delete [] a;                        // deallocate old memory
    a = temp;
}
```

Program 3.3: Template function to change the size of a 1-dimensional array

3.1.2 Using Templates to Represent Container Classes

A *container class* is a class that represents a data structure that contains or stores a number of data objects. Objects can usually be added to or deleted from a container class. The array is an example of a container class as it is used to store a number of objects. We begin by introducing the container class *Bag*. Our specification for the class *Bag* is that it is a data structure into which objects can be inserted and from which objects can be deleted, like a bag of groceries. A bag can have multiple occurrences of the same element, but we do not care about the position of an element; nor do we care which element is removed when a delete operation is performed. We implement *Bag* by using a C++ array to hold its objects. Program 3.4 contains the class definition of a *Bag* of integers. The

default initial capacity of a bag is 10. When a *Bag* function is unable to perform its task, it is to throw an exception.

```

class Bag
{
public:
    Bag (int bagCapacity = 10); // constructor
    ~Bag();                     // destructor

    int Size() const;           // return number of elements in bag
    bool IsEmpty() const;      // return true if the bag is empty; false otherwise
    int Element() const;       // return an element that is in the bag

    void Push(const int);       // insert an integer into the bag
    void Pop();                 // delete an integer from the bag

private:
    int *array;
    int capacity;              // capacity of array
    int top;                   // array position of top element
};

```

Program 3.4: Definition of the class *Bag* containing integers

We implement insertion into the *Bag* by storing the element that is to be inserted into the first available position in the array. In case the array is full, its capacity is doubled. Deletion is implemented by deleting the element in the middle position of the array, and then moving all elements to its right, one position to the left. The function *Element* is implemented so as to return the element that will be deleted in case a *Pop* is done. Our implementation decisions for choosing the positions at which an element will be inserted or deleted as well as for choosing the element returned by *Element* are arbitrary. We could have just as easily decreed that elements should be inserted into the middle position, deleted from the last position, and that *Element* return the integer in the first position of the array. Program 3.5 contains our implementation of *Bag* operations.

Most of the operations of *Bag* need no explanation. We have declared member functions *Size()*, *IsEmpty()*, and *Element* as **inline** because of their small length (each contains only two lines of code). This eliminates the overhead of performing a function call. The three functions have the **const** attribute as they do not change the bag object upon which they operate.

```

Bag::Bag (int bagCapacity): capacity ( bagCapacity ) {
    if (capacity < 1) throw "Capacity must be > 0";
    array = new int[capacity];
    top = -1;
}

Bag::~Bag() { delete [ ] array;}

inline int Bag::Size() const {return top+1;}

inline bool Bag::IsEmpty() const {return size == 0;}

inline int Bag::Element() const {
    if (IsEmpty()) throw "Bag is empty";
    return array [0];
}

void Bag::Push(const int x) {
    if (capacity == top+1) ChangeSize1D(array, capacity, 2 * capacity);
    capacity *= 2;
    array[++top] = x;
}

void Bag::Pop() {
    if (IsEmpty()) throw "Bag is empty, cannot delete";
    int deletePos = top / 2;
    copy (array + deletePos + 1, array + top + 1, array + deletePos);
    // compact array
    top--;
}

```

Program 3.5: Implementation of operations of *Bag*

As we have defined it, *Bag* can be used to store integers only. We would like to implement *Bag* using templates so that *Bag* can be used to store objects of any data type. Container classes are particularly suitable for implementation using templates because the algorithms for basic container class operations are usually independent of the type of objects that the container class contains. For example, the algorithm to add an element to *Bag* does not depend on the type of its elements. Program 3.6 contains a template class definition for *Bag* and

Program 3.7 contains the implementation of some of its operations. These are obtained from Programs 3.4 and 3.5, respectively, by prefixing the class definition of *Bag* and the definitions of all its member functions that are outside the class (in our example, all member functions are defined outside the class definition) with the statement:

```
template<class T>
```

Next, *int* is replaced by *T* when it refers to an object being stored in *Bag*. Finally, in all member function headers, *Bag::* is replaced by *Bag<T>::*. Notice, also, that the single input parameter to *Push* is made a constant reference. This is done because it is now possible that *T* represents a large object, and considerable overhead may be incurred if it is passed by value.

```
template <class T>
class Bag
{
public:
    Bag (int bagCapacity = 10);
    ~Bag();

    int Size() const;
    bool IsEmpty() const;
    T& Element() const;

    void Push(const T&);
    void Pop();

private:
    T *array;
    int capacity;
    int top;
};
```

Program 3.6: Definition of template class *Bag*

The following statements instantiate the template class *Bag* to *int* and *Rectangle*, respectively. So, *a* is a *Bag* of integers and *r* is a *Bag* of *Rectangles*.

```
Bag<int> a ;
Bag<Rectangle> r ;
```

```

template <class T>
Bag<T>::Bag (int bagCapacity): capacity ( bagCapacity ) {
    if (capacity < 1) throw "Capacity must be > 0";
    array = new T[capacity];
    top = -1;
}

template <class T>
Bag<T>::~~Bag() {delete [ ] array; }

template <class T>
void Bag<T>::Push(const T& x) {
    if (capacity == top+1)
    {
        ChangeSize1D(array, capacity, 2 * capacity);
        capacity *= 2;
    }
    array[++top] = x;
}

template <class T>
void Bag<T>::Pop() {
    if (IsEmpty()) throw "Bag is empty, cannot delete";
    int deletePos = top / 2;
    copy(array + deletePos + 1, array + top + 1, array + deletePos);
        // compact array
    array[top--].~T(); // destructor for T
}

```

Program 3.7: Implementation of some operations of *Bag*

3.2 THE STACK ABSTRACT DATA TYPE

We shall now study two data structures that are frequently used in computer programs. These data structures, the stack and the queue, are special cases of the more general data structure *ordered list* that we discussed in Section 2.3. In this section we begin by defining the ADT *Stack* and follow with its implementation. In Section 3.3 we look at the queue.

A *stack* is an ordered list in which insertions (also known as additions, puts, and pushes) and deletions (also known as removals and pops) are made at

one end called the *top*. Given a stack $S = (a_0, \dots, a_{n-1})$, we say that a_0 is the bottom element, a_{n-1} is the top element, and a_i is on top of element a_{i-1} , $0 < i < n$. The restrictions on the stack imply that if we add the elements A, B, C, D, E to the stack, in that order, then E is the first element we delete from the stack. Figure 3.1 illustrates this sequence of operations. Since the last element inserted into a stack is the first element removed, a stack is also known as a *Last-In-First-Out (LIFO)* list.

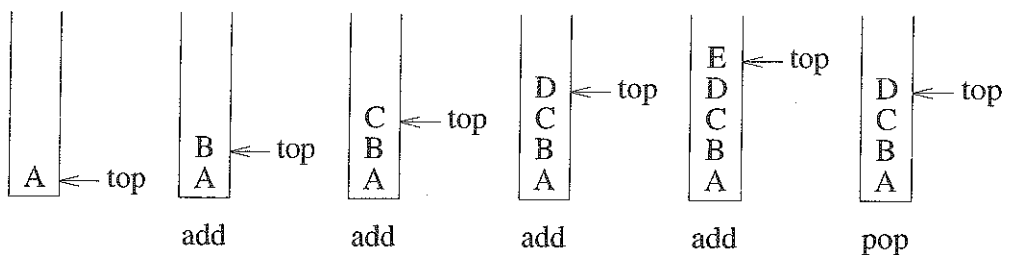


Figure 3.1: Inserting and deleting elements in a stack

Example 3.1 [System stack]: Before we discuss the stack ADT, we look at a special stack, called the system stack, that is used by a program at runtime to process function calls. Whenever a function is invoked, the program creates a structure, referred to as an *activation record* or a *stack frame*, and places it on top of the system stack. Initially, the activation record for the invoked function contains only a pointer to the previous stack frame and a return address. The previous stack frame pointer points to the stack frame of the invoking function; the return address contains the location of the statement to be executed after the function terminates. Since only one function executes at any given time, the function whose stack frame is on top of the system stack is chosen. If this function invokes another function, the local variables and the parameters of the invoking function are added to its stack frame. A new stack frame is then created for the invoked function and placed on top of the system stack. When this function terminates, its stack frame is removed and the processing of the invoking function, which is again on top of the stack, continues. A simple example illustrates this process.

Assume that we have a main function that invokes function $a1$. Figure 3.2(a) shows the system stack before $a1$ is invoked; Figure 3.2(b) shows the system stack after $a1$ has been invoked. Frame pointer fp is a pointer to the current stack frame. The system also maintains separately a stack pointer, sp , which we have not illustrated.

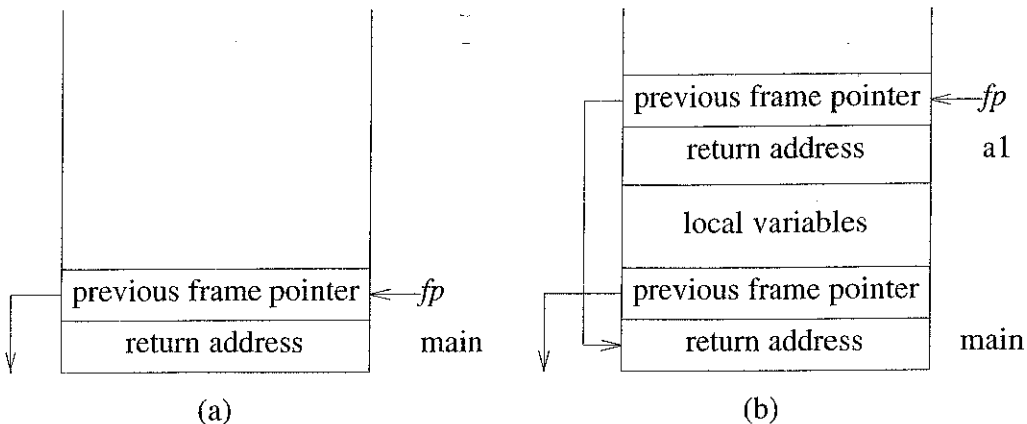


Figure 3.2: System stack after function call

Since all functions are stored similarly in the system stack, it makes no difference if the invoking function calls itself. That is, a recursive call requires no special strategy; the runtime program simply creates a new stack frame for each recursive call. However, recursion can consume a significant portion of the memory allocated to the system stack; it could consume the entire available memory. □

Our discussion of the system stack suggests a basic set of operations, including insert an item, delete an item, and check for stack empty. These are given in the ADT specification (ADT 3.1). When an ADT function is unable to perform its task, it is to throw an exception.

The easiest way to implement the stack ADT is by using an array, say *stack* []. The first, or bottom, element of the stack is stored in *stack* [0], the second in *stack* [1], and the *i*th in *stack* [*i*−1]. Associated with the array is a variable, *top*, which points to the top element in the stack. Initially, *top* is set to −1 to denote an empty stack. This results in the following data member declarations and constructor definition of *Stack*:

private:

```
T*stack;           // array for stack elements
int top;           // array position of top element
int capacity;      // capacity of stack array
```

```

template <class T>
class Stack
{ // A finite ordered list with zero or more elements.
public:
    Stack (int stackCapacity = 10);
    // Create an empty stack whose initial capacity is stackCapacity.

    bool IsEmpty() const;
    // If number of elements in the stack is 0, return true else return false.

    T& Top() const;
    // Return top element of stack.

    void Push (const T& item);
    // Insert item into the top of the stack.

    void Pop();
    // Delete the top element of the stack.
};

```

ADT 3.1: Abstract data type *Stack*

```

template <class T>
Stack<T>::Stack (int stackCapacity) : capacity (stackCapacity)
{
    if (capacity < 1) throw "Stack capacity must be > 0";
    stack = new T[capacity];
    top = -1;
}

```

The member functions *IsEmpty*() and *Top*() are implemented as follows:

```

template <class T>
inline bool Stack<T>::IsEmpty() const { return top == -1;}

template <class T>
inline T& Stack<T>::Top() const
{
    if (IsEmpty()) throw "Stack is empty";
    return stack[top];
}

```

138 Stacks and Queues

The *Push* and *Pop* functions are shown in Programs 3.8 and 3.9.

```
template <class T>
void Stack<T>::Push (const T& x)
{ // Add x to the stack.
    if (top == capacity - 1)
    {
        ChangeSize1D(stack, capacity, 2*capacity);
        capacity *= 2;
    }
    stack[++top] = x;
}
```

Program 3.8: Adding to a stack

```
template <class T>
void Stack<T>::Pop()
{ // Delete top element from the stack.
    if (IsEmpty()) throw "Stack is empty. Cannot delete.";
    stack[top--].~T(); // destructor for T
}
```

Program 3.9: Deleting from a stack

EXERCISES

1. Extend the stack ADT by adding functions to output a stack; split a stack into two stacks with one containing the bottom half elements and the second the remaining elements; and to combine two stacks into one by placing all elements of the second stack on top of those of the first stack. Write C++ code for your new functions.
2. Consider the railroad switching network given in Figure 3.3. Railroad cars numbered 1, 2, 3, ..., n are initially in the top right track segment (in this order, left to right). Railroad cars can be moved into the vertical track segment one at a time from either of the horizontal segments and then moved from the vertical segment to any one of the horizontal segments. The vertical segment operates as a stack as new cars enter at the top and cars depart the vertical segment from the top. For instance, if $n = 3$, we could move

car 1 into the vertical segment, move 2 in, move 3 in, and then take the cars out producing the new order 3, 2, 1. For $n = 3$ and 4 what are the possible permutations of the cars that can be obtained? Are any permutations not possible?

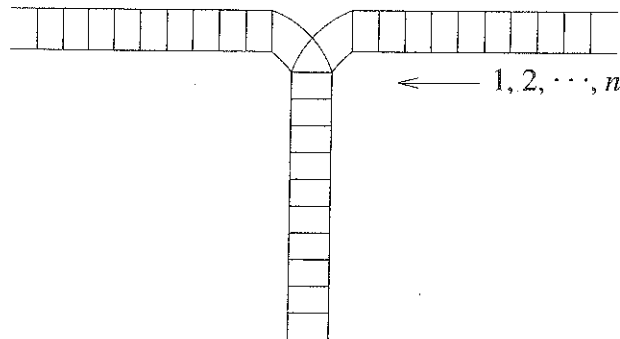


Figure 3.3: Railroad switching network

3.3 THE QUEUE ABSTRACT DATA TYPE

A *queue* is an ordered list in which insertions (also called additions, puts, and pushes) and deletions (also called removals and pops) take place at different ends. The end at which new elements are added is called the *rear*, and that from which old elements are deleted is called the *front*. The restrictions on a queue imply that if we insert A, B, C, D , and E in that order, then A is the first element deleted from the queue. Figure 3.4 illustrates this sequence of events. Since the first element inserted into a queue is the first element removed, queues are also known as *First-In-First-Out (FIFO)* lists. The ADT specification of the queue appears in ADT 3.2. When an ADT function is unable to perform its task, it is to throw an exception.

Analogous to the representation of a stack in Section 3.2, we may use an array `queue[]` with the first (or front) element of the queue in `queue[0]`, the next in `queue[1]`, and so on. Figure 3.5(a) shows a 3-element queue represented in this way and Figure 3.5(b) shows the queue after an element has been deleted. Figure 3.5(c) shows the result of adding an element to the queue of Figure 3.5(b). Since a delete or pop removes the front element, which is in `queue[0]`, each delete requires us to shift the remaining elements to the left. Hence, it takes $\Theta(n)$ time to pop an element from a queue that has n elements; an insertion or push, on the other hand, can be done in $\Theta(1)$ time exclusive of the time for any array resizing

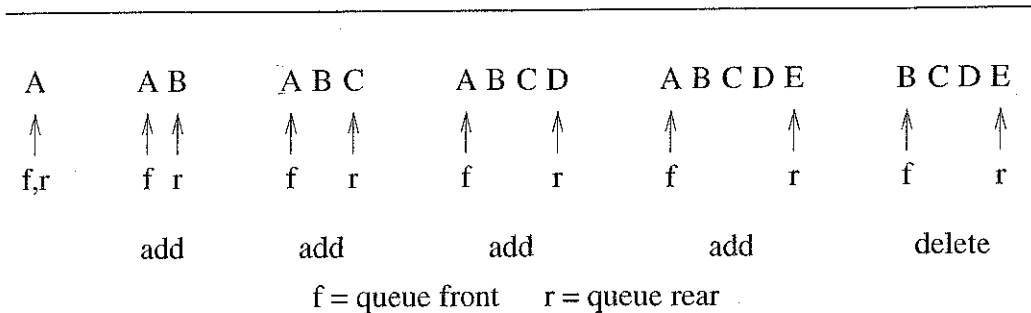


Figure 3.4: Inserting and deleting elements in a queue

```

template <class T>
class Queue
{ // A finite ordered list with zero or more elements.
public:
    Queue(int queueCapacity = 10);
    // Create an empty queue whose initial capacity is queueCapacity

    bool IsEmpty() const;
    // If number of elements in the queue is 0, return true else return false.

    T& Front() const;
    // Return the element at the front of the queue.

    T& Rear() const;
    // Return the element at the rear of the queue.

    void Push (const T& item);
    // Insert item at the rear of the queue.

    void Pop();
    // Delete the front element of the queue.
};

```

ADT 3.2: Abstract data type *Queue*

that may be needed.

To pop an element in $\Theta(1)$ time, we must relax the requirement that *queue* [0] contain the front element of the queue. With this relaxation, we use a variable, *front*, to keep track of the location of the front element. The queue

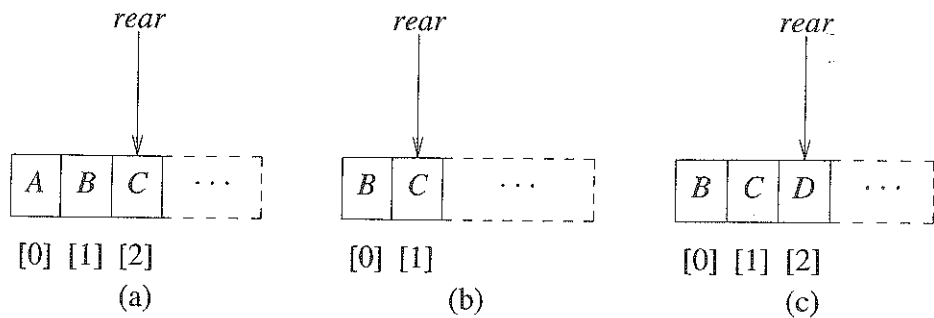


Figure 3.5: Queues represented with front element in *queue* [0]

elements are in *queue* [*front*], ..., *queue* [*rear*]. Figure 3.6 shows three queues represented in this way. The queue of Figure 3.6(b) results from the deletion of the front element of the queue of Figure 3.6(a) and that of Figure 3.6(c) results from adding an element to the queue of Figure 3.6(b).

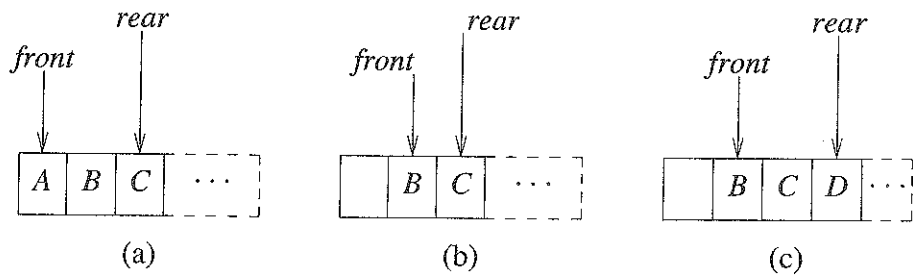


Figure 3.6: Queues represented with front element in *queue* [*front*]

Suppose that the capacity of the array *queue* is *capacity*. The representation of Figure 3.6 runs into a problem when *rear* equals *capacity* - 1 and *front* > 0 as in Figure 3.7(a). How do we add an element to this configuration without increasing array capacity? One possibility is to shift all elements to the left end of the queue (as in Figure 3.7(b)) and create space at the right end. This shifting takes time proportional to the size (i.e., number of elements) of the queue.

The worst-case add and delete times (assuming no array resizing is needed) become $\Theta(1)$ when we permit the queue to wrap around the end of the array. At this time it is convenient to think of the array positions as arranged in a circle

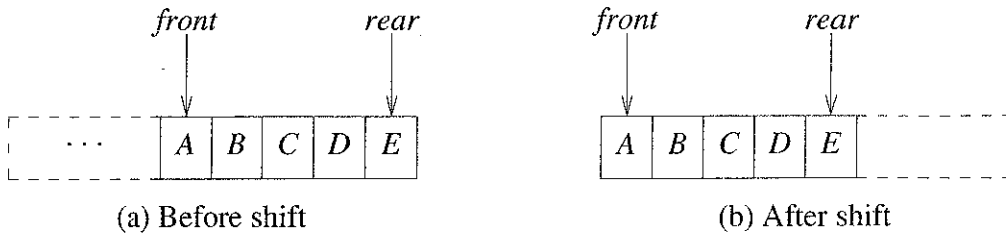


Figure 3.7: Shifting queue elements to the left

(Figure 3.8) rather than in a straight line (Figure 3.7). In Figure 3.8, we have changed the convention for the variable *front*. This variable now points one position counterclockwise from the location of the front element in the queue. The convention for *rear* is unchanged. This change simplifies the codes slightly.

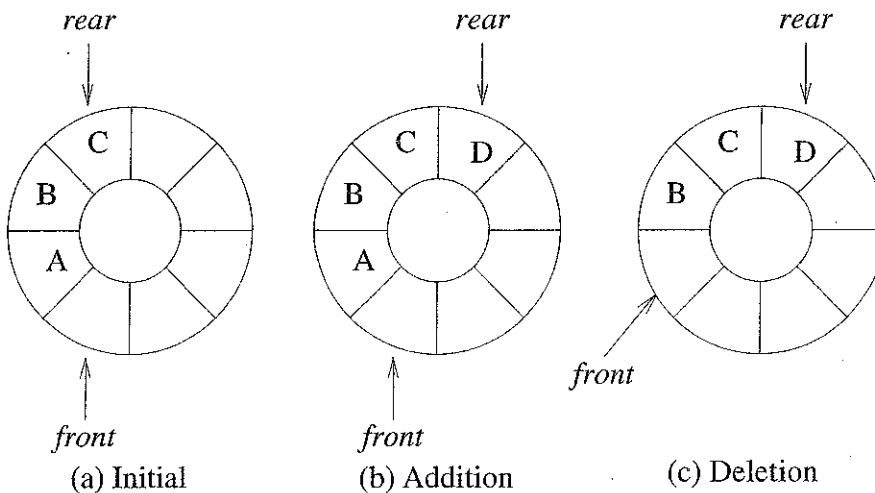


Figure 3.8: Circular queue

When the array is viewed as a circle, each array position has a next and a previous position. The position next to position $\text{capacity} - 1$ is 0, and the position that precedes 0 is $\text{capacity} - 1$. When the queue *rear* is at $\text{capacity} - 1$, the next element is put into position 0. To work with a circular queue, we must be able to move the variables *front* and *rear* from their current position to the next

position (clockwise). This may be done using code such as

```
if (rear == capacity - 1) rear = 0;
else rear++;
```

Using the modulus operator, which computes remainders, this code is equivalent to $(rear+1) \% capacity$. With our conventions for *front* and *rear*, we see that the front element of the queue is located one position clockwise from *front* and the rear element is at position *rear*.

Suppose we implement the ADT *Queue* using an array in a circular fashion. We may use the following data member declarations and constructor.

private:

```
T *queue;      // array for queue elements
int front,     // one counterclockwise from front
    rear,      // array position of rear element
    capacity;  // capacity of queue array
```

```
template <class T>
Queue<T>::Queue (int queueCapacity) : capacity (queueCapacity)
{
    if (capacity < 1) throw "Queue capacity must be > 0";
    queue = new T[capacity];
    front = rear = 0;
}
```

To determine a suitable test for an empty queue, we experiment with the queues of Figure 3.8. To delete an element, we advance *front* one position clockwise and to add an element, we advance *rear* one position clockwise and insert at the new position. If we perform 3 deletions from the queue of Figure 3.8(c) in this fashion, we will see that the queue becomes empty and that $front = rear$. When we do 5 additions to the queue of Figure 3.8(b), the queue becomes full and $front = rear$. So, we cannot distinguish between an empty and a full queue. To avoid the resulting confusion, we shall increase the capacity of a queue just before it becomes full. Consequently, $front == rear$ iff the queue is empty. The member functions *IsEmpty()*, *Front()*, and *Rear()* are implemented as follows:

```
template <class T>
inline bool Queue<T>::IsEmpty() {return front == rear;}
```

```
template <class T>
inline T& Queue<T>::Front()
{
```

144 Stacks and Queues

```
        if (IsEmpty()) throw "Queue is empty. No front element";
        return queue [(front + 1) % capacity];
    }

    template <class T>
    inline T& Queue<T>::Rear()
    {
        if (IsEmpty()) throw "Queue is empty. No rear element";
        return queue [rear];
    }
```

Program 3.10 gives the code to add an element to a queue. This code uses custom code to double the capacity of *queue* when necessary. To see the need for custom code to double capacity, consider the full queue of Figure 3.9(a). This figure shows a queue with seven elements in an array whose capacity is 8. To visualize array doubling when a circular queue is used, it is better to flatten out the array as in Figure 3.9(b). Figure 3.9(c) shows the array after array doubling by *ChangeSize1D* (Program 3.3).

```
template <class T>
void Queue<T>::Push(const& x)
{ // Add x at rear of queue.
    if ((rear + 1) % capacity == front)
        { // queue full, double capacity
            // code to double queue capacity comes here
        }

    rear = (rear + 1) % capacity;
    queue [rear] = x;
}
```

Program 3.10: Adding to a queue

To get a proper circular queue configuration, we must slide the elements in the right segment (i.e., elements *A* and *B*) to the right end of the array as in Figure 3.10(d). The array doubling and the slide to the right together copy at most $2 * \text{capacity} - 2$ elements. The number of elements copied can be limited to $\text{capacity} - 1$ by customizing the array doubling code so as to obtain the configuration of Figure 3.10(e). This configuration may be obtained as follows:

- (1) Create a new array *newQueue* of twice the capacity.

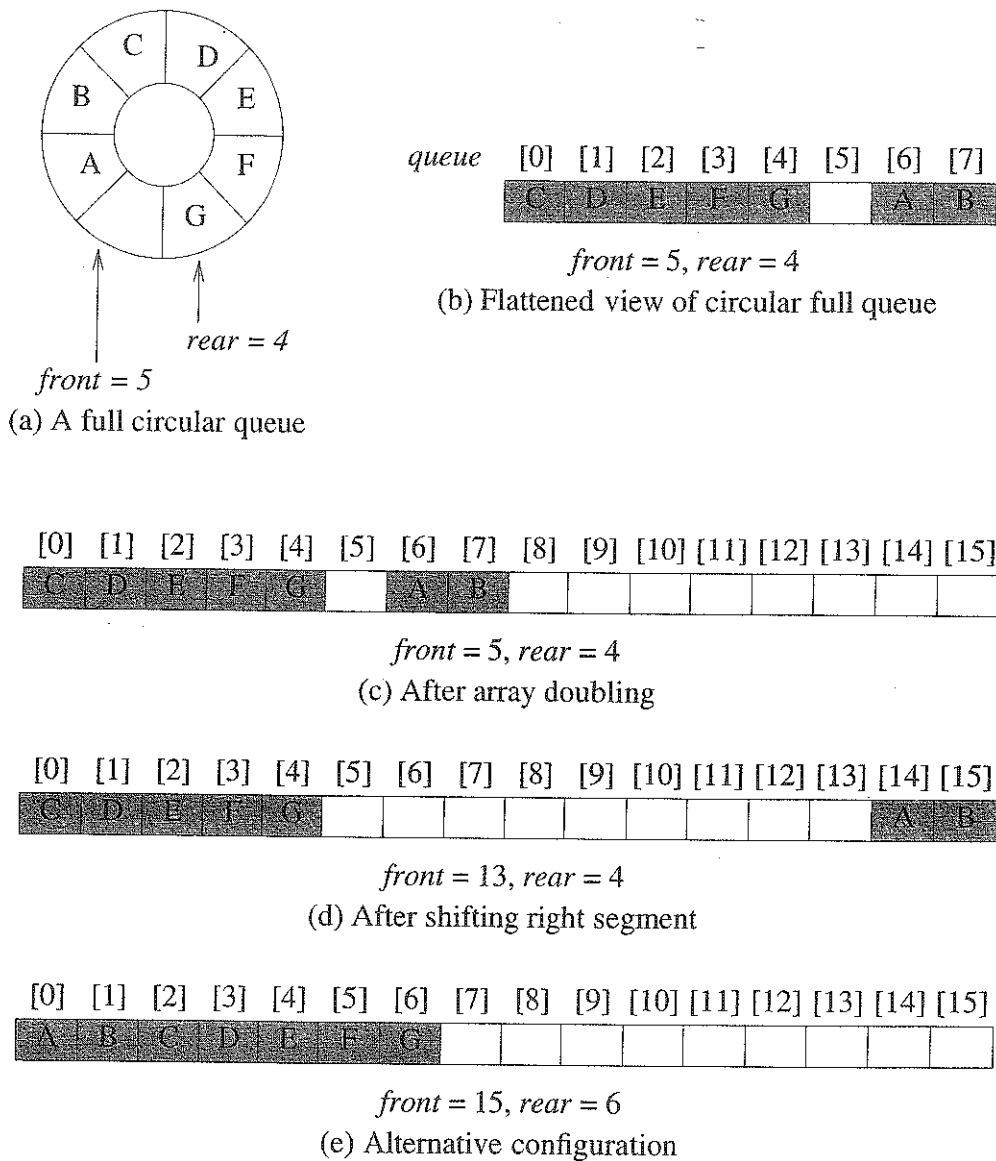


Figure 3.9: Doubling queue capacity

146 Stacks and Queues

- (2) Copy the second segment (i.e., the elements *queue*[*front*+1] through *queue*[*capacity*-1]) to positions in *newQueue* beginning at 0.
- (3) Copy the first segment (i.e., the elements *queue*[0] through *queue*[*rear*]) to positions in *newQueue* beginning at *capacity*-*front*-1.

The code of Program 3.11 obtains the configuration of Figure 3.10(e). Program 3.12 gives the code to delete or pop an element from a queue. Its complexity is $O(1)$.

```
// allocate an array with twice the capacity
T* newQueue = new T[2*capacity];

// copy from queue to newQueue
int start = (front + 1) % capacity;
if (start < 2)
    // no wrap around
    copy(queue + start, queue + start + capacity - 1, newQueue);
else
    { // queue wraps around
        copy(queue + start, queue + capacity, newQueue);
        copy(queue, queue + rear + 1, newQueue + capacity - start);
    }

// switch to newQueue
front = 2*capacity - 1;
rear = capacity - 2;
capacity *= 2;
delete [] queue;
queue = newQueue;
```

Program 3.11: Doubling queue capacity

One way to use all positions in the array *queue* is to use an additional variable, *lastOp*, to record the last operation performed on the queue. This variable is initialized to "Pop." Following each addition, it is set to "Push" and following each deletion, it is set to "Pop." Now when *front* == *rear*, we can determine whether the queue is empty or full by examining the value of *lastOp*. If *lastOp* is "Push," then the queue is full. Otherwise, it is empty. The use of the variable *lastOp* as described above does, however, slow down the *Push* and *Pop* functions. Since the *Push* and *Pop* functions will be used many times in any problem

```

template <class T>
void Queue<T>::Pop()
{ // Delete front element from queue.
    if (IsEmpty()) throw "Queue is empty. Cannot delete.";
    front = (front + 1) % capacity;
    queue [front].~T(); // destructor for T
}

```

Program 3.12: Deleting from a queue

involving queues, the loss of one queue position will be more than made up for by the reduction in computing time. Hence, we favor the implementations of Programs 3.10 and 3.12 over those that result from the use of the variable *LastOp*.

EXERCISES

1. Rewrite functions *Push* and *Pop* (Programs 3.10 and 3.12) using the variable *lastOp* as discussed in this section. The queue should now be able to hold up to *capacity* elements. The complexity of each of your functions should be $\Theta(1)$ (exclusive of the time taken to double queue capacity when needed).
2. To the class *Queue*, add functions to return the size and capacity of a queue.
3. To the class *Queue*, add a function to split a queue into two queues. The first queue is to contain every other element beginning with the first; the second queue contains the remaining elements. The relative order of queue elements is unchanged. What is the complexity of your function?
4. To the class *Queue*, add a function to merge two queues into one by alternately taking elements from each queue. The relative order of queue elements is unchanged. What is the complexity of your function?
5. A double-ended queue (deque) is a linear list in which additions and deletions may be made at either end. Obtain a data representation mapping a deque into a one-dimensional array. Write C++ template functions to add and delete elements from either end of the deque and also to return an element from either end.
6. A linear list is being maintained circularly in an array with *front* and *rear* set up as for circular queues.
 - (a) Obtain a formula in terms of the array capacity, *front*, and *rear*, for

the number of elements in the list.

- (b) Write a C++ template function to delete the k th element in the list.
- (c) Write a C++ template function to insert an element y immediately after the k th element. Use array doubling in case you need to increase the capacity of the element array.
- (d) Develop a complete template class *List* that includes the functions of (b) and (c) as well as a constructor and destructor. Your class also should include functions to return the i th element in the list and the size of the list. An *IsEmpty* function should be included as well. Test all functions.

What is the time complexity of your functions for (b) and (c)?

3.4 SUBTYPING AND INHERITANCE IN C++

Inheritance is used to express *subtype* relationships between ADTs. This is also referred to as the IS-A relationship. We say that a data object of Type B IS-A data object of Type A if B is more specialized than A or A is more general than B ; that is, all B s are A s, but not all A s are B s; or the set of all objects of Type B is a subset of the set of all objects of Type A . For example, *Chair* IS-A *Furniture*, *Lion* IS-A *Mammal*, *Rectangle* IS-A *Polygon*.

Consider the relationship between *Bag* and *Stack*. A bag is simply a data structure into which elements can be inserted and from which elements can be deleted. A stack is also a data structure into which elements can be inserted and from which elements can be deleted. However, a stack is more specialized in that it requires that elements be deleted in last-in first-out order. So, a stack can be used instead of a bag, but a bag cannot be used for an application that requires a stack. So, *Stack* IS-A *Bag* or *Stack* is a subtype of *Bag*. The IS-A relationship is a fundamental, conceptual relationship between the specifications of two ADTs. The relationship is not affected by the implementation of either ADT. So, *Stack* IS-A *Bag* is true, even if their implementations are changed (as long as their ADT specifications are met).

C++ provides a mechanism to express the IS-A relationship called *public inheritance*. Program 3.13 shows how public inheritance is implemented in C++. The declaration `": public Bag"` appended to class name "*Stack*" implies that *Stack* publicly inherits from *Bag*. Here, *Bag*, representing the more general ADT is called the *base class*, while *Stack*, representing the more specialized ADT is called the *derived class*. You will notice that some member functions are preceded by the keyword **virtual**. We will discuss this in the next chapter.

An important consequence of inheritance is that the derived class (*Stack*) inherits

```

class Bag
{
public:
    Bag (int bagCapacity = 10);
    virtual ~Bag();

    virtual int Size() const;
    virtual bool IsEmpty() const;
    virtual int Element() const;

    virtual void Push(const int);
    virtual void Pop();
protected:
    int *array;
    int capacity;
    int top;
};

class Stack : public Bag
{
public:
    Stack (int stackCapacity = 10);
    ~Stack();
    int Top() const;
    void Pop();
};

```

Program 3.13: Definition of *Bag* and *Stack*

all the members (data and functions) of the base class (*Bag*). However, only the non-private members of the base class are accessible to the derived class. This means that any members in *Bag* that are protected or public may be accessed by *Stack*. So, *array*, *capacity*, and *top* may be accessed by *Stack* even though they are not defined in the class definition of *Stack*. Similarly, *Size*, *IsEmpty*, *Element*, and *Push* are accessible to *Stack* even though they are not defined in *Stack*.

Another important consequence of public inheritance is that inherited public and protected members of the base class have the same level of access in the derived class as they did in the base class. So all three data members are protected members of *Stack*. The functions *Size*, *IsEmpty*, *Element*, and *Push* are all public members of *Stack*.

150 Stacks and Queues

The member functions inherited by *Stack* from *Bag* have the same prototypes. This is a reuse of the *interface* of a base class. Notice also that the only operation that has a different implementation in *Bag* and *Stack* is *Pop*. The other operations all have identical implementations. C++ allows us to reuse the base-class implementation of an operation in a derived class. In the event that the implementation of a derived class operation must be different from the base class implementation, it is also possible to override the base class implementation. In our example, only *Pop* is redefined in the derived class. (The constructor and destructor cannot be inherited and therefore must also be redefined in the derived class.) The implementations of all *Bag* operations are assumed to be identical to those in Program 3.5. In Program 3.14, we show the member functions of *Stack* that need to be reimplemented. The implementations of all other functions are inherited from *Bag* and hence do not have to be reimplemented.

```
Stack::Stack(int stackCapacity) : Bag(stackCapacity) { }
// Constructor for Stack calls constructor for Bag.

Stack::~Stack() { }
// Destructor for Bag is automatically called when Stack
// is destroyed. This ensures that array is deleted.

int Stack::Top() const
{
    if (IsEmpty()) throw "Stack is empty.";
    return array[top];
}

void Stack::Pop()
{
    if (IsEmpty()) throw "Stack is empty. Cannot delete.";
    top--;
}
```

Program 3.14: Implementation of *Stack* operations

The following code fragment illustrates how inheritance works:

```
Bag b(3); // uses Bag constructor to create array of capacity 3
Stack s(3); // uses Stack constructor to create array of capacity 3
```

```
b . Push(1); b . Push(2); b . Push(3);
// use Bag::Push.
```

```
s . Push(1); s . Push(2); s . Push(3);
// Stack::Push not defined, so use Bag::Push.
```

```
b . Pop(); // uses Bag::Pop, which calls Bag::IsEmpty
s . Pop();
// uses Stack::Pop, which calls Bag::IsEmpty because IsEmpty
// has not been redefined in Stack.
```

All operations on base class objects (such as *b* in the example) work in exactly the same way they would have, had there been no inheritance. Operations on derived class objects (such as *s* in the example) work differently: if an operation is defined in *Stack*, then that definition is used. If an operation is not defined in *Stack*, then the operation defined in the class that *Stack* inherits from (*Bag* in our example) is used. After executing the above code, *b* contains 1 and 3, while *s* contains 1 and 2.

Note that the following also are permissible:

```
s . Size(); // uses Bag::Size
```

```
s . Element(); // uses Bag::Element
```

If we do not wish to provide access to certain base class functions, then we must redefine these functions in the derived class. So, for example, we may redefine *Element* in *Stack* so as to throw an exception.

The *Queue* ADT is also a subtype of *Bag*. It is more specialized than *Bag* because it requires that elements be deleted in first-in first-out order. Therefore, *Queue* can also be represented as a derived class of *Bag*. However, there is less similarity between the implementations of *Bag* and *Queue* than there is between *Bag* and *Stack*. Like *Bag* and *Stack*, *Queue* uses an array, but unlike *Bag* and *Stack*, *Queue* requires the data members *front* and *rear*. The implementations of the operations *IsEmpty*, *Push*, and *Pop* are different from those of *Bag*. This means that a larger number of operations need to be redefined in *Queue* than were redefined in *Stack*. Even though there is not much reuse of implementation, there is still reuse of interfaces because the redefined functions of *Queue* have the same prototypes as the original functions in *Bag*. The exercises explore a full implementation of *Queue* as a derived class of *Bag*.

EXERCISES

1. Implement *Stack* as a publicly derived class of *Bag* using templates.
2. Implement *Queue* as a publicly derived class of *Bag* using templates.
3. A double-ended queue (deque) is a linear list in which additions and deletions may be made at either end. Implement the class *Deque* as a publicly derived class of *Queue*. The class *Deque* must have public functions (either via inheritance from *Queue* or by direct implementation in *Deque*) to add and delete elements from either end of the deque and also to return an element from either end. The complexity of each function (exclusive of array doubling) should be $\Theta(1)$.
4. For each of the following pairs state whether they are linked by the IS-A relationship. Justify your answer.
 - (a) *Rectangle* and *Trapezium*.
 - (b) *Rectangle* and *Circle*.
 - (c) *Lion* and *Tiger*.
 - (d) *Stack* and *Ordered List*.
 - (e) *Queue* and *Ordered List*.

3.5 A MAZING PROBLEM

The rat in a maze experiment is a classical one from experimental psychology. A rat (or mouse) is placed through the door of a large box without a top. Walls are set up so that movements in most directions are obstructed. The rat is carefully observed by several scientists as it makes its way through the maze until it eventually reaches the exit. There is only one way out, but at the end is a nice chunk of cheese. The idea is to run the experiment repeatedly until the rat will zip through the maze without taking a single false path. The trials yield its learning curve.

We can write a computer program for getting through a maze, and it will probably not be any smarter than the rat on its first try through. It may take many false paths before finding the right one. But the computer can remember the correct path far better than the rat. On its second try it should be able to go right to the end with no false paths taken, so there is no sense re-running the program. Why don't you sit down and try to write this program yourself before you read on and look at our solution. Keep track of how many times you have to go back and correct something. This may give you an idea of your own learning curve as we re-run the experiment throughout the book.

Let us represent the maze by a two-dimensional array, *maze*[*i*] [*j*], where

$1 \leq i \leq m$ and $1 \leq j \leq p$. A value of 1 implies a blocked path, and a 0 means one can walk right on through. We assume that the rat starts at $maze[1][1]$, and the exit is at $maze[m][p]$. An example is given in Figure 3.11.

entrance	0	1	0	0	0	1	1	0	0	0	1	1	1	1	1
	1	0	0	0	1	1	0	1	1	1	0	0	1	1	1
	0	1	1	0	0	0	0	1	1	1	1	0	0	1	1
	1	1	0	1	1	1	1	0	1	1	0	1	1	0	0
	1	1	0	1	0	0	1	0	1	1	1	1	1	1	1
	0	0	1	1	0	1	1	1	0	1	0	0	1	0	1
	0	0	1	1	0	1	1	1	0	1	0	0	1	0	1
	0	1	1	1	1	0	0	1	1	1	1	1	1	1	1
	0	0	1	1	0	1	1	0	1	1	1	1	1	0	1
	1	1	0	0	0	1	1	0	1	1	0	0	0	0	0
	0	0	1	1	1	1	1	0	0	0	1	1	1	1	0
	0	1	0	0	1	1	1	1	1	0	1	1	1	1	0
															exit

Figure 3.11: An example maze (can you find a path?)

With the maze represented as a two-dimensional array, the location of the rat in the maze can at any time be described by the row, i , and the column, j , of its position. Now let us consider the possible moves the rat can make from a point $[i][j]$ in the maze. Figure 3.12 shows the possible moves from any point $[i][j]$. The position $[i][j]$ is marked by an X. If all the surrounding squares have a 0, then the rat can choose any of these eight squares as its next position. We call these eight directions by the names of the points on a compass: north, northeast, east, southeast, south, southwest, west, and northwest, or N, NE, E, SE, S, SW, W, and NW.

We must be careful here because not every position has eight neighbors. If $[i][j]$ is on a border where either $i = 1$ or m , or $j = 1$ or p , then less than eight, and possibly only three, neighbors exist. To avoid checking for border conditions, we surround the maze by a border of ones. The array will therefore be declared as $maze[m+2][p+2]$. Another device that will simplify the problem is to predefine the possible directions to move in a table, *move*, as in Figure 3.13. The data types needed are

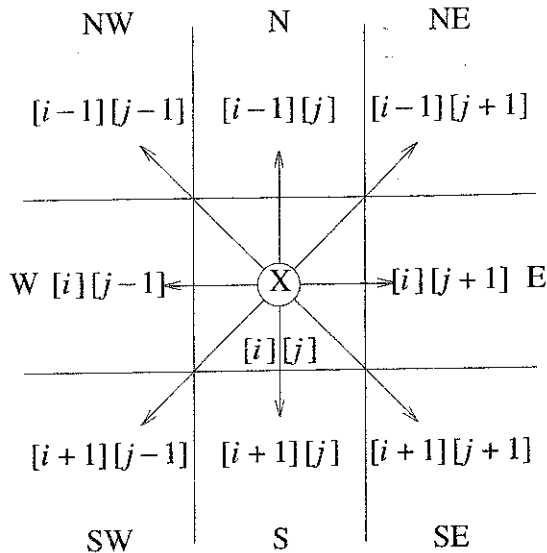


Figure 3.12: Allowable moves

```

struct offsets
{
    int a, b ;
};

enum directions { N, NE, E, SE, S, SW, W, NW };
offsets move[8] ;

```

If we are at position $[i][j]$ in the maze and we wish to find the position $[g][h]$ that is southwest of us, then we set

$$g = i + \text{move}[\text{SW}].a; \quad h = j + \text{move}[\text{SW}].b;$$

For example, if we are at position $[3][4]$, then the position to the southwest is given by $[3 + 1 = 4] [4 + (-1) = 3]$.

As we move through the maze, we may have the chance to go in several directions. Not knowing which one to choose, we pick one but save our current position and the direction of the last move in a list. This way, if we have taken a false path, we can return and try another direction. With each new location we will examine the possibilities, starting from the north and looking clockwise. Finally, in order to prevent us from going down the same path twice, we use

<i>q</i>	<i>move</i> [<i>q</i>]. <i>a</i>	<i>move</i> [<i>q</i>]. <i>b</i>
N	-1	0
NE	-1	1
E	0	1
SE	1	1
S	1	0
SW	1	-1
W	0	-1
NW	-1	-1

Figure 3.13: Table of moves

another array, *mark*[*m* + 2][*p* + 2], which is initially zero. *mark*[*i*][*j*] is set to 1 once we arrive at that position. We assume *maze*[*m*][*p*] = 0, since otherwise there is no path to the exit. Program 3.15 is a first pass at an algorithm.

This is not a C++ program, and yet it describes the essential processing without too much detail. The use of indentation for delineating important blocks of code plus the use of C++ reserved words make the looping and conditional tests transparent.

What remains to be pinned down? Using the three arrays *maze*, *mark*, and *move*, we need only specify how to represent the list of triples (*i*, *j*, *dir*). Since the algorithm calls for removing first the most recently entered triple, this list should be a stack. To avoid doubling array capacity during a stack insertion, we need to choose a sufficiently large initial capacity for the stack. Since each position in the maze is visited at most once, at most *mp* elements can be placed into the stack. Thus, an initial capacity of *mp* is a safe but somewhat conservative value to use for the initial capacity of the stack. The maze of Figure 3.14 has only one path from entrance to exit. It has $\lceil m/2 \rceil (p-2) + 2$ positions. Thus, *mp* is not too crude a bound. We are now ready to give a precise path construction function (Program 3.16).

The arrays *maze*, *mark*, and *move*, are assumed global to *Path*. Further, *stack* is defined to be a stack of *Items* where the struct *Items* is defined as

```
struct Items {
    int x, y, dir;
};
```

The struct *Items* has a constructor (not shown) that sets the values of *x*, *y* and *dir*. The operator << is overloaded for both *Stack* and *Items* as shown in Program

```

initialize list to the maze entrance coordinates and direction east;
while (list is not empty)
{
    (i,j,dir) = coordinates and direction from end of list;
    delete last element of list;
    while (there are more moves from (i,j))
    {
        (g,h) = coordinates of next move;
        if ((g == m) && (h == p)) success;
        if ( (!maze [g][h]) // legal move
            && (!mark [g][h]) // haven't been here before
        {
            mark [g][h] = 1;
            dir = next direction to try;
            add (i,j,dir) to end of list;
            (i,j,dir) = (g,h,N);
        }
    }
}
cout << "No path in maze." << endl;

```

Program 3.15: First pass at finding a path through a maze

3.17. We assume that the operator << has been given access to the private data members of *Stack* through the use of the **friend** declaration.

Analysis of *Path*: Now, what can we say about the computing time of the function *Path*? It is interesting that even though the problem is easy to grasp, it is difficult to make any but the most trivial statement about the computing time. The reason for this is that the number of iterations of the main **while** loop is entirely dependent upon the given maze. What we can say is that each new position [*i*][*j*] that is visited gets marked, so paths are never taken twice. There are at most eight iterations of the inner **while** loop for each marked position. Each iteration of the inner **while** loop takes a fixed amount of time, $O(1)$, and if the number of zeros in *maze* is *z*, then at most *z* positions can get marked. Since *z* is bounded above by *mp*, the computing time is $O(mp)$. (In actual experiments, however, the rat may be inspired by the watching psychologists and the invigorating odor from the cheese at the exit. It might reach its goal by examining far fewer paths than those examined by function *Path*. This may happen despite the fact that the rat has no pencil and only a very limited mental stack. It is

entrance	0	0	0	0	0	0	0	0	1
	1	1	1	1	1	1	1	1	0
	1	0	0	0	0	0	0	0	1
	0	1	1	1	1	1	1	1	1
	1	0	0	0	0	0	0	0	1
	1	1	1	1	1	1	1	1	0
	1	0	0	0	0	0	0	0	1
	0	1	1	1	1	1	1	1	1
	1	0	0	0	0	0	0	0	0
	1	0	0	0	0	0	0	0	0
exit									

Figure 3.14: A maze with a long path

difficult to incorporate the effect of the cheese odor and the cheering of the psychologists into a computer program.) Note that the array *mark* can be eliminated altogether and *maze*[*g*][*h*] changed to 1 instead of setting *mark*[*g*][*h*] to 1, but this will destroy the original maze. □

EXERCISES

- Find a path through the maze of Figure 3.11.
 - Trace out the action of function *Path* (Program 3.16) on the maze of Figure 3.11. Compare this to your own attempt in (a).
- What is the maximum path length from start to finish for any maze of dimensions $m \times p$?
- Write and test recursive version of *Path*. What is the time complexity of your recursive version?

3.6 EVALUATION OF EXPRESSIONS

3.6.1 Expressions

When pioneering computer scientists conceived the idea of higher-level programming languages, they were faced with many hurdles. One of these was how to generate machine-language instructions to evaluate an arithmetic expression. An assignment statement such as

```

void Path(const int m, const int p)
{
    // Output a path (if any) in the maze; maze[0][i] = maze [m + 1][i] =

    // start at (1,1)
    mark[1][1] = 1;
    Stack<Items> stack(m*p);
    Items temp(1, 1, E);
    // set temp.x, temp.y, and temp.dir
    stack.Push(temp);

    while (!stack.IsEmpty())
    {
        // stack not empty
        temp = stack.Top ();
        stack.Pop (); // unstack
        int i = temp.x; int j = temp.y; int d = temp.dir;
        while (d < 8) // move forward
        {
            int g = i + move[d].a; int h = j + move[d].b;
            if ((g == m) && (h == p)) { // reached exit
                // output path
                cout << stack;
                cout << i << " " << j << endl; // last two squares on the path
                cout << m << " " << p << endl;
                return;
            }
            if ((!maze[g][h]) && (!mark[g][h])) { // new position
                mark[g][h] = 1;
                temp.x = i; temp.y = j; temp.dir = d+1;
                stack.Push(temp); // stack it
                i = g; j = h; d = N; // move to (g,h)
            }
            else d++; // try next direction
        }
    }
    cout << "No path in maze." << endl;
}

```

Program 3.16: Finding a path through a maze

```

template <class T>
ostream& operator<<(ostream& os, Stack<T>& s)
{
    os << "top = " << s.top << endl;
    for (int i = 0 ; i <= s.top; i++)
        os << i << ":" << s.stack[i] << endl;
    return os;
}

ostream& operator<<(ostream& os, Items& item)
{
    return os << item.x << "," << item.y << "," << item.dir;
}

```

Program 3.17: Overloading <<

$$X = A/B - C + D * E - A * C$$

might have several meanings, and even if it were uniquely defined, say by a full use of parentheses, it still seemed a formidable task to generate a correct instruction sequence. Fortunately the solution we have today is both elegant and simple. Moreover, it is so simple that this aspect of compiler writing is really one of the more minor issues.

An expression is made up of operands, operators, and delimiters. The expression above has five operands: *A*, *B*, *C*, *D*, and *E*. Though these are all one-letter variables, operands can be any legal variable name or constant in our programming language. In any expression, the values that variables take must be consistent with the operations performed on them. These operations are described by the operators. In most programming languages there are several kinds of operators that correspond to the different kinds of data a variable can hold. First, there are the basic arithmetic operators: plus, minus, times, and divide (+, −, *, /). Other arithmetic operators include unary minus, and %. A second class is the relational operators: <, <=, ==, <>, >=, and >. These are usually defined to work for arithmetic operands, but they can just as easily work for character string data. ('CAT' is less than 'DOG' since it precedes 'DOG' in alphabetical order.) The result of an expression that contains relational operators is one of two constants: **true** or **false**. Such an expression is called Boolean, named after the mathematician George Boole, the father of symbolic logic. There also may be logical operators such as &&, ||, and !.

The first problem with understanding the meaning of an expression is to decide in what order the operations are carried out. This means that every

160 Stacks and Queues

language must uniquely define such an order. For instance, if $A = 4$, $B = C = 2$, $D = E = 3$, then in the above equation we might want X to be assigned the value

$$\begin{aligned} & ((4/2) - 2) + (3 * 3) - (4 * 2) \\ &= 0 + 9 - 8 \\ &= 1 \end{aligned}$$

However, the true intention of the programmer might have been to assign X the value

$$\begin{aligned} & (4/(2 - 2 + 3)) * (3 - 4) * 2 \\ &= (4/3) * (-1) * 2 \\ &= -2.66666666 \end{aligned}$$

Of course, the programmer could specify the latter order of evaluation by using parentheses:

$$X = ((A/(B - C + D)) * (E - A) * C$$

To fix the order of evaluation, we assign to each operator a priority. Then within any pair of parentheses we understand that operators with the highest priority will be evaluated first. A set of sample priorities from C++ is given in Figure 3.15. The highest priority is 1.

priority	operator
1	unary minus, !
2	*, /, %
3	+, -
4	<, <=, >=, >
5	==, !=
6	&&
7	

Figure 3.15: Priority of operators in C++

Unary minus and the logical not (!) have top priority, followed by *, /, and %. When we have an expression where two adjacent operators have the same priority, we need a rule to tell us which one to perform first. For example, do we want the value of $A/B*C$ to be understood as $(A/B) * C$ or $A/(B*C)$? Convince yourself that there will be a difference by trying $A = B = C = 2$. The C++ rule is that for all priorities, evaluation of operators of the same priority will proceed left to right. So, $A/B*C$ will be evaluated as $(A/B) * C$. Remember that by

using parentheses we can override these rules, as expressions are always evaluated with the innermost parenthesized expression first.

Now that we have specified priorities and rules for breaking ties we know how $X = A/B - C + D * E - A * C$ will be evaluated, namely, as

$$X = (((A/B) - C) + (D * E)) - (A * C)$$

3.6.2 Postfix Notation

How can a compiler accept an expression and produce correct code? The answer is given by reworking the expression into a form we call *postfix notation*. If e is an expression with operators and operands, the conventional way of writing e is called *infix*, because the operators come *in-between* the operands. (Unary operators precede their operand.) The *postfix* form of an expression calls for each operator to appear *after* its operands. For example,

infix $A * B / C$ has postfix $AB * C /$

If we study the postfix form of $A * B / C$, we see that the multiplication comes immediately after its two operands A and B . Now imagine that $A * B$ is computed and stored in T . Then we have the division operator, $/$, coming immediately after its two operands T and C .

Let us look at our previous example

infix: $A/B - C + D * E - A * C$
postfix: $AB / C - DE * + AC * -$

and trace out the meaning of the postfix expression.

Suppose that every time we compute a value, we store it in the temporary location T_i , $i \geq 1$. If we read the postfix expression left to right, the first operation is division. The two operands that precede this are A and B . So, the result of A/B is stored in T_1 , and the postfix expression is modified as in Figure 3.16. This figure also gives the remaining sequence of operations. The result is stored in T_6 . Notice that if we had parenthesized the expression, this would change the postfix only if the order of normal evaluation were altered. Thus, $(A/B) - C + (D * E) - A * C$ will have the same postfix form as the previous expression without parentheses. But $(A/B) - (C + D) * (E - A) * C$ will have the postfix form $AB / CD + EA - * C * -$.

What are the virtues of postfix notation that enable easy evaluation of expressions? To begin with, the need for parentheses is eliminated. Second, the priority of the operators is no longer relevant. The expression may be evaluated by making a left to right scan, stacking operands, and evaluating operators using

162 Stacks and Queues

operation	postfix
$T_1 = A/B$	$T_1 C - DE * + AC * -$
$T_2 = T_1 - C$	$T_2 DE * + AC * -$
$T_3 = D * E$	$T_2 T_3 + AC * -$
$T_4 = T_2 + T_3$	$T_4 AC * -$
$T_5 = A * C$	$T_4 T_5 -$
$T_6 = T_4 - T_5$	T_6

Figure 3.16: Postfix evaluation

as operands the correct number from the stack and finally placing the result onto the stack (see Program 3.18). This evaluation process is much simpler than attempting direct evaluation from infix notation.

```
void Eval(Expression e)
{
    // Evaluate the postfix expression e. It is assumed that the last token (a token
    // is either an operator, operand, or '#') in e is '#'. A function NextToken is
    // used to get the next token from e. The function uses the stack stack
    Stack<Token> stack; // initialize stack
    for (Token x = NextToken(e); x != '#'; x = NextToken(e))
        if (x is an operand) stack.Push(x) // add to stack
        else { // operator
            remove the correct number of operands for operator x from stack;
            perform the operation x and store the result (if any) onto the stack;
        }
}
```

Program 3.18: Evaluating postfix expressions

3.6.3 Infix to Postfix

To see how to devise an algorithm for translating from infix to postfix, note that the order of the operands in both forms is the same. In fact, it is simple to describe an algorithm for producing postfix from infix:

- (1) Fully parenthesize the expression.
-

- (2) Move all operators so that they replace their corresponding right parentheses.
- (3) Delete all parentheses.

For example, $A/B - C + D * E - A * C$, when fully parenthesized, yields

$$(((A/B) - C) + (D * E)) - (A * C)$$

The arcs join an operator and its corresponding right parenthesis. Steps 2 and 3 yield

$$AB/C - DE * + AC * -$$

Since the order of the operands is the same in infix and postfix, when we scan an expression for the first time, we can form the postfix by immediately passing any operands to the output. To handle the operators, we store them in a stack until it is time to pass them to the output.

For example, since we want $A + B * C$ to yield $ABC * +$, our algorithm should perform the following sequence of stacking (these stacks will grow to the right):

next token	stack	output
none	empty	none
A	empty	A
+	+	A
B	+	AB

At this point the algorithm must determine if $*$ gets placed on top of the stack or if the $+$ gets taken off. Since $*$ has higher priority, we should stack $*$, producing

*	++	AB
C	++	ABC

Now the input expression is exhausted, so we output all remaining operators in the stack to get

$$ABC * +$$

For another example, $A * (B + C) * D$ has the postfix form $ABC + * D *$, and so the

164 Stacks and Queues

algorithm should behave as

next token	stack	output
none	empty	none
A	empty	A
*	*	A
(	*(	A
B	*(	AB
+	*(+	AB
C	*(+	ABC

At this point we want to unstack down to the corresponding left parenthesis and then delete the left and right parentheses. This gives us

)	*	ABC +
*	*	ABC ++
D	*	ABC ++ D
done	empty	ABC ++ D *

These examples motivate a priority-based scheme for stacking and unstacking operators. The left parenthesis complicates things, since when it is not in the stack, it behaves as an operator with high priority, whereas once it gets in, it behaves as one with low priority (no operator other than the matching right parenthesis should cause it to get unstacked). We establish two priorities for operators: *isp* (in-stack priority) and *icp* (in-coming priority). The *isp* and *icp* of all operators in Figure 3.15 remain unchanged. In addition, we assume that *isp*('(') returns 8, *icp*('(') returns 0, and *isp*(')') returns 8. These priorities result in the following rule: *Operators are taken out of the stack as long as their in-stack priority is numerically less than or equal to the in-coming priority of the new operator.* Our function to transform from infix to postfix is given in Program 3.19.

Analysis of Postfix: The function makes only a left-to-right pass across the input. The time spent on each operand is $O(1)$. Each operator is stacked and unstacked at most once. Hence, the time spent on each operator is also $O(1)$. So, the complexity of *Postfix* is $\Theta(n)$, where n is the number of tokens in the expression. $\square$

```

void Postfix (Expression e)
{
    // Output the postfix form of the infix expression e. NextToken
    // is as in function Eval (Program 3.18). It is assumed that
    // the last token in e is '#' Also, '#' is used at the bottom of the stack
    Stack<Token> stack; // initialize stack
    stack.Push('#');
    for (Token x = NextToken(e); x != '#'; x = NextToken(e))
        if (x is an operand) cout << x;
        else if (x == '(')
            { // unstack until '('
                for (; stack.Top() != '('; stack.Pop())
                    cout << stack.Top();
                stack.Pop(); // unstack '('
            }
        else { // x is an operator
            for (; isp(stack.Top()) <= icp(x); stack.Pop())
                cout << stack.Top();
            stack.Push(x);
        }
    }

    // end of expression; empty the stack
    for (; !stack.IsEmpty(); cout << stack.Top(), stack.Pop();)
        cout << endl;
}

```

Program 3.19: Converting from infix to postfix form

EXERCISES

- Write the postfix form of the following expressions:
 - $A * B * C$
 - $-A + B - C + D$
 - $A * -B + C$
 - $(A + B) * D + E / (F + A * D) + C$
 - $A \&\& B \parallel C \parallel ! (E > F)$ (assuming C++ precedence)
 - $!(A \&\& !((B < C) \parallel (C > D))) \parallel (C < E)$

166 Stacks and Queues

2. Use the priorities of Figure 3.15 together with those for '(' and '#' to answer the following:
 - (a) In function *Postfix* (Program 3.19), what is the maximum number of elements that can be on the stack at any time if the input expression e has n operators and delimiters?
 - (b) What is the answer to (a) if e has n operators and the depth of nesting of parentheses is at most 6?
3. Another expression form that is easy to evaluate and is parenthesis-free is known as *prefix*. In this way of writing expressions, the operators precede their operands. For example:

infix	prefix
$A * B / C$	$/*ABC$
$A / B - C + D * E - A * C$	$- + - / ABC * DE * AC$
$A * (B + C) / D - G$	$-/* A + BCDG$

Notice that the order of operands is not changed in going from infix to prefix.

- (a) What is the prefix form of the expressions in Exercise 1?
- (b) Write a C++ function to evaluate a prefix expression e . (Hint: Scan e right to left and assume that the leftmost token of e is '#'.)
- (c) Write a C++ function to transform an infix expression e into its prefix equivalent. Assume that the input expression e begins with a '#' and that the prefix expression should begin with a '#'.

What is the time complexity of your functions for (b) and (c)? How much space is needed by each of these functions?

4. Write a C++ function to transform from prefix to postfix. Carefully state any assumptions you make regarding the input. How much time and space does your function take?
5. Do the preceding exercise, but this time for a transformation from postfix to prefix.
6. Write a C++ function to generate fully parenthesized infix expressions from their postfix form. What is the complexity (time and space) of your function?
7. Do the preceding exercise starting from prefix form.

3.7 ADDITIONAL EXERCISES

1. [*Programming Project*] [Landweber] People have spent so much time playing card games of solitaire that the gambling casinos are now capitalizing on this human weakness. A form of solitaire is described below. Your assignment is to write a computer program to play the game, thus freeing hours of time for people to return to more useful endeavors.

To begin the game, 28 cards are dealt into seven piles. The leftmost pile has one card, the next two cards, and so forth, up to seven cards in the rightmost pile. Only the uppermost card of each of the seven piles is turned face up. The cards are dealt left to right, one card to each pile, dealing to one less pile each time, and turning the first card in each round face up. On the topmost face-up card of each pile you may build in descending sequences red on black or black on red. For example, on the 9 of spades you may place either the 8 of diamonds or the 8 of hearts. All face-up cards on a pile are moved as a unit and may be placed on another pile according to the bottommost face-up card. For example, the 7 of clubs on the 8 of hearts may be moved as a unit onto the 9 of clubs or the 9 of spades.

Whenever a face-down card is uncovered, it is turned face up. If one pile is removed completely, a face-up king may be moved from a pile (together with all cards above it) or the top of the waste pile (see below) into the vacated space. There are four output piles, one for each suit, and the object of the game is to get as many cards as possible into the output piles. Each time an ace appears at the top of a pile or the top of the stack, it is moved into the appropriate output pile. Cards are added to the output piles in sequence, the suit for each pile being determined by the ace on the bottom.

From the rest of the deck, called the stock, cards are turned up one by one and placed face up on a waste pile. You may always play cards off the top of the waste pile, but only one at a time. Begin by moving a card from the stock to the top of the waste pile. If there is ever more than one possible play to be made, the following order must be observed:

- (a) Move a card from the top of a playing pile or from the top of the waste pile to an output pile. If the waste pile becomes empty, move a card from the stock to the waste pile.
- (b) Move a card from the top of the waste pile to the leftmost playing pile to which it can be moved. If the waste pile becomes empty, move a card from the stock to the waste pile.
- (c) Find the leftmost playing pile that can be moved and place it on top

of the leftmost playing pile to which it can be moved.

- (d) Try (a), (b), and (c) in sequence, restarting with (a) whenever a move is made.
- (e) If no move is made via (a) through (d), move a card from the stock to the waste pile and retry (a).

Only the topmost card of the playing piles or the waste pile may be played to an output pile. Once played on an output pile, a card may not be withdrawn to help elsewhere. The game is over when either all the cards have been played to the output, or the stock pile has been exhausted and no more cards can be moved.

When played for money, the player pays the house \$52 at the beginning and wins \$5 for every card played to the output piles. Write your program so that it will play several games and determine your net winnings. Use a random number generator to shuffle the deck. Output a complete record of two games in easily understood form. Include as output the number of games played and the net winnings (+ or -).

2. [**Programming Project**] [Landweber] We want to simulate an airport landing and takeoff pattern. The airport has three runways, runway 1, runway 2, and runway 3. There are four landing holding patterns, two for each of the first two runways. Arriving planes will enter one of the holding pattern queues, where the queues are to be as close in size as possible. When a plane enters a holding queue, it is assigned an integer *id* number and an integer giving the number of time units the plane can remain in the queue before it must land (because of low fuel level). There is also a queue for takeoffs for each of the three runways. Planes arriving in a takeoff queue are also assigned an integer *id*. The takeoff queues should be kept approximately the same size.

At each time, up to three planes may arrive at the landing queues and up to three planes may arrive at the takeoff queues. Each runway can handle one takeoff or landing at each time slot. Runway 3 is to be used for takeoffs except when a plane is low on fuel. At each time unit, planes in either landing queue whose air time has reached zero must be given priority over other landings and takeoffs. If only one plane is in this category, runway 3 is to be used. If more than one, then the other runways are also used (at each time, at most three planes can be serviced in this way).

Use successive even (odd) integers for *id*'s of planes arriving at takeoff (landing) queues. At each time unit assume that arriving planes are entered into queues before takeoffs or landings occur. Try to design your algorithm so that neither landing nor takeoff queues grow excessively. However, arriving planes must be placed at the ends of queues. Queues cannot be

reordered.

The output should clearly indicate what occurs at each time unit. Periodically output (a) the contents of each queue; (b) the average takeoff waiting time; (c) the average landing waiting time; (d) the average flying time remaining on landing; and (e) the number of planes landing with no fuel reserve. (b) and (c) are for planes that have taken off or landed, respectively. The output should be self-explanatory and easy to understand (and uncluttered).

The input can be from a terminal, a file, or it can be generated by a random number generator. For each time unit the input consists of the number of planes arriving at take-off queues, the number of planes arriving at landing queues, and the remaining flying times for each plane arriving at a landing queue.

CHAPTER 4

Linked Lists

4.1 SINGLY LINKED LISTS AND CHAINS

In the previous chapters, we studied the representation of simple data structures using an array and a sequential mapping. These representations had the property that successive nodes of the data object were stored a fixed distance apart. Thus, (1) if the element a_{ij} of a table was stored at location L_{ij} , then $a_{i,j+1}$ was at the location $L_{ij} + 1$; (2) if the i th element in a queue was at location L_i , then the $(i + 1)$ th element was at location $(L_i + 1) \% n$ for the circular representation; (3) if the topmost node of a stack was at location L_T , then the node beneath it was at location $L_T - 1$, and so on. These sequential storage schemes proved adequate for the tasks we wished to perform (accessing an arbitrary node in a table, insertion or deletion of stack and queue elements). However, when a sequential mapping is used for ordered lists, operations such as insertion and deletion of arbitrary elements become expensive. For example, consider the following list of three-letter English words ending in AT:

(BAT, CAT, EAT, FAT, HAT, JAT, LAT, MAT, OAT, PAT, RAT, SAT, VAT, WAT)

To make this list more complete we naturally want to add the word GAT, which means gun or revolver. If we are using an array and a sequential mapping to keep this list, then the insertion of GAT will require us to move elements already in the list either one location higher or lower. We must move either HAT, JAT, LAT, ..., WAT or BAT, CAT, EAT, and FAT. If we have to do many such insertions into the middle, neither alternative is attractive because of the amount of data movement. Excessive data movement also is required for deletions. Suppose we decide to remove the word LAT, which refers to the Latvian monetary unit. Then again, we have to move many elements so as to maintain the sequential representation of the list.

An elegant solution to this problem of data movement in *sequential* representations is achieved by using *linked* representations. Unlike a sequential representation, in which successive items of a list are located a fixed distance apart, in a linked representation these items may be placed anywhere in memory. In other words, in a sequential representation the order of elements is the same as in the ordered list, whereas in a linked representation these two sequences need not be the same. To access list elements in the correct order, with each element we store the address or location of the next element in that list. Thus, associated with each data item in a linked representation is a pointer or link to the next item. In general, a linked list is comprised of nodes; each node has zero or more data fields and one or more link or pointer fields.

Figure 4.1 shows how some of the elements in our list of three-letter words may be represented in memory by using pointers. The elements of the list are stored in a one-dimensional array called *data*, but the elements no longer occur in sequential order, BAT before CAT before EAT, and so on. Instead we relax this restriction and allow them to appear anywhere in the array and in any order. To remind us of the real order, a second array, *link*, is added. The values in this array are pointers to elements in the *data* array. For any i , $data[i]$ and $link[i]$ together comprise a node. Since the list starts at $data[8] = \text{BAT}$, let us set a variable $first = 8$. $link[8]$ has the value 3, which means it points to $data[3]$, which contains CAT. Since $link[3] = 4$, the next element, EAT, in the list is in $data[4]$. The element after EAT is in $data[link[4]]$. By continuing in this way we can list all the words in the proper order. We recognize that we have come to the end of our ordered list when *link* equals zero. To ensure that a *link* of zero always signifies the end of a list, we do not use position zero of *data* to store a list element.

It is customary to draw linked lists as an ordered sequence of nodes with links being represented by arrows, as in Figure 4.2. Notice that we do not explicitly put in the values of the pointers but simply draw arrows to indicate they are there. The arrows reinforce in our own mind the facts that (1) the nodes do not actually reside in sequential locations and (2) the actual locations of nodes are immaterial. Therefore, when we write a program that works with lists, we do not

	<i>data</i>	<i>link</i>
1	HAT	15
2		
3	CAT	4
4	EAT	9
5		
6		
7	WAT	0
8	BAT	3
9	FAT	1
10		
11	VAT	7
	.	.
	.	.
	.	.

Figure 4.1: Nonsequential list representation

look for a specific address except when we test for zero. The linked structures of Figures 4.1 and 4.2 are called singly linked lists or chains. In a *singly linked list*, each node has exactly one pointer field. A *chain* is a singly linked list that is comprised of zero or more nodes. When the number of nodes is zero, the chain is empty. The nodes of a non-empty chain are ordered so that the first node links to the second node; the second to the third; and so on. The last node of a chain has a 0 link.

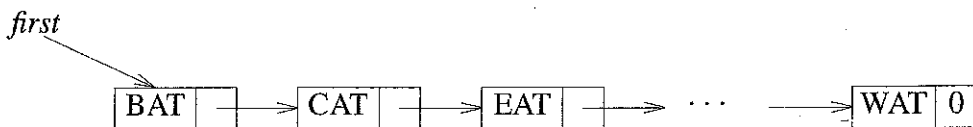


Figure 4.2: Usual way to draw a linked list

Let us now see why it is easier to make insertions and deletions at arbitrary

positions using a linked list rather than a sequential list. To insert the data item GAT between FAT and HAT, the following steps are adequate:

- (1) Get a node *a* that is currently unused.
- (2) Set the *data* field of *a* to GAT.
- (3) Set the *link* field of *a* to point to the node after FAT, which contains HAT.
- (4) Set the *link* field of the node containing FAT to *a*.

Figure 4.3(a) shows how the arrays *data* and *link* will be changed after we insert GAT. Figure 4.3(b) shows how we can draw the insertion using our arrow notation. Dashed arrows are new ones. The important thing to notice is that when we insert GAT, we do not have to move any elements that are already in the list. We have overcome the need to move data at the expense of the storage needed for the field *link*. Usually, this penalty is not too severe. When each list element is large, significant time is saved by not having to move elements during an insert or delete.

Now suppose we want to delete GAT from the list. All we need to do is find the element that immediately precedes GAT, which is FAT, and set *link*[9] to the position of HAT which is 1. Again, there is no need to move the data around. Even though the link of GAT still contains a pointer to HAT, GAT is no longer in the list as it cannot be reached by starting at the first element of list and following links (see Figure 4.4).

4.2 REPRESENTING CHAINS IN C++

In the following subsections, we shall see the basic techniques used to implement chains in C++. In the next section, we generalize the representation using templates.

4.2.1 Defining A Node in C++

To define the structure of a node, we need to know the type of each of its fields. The *data* field in the example of the previous section is simply a three-character string, and the *link* field is a pointer to another node.

If the type of the node in our earlier example is denoted by *ThreeLetterNode*, then the data type *ThreeLetterNode* may be defined as a class as follows:

```
class ThreeLetterNode {
private:
    char data[3];
    ThreeLetterNode *link;
};
```

Example 4.1 defines two node structures that can be combined with each

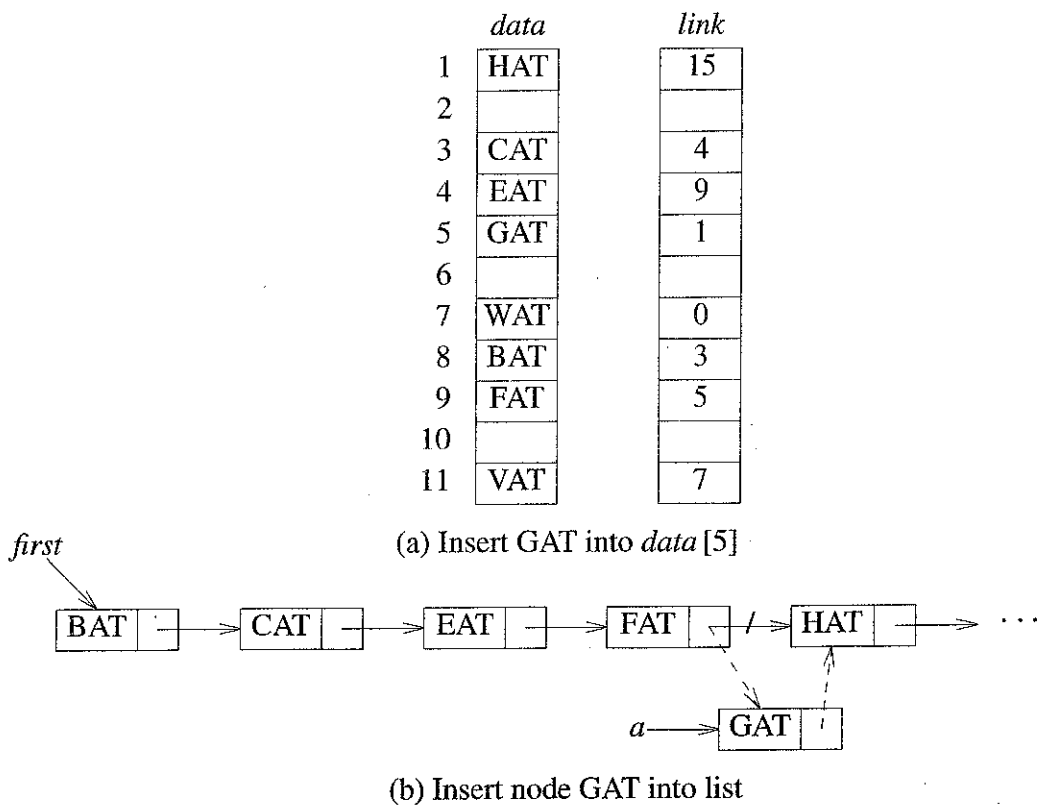


Figure 4.3: Inserting into a linked list

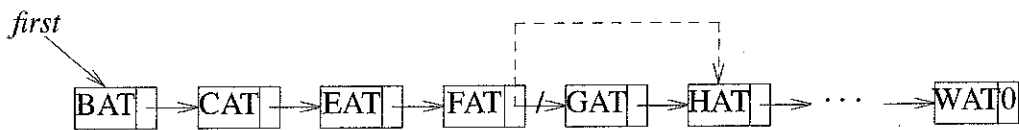


Figure 4.4: Delete GAT

other to form a complex list structure.

Example 4.1: The class definitions

```
class NodeA {
private:
    int data1;
    char data2;
    float data3;
    NodeA *linka;
    NodeB *linkb;
};

class NodeB {
private:
    int data;
    NodeB *link;
};
```

define *NodeA* to consist of three data fields and two link fields, whereas nodes of type *NodeB* consist of one data field and one link field. Note that the *linkb* data member of nodes of type *NodeA* must point to nodes of type *NodeB*. Figure 4.5 shows a linked list that has one node of each type. □

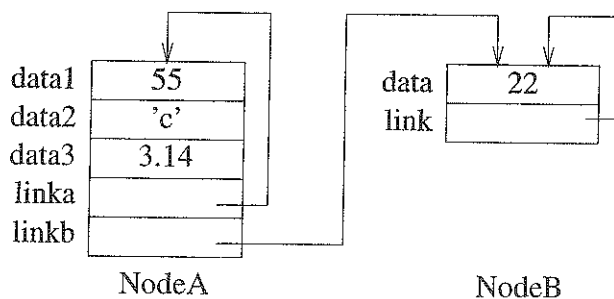


Figure 4.5: Illustration of the node structures *NodeA* and *NodeB*

4.2.2 Designing a Chain Class in C++

We have seen how to represent a single node in C++. Now, we shall design a C++ representation for a chain of nodes. The basic elements of our design are especially important because they apply not just to chains, but also to other linked data structures that we shall study later. As is typical with any design process, we shall explore different alternatives, some of which will turn out to be infeasible or undesirable.

Design Attempt 1: The variable *first*, in the *ThreeLetterNode* example, is declared as

```
ThreeLetterNode *first;
```

The data members of the node pointed to by *first* may be referenced in the following way:

```
first→data, first→link
```

and the components of *data* are referenced as

```
first→data [0], first→data [1], first→data [2]
```

This is shown diagrammatically in Figure 4.6.

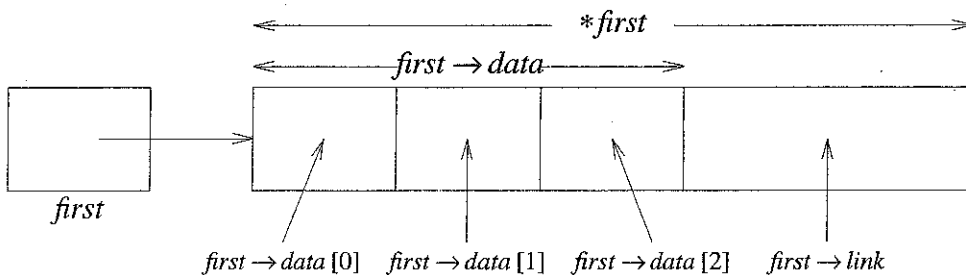


Figure 4.6: Referencing the data members of a node

There is, however, a flaw in our implementation of *ThreeLetterNode*. Notice that the data members *data* and *link* are declared as **private**. This is in keeping with the data encapsulation principles discussed in Chapter 1. The consequence of declaring the data members *data* and *link* as **private** is that the expressions *first*→*link*, *first*→*data* [0], *first*→*data* [1], and *first*→*data* [2] will all result in compile-time errors because private data members cannot be accessed

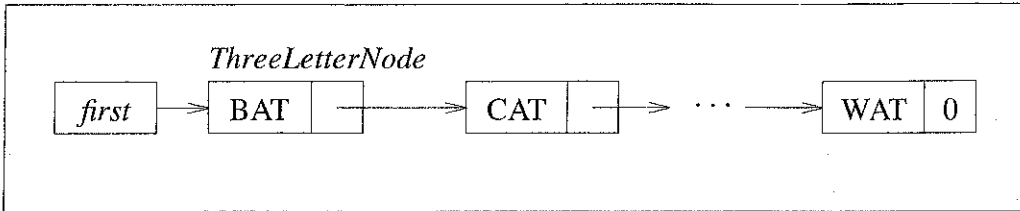
from outside the class!

Design Attempt 2: How do we overcome the drawback of our first attempt? Making the data members public would do the trick, but that violates data encapsulation. A solution might be to define public member functions *GetLink()*, *SetLink()*, *GetData()*, and *SetData()* on *ThreeLetterNode* which are used to indirectly access the data members. This is not a good solution, however, because this solution allows one to read and change these data members from anywhere in the program. The ideal solution would only permit those functions that perform list manipulation operations (like inserting a node into or deleting a node from a chain) access to the data members of *ThreeLetterNode*.

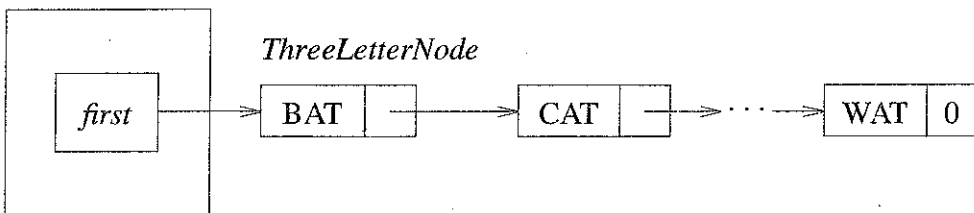
Design Attempt 3: Let us try to tackle this data structure design problem from a different perspective: the data structure we are interested in implementing is a chain of three-letter words. This suggests that our program should contain a class corresponding to the entire chain. Let us call this class *ThreeLetterChain*. This class should contain member functions that carry out list manipulation operations such as insert and delete. Thus, we will be using a composite of two classes, *ThreeLetterNode* and *ThreeLetterChain*, to implement our chain. The conceptual relationship between the two classes may be characterized by the following statement: an object of the class *ThreeLetterChain* consists of zero or more objects of the class *ThreeLetterNode*; or, *ThreeLetterChain* HAS-A *ThreeLetterNode*.

Definition: We say that a data object of type *A* HAS-A data object of type *B* if *A* conceptually contains *B* or *B* is a part of *A*. For example, *Computer* HAS-A *Processor*, *Book* HAS-A *Page*, and so on. HAS-A relationships between ADTs are usually expressed by making the contained class a member of the containing class. This is possible when the type *A* object contains a fixed number of type *B* objects. □

Figure 4.7 shows the *conceptual* relationship between the two classes. Figure 4.7 suggests that an object of class *ThreeLetterChain* *physically* contains many objects of class *ThreeLetterNode*; that is objects of *ThreeLetterNode* are declared as data members of *ThreeLetterChain*. But, the number of nodes in a chain is not a constant; on the contrary, it varies with each insert or delete operation performed on the list. So, it is impossible to know in advance the number of *ThreeLetterNodes* to include in *ThreeLetterChain*. For this reason, the *ThreeLetterChain* class is defined so that it only contains the *access pointer*, *first*, which points to the first node in the list. Figure 4.8 shows the *actual* relationship between *ThreeLetterChain* and *ThreeLetterNode* objects. It shows that *ThreeLetterNode* objects are not physically contained inside *ThreeLetterChain*. The only

ThreeLetterChain**Figure 4.7:** Conceptual relationship between *ThreeLetterChain* and *ThreeLetterNode*

data member contained in *ThreeLetterChain* is the pointer *first*.

ThreeLetterChain**Figure 4.8:** Actual relationship between *ThreeLetterChain* and *ThreeLetterNode*

We are now finally ready to propose a solution to our dilemma of how to define the class *ThreeLetterNode* so that only list manipulation operations have access to the data members of its nodes. This is achieved by declaring *ThreeLetterChain* to be a **friend** of *ThreeLetterNode*. Only member functions of *ThreeLetterChain* and *ThreeLetterNode* can now access the private data members of *ThreeLetterNode*. The class definitions of *ThreeLetterChain* and *ThreeLetterNode* are shown in Program 4.1.

Nested Classes: An alternative method to represent a chain in C++ is to use nested classes, where one class is defined inside the definition of another class. Here, class *ThreeLetterNode* is defined inside the private portion of the definition of class *ThreeLetterChain*, as shown in Program 4.2. This ensures that

```
class ThreeLetterChain; // forward declaration
```

```
class ThreeLetterNode {
friend class ThreeLetterChain;
private:
    char data[3];
    ThreeLetterNode *link;
};
```

```
class ThreeLetterChain {
public:
    // Chain manipulation operations
    .
    .
private:
    ThreeLetterNode *first;
};
```

Program 4.1: Composite classes

ThreeLetterNode objects cannot be accessed outside class *ThreeLetterChain*. Notice that the data members of *ThreeLetterNode* are public. This ensures that they can be accessed by member functions of *ThreeLetterChain*. Using nested classes achieves the same effect as our previous approach, which uses composite classes. We use the composite class approach in the rest of the text. One of our reasons for preferring composite classes over nested classes is that a single node class can be used by many different classes (so long as each is a **friend** of the node class).

4.2.3 Pointer Manipulation in C++

Nodes of a predefined type may be created using the C++ command **new**. If *f* is of type *ThreeLetterNode** then following the call *f* = **new** *ThreeLetterNode*, **f* denotes the node of type *ThreeLetterNode* that is created. Similarly, if *a* and *b* are of type *NodeA** and *NodeB**, respectively, (Example 4.1) then following the execution of the statements

```

class ThreeLetterChain {
public:
    // Chain Manipulation operations

private:
    class ThreeLetterNode { // nested class
    public:
        char data[3];
        ThreeLetterNode *link;
    };
    ThreeLetterNode *first;
};

```

Program 4.2: Nested classes

```

a = new NodeA;
b = new NodeB;

```

**a*, and **b*, respectively, denote the nodes of type *NodeA* and *NodeB* that are created. These nodes may be deleted in the following way:

```

delete f; delete a; delete b;

```

C++ also allows pointer variables to be assigned the null pointer constant 0 (or **NULL**). This constant is used to denote a pointer that points to no node (e.g., the *link* data member in the last node of Figure 4.3(b)) or an empty list (as in *first* = 0). Addition of integers to pointer variables is permitted in C++ (but is generally only used in the context of arrays). Thus, if *x* is a pointer variable, the expression *x* + 1 is valid, but may have no logical meaning. Two pointer variables of the same type may also be compared to see if both point to the same node. Thus, if *x* and *y* are pointer variables of the same type, then the expressions *x* == *y*, *x* != *y*, *x* == 0, *x* != 0 are all valid. The effect of the assignments *x* = *y* and **x* = **y* on the initial configuration of Figure 4.9(a) is given in Figure 4.9(b) and (c). When a pointer is output in C++ using **cout**, the location in memory that the pointer is addressing is output.

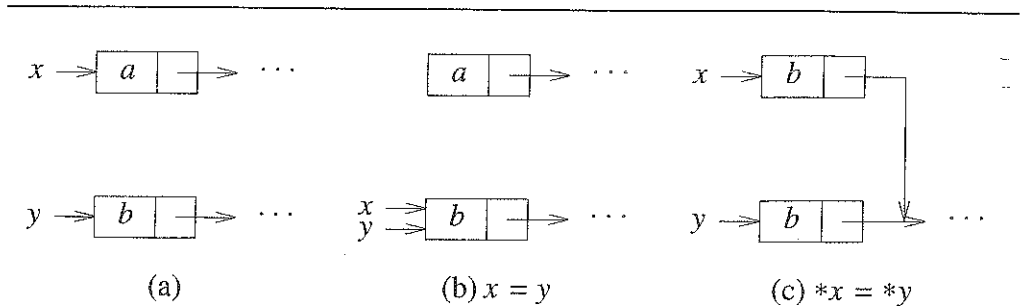


Figure 4.9: Effect of pointer assignments

4.2.4 Chain Manipulation Operations

In this subsection, we look at three functions that manipulate chains. These functions use the classes *ChainNode* and *Chain*, where *ChainNode* is defined as:

```
class ChainNode {
friend class Chain;
public:
    ChainNode(int element = 0, ChainNode* next = 0)
        // 0 is the default value for element and next
        {data = element; link = next;}
private:
    int data;
    ChainNode *link;
};
```

The access pointer *first*, which points to the first node in the chain, is a private data member of *Chain*. It is assumed that the three functions we are about to develop are declared in the **public** segment of the class definition of *Chain*.

Example 4.2: Function *Chain::Create2* (Program 4.3) creates a chain with two nodes. The *data* field of the first node is set to 10 and that of the second to 20. Function *Create2* uses the constructor for *ChainNode* to initialize the fields of the two newly created nodes. The resulting list structure is shown in Figure 4.10. □

Example 4.3: Let *first* be a pointer to the first node of a chain. *first* == 0 iff the chain is empty (i.e., there are no nodes on the chain). Let *x* be a pointer to some arbitrary node in the chain. Program 4.4 inserts a node with *data* field 50

```

void Chain::Create2()
{
    // create and set fields of second node
    ChainNode* second = new ChainNode (20,0);

    // create and set fields of first node
    first = new ChainNode (10, second);
}

```

Program 4.3: Creating a two-node list



Figure 4.10: A two-node list

following the node pointed to by x except when the chain is empty. Once again, the constructor for *ChainNode* is used to initialize the fields of the new node. The resulting chain structures for the two cases $first == 0$ and $first != 0$ are shown in Figure 4.11. □

```

void Chain::Insert50(ChainNode *x)
{
    if (first)
        // insert after x
        x->link = new ChainNode (50, x->link);
    else
        // insert into empty list
        first = new ChainNode (50);
}

```

Program 4.4: Inserting a node

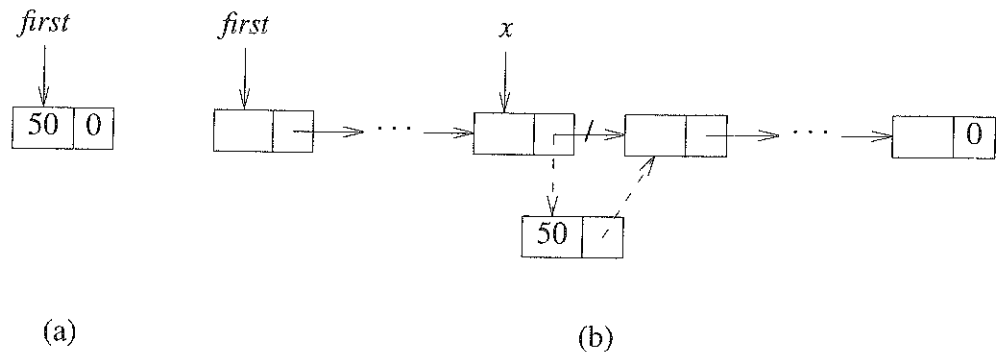


Figure 4.11: Inserting into an empty and nonempty list

Example 4.4: Let *first* and *x* be as in Example 4.3. Let *y* point to the node (if any) that precedes *x*, and let *y* == 0 iff *x* == *first*. Function *Chain::Delete* (Program 4.5) deletes node *x* from the chain. □

```
void Chain::Delete(ChainNode *x, ChainNode *y)
{
    if (x == first) first = first->link;
    else y->link = x->link;
    delete x;
}
```

Program 4.5: Deleting a node

EXERCISES

The following exercises are based on the definitions of Section 4.2.4. All functions are to be implemented as member functions of *Chain* and are therefore assumed to have access to the data members of *ChainNode*.

1. Write a C++ function *length* to count the number of nodes in a chain. What is the time complexity of your function?
2. Let *x* be a pointer to an arbitrary node in a chain. Write a C++ function to delete this node from the chain. If *x* == *first*, then *first* should be reset to point to the new first node in the chain. What is the time complexity of

your function?

3. Write a C++ function to delete every other node beginning with node *first* (i.e., the first, third, fifth, and so on nodes of the chain are deleted). What is the time complexity of your function?
4. Let $x = (x_1, x_2, \dots, x_n)$ and $y = (y_1, y_2, \dots, y_m)$ be two chains. Write a C++ function to merge the two chains together to obtain the chain $z = (x_1, y_1, x_2, y_2, \dots, x_m, y_m, x_{m+1}, \dots, x_n)$ if $m \leq n$ and $z = (x_1, y_1, x_2, y_2, \dots, x_n, y_{n+1}, \dots, y_m)$ if $m > n$. Following the merge, x and y should represent empty chains because each node initially in x or y is now in z . No additional nodes may be used. What is the time complexity of your function?
5. Let $x = (x_1, x_2, \dots, x_n)$ and $y = (y_1, y_2, \dots, y_m)$ be two chains. Assume that in each chain, the nodes are in nondecreasing order of their data-field values. Write a C++ function to merge the two chains to obtain a new chain z in which the nodes are also in this order. Following the merge, x and y should represent empty chains because each node initially in x or y is now in z . No additional nodes may be used. What is the time complexity of your function?
6. It is possible to traverse a chain in both directions (i.e., left to right and a restricted right-to-left traversal) by reversing the links during the left-to-right traversal. A possible configuration under this scheme is given in Figure 4.12. Assume that l and r are data members of *Chain*.

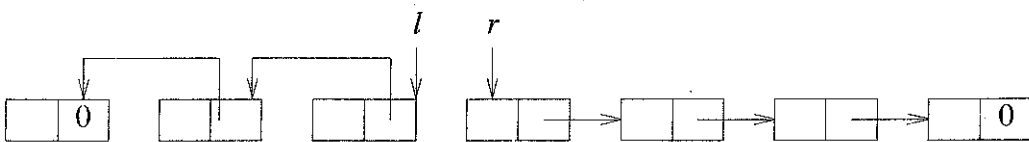


Figure 4.12: Possible configuration for a chain traversed in both directions

The pointer r points to the node currently being examined and l to the node on its left. Note that all nodes to the left of r have their links reversed.

- (a) Write a C++ function to move pointer r , n nodes to the right from any given position (l, r) . If n is greater than the number of nodes to the right, set r to 0 and l to the rightmost node in the chain.
- (b) Write a C++ function to move r , n nodes to the left from any given position (l, r) . If n is greater than the number of nodes to the left, set l

to 0 and r to the leftmost node in the chain.

4.3 THE TEMPLATE CLASS CHAIN

The cost of developing software systems is significant. A contributor is the large number of programmer-hours required to develop, test, and maintain software. Thus, there is a need for software development strategies that reduce the number of person-hours spent without sacrificing the quality of the software. One technique for this is software reuse. The basic principle of software reusability is that when we initially design and develop software, we must try and do so in a manner that makes it possible to reuse the software in the future. This typically requires a greater investment of time when the software is first developed but pays dividends in terms of time saved when the software is reused.

In this section, we shall enhance the chain class of the previous section so that it becomes more reusable. If we are later required to develop software for an application that uses chains, we will be able to reuse our enhanced chain class.

4.3.1 Implementing Chains with Templates

We have already seen that the template mechanism can be used to make a container class more reusable. A chain is clearly a container class, and is therefore a good candidate for implementation with templates. Program 4.6 contains the template definition of our enhanced chain class. Notice that *Chain*<*T*> has been made a friend of *ChainNode*<*T*>. This means that the members of *ChainNode*<int> can be accessed by members of *Chain*<int>, but not by members of, say, *Chain*<float>. Similarly, notice that *first* is declared to be a pointer to an object of type *ChainNode*<*T*>. This ensures that an object of type *Chain*<int>, say, only consists of nodes of type *ChainNode*<int>.

An empty chain of integers *intlist* would be defined as:

```
Chain<int> intlist;
```

4.3.2 Chain Iterators

An *iterator* is an object that is used to access the elements of a container class one by one. The following discussion motivates the need for iterators of a container class. Consider the following operations that one might wish to perform on a container class *C*, all of whose elements are integers.

```
template <class T> class Chain; // forward declaration
```

```
template <class T>
class ChainNode {
friend class Chain <T>;
private:
    T data;
    ChainNode <T> *link;
};
```

```
template <class T>
class Chain {
public:
    Chain () {first = 0;}; // constructor initializing first to 0
    // Chain manipulation operations

private:
    ChainNode <T> *first;
};
```

Program 4.6: Template definition of chains

- (1) Output all integers in C .
- (2) Obtain the maximum, minimum, mean, median, or mode of all integers in C .
- (3) Obtain the sum, product, or sum of squares of all integers in C .
- (4) Obtain all integers in C that satisfy some property P (for example, P could be *is a positive integer*, *is a square of an integer*, etc.).
- (5) Obtain the integer x from C such that, for some function f , $f(x)$ is maximum.

Notice that the solutions to each of these problems require you to examine all elements of the container class. The pseudo-code for these problems typically takes the following form:

```

initialization step;
for each item in C
{
    currentItem = current item of C;
    do something with currentItem;
}
postprocessing step;

```

For example, to find the maximum of all elements in the container class, we would use the following pseudo-code:

```

1 int x = std::numeric_limits<int>::min(); // initialization, must include <limits>
2 for each item in C
3 {
4     currentItem = current item of C;
5     x = max(currentItem, x);           // do something
6 }
7 return x;                             // postprocessing step

```

Program 4.7: Pseudo-code for computing maximum element

The C++ implementation of lines 2 and 4 depends on the container class being used. For an array *a* of size *n*, the implementation of these lines is:

```

2 for (int i = 0; i < n; i++)
4 currentItem = a[i];

```

For a nonempty chain of integers, the implementation of lines 2 and 4 is:

```

2 for ( ChainNode<int> *ptr = first; ptr != 0; ptr = ptr->link)
4 currentItem = ptr->data;

```

Operations such as finding the max element have to be implemented as member functions of the particular container class as these operations access private data members of the container class. There are some drawbacks to this approach. For example, consider the container class *Chain* <*T*>.

(1) Since *Chain* <*T*> is a template class, all of its operations should preferably be independent of the type of object to which *T* is instantiated. However, operations that are meaningful for one instantiation of *T* may not be meaningful for

188 Linked Lists

another instantiation. For example, computing the sum of all elements in a chain makes sense when $T = \text{int}$, but not when $T = \text{Rectangle}$.

(2) The number of member functions of $\text{Chain} <T>$ can become quite large, making the class definition rather unwieldy. A class definition should be as compact as possible so that it is easier for a class user to understand. Moreover, classes are often part of a class library designed by one programmer or programming team. It is not feasible for the class designer to predict all the operations required by a particular user of the class. If a user requires an operation that is not supported by the class, he or she would be forced to add that operation to the container class definition. This is not considered to be a good practice since the resulting class definition may not be consistent with the objectives with which the original class was designed.

(3) Even if it is acceptable for the user to add member functions to a class, he or she would have to learn how to sequence through the elements of the container class, which would entail learning how the container class is implemented.

These arguments suggest that container classes be equipped with iterators that provide systematic access to the elements of the object. Users can employ these iterators to implement their own functions depending upon the particular application. Typically, an iterator is implemented as a nested class of the container class.

C++ Iterators

In C++, an *iterator* is a pointer to an element of an object (for example, a pointer to an element of an array or chain). As the name suggests, an iterator permits you to go (or iterate) through the elements of the object one by one. Program 4.8 shows how to use a pointer (or iterator) y to an array element to iterate through the array's elements. The datatype of the pointer y is int^* , which indicates that y points to elements of type int . In the **for** loop header, y is initialized to point to the first element, $x[0]$, in the array $x[]$ (technically, the variable x is a pointer to the first element of the array). The expression $y++$ increments the pointer so that it advances to the next element of the array. Similarly, $x + 3$ is a pointer 3 positions from x ; that is, it points one position past the last element $x[2]$ of the array. So, in the **for** loop of Program 4.8, y iterates through elements in the range $[x, x + 3)$. The expression $*y$ dereferences y so as to get the element pointed to by y . The program outputs $x[0:2]$.

The following code is equivalent to the **for** loop of Program 4.8.

```

void main()
{
    int x[3] = {0, 1, 2};

    // use a pointer y to iterate through the array x
    for (int* y = x; y != x + 3; y++)
        cout << *y << " ";
    cout << endl;
}

```

Program 4.8: Using an array iterator

```

for (int i = 0; i != 3; i++)
    cout << x[i] << " ";

```

Although you may find this code more transparent than that of Program 4.8, the code of Program 4.8 is generalized easily to output the elements of any object for which an iterator is defined. For example, the code

```

for (Iterator i = start; i != end; i++)
    cout << *i << " ";

```

outputs all elements in the range $[start, end)$. In this code *Iterator* is the datatype of the iterator, *start* is the iterator value for the first element in the range and *end* is the value the iterator has when incremented one past the last element to be output.

The concept of an iterator is fundamental to writing generic code in C++. Program 4.9 gives a possible code for the STL *copy* function. This code may be used to copy elements of any object that has an iterator for which the operators *!=*, and *++* (postincrement) as well as the dereferencing operator (***) have been defined. Different generic codes we write require our iterator to have different capabilities. For example, the STL *copy_backward* function requires us to decrement the value of the iterator.

To simplify iterator development and categorization of generic iterator-based codes, the C++ STL defines five categories of iterators: input, output, forward, bidirectional and random access. All iterators support the equality operators *==* and *!=* as well as the dereference operator ***. Input iterators additionally provide read access to the elements pointed at and support the pre- and post-increment operator *++*. Output iterators provide write access to the elements and also permit iterator advancement via the *++* operator. Forward iterators may be

```

template <class Iterator>
void copy (Iterator start, Iterator end, Iterator to)
{ // copy from [start, end) to [to, to + end - start)
    while (start != end)
        { *to = *start; start++; to++; }
}

```

Program 4.9: Possible code for STL *copy* function

advanced using the increment operator ++ while bidirectional iterators may be incremented as well as decremented (--). Random access iterators are the most general. They permit pointer jumps by arbitrary amounts as well as pointer arithmetic. C++ array iterators such as *y* in Program 4.8 are random access iterators.

A Forward Iterator for *Chain*

We may implement a forward iterator class for *Chain* as in Program 4.10. Our implementation requires that *ChainIterator* be a public nested member class of *Chain*.

Additionally, we add the following public member functions to *Chain*.

```

ChainIterator begin() {return ChainIterator(first);}
ChainIterator end() {return ChainIterator(0);}

```

which respectively return iterators initialized to the first node of a list and one past the last node. We may initialize an iterator object *yi* to the start of a chain of integers *y* using the statement

```
Chain<int>::ChainIterator yi = y.begin();
```

and we may sum the elements in this chain using the STL algorithm *accumulate* and the statement

```
sum = accumulate (y.begin (), y.end (), 0);
```

4.3.3 Chain Operations

Perhaps the most important decision when designing a reusable class is choosing which operations to include. It is important to provide enough operations so that the class can be used in many applications. It is also important not to include too many operations because this will make the class bulky. Examples of operations that would be included in most reusable classes are constructors (including

```

class ChainIterator {
public:
    // typedefs required by C++ for a forward iterator omitted

    // constructor
    ChainIterator(ChainNode <T>* startNode = 0)
        {current = startNode;}

    // dereferencing operators
    T& operator*() const {return current->data;}
    T* operator->() const {return &current->data;}

    // increment
    ChainIterator& operator++() // preincrement
        {current = current->link; return *this;}
    ChainIterator operator++(int) // postincrement
        {
            ChainIterator old = *this;
            current = current->link;
            return old;
        }

    // equality testing
    bool operator!=(const ChainIterator right) const
        {return current != right.current;}
    bool operator==(const ChainIterator right) const
        {return current == right.current;}
private:
    ChainNode <T>* current;
};

```

Program 4.10: A forward iterator for *Chain*

default and copy constructors), a destructor, an assignment operator (**operator=**), the test-for-equality operator (**operator==**), and operators to input and output a class object (obtained by overloading **operator>>** and **operator<<**, respectively). Other operators may also be overloaded depending on the class.

A chain class should provide functions to insert and delete elements. There could be a number of variations under each of these categories. For example, there could be an insertion function that inserts an element at the front of a chain,

192 Linked Lists

another that inserts at the end, as in Program 4.11, and yet another that inserts after the i th element of the chain. To insert efficiently at the end of a chain, we add a private data member *last* to the class *Chain*<*T*>. This data member points to the last node in the chain.

```
template <class T>
void Chain<T>::InsertBack(const T& e)
{
    if (first) { // nonempty chain
        last → link = new ChainNode<T>(e);
        last = last → link;
    }
    else first = last = new ChainNode<T>(e);
}
```

Program 4.11: Inserting at the back of a list

Program 4.12 concatenates two chains. Like the function *InsertBack*, the function *Concatenate* assumes that *Chain* has been augmented with the private data member *last*. The complexity of this function is $O(1)$.

```
template <class T>
void Chain<T>::Concatenate(Chain<T>& b)
{ // b is concatenated to the end of *this.
    if (first) (last → link = b.first; last = b.last;)
    else { first = b.first; last = b.last; }
    b.first = b.last = 0;
}
```

Program 4.12: Concatenating two chains

Another useful function is one that reverses the order of elements in a chain (Program 4.13). This function is especially interesting because it does an “in-place” reversal of the elements using three pointers; no element is physically moved during the reversal, only pointers are changed.

You should try out Program 4.13 on at least three sample chains, the empty chain and chains of length 1 and 2, to convince yourself that you understand the mechanism. For a chain with $m \geq 1$ nodes, the **while** loop is executed m times and each execution of this loop takes $O(1)$ time. So, the computing time is linear or $O(m)$.

```

template <class T>
void Chain <T>::Reverse ()
{ // A chain is reversed so that  $(a_1, \dots, a_n)$  becomes  $(a_n, \dots, a_1)$ .
  ChainNode <T> *current = first,
               *previous = 0; // previous trails current
  while (current) {
    ChainNode <T> *r = previous;
    previous = current;      // r trails previous
    current = current->link;  // current moves to next node
    previous->link = r;       // link previous to preceding node
  }
  first = previous;
}

```

Program 4.13: Reversing a list

4.3.4 Reusing a Class

Having designed and developed a reusable class for chains, we should now try to reuse this class in applications that require chains. In Section 4.7, we illustrate how polynomials can be implemented by reusing the chain class.

Having said that, however, there are some scenarios where one should not attempt to reuse a class, but rather design a new one. We present two such scenarios below:

- (1) One of the disadvantages of reusing a class (say C_1) to implement another class (say C_2) is that, sometimes, this is less efficient than directly implementing class C_2 . If efficiency is of paramount importance, the latter approach is usually the better course of action. In Section 4.6, we use chains to implement stacks and queues. Since each is an important data structure, which will itself be reused in other applications, we implement them directly to make each as efficient as possible.
- (2) Another scenario in which classes should not be reused is when the operations required by the application are complex and specialized, and therefore not offered by the class. In this situation, also, it is preferable to develop the application directly rather than to "force" the application to reuse an existing class. In Section 4.8, we use chains to solve the problem of finding equivalence classes. The chain operations required by our algorithm are specialized and not likely to be implemented in a reusable chain class. Hence, we implement equivalence classes directly.

EXERCISES

The following exercises assume that chains are defined as in Program 4.6. All functions are to be implemented by employing *ChainIterator* to traverse a chain.

1. Write a C++ template function to output all elements of a chain by overloading the output operator `<<`. Assume that the operator `<<` has been overloaded for the data type *T*.
2. Write a C++ function to compute the minimum of all elements of a chain of integers.
3. Write a C++ function to copy the elements of a chain into an array. Use the STL function *copy* together with array and chain iterators.
4. Let $x_1, x_2, \dots, x_n$ be the elements of a chain. Each x_i is an integer. Write a C++ function to compute the expression $\sum_{i=1}^{n-5} (x_i * x_{i+5})$. [Hint: use two iterators to simultaneously traverse the chain.]
5. Fully code and test the C++ template class *Chain*<*T*>. You must include a constructor, which constructs an initially empty chain; a destructor, which deletes all nodes in the chain; functions to insert at either end of the chain; functions *Front* and *Back* that, respectively, return the first and last elements of the list; a function *Get*(*i*) that returns the *i*th element in the list; functions to delete from either end; functions to insert as the *i*th element and to delete the *i*th element; and a forward iterator.

4.4 CIRCULAR LISTS

A *circular list* (or, more precisely, a singly-linked circular list) may be obtained by modifying a chain so that the *link* field of the last node points to the first node. Figure 4.13 shows how the chain of Figure 4.2 is modified to form a circular list.

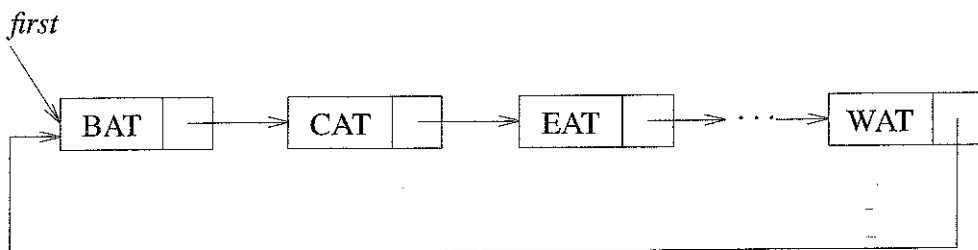


Figure 4.13: A circular list

The circular list structure has some advantages over the chain (as we shall see later). The C++ implementation of circular lists is very similar to that for chains except for some minor differences: To check whether a pointer *current* points to the last node of a circular list, we check for $(current \rightarrow link == first)$ instead of $(current \rightarrow link == 0)$. Secondly, the functions for insertion into and deletion from a circular list must ensure that the *link* field of the last node points to the first node of the list upon completion.

Let us take a look at an operation on a circular list. Suppose we want to insert a new node at the front of the list of Figure 4.14. We have to change the *link* of the node containing x_3 , which requires that we move down the entire length of the list until we find the last node. It is more convenient if the access pointer of a circular list points to the last node rather than to the first (Figure 4.15).

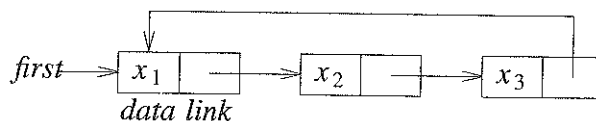


Figure 4.14: Example of a circular list

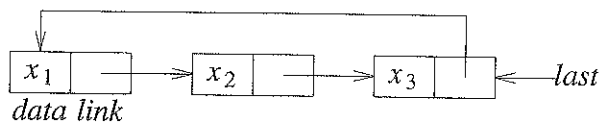


Figure 4.15: Pointing to the last node of a circular list

Now we can write functions that insert at the front (Program 4.14) or at the back of a circular list in $O(1)$ time. To insert e at the back, one only needs to add the statement $last = newNode$ to the *else* clause of *InsertFront*. The code of Program 4.14 assumes the existence of the template class *CircularList* similar to the class *Chain* of Program 4.6. It is assumed that *CircularList* contains the private data member *last* that points to the last node of the list and that *CircularList* is a *friend* of *ChainNode*. In some applications, using the structure of Figure 4.13

```

template <class T>
void CircularList<T>::InsertFront(const T& e)
{ // Insert the element e at the "front" of the circular
  // list *this, where last points to the last node in the list.
  ChainNode<T> *newNode = new ChainNode<T>(e);
  if (last) { // nonempty list
    newNode->link = last->link;
    last->link = newNode;
  }
  else { // empty list
    last = newNode;
    newNode->link = newNode;
  }
}

```

Program 4.14: Inserting at the front of a circular list

causes problems as the empty list has to be handled as a special case. To avoid such special cases, we introduce a dummy node, called the *header* node, into each circular list (i.e., each instance of a circular list will contain one additional node). Thus, the empty list will have the representation of Figure 4.16(a) and the list of Figure 4.13 will have the representation of Figure 4.16(b).

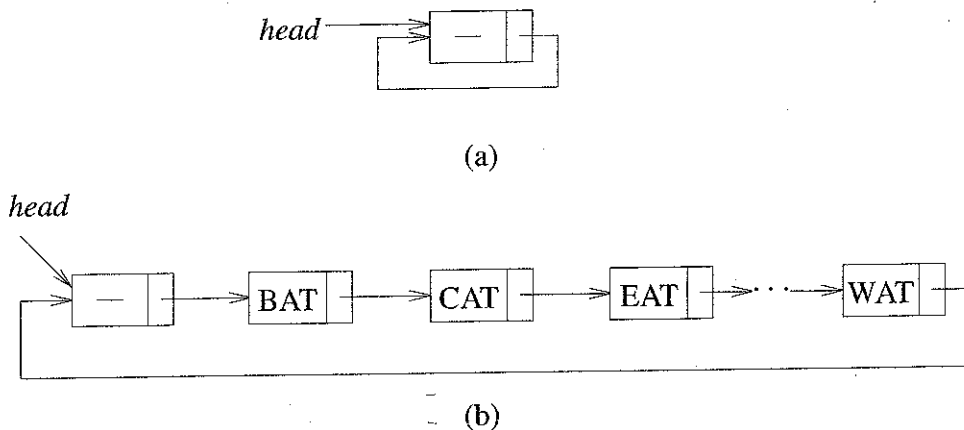


Figure 4.16: Circular lists with a header node

EXERCISES

1. Do Exercise 1 of Section 4.2 for the case of circularly linked lists.
2. Do Exercise 2 of Section 4.2 for the case of circularly linked lists.
3. Do Exercise 3 of Section 4.2 for the case of circularly linked lists.
4. Do Exercise 4 of Section 4.2 for the case of circularly linked lists.
5. Do Exercise 5 of Section 4.2 for the case of circularly linked lists.
6. Do Exercise 6 of Section 4.2 for the case of circularly linked lists.
7. Do Exercise 1 of Section 4.3 for the case of circularly linked lists.
8. Do Exercise 2 of Section 4.3 for the case of circularly linked lists.
9. Do Exercise 3 of Section 4.3 for the case of circularly linked lists.
10. Do Exercise 4 of Section 4.3 for the case of circularly linked lists.
11. Repeat the previous exercise for circularly linked lists with a header node. This time, assume that *CircularListWithHeader* contains a private data member, *header*, that points to the header node.

4.5 AVAILABLE SPACE LISTS

The destructors for chains and circular lists take time linear in the length of the chain or linear list as they delete nodes one at a time. The run time of these destructors may be reduced to $O(1)$ if we maintain our own chain of free (or deleted) nodes. When a new node is needed, we may examine our chain of free nodes. If this chain is not empty, then one of the nodes on it may be made available for use. Only when this chain is empty do we need to invoke `new` to create a new node.

Consider the class *CircularList* of Section 4.4. Let *av* be a static class member of *CircularList* $\langle T \rangle$ of type *ChainNode* $\langle T \rangle^*$ that points to the first node in our chain of nodes that have been “deleted.” (We implement the chain of deleted nodes directly rather than reusing our class *Chain* $\langle T \rangle$ for efficiency reasons.) Our chain of deleted nodes will henceforth be called the *available-space list* or *av list*. Initially, *av* = 0. Instead of using the functions `new` and `delete`, we shall now use the functions *CircularList* $\langle T \rangle::GetNode$ (Program 4.15) and *CircularList* $\langle T \rangle::RetNode$ (Program 4.16).

As illustrated by function *CircularList* $\langle T \rangle::\sim CircularList$ (Program 4.17), a circular list may now be deleted in a fixed amount of time independent of the number of nodes on the list. Figure 4.17 is a schematic showing the link changes involved in deleting a circular list.

A chain may be deleted in $O(1)$ time if we know its first and last nodes. The instructions:

```

template <class T>
ChainNode <T>* CircularList <T>::GetNode()
{
    // Provide a node for use.
    ChainNode <T>* x;
    if (av) {x = av; av = av->link;}
    else x = new ChainNode <T>;
    return x;
}

```

Program 4.15: Getting a node

```

template <class T>
void CircularList <T>::RetNode(ChainNode <T>*& x)
{
    // Free the node pointed to by x.
    x->link = av;
    av = x;
    x = 0;
}

```

Program 4.16: Returning a node

```

template <class KeyT>
void CircularList <T>::~CircularList ()
{
    // Delete the circular list.
    if (last) {
        ChainNode <T>* first = last->link;
        last->link = av; // last node linked to av
        av = first; // first node of list becomes front of av list
        last = 0;
    }
}

```

Program 4.17: Deleting a circular list

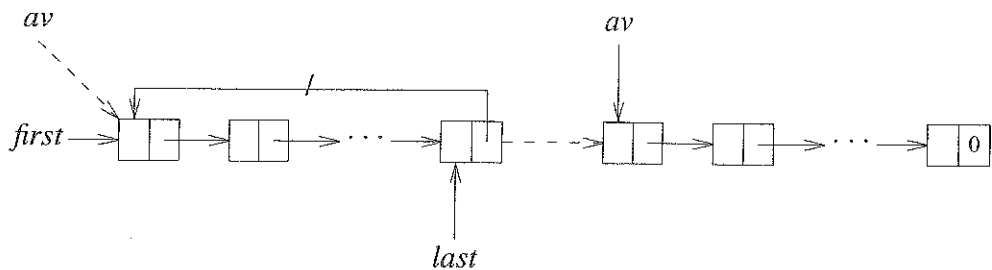


Figure 4.17: Dashed arrows indicate changes involved in deleting a circular list

```
last->link = av;
av = first;
```

accomplish this. When we have our own available space list, we may do other tasks quickly as well. For example, we can generalize the *GetNode* function so that *GetNode*(*n*) returns a chain or circular list with *n* nodes. Since the available space list already has most of the links correctly set, we can reduce the time spent setting links when creating chains and circular lists of known length.

4.6 LINKED STACKS AND QUEUES

We have already seen how to represent stacks and queues sequentially. In this section we examine how to use linked lists to represent stacks and queues. Figure 4.18 shows a linked stack and a linked queue.

Notice that the direction of links for both the stack and the queue is such that it facilitates insertion and deletion of nodes. In the case of Figure 4.18(a), you can easily add a node at the top or delete one from the top. In Figure 4.18(b), you can easily add a node at the rear, and both addition and deletion can be performed at the front, although for a queue we do not want to add nodes at the front. The public members of the linked stack and queue classes *LinkedStack* and *LinkedQueue* are, respectively, the same as those given in the stack and queue ADTs, ADT 3.1 and ADT 3.2. *LinkedStack* has the single private data member *top*, which points to the top node of the stack and *LinkedQueue* has two private data members *front* and *rear*, which, respectively, point to the first and last nodes of the queue. Both classes use nodes of type *ChainNode* <*T*> (Section 4.3) and it is assumed that both classes are declared as **friends** of

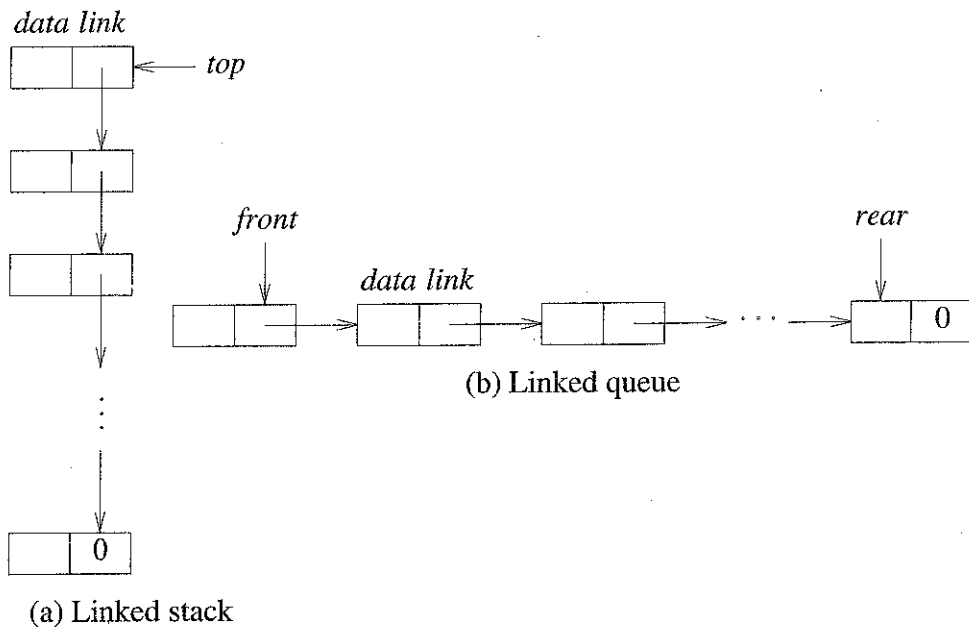


Figure 4.18: Linked stack and queue

ChainNode <T>. The constructor for *LinkedStack* sets *top* to 0 (NULL) and that for *LinkedQueue* sets both *front* and *rear* to 0. The codes for the ADT functions *IsEmpty*, *Top*, *Front* and *Rear* are quite similar to those for the case when an array was used to represent a stack and a queue. Functions for insertion and deletion from linked stacks and queues are presented in Programs 4.19 - 4.22.

```
template <class T>
void LinkedStack<T>::Push(const T& e) {
    top = new ChainNode<T>(e, top);
}
```

Program 4.19: Adding to a linked stack

```

template <class T>
void LinkedStack<T>::Pop()
{ // Delete top node from the stack.
    if (IsEmpty()) throw "Stack is empty. Cannot delete.";
    ChainNode<T> *delNode = top;
    top = top->link;    // remove top node
    delete delNode;    // free the node
}

```

Program 4.20: Deleting from a linked stack

```

template <class T>
void LinkedQueue<T>::Push(const T& e)
{
    if (IsEmpty()) front = rear = new ChainNode(e, 0); // empty queue
    else rear = rear->link = new ChainNode(e, 0); // attach node and update rear
}

```

Program 4.21: Adding to a linked queue

```

template <class T>
void LinkedQueue<T>::Pop()
{ // Delete first element in queue.
    if (IsEmpty()) throw "Queue is empty. Cannot delete.";
    ChainNode<T> *delNode = front;
    front = front->link;    // remove first node from chain
    delete delNode;        // free the node
}

```

Program 4.22: Deleting from a linked queue

EXERCISES

1. Develop and test a complete C++ template class for linked stacks.
2. Develop and test a complete C++ template class for linked queues.
3. Consider the hypothetical data object X2. X2 is a linear list with the restriction that although insertions to the list may be made at either end, deletions can be made from one end only. Design a linked-list representation

202 Linked Lists

for X2. Develop and test a complete C++ template class implementation for X2.

4. Develop and test a C++ template class that implements the queue ADT using a circularly linked list.
5. Implement the stack data structure as a derived class of the class *List<T>* of Section 4.3. Test your code.
6. Implement the queue data structure as a derived class of the class *List<T>* of Section 4.3. Test your code.

4.7 POLYNOMIALS

4.7.1 Polynomial Representation

Let us tackle a reasonable-sized problem using linked lists. This problem, the manipulation of symbolic polynomials, has become a classic example of the use of list processing. In general, we want to represent the polynomial $a(x) = a_m x^{e_m} + \cdots + a_1 x^{e_1}$, where the a_i are nonzero coefficients and the exponents e_i are nonnegative integers such that $e_m > e_{m-1} > \cdots > e_2 > e_1 \geq 0$. We shall define a *Polynomial* class to implement polynomials. Since a polynomial is to be represented by a list, we have *Polynomial* IS-IMPLEMENTED-BY *List*.

Definition: We say that a data object of Type *A* IS-IMPLEMENTED-IN-TERMS-OF a data object of Type *B* if the Type *B* object is central to the implementation of the Type *A* object. This relationship is usually expressed by declaring the Type *B* object as a data member of the Type *A* object. $\square$

Thus, we shall make the chain object *poly* a data member of *Polynomial*. For this, we employ the chain template class of Program 4.6. Each *ChainNode* will represent a term in the polynomial. To do this, the list template *T* is instantiated to **struct** *Term*; where, *Term* consists of two data members *coef* and *exp*. We use **struct** rather than **class** to define *Term* to emphasize that the data members of *Term* are public. The data members of *Term* are made public so that any function that has access to a *Term* object also has access to its data members. This does not violate data encapsulation for *Polynomial* because the linked list and its contents, including all *Term* objects are all private and cannot be accessed. Assuming that the coefficients are integers, the required class declarations are developed in Program 4.23.

Note that *ChainNodes* contain two fields *data* and *link*, and, since *data* is of type *Term*, *data* contains two fields *coef* and *exp*. For clarity, we draw the nodes in a polynomial as below.

```

struct Term
{
    // All members of Term are public by default.
    int coef;    // coefficient
    int exp;    // exponent
    Term Set(int c, int e) { coef = c; exp = e; return *this; };
};

class Polynomial {
public:
    // public functions defined here
private:
    Chain <Term> poly;
};

```

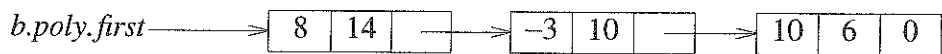
Program 4.23: Polynomial class definition

coef	exp	link
------	-----	------

Figure 4.19 shows the representation for the polynomials $a = 3x^{14} + 2x^8 + 1$ and $b = 8x^{14} - 3x^{10} + 10x^6$.



(a)



(b)

Figure 4.19: Representation of $3x^{14} + 2x^8 + 1$ and $8x^{14} - 3x^{10} + 10x^6$

4.7.2 Adding Polynomials

To add two polynomials a and b , we use the list iterators ai and bi to examine their terms starting at the nodes pointed to by $a.first$ and $b.first$. If the exponents of two terms are equal, then the coefficients are added. A new term is created for the result if the sum of the coefficients is not zero. If the exponent of the current term in a is less than the exponent of the current term in b , then a copy of the term in b is created and attached to the *Polynomial* object c . The iterator ib is advanced to the next term in b . Similar action is taken on a if $ai \rightarrow exp > bi \rightarrow exp$. Figure 4.20 illustrates this addition process on the polynomials a and b of the example of Figure 4.19.

Each time a new node is created, its *coef* and *exp* data members are set and the node is appended to the end of c by function *InsertBack* (Program 4.11). The complete addition code is specified by the function **operator+()** (Program 4.24).

This is our first complete example of the use of list processing, so it should be carefully studied. The basic algorithm is straightforward, using a merging process that streams along the two polynomials, either copying terms or adding them to the result. Thus, the main **while** loop of lines 7-21 has three cases depending upon whether the exponents of the terms are $=$, $<$, or $>$.

Analysis of operator+(): The following tasks contribute to the computing time:

- (1) coefficient additions
- (2) exponent comparisons
- (3) inserting a term at the end of a chain.

Let us assume that each of these three tasks, if done once, takes a single unit of time. The total time taken by **operator+()** is then determined by the number of times these tasks are performed. This number clearly depends on how many terms are present in the polynomials a (***this**) and b . Assume that a and b have m and n terms, respectively:

$$a(x) = a_m x^{e_m} + \cdots + a_1 x^{e_1}, \quad b(x) = b_n x^{f_n} + \cdots + b_1 x^{f_1}$$

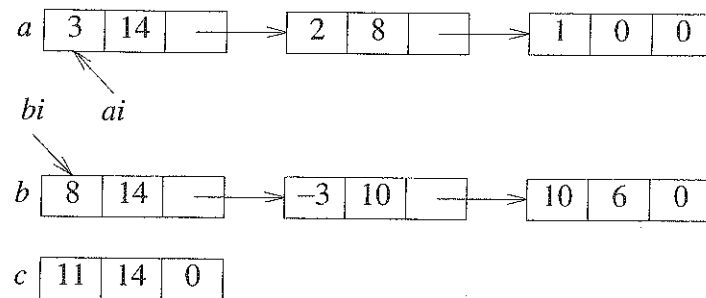
where

$$a_i, b_i \neq 0 \text{ and } e_m > \cdots > e_1 \geq 0, f_n > \cdots > f_1 \geq 0$$

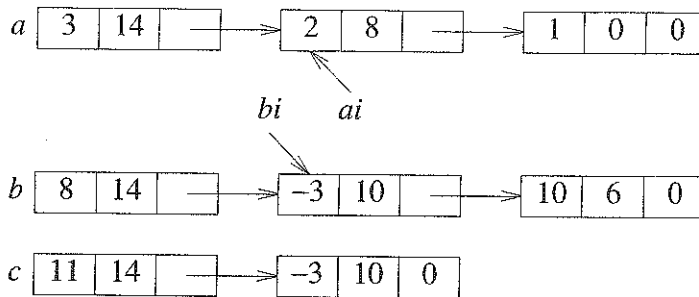
Then clearly the number of coefficient additions can vary as

$$0 \leq \text{coefficient additions} \leq \min\{m, n\}$$

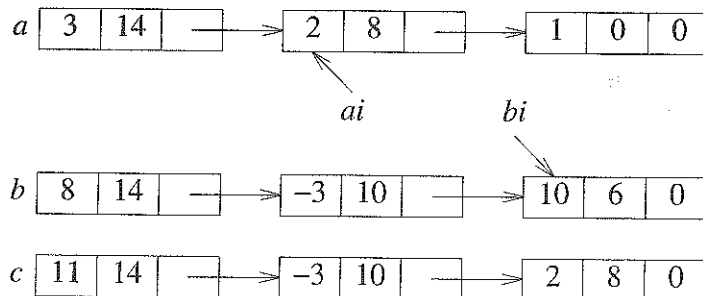
The lower bound is achieved when none of the exponents are equal; the upper



(i) $ai \rightarrow exp == bi \rightarrow exp$



(ii) $ai \rightarrow exp < bi \rightarrow exp$



(iii) $ai \rightarrow exp > bi \rightarrow exp$

Figure 4.20: Generating the first three terms of $c = a + b$

bound is achieved when the exponents of one polynomial are a subset of the exponents of the other polynomial.

As for exponent comparisons, one comparison is made on each iteration of the first **while** loop. On each iteration either ai or bi or both move to the next

```

1 Polynomial Polynomial::operator+(const Polynomial& b) const
2 { // Polynomials *this (a) and b are added and the sum returned.
3     Term temp;
4     Chain<Term>::ChainIterator ai = poly.begin(),
5                                     bi = b.poly.begin();
6     Polynomial c;
7     while (ai && bi) { // current nodes are not null
8         if (ai->exp == bi->exp) {
9             int sum = ai->coef + bi->coef;
10            if (sum) c.poly.InsertBack(temp.Set(sum, ai->exp));
11            ai++; bi++; // advance to next term
12        }
13        else if (ai->exp < bi->exp) {
14            c.poly.InsertBack(temp.Set(bi->coef, bi->exp));
15            bi++; // next term of b
16        }
17        else {
18            c.poly.InsertBack(temp.Set(ai->coef, ai->exp));
19            ai++; // next term of a
20        }
21    }
22    while (ai) { // copy rest of a
23        c.poly.InsertBack(temp.Set(ai->coef, ai->exp));
24        ai++;
25    }
26    while (bi) { // copy rest of b
27        c.poly.InsertBack(temp.Set(bi->coef, bi->exp));
28        bi++;
29    }
30    return c;
31 }

```

Program 4.24: Adding two polynomials

term in their respective polynomials. Since the total number of terms is $m + n$, the number of iterations and hence the number of exponent comparisons is bounded by $m + n$. You can easily construct a case when $m + n - 1$ comparisons will be necessary — e.g., $m = n$ and

$$e_n > f_n > e_{n-1} > f_{n-1} > \cdots > e_2 > f_2 > e_1 > f_1$$

The maximum number of terms in c is $m + n$, so no more than $m + n$ nonzero terms are inserted at the end of c . In summary, the maximum number of executions of any of the statements in **operator+**() is bounded above by $m + n$. Therefore, the computing time is $O(m + n)$. Since any algorithm that adds two polynomials must look at each nonzero term at least once, our code for **operator+**() is optimal to within a constant factor. $\square$

4.7.3 Circular List Representation of Polynomials

Figure 4.21 shows the circular list representation of a polynomial. The structure of Figure 4.21, however, causes some problems during addition and other polynomial operations, as the zero polynomial has to be handled as a special case. To avoid such special cases we introduce a header node into each polynomial (i.e., each polynomial, zero or nonzero, will contain one additional node). The *exp* and *coef* data members of this node will not be relevant. Thus, the zero polynomial will have the representation of Figure 4.22(a), and $a = 3x^{14} + 2x^8 + 1$ will have the representation of Figure 4.22(b).

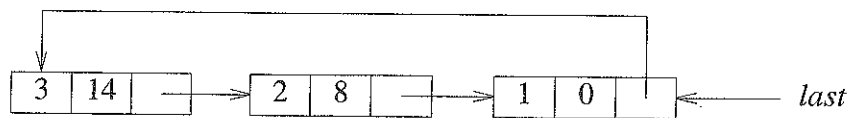


Figure 4.21: Circular list representation of $3x^{14} + 2x^8 + 1$

Using this circular list with header representation, the addition algorithm takes the form given in Program 4.25. This algorithm assumes that the *exp* field of the header node of a polynomial is -1 and that the *begin()* function for *CircularListWithHeader* returns an iterator that points to the node $head \rightarrow link$. Now when all terms of a (*this) have been examined, ai is at the header and $ai \rightarrow exp = -1$. Since $-1 \leq bi \rightarrow exp$, the remaining terms of b can be copied by further executions of the **while** loop. The same is true if all terms of b are examined before those of a . This implies that there is no need for additional code to copy the remaining terms as in Program 4.24.

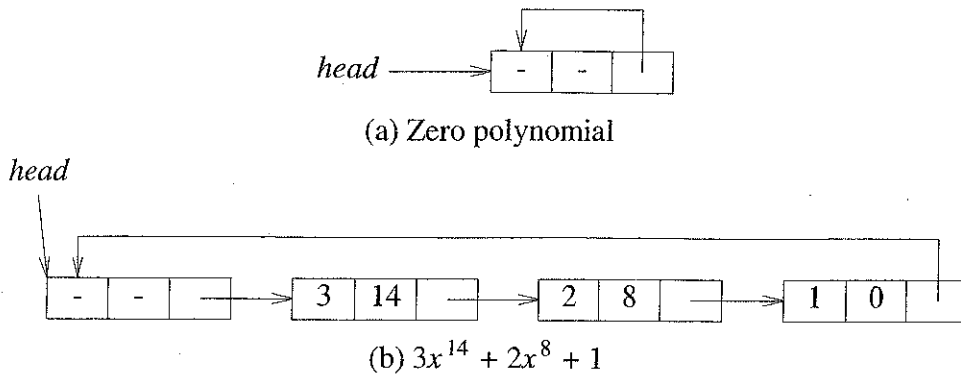


Figure 4.22: Example polynomials

EXERCISES

1. When a class object is passed by value, it is copied into the function's local store. The algorithm for copying the object is specified by the *copy constructor*. If the object's class definition does not define the copy constructor, the default copy constructor is used. The default copy constructor only copies the data members of the object. Define a copy constructor

Polynomial(const *Polynomial*& p)

that copies all the terms of the polynomial p into **this*. Do not forget to delete all terms that may be in **this* prior to the invocation of the copy constructor.

2. Overload the C++ input operator `<<` so that objects of type *Polynomial* may be input using this operator. Your code should read in n pairs of coefficients and exponents, (c_i, e_i) , $1 \leq i \leq n$, of a univariate polynomial and convert the polynomial into the circularly linked list representation described in this section. Assume that $e_i > e_{i+1}$, $1 \leq i < n$, and that $c_i \neq 0$, $1 \leq i \leq n$. Show that this operation can be performed in time $O(n)$.
3. Let a and b be two polynomials represented as circular lists with header nodes. Write a C++ function to compute the product polynomial $c = a * b$. Your code should leave a and b unaltered. Show that if n and m are the number of terms in a and b , respectively, then this multiplication can be carried out in time $O(nm^2)$ or $O(mn^2)$. If a and b are dense, show that the multiplication takes $O(mn)$.

```

1 Polynomial Polynomial::operator+(const Polynomial& b) const
2 // Polynomials *this (a) and b are added and the sum returned.
3     Term temp;
4     CircularListWithHeader<Term>::Iterator ai = poly.begin(),
5                                     bi = b.poly.begin();
6     Polynomial c; // assume constructor sets head->exp = -1
7     while (1) {
8         if (ai->exp == bi->exp) {
9             if (ai->exp == -1) return c;
10            int sum = ai->coef + bi->coef;
11            if (sum) c.poly.InsertBack(temp.Set(sum, ai->exp));
12            ai++; bi++; // advance to next term
13        }
14        else if (ai->exp < bi->exp) {
15            c.poly.InsertBack(temp.Set(bi->coef, bi->exp));
16            bi++; // next term of b
17        }
18        else {
19            c.poly.InsertBack(temp.Set(ai->coef, ai->exp));
20            ai++; // next term of a
21        }
22    }
23 }

```

Program 4.25: Adding two polynomials represented as circular lists with header nodes

4. Write a C++ function to evaluate a polynomial at the point x , where x is a real number. Assume that the polynomial is represented as a circularly linked list with a header node.
5. [**Programming Project**] Develop a C++ class *Polynomial* to represent and manipulate univariate polynomials with integer coefficients (use circular linked lists with header nodes). Each term of the polynomial will be represented as a node. Thus, a node in this system will have three data members as below:

coef	exp	link
------	-----	------

Each polynomial is to be represented as a circular list with header node. To delete polynomials efficiently, we need to use an available-space list and associated functions as described in Section 4.5. The external (i.e., for input or output) representation of a univariate polynomial will be assumed to be a sequence of integers of the form: $n, c_1, e_1, c_2, e_2, c_3, e_3, \dots, c_n, e_n$, where e_i represents an exponent and c_i a coefficient; n gives the number of terms in the polynomial. The exponents are in decreasing order— $e_1 > e_2 > \dots > e_n$.

Write and test the following functions:

- (a) *istream& operator>>(istream& is, Polynomial& x)*: Read in an input polynomial and convert it to its circular list representation using a header node.
- (b) *ostream& operator<<(ostream& os, Polynomial& x)*: Convert x from its linked list representation to its external representation and output it.
- (c) *Polynomial::Polynomial(const Polynomial& a)* [Copy Constructor]: Initialize the polynomial **this* to the polynomial a .
- (d) *const Polynomial& Polynomial::operator=(const Polynomial& a)* **const** [Assignment Operator]: Assign polynomial a to **this*.
- (e) *Polynomial::~~Polynomial()* [Destructor]: Return all nodes of the polynomial **this* to the available-space list.
- (f) *Polynomial operator+ (const Polynomial& b) const* [Addition]: Create and return the polynomial **this* + b .
- (g) *Polynomial operator- (const Polynomial& b) const* [Subtraction]: Create and return the polynomial **this* - b .
- (h) *Polynomial operator*(const Polynomial& b) const* [Multiplication]: Create and return the polynomial **this* * b .
- (i) *float Polynomial::Evaluate(float x) const*: Evaluate the polynomial **this* at x and return the result.

4.8 EQUIVALENCE CLASSES

Let us put together some of the ideas discussed so far on linked and sequential representations to solve a problem that arises in the design and manufacture of very large-scale integrated (VLSI) circuits. One of the steps in the manufacture of a VLSI circuit involves exposing a silicon wafer using a series of masks. Each

mask consists of several polygons. Polygons that overlap are electrically equivalent. Electrical equivalence specifies a relationship among mask polygons. This relation has several properties that it shares with other relations, such as the conventional mathematical equivalence. Suppose we denote an arbitrary relation by the symbol $\equiv$, and suppose that

- (1) For any polygon x , $x \equiv x$ (e.g., x is electrically equivalent to itself). Thus, $\equiv$ is *reflexive*.
- (2) For any two polygons x and y , if $x \equiv y$, then $y \equiv x$. Thus, the relation $\equiv$ is *symmetric*.
- (3) For any three polygons x , y , and z , if $x \equiv y$ and $y \equiv z$, then $x \equiv z$ (e.g., if x and y are electrically equivalent and y and z are also, then so also are x and z). The relation $\equiv$ is *transitive*.

Definition: A relation $\equiv$ over a set S , is said to be an *equivalence relation* over S iff it is symmetric, reflexive, and transitive over S . $\square$

Equivalence relations are numerous. For example, the "equal to" ($=$) relationship is an equivalence relation, since (1) $x = x$, (2) $x = y$ implies $y = x$, and (3) $x = y$ and $y = z$ implies $x = z$. One effect of an equivalence relation is to partition the set S into equivalence classes such that two members x and y of S are in the same equivalence class iff $x \equiv y$. For example, if we have 12 polygons numbered 0 through 11 and the following overlap pairs are defined:

$$0 \equiv 4, 3 \equiv 1, 6 \equiv 10, 8 \equiv 9, 7 \equiv 4, 6 \equiv 8, 3 \equiv 5, 2 \equiv 11, \text{ and } 11 \equiv 0$$

then, as a result of the reflexivity, symmetry, and transitivity of the relation $\equiv$, we get the following partitioning of the 12 polygons into three equivalence classes:

$$\{0, 2, 4, 7, 11\}; \{1, 3, 5\}; \{6, 8, 9, 10\}$$

These equivalence classes are important, as each equivalence class defines a *signal net*. The signal nets can be used to verify the correctness of the masks.

Our algorithm to determine equivalence classes works in essentially two phases. In the first phase the equivalence pairs (i, j) are read in and stored. In phase two we begin at 0 and find all pairs of the form $(0, j)$. The values 0 and j are in the same class. By transitivity, all pairs of the form (j, k) imply k is in the same class as 0. We continue in this way until the entire equivalence class containing 0 has been found and output. Then we find an object not yet output. This is in a new equivalence class. The objects in this equivalence class are found as before and output.

The first design for our equivalence class algorithm might be as in Program 4.26. Let m and n represent the number of input pairs and the number of objects, respectively. We need to select a data structure to hold these pairs. Easy random access would suggest a Boolean array, say $\text{pairs}[n][n]$. The element $\text{pairs}[i][j] =$

```

void Equivalence()
{
    initialize;
    while more pairs
    {
        input the next pair (i,j);
        process this pair;
    }
    initialize for output;
    for (each object not yet output)
        output the equivalence class that contains this object;
}

```

Program 4.26: First version of equivalence algorithm

true if and only if (i,j) is an input pair. However, using such an array could potentially be quite wasteful of space since very few of the array elements may be used. Any algorithm that uses this data structure would require $\Theta(n^2)$ time, just to initialize the array.

These considerations lead us to consider a chain to represent each row of the aforementioned array *pairs*. Each node on a chain requires only a *data* and a *link* field. However, we still need random access to the i th row. For this, we use a one-dimensional array, *first*[n], with *first*[i] a pointer to the first node in the chain for row i .

Looking at the second phase of the algorithm we need a mechanism that tells us whether or not object i is yet to be printed. A Boolean array, *out*[n], can be used for this. Program 4.27 gives the next refinement of our equivalence algorithm.

Let us simulate the algorithm, as we have it so far, on the input data set

$0 \equiv 4, 3 \equiv 1, 6 \equiv 10, 8 \equiv 9, 7 \equiv 4, 6 \equiv 8, 3 \equiv 5, 2 \equiv 11, \text{ and } 11 \equiv 0$

After the **while** loop is completed the chains look as in Figure 4.23. For each relation $i \equiv j$, two nodes are used. *first*[i] points to a chain of nodes that contains every number directly equivalent to i by an input relation.

In phase two, we scan the *first* array in the order $i, 0 \leq i < n$. For each i such that *out*[i] is **false**, the elements in the list *first*[i] are output. To enable the processing of the remaining elements which, by transitivity, belong in the same class as i , a linked stack is created. Thus, a node may initially be on one of the chains given by *first*[] and later be on a linked stack of elements waiting to be processed. The complete function is given in Program 4.28.

```

void Equivalence()
{
    read n;           // read in number of objects
    initialize first[0:n-1] to 0 and out[0:n-1] to false;
    while more pairs // input pairs
    {
        read the next pair (i,j);
        put j on the chain first[i];
        put i on the chain first[j];
    }
    for (i = 0; i < n; i++)
        if (!out[i]) {
            out[i] = true;
            output the equivalence class that contains object i;
        }
}

```

Program 4.27: A more detailed version of equivalence algorithm

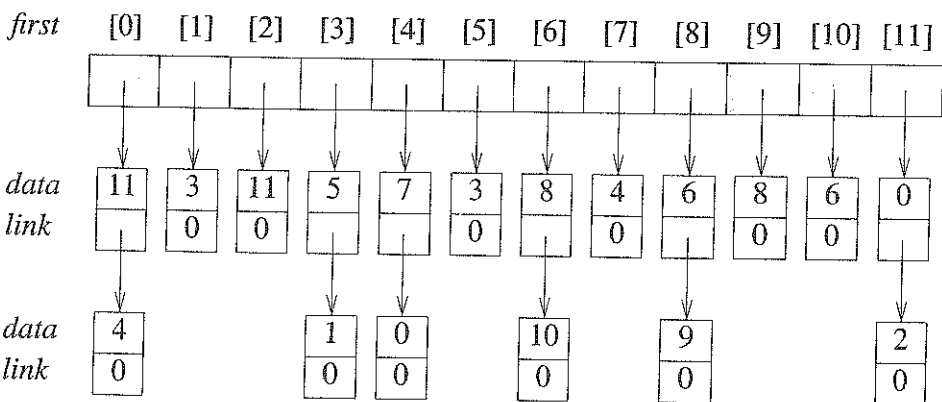


Figure 4.23: Lists after pairs have been input

```

class ENode {
friend void Equivalence ();
public:
    ENode(int d = 0) // constructor
        {data = d; link = 0;}
private:
    int data;
    ENode *link;
};

void Equivalence ()
{
    // Input the equivalence pairs and output the equivalence classes.
    ifstream inFile( "equiv.in", ios::in); // "equiv.in" is the input file
    if (!inFile) throw "Cannot open input file.";
    int i, j, n;
    inFile >> n; // read number of objects
    // initialize first and out
    ENode **first = new ENode* [n];
    bool *out = new bool [n];
    // use STL function fill to initialize
    fill(first, first + n, 0);
    fill(out, out + n, false);

    // Phase 1: input equivalence pairs
    inFile >> i >> j;
    while (inFile.good()) { // check end of file
        first[i] = new ENode(j, first[i]);
        first[j] = new ENode(i, first[j]);
        inFile >> i >> j;
    }

    // Phase 2: output equivalence classes
    for (i = 0; i < n; i++)
        if (!out[i]) { // needs to be output
            cout << endl << "A new class: " << i;
            out[i] = true;
            ENode *x = first[i]; ENode *top = 0; // initialize stack
            while (1) { // find rest of class
                while (x) { // process the list
                    j = x->data;
                    if (!out[j]) {

```

```

        cout << ", " << j;
        out[j] = true;
        ENode *y = x->link;
        x->link = top;
        top = x;
        x = y;
    }
    else x = x->link;
} // end of while(x)
if (!top) break;
x = first[top->data];
top = top->link; // unstack
} // end of while(1)
} // end of if (!out[i])
for (i = 0; i < n; i++)
    while (first[i]) {
        ENode *delnode = first[i];
        first[i] = delnode->link;
        delete delnode;
    }
delete [ ] first; delete [ ] out;
}

```

Program 4.28: C++ function to find equivalence classes

Analysis of *Equivalence*: The initialization of *first* and *out* takes $O(n)$ time. The processing of each input pair in phase 1 takes a constant amount of time. Hence, the total time for this phase is $O(n + m)$, where m is the number of input pairs. In phase 2 each node is put onto the linked stack at most once. Since there are only $2m$ nodes and the **for** loop is executed n times, the time for this phase is $O(m + n)$. Hence, the overall computing time is $O(m + n)$. Any algorithm that processes equivalence relations must look at all m equivalence pairs and at all n objects at least once. Thus, no algorithm can have a computing time less than $\Theta(m + n)$. This means that function *Equivalence* is optimal to within a constant factor. Unfortunately, the space required by the algorithm is also $O(m + n)$. In Chapter 5 we shall see an $O(n)$ -space solution to this problem.

EXERCISES

1. Rewrite function *Equivalence* (Program 4.28) using an array of objects of type *Chain* <int> rather than an array of type *ENode**. Also, use an object of type *Stack* <int> (see Section 3.2) rather than the custom linked stack used in Program 4.28. What can you say about the expected relative

performance of your function and Program 4.28? Which requires more space?

2. Rewrite function *Equivalence* (Program 4.28) using dynamic arrays and array doubling in place of chains (see Sections 2.5 and 3.1.1). Compare the worst-case space and time complexities of the two versions of *Equivalence*.

4.9 SPARSE MATRICES

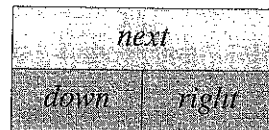
4.9.1 Sparse Matrix Representation

In Chapter 2, we saw that when matrices were sparse (i.e., many of the entries were zero), then much space and computing time could be saved if only the nonzero terms were retained explicitly. In the case where these nonzero terms did not form any “nice” pattern such as a triangle or a band, we devised a sequential scheme in which each nonzero element was represented by a structure with three data members: row, column, and value. The sequential representation of Chapter 2 permits easy access of matrix terms by row. However, accessing all the terms in a specific column of a matrix is difficult. To provide easy access both by row and by column, we devise a linked representation for a sparse matrix. In the data representation we use, each nonzero element is in two circular lists; one is a row list and the other a column list. So, we have a circular list for each row and each column of the matrix. Each circular list has a header node.

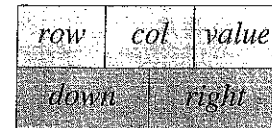
Our sparse matrix representation uses nodes of the type *MatrixNode*. This class has a Boolean field *head*, which is used to distinguish between header nodes and nodes that represent nonzero elements. Each header node has three additional fields: *down*, *right*, and *next*. The total number of header nodes is $\max \{\text{number of rows, number of columns}\}$. The header node for row i is also the header node for column i . The *down* field of a header node is used to link into a column list; the *right* field is used to link into a row list. The *next* field links the header nodes together.

All other nodes have six fields: *head*, *row*, *col*, *value*, *down*, and *right* (Figure 4.24). The *down* field is used to link to the next nonzero term in the same column and the *right* field links to the next nonzero term in the same row. Thus, if $a_{ij} \neq 0$, then there is a node with *head* = **false**, *value* = a_{ij} , *row* = i , and *col* = j . This node is linked into the circular linked lists for row i and column j . Hence, the node is simultaneously in two different lists.

As noted earlier, each header node is in three lists: a row list, a column list, and a list of header nodes. The list of header nodes itself has a header node, H , that is identical to the nodes used to represent nonzero elements. The *row* and



(a) header node



(b) element node

head field is not shown

Figure 4.24: Node structure for sparse matrices

col fields of this node are used to store the matrix dimensions. The entire matrix is represented by class *Matrix* with the data member *headnode*, which points to *H*.

$$\begin{bmatrix} 2 & 0 & 0 & 0 \\ 4 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \\ 8 & 0 & 0 & 1 \\ 0 & 0 & 6 & 0 \end{bmatrix}$$

Figure 4.25: 5×4 sparse matrix *a*

The linked structure obtained for the 5×4 matrix, *a*, of Figure 4.25 is shown in Figure 4.26. Although Figure 4.26 does not show the values of the *head* fields, these values are readily determined from the node structure shown. For each nonzero term of *a*, we have one six-field node that is in exactly one column list and one row list. The header nodes are marked H0 to H4 and are drawn twice to simplify the figure. As seen in the figure, the *right* field of *headnode* is used to link into the list of header nodes. Notice that the whole matrix may be referenced through the header node, *headnode*, of the list of header nodes.

If we wish to represent an $n \times m$ sparse matrix with *r* nonzero terms, the number of nodes needed is $\max\{n, m\} + r + 1$. Although each node may require several words of memory, the total storage needed will be less than nm for sufficiently small *r*.

The required node structure may be defined in C++ using an anonymous

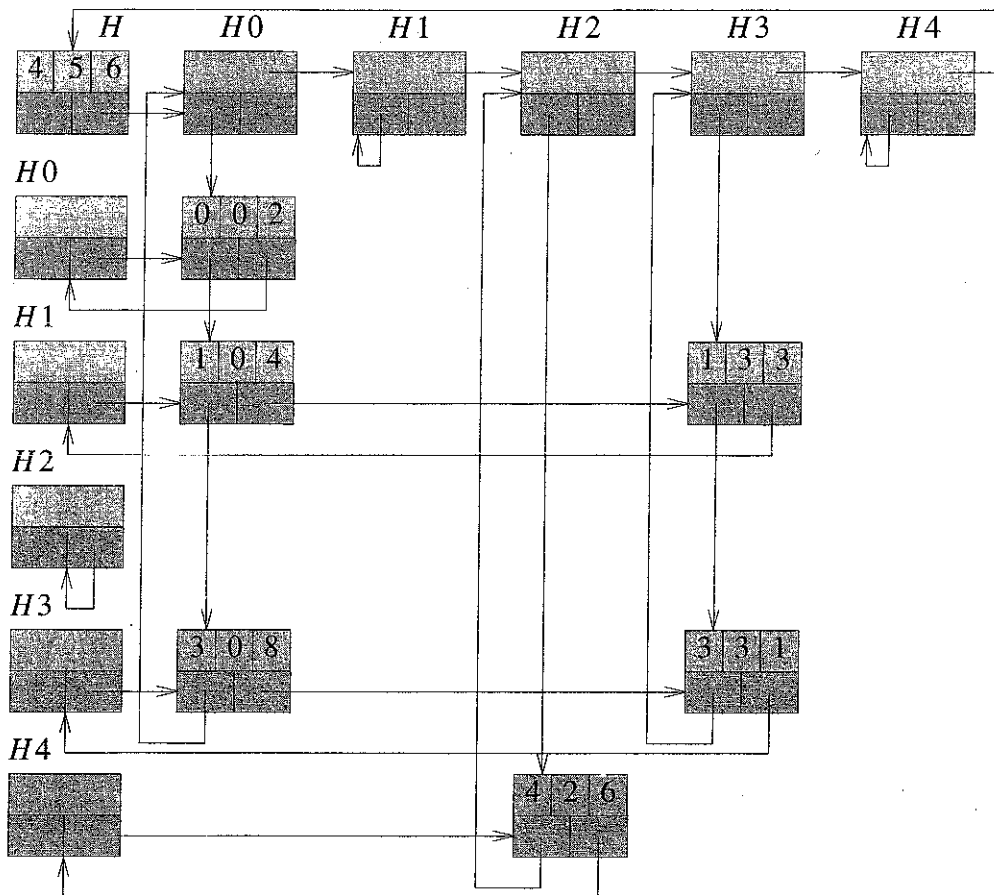


Figure 4.26: Linked representation of the sparse matrix of Figure 4.25 (the *head* field of a node is not shown)

union as in Program 4.29. Since all nodes contain fields *head*, *down* and *right*, these are declared outside the anonymous union. Header nodes also contain *next* while all other nodes contain *row*, *col*, and *value*. These fields are all incorporated into the **struct** *Triple*. Space is allocated by the union declaration for the larger of the fields *next* and *triple* (which is an object of type *Triple*).

```

struct Triple {int row, col, value;};
class Matrix; // forward declaration
class MatrixNode {
friend class Matrix;
friend istream& operator>>(istream&, Matrix&); // for reading in a matrix
private:
    MatrixNode *down, *right;
    bool head;
    union { // anonymous union
        MatrixNode *next;
        Triple triple;
    };
    MatrixNode(bool, Triple *); // constructor
};

MatrixNode::MatrixNode(bool b, Triple * t) // constructor
{
    head = b;
    if (b) {right = down = this;} // row/column header node
    else triple = *t; // element node or header node for list of header nodes
}

class Matrix {
friend istream& operator>>(istream&, Matrix&);
public:
    ~Matrix(); // destructor
private:
    MatrixNode *headnode;
};

```

Program 4.29: Class definitions for sparse matrices

4.9.2 Sparse Matrix Input

The first operation we shall consider is that of reading in a sparse matrix and obtaining its linked representation. We assume that the first input line gives us the number of rows, the number of columns, and the number of nonzero terms in the matrix. Each subsequent input line is a triple of the form (i, j, a_{ij}) . These triples consist of the *row*, *col*, and *value* data members of the nonzero terms of the matrix. We also assume that these triples are ordered by rows and within rows by columns.

220 Linked Lists

For example, the input for the 5×4 sparse matrix of Figure 4.25, which has six nonzero terms, would take the following form: 5, 4, 6; 0, 0, 2; 1, 0, 4; 1, 3, 3; 3, 0, 8; 3, 3, 1; 4, 2, 6. The code for the input operator `>>` (Program 4.30) makes use of an auxiliary array *head*, of size $\max(s.\text{row}, s.\text{col})$. *head*[*i*], which is a pointer to the header node for column *i* and hence also for row *i*, enables efficient random access to columns while the input matrix is set up. Function `operator>>` first creates the header nodes (but doesn't link them together) and then inputs the matrix elements one at a time. Each element is added to its row and column lists upon input. The *next* field of header node *i* is used initially to keep track of the last node in column *i*. Eventually, in line 31, the header nodes are linked together through this field.

Analysis of `operator>>`: Assuming that `new` works in $O(1)$ time, all the header nodes may be set up in $O(\max\{n, m\})$ time, where *n* is the number of rows and *m* the number of columns in the matrix being input. Each nonzero term is set up in $O(1)$ time because of the use of the variable *last* and a random access scheme for the bottommost node in each column list. Hence, the `for` loop of lines 15-26 takes $O(r)$ time. The rest of the algorithm takes $O(\max\{n, m\})$ time. The total time is therefore $O(\max\{n, m\} + r) = O(n + m + r)$. Note that this time is asymptotically better than the input time of $O(nm)$ for an $n \times m$ matrix using a two-dimensional array but slightly worse than that for the sequential sparse method of Section 2.3. $\square$

4.9.3 Deleting a Sparse Matrix

All the nodes of a sparse matrix may be returned one at a time using `delete`. A faster way to return the nodes is to set up an available-space list (Section 4.5). Assume that *av* points to the first node of this list and that this list is linked through the field *right*. Function `Matrix::~Matrix()` (Program 4.31) deletes a sparse matrix in an efficient way.

Analysis of `~Matrix()`: Since each node is in only one row list, it is sufficient to return all the row lists of the matrix. Each row list is circularly linked through the field *right*. Thus, nodes need not be returned one by one, as a circular list can be deleted in $O(1)$ time. The computing time for Program 4.31 is readily seen to be $O(n + m)$. Note that even if the available-space list had been linked through the field *down*, deleting still could have been carried out in $O(n + m)$ time. $\square$

```

1 istream& operator>>(istream& is, Matrix& matrix)
2 {// Read in a matrix and set up its linked representation.
3   Triple s;
4   is >> s.row >> s.col >> s.value; // matrix dimensions
5   int p = max(s.row, s.col);
6   // set up header node for list of header nodes.
7   matrix.headnode = new MatrixNode(false, &s);
8   if (p == 0) { matrix.headnode->right = matrix.headnode; return is;}
9   // at least one row or column
10  MatrixNode **head = new MatrixNode * [p];
11  for (int i = 0; i < p; i++)
12    head[i] = new MatrixNode(true, 0);
13  int currentRow = 0;
14  int MatrixNode *last = head[0]; // last node in current row
15  for (i = 0; i < s.value; i++) // input triples
16  {
17    Triple t;
18    is >> t.row >> t.col >> t.value;
19    if (t.row > currentRow) { // close current row
20      last->right = head[currentRow];
21      currentRow = t.row;
22      last = head[currentRow];
23    } // end of if
24    last->right = new MatrixNode(false, &t); // link new node into row list
25    head[t.col]->next = head[t.col]->next->down = last; // link into column list
26  } // end of for
27  last->right = head[currentRow]; // close last row
28  for (i = 0; i < s.col; i++) head[i]->next->down = head[i]; // close all column lists
29  // link the header nodes together
30  for (i = 0; i < p - 1; i++) head[i]->next = head[i + 1];
31  head[p - 1]->next = matrix.headnode;
32  matrix.headnode->right = head[0];
33  delete [ ] head;
34  return is;
35  }

```

Program 4.30: Reading in a sparse matrix

```

Matrix::~Matrix ()
{
    // Return all nodes to the av list. This list is a chain linked via the right
    // field. av is a static variable that points to the first node of the av list.
    if (!headnode) return; // no nodes to delete
    MatrixNode *x = headnode →right;
    headnode →right = av; av = headnode; // return headnode
    while (x != headnode) { // erase by rows
        MatrixNode *y = x →right;
        x →right = av;
        av = y;
        x = x →next; // next row
    }
    headnode = 0;
}

```

Program 4.31: Deleting a sparse matrix

EXERCISES

In Exercises 1 to 5, the sparse matrix representation described in this section is assumed.

1. Write the C++ function, **operator+(const Matrix& b) const**, which returns the matrix ***this** + *b*. Show that if ***this** and *b* are $n \times m$ matrices with r_a and r_b nonzero terms, then this addition can be carried out in $O(n + m + r_a + r_b)$ time.
 2. Write the C++ function, **operator*(const Matrix& b) const**, which returns the matrix ***this** * *b*. If *a* is an $n \times m$ matrix with r_a nonzero terms and if ***this** is an $n \times m$ matrix with r_a nonzero terms and *b* is an $m \times p$ matrix with r_b nonzero terms, then this multiplication can be done in $O(pr_a + nr_b)$ time. Can you think of a way to do the multiplication in $O(\min\{pr_a, nr_b\})$ time?
 3. Write the C++ function **operator<<()**, which outputs a sparse matrix as triples (i, j, a_{ij}) . The triples are to be output by rows and within rows by columns. Show that this operation can be performed in time $O(n + r_a)$ if there are r_a nonzero terms in the matrix. n is the number of rows in the matrix.
 4. Write the C++ function, *Transpose()*, which transposes a sparse matrix. What is the computing time of your function?
-

5. Write and test a copy constructor for sparse matrices. What is the computing time of your copy constructor?
6. **[Programming Project]** A simpler and more efficient representation for sparse matrices can be obtained when one is limited to the operations of addition, subtraction, and multiplication. Now, nodes have the data members *rowLink*, *colLink*, *row*, *col*, and *value*. Each nonzero term is represented by a node. These nodes are linked together to form two circular lists. The first list, the row list, is made up by linking nodes by rows and within rows by columns. The linking is done via the *right* data member. The second list, the column list, is made up by linking nodes via the *down* data member. In this list, nodes are linked by columns and within columns by rows. These two lists share a common header node. In addition, a node is added to contain the dimensions of the matrix. Draw the resulting representation for the matrix of Figure 4.25.

Write a C++ class for this representation. You must include the following functions. What is the computing time of each of your functions? How do these times compare with the corresponding times for the representation of this section?

- (a) *istream& operator>>(istream& os, Matrix& m):*
Read in the matrix and set it up according to the representation of this section. The first input line gives the matrix dimensions. The next several lines contain one triple, (row, column, value), each. The last triple ends the input file. These triples are in increasing order by rows. Within rows, the triples are in increasing order of columns. The data is to be read in one line at a time and converted to internal representation.
- (b) *ostream& operator<<(ostream& os, const Matrix& m):*
Output the terms of *m*. To do this, you will have to design a suitable output format. The output should be ordered by rows and within rows by columns.
- (c) *Matrix::Matrix(const Matrix& a)* [Copy Constructor]:
Initialize the sparse matrix **this* to the sparse matrix *a*.
- (d) *const Matrix& Matrix::operator=(const Matrix& a) const* [Assignment Operator]: Assign sparse matrix *a* to **this*.
- (e) *Matrix::~Matrix()* [Destructor]:
Return all nodes of the sparse matrix **this* to the available-space list.
- (f) *Matrix Matrix::operator+(const Matrix& b) const:*
Create and return the sparse matrix **this* + *b*.
- (g) *Matrix Matrix::operator-(const Matrix& b) const:*
Create and return the sparse matrix **this* - *b*.

- (h) *Matrix Matrix::operator* (const Matrix& b) const:*
Create and return the sparse matrix **this * b*.
 - (i) *Matrix Matrix::Transpose () () const:*
Create and return the transpose of **this*.
7. Compare the sparse representations of Exercise 6 and this section with respect to the time needed to output the elements in an arbitrary row or column.
 8. [**Programming Project**] Implement a complete linked-list system to perform arithmetic on sparse matrices using the representation of this section. You must include all the functions listed in Exercise 6.

4.10 DOUBLY LINKED LISTS

So far we have been working chiefly with chains and singly linked circular lists. For some problems these would be too restrictive. One difficulty with these lists is that if we are pointing to a specific node, say *p*, then we can move only in the direction of the links. The only way to find the node that precedes *p* is to start at the beginning of the list. The same problem arises when one wishes to delete an arbitrary node from a singly linked list. As can be seen from Example 4.4, easy deletion of an arbitrary node requires knowing the preceding node. If we have a problem in which it is necessary to move in either direction or in which we must delete arbitrary nodes, then it is useful to have doubly linked lists. Each node now has two link fields, one linking in the forward direction and the other linking in the backward direction.

A node in a doubly linked list has at least two fields—e.g., *left* (left link), and *right* (right link). Usually, each node will have a *data* field as well. A doubly linked list may or may not be circular and may or may not have a header node. A sample doubly linked circular list with header node is shown in Figure 4.27. This list has a header node plus three additional nodes. As was true in the earlier sections, header nodes are convenient for the algorithms. Now suppose that *p* points to any node in a doubly linked list. So, $p == p \rightarrow \text{left} \rightarrow \text{right} == p \rightarrow \text{right} \rightarrow \text{left}$. This formula reflects the essential virtue of this structure—namely, that one can go back and forth with equal ease. When header nodes are used, an empty list has exactly one node—the header node (Figure 4.28). Program 4.32 contains the class definition of a doubly linked list of integers. To work with these lists we must be able to insert and delete nodes. Function *Dbllist::Delete* (Program 4.33) deletes node *x* from the list. Following the deletion, *x* points to a node that is no longer part of the list. Figure 4.29 shows how the function works on a doubly linked list with only a single node. Even though the *right* and *left* data members of node *x* still point to the header node, this node

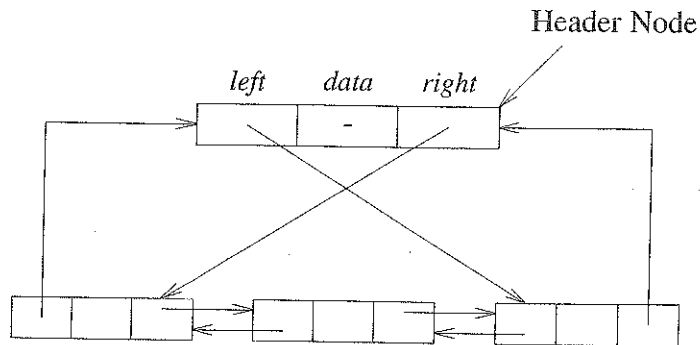


Figure 4.27: Doubly linked circular list with header node

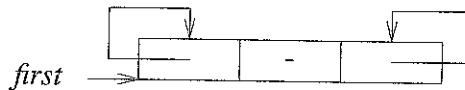


Figure 4.28: Empty doubly linked circular list with header node

has effectively been removed, as there is no way to access x through *first*. Insertion is only slightly more complex (see Program 4.34).

EXERCISES

1. Let x be a node in a singly linked circular list. Write a C++ function to delete the data in this node. Following the deletion, the number of nodes in the list is one less than before the deletion. Your function must run in $O(1)$ time. (Hint: Instead of deleting the node x , copy the data from the node, if any, that is next to x and delete that node instead.)
2. Write a function `void DbList::Concatenate(DbList m)` to concatenate the two lists `*this` and `m`. On completion of the function, the resulting list should be stored in `*this` and the list `m` should contain the empty list. Your function must run in $O(1)$ time.

```

class Dbllist;

class DbllistNode {
friend class Dbllist;
private:
    int data;
    DbllistNode *left, *right;
};

class Dbllist {
public:
    // List manipulation operations
    .
    .
private:
    DbllistNode *first; // points to header node
};

```

Program 4.32: Class definition of a doubly linked list

```

void Dbllist::Delete(DbllistNode *x)
{
    if (x == first) throw "Deletion of header node not permitted";
    else {
        x->left->right = x->right;
        x->right->left = x->left;
        delete x;
    }
}

```

Program 4.33: Deletion from a doubly linked circular list

3. Devise a linked representation for a list in which insertions and deletions can be made at either end in $O(1)$ time. Such a structure is called a *deque*. Write functions to insert and delete at either end.
-

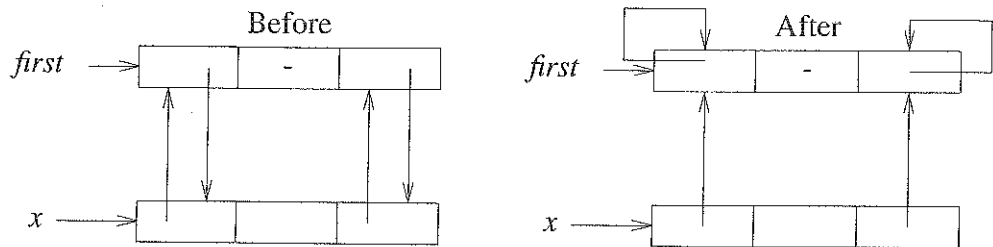


Figure 4.29: Deletion from a doubly linked circular list

```

void DbList::Insert(DbListNode *p, DbListNode *x)
{
  // insert node p to the right of node x
  p->left = x; p->right = x->right;
  x->right->left = p; x->right = p;
}

```

Program 4.34: Insertion into a doubly linked circular list

4. Consider the operation XOR (exclusive OR, also written as $\oplus$) defined as follows (for i, j binary):

$$i \oplus j = \begin{cases} 0 & \text{if } i \text{ and } j \text{ are identical} \\ 1 & \text{otherwise} \end{cases}$$

This definition differs from the usual OR of logic, which is defined as

$$i \text{ OR } j = \begin{cases} 0 & \text{if } i = j = 0 \\ 1 & \text{otherwise} \end{cases}$$

The definition can be extended to the case in which i and j are binary strings (i.e., take the XOR of corresponding bits of i and j). So, for example, if $i = 10110$ and $j = 01100$, then $i \text{ XOR } j = i \oplus j = 11010$. Note that

$$a \oplus (a \oplus b) = (a \oplus a) \oplus b = b$$

and

$$(a \oplus b) \oplus b = a \oplus (b \oplus b) = a$$

This notation gives us a space-saving device for storing the right and left links of a doubly linked list. The nodes will now have only two data members: *data* and *link*. If *l* is to the left of node *x* and *r* to its right, then $x \rightarrow \text{link} = l \oplus r$. If *x* is the leftmost node of a non-circular list, *l* = 0, and if *x* is the rightmost node, *r* = 0. For a new doubly linked list class in which the link field of each node is the exclusive or of the addresses of the nodes to its left and right, do the following.

- (a) Write a C++ function to traverse the doubly linked list from left to right, printing out the contents of the *data* field of each node.
 - (b) Write a C++ function to traverse the list from right to left, printing out the contents of the *data* field of each node.
5. **[Programming Project]:** Implement a C++ template class for doubly linked circular lists with header nodes. You must include a constructor, copy constructor and destructor as well as functions to insert and delete. A bidirectional iterator must be included as well.

4.11 GENERALIZED LISTS

4.11.1 Representation of Generalized Lists

In Chapter 2 a linear list was defined to be a finite sequence of $n \geq 0$ elements, $a_0, \dots, a_{n-1}$, which we write as $(a_0, \dots, a_{n-1})$. The elements of a linear list are restricted to atoms; thus, the only structural property a linear list has is that of position (i.e., a_i precedes a_{i+1} , $0 \leq i < n-1$). Relaxing this restriction on the elements of a list and permitting them to have a structure of their own leads to the notion of a generalized list. Now, the elements a_i , $0 \leq i \leq n-1$, may be either atoms or lists.

Definition: A *generalized list*, is a finite sequence of $n \geq 0$ elements, $a_0, \dots, a_{n-1}$, where a_i is either an atom or a list. The elements a_i , $0 \leq i \leq n-1$, that are not atoms are said to be the *sublists*. $\square$

Let $A = (a_0, \dots, a_{n-1})$ be a list. *A* is the *name* of the list $(a_0, \dots, a_{n-1})$, and *n* is its *length*. By convention, all list names are represented by capital letters. Lowercase letters are used to represent atoms. If $n \geq 1$, then a_0 is the *head* of *A*, and $(a_1, \dots, a_{n-1})$ is the *tail* of *A*. Some examples of generalized lists are

(1) $A = ()$: the null, or empty, list; its length is zero.

(2) $B = (a, (b, c))$: a list of length two; its first element is the atom *a*, and its

second element is the linear list (b,c) .

(3) $C = (B,B,())$: a list of length three whose first two elements are the list B , and the third element is the null list.

(4) $D = (a,D)$: a recursive list of length two; D corresponds to the infinite list $(a, (a, (a, \dots)))$.

A is the empty list. For list B , we have $head(B) = 'a'$ and $tail(B) = ((b,c))$; $tail(B)$ also has a head and tail, which are (b,c) and $()$, respectively. Looking at list C , we see that $head(C) = B$ and $tail(C) = (B,())$. Continuing, we have $head(tail(C)) = B$ and $tail(tail(C)) = (())$, both of which are lists.

Two important consequences of our definition for a list are (1) lists may be shared by other lists as in example (3), where list B makes up two of the sublists of C ; and (2) lists may be recursive as in example (4). The implications of these two consequences for the data structures needed to represent lists will become evident.

First, let us restrict ourselves to the situation in which the lists being represented are neither shared nor recursive. To see where this notion of a list may be useful, consider the problem of representing polynomials in several variables. For example,

$$P(x,y,z) = x^{10}y^3z^2 + 2x^8y^3z^2 + 3x^8y^2z^2 + x^4y^4z + 6x^3y^4z + 2yz$$

Each term of P may be represented using a structure that has four fields: *coef*, *exp_x*, *exp_y*, and *exp_z*. Going this route would require us to define one structure for polynomials in one variable, another for those in two variables, and so on. Alternatively, we could define a structure with a large number of exponent fields and use only as many as needed. Neither of these solutions is elegant.

If we rewrite $P(x,y,z)$ as

$$((x^{10} + 2x^8)y^3 + 3x^8y^2)z^2 + ((x^4 + 6x^3)y^4 + 2y)z$$

we see that $P(x,y,z)$ may be viewed as a generalized list. Every polynomial can be written in this fashion, factoring out a main variable z , followed by a second variable y , and so on. Looking carefully now at $P(x,y,z)$, we see that there are two terms in the variable z , Cz^2 and Dz , where C and D are polynomials themselves, but in the variables x and y . Looking more closely at $C(x,y)$, we see that it is of the form $Ey^3 + Fy^2$, where E and F are polynomials in x . Continuing in this way, we see that every polynomial consists of a variable plus coefficient-exponent pairs. Each coefficient is itself a polynomial (in one less variable) if we regard a single numerical coefficient as a polynomial in zero variables.

We see that every polynomial, regardless of the number of variables in it, can be represented using nodes of the type *PolyNode*, defined as

230 Linked Lists

```
enum Triple {var, ptr, no};
class PolyNode
{
    PolyNode *next;           // link field
    int exp;
    Triple trio;
    union {
        char vble;
        PolyNode *down;       // link field
        int coef;
    };
};
```

In this representation, there are three types of nodes, depending on the value of *trio*. If *trio* == *var*, then the node is the header node for a list; in this case, the field *vble* is used to indicate the name of the variable on which that list is based and the *exp* field is set to 0. (Note that the type of the data member *vble* can be changed to **int** if all variables are kept in a table and *vble* just gives the corresponding table index.) If *trio* == *ptr*, then the coefficient is itself a list and is pointed by the field *down*. If *trio* == *no*, then the coefficient is an integer and is stored in the field *coef*. In the last two cases, *exp* represents the exponent of the variable on which that list is based.

Figure 4.30 gives the representation of the polynomial $3x^2y$. Here *first* points to a header node, which indicates that the upper list is based on *y*. Hence the next term in the list refers to the term $y^1 = y$. The coefficient of this term is the polynomial represented by the lower list. This polynomial is $3x^2$. Figure 4.31 gives the representation of the polynomial $P(x,y,z)$ defined earlier. For simplicity, the *trio* field is omitted from Figure 4.31. The value of this field for each node is self-evident.

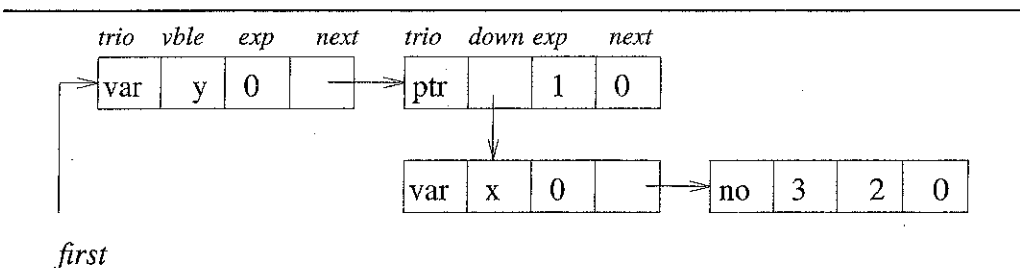


Figure 4.30: Representation of $3x^2y$

Every generalized list can be represented using the node structure

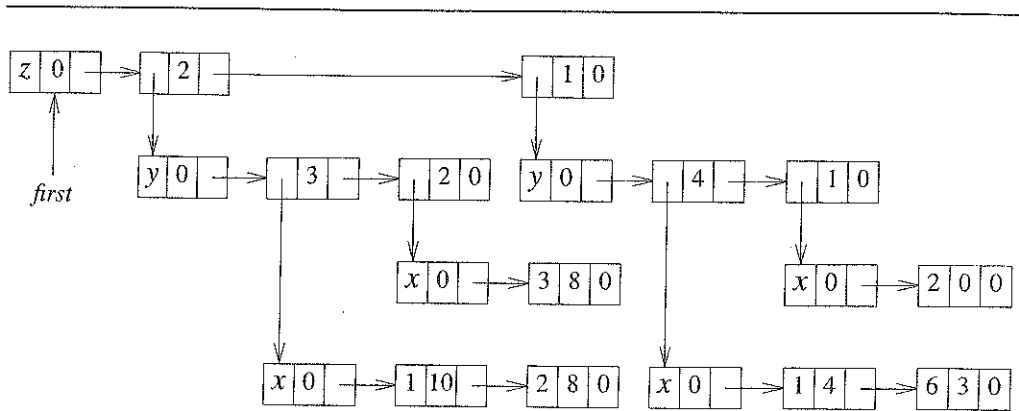


Figure 4.31: $((x^{10} + 2x^8)y^3 + 3x^8y^2)z^2 + ((x^4 + 6x^3)y^4 + 2y)z$

<code>tag = false/true</code>	<code>data/down</code>	<code>next</code>
-------------------------------	------------------------	-------------------

This data structure may be defined in C++ as:

template <class *T*> **class** *GenList*; // forward declaration

```

template <class T>
class GenListNode {
friend class GenList <T>;
private:
    GenListNode <T> *next;
    bool tag;
    union {
        T data;
        GenListNode <T> *down;
    };
};

```

```

template <class T>
class GenList {
public:
    // List manipulation operations

private:
    GenListNode <T> *first;
};

```

232 Linked Lists

The *data/down* field holds an atom if *head(A)* is an atom and holds a pointer to the list representation of *head(A)* if *head(A)* is a list. (The exercises examine how a multivariate polynomial may be stored using this structure.) Using this node structure, the example lists (1) to (4) (page 228) have the representation shown in Figure 4.32.

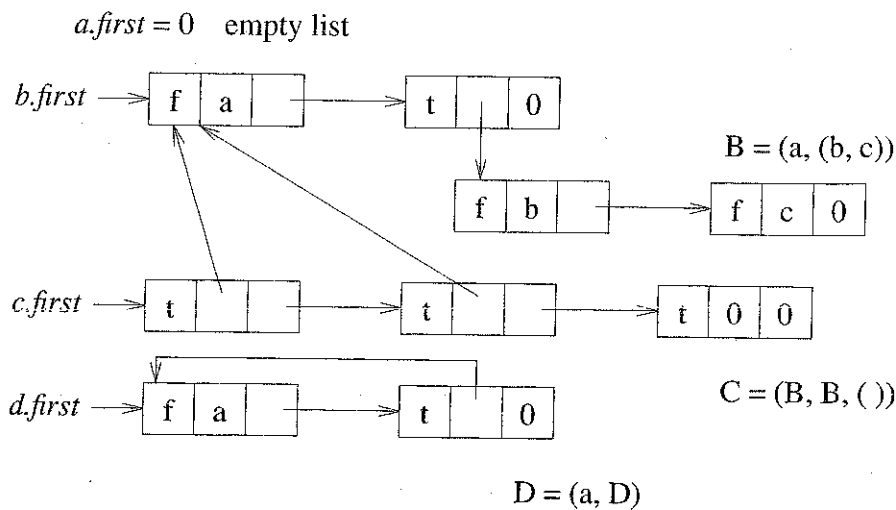


Figure 4.32: Representation of lists (1) to (4) (page 228); an *f* in the *tag* field represents the value **false**, whereas a *t* represents the value **true**)

4.11.2 Recursive Algorithms for Lists

When a data object is defined recursively, it is often easy to describe recursive algorithms that work on these objects. Recursive algorithms for C++ objects typically consist of two components—the recursive function itself (the *workhorse*) and a second function that invokes the recursive function at the top level (the *driver*). The driver is declared as a public member function while the workhorse is declared as a private member function.

4.11.2.1 Copying a List

To see how recursion is useful, let us write a function (Program 4.35) that produces an exact copy of a nonrecursive list in which no sublists are shared.

```

// Driver
void GenList <T>::Copy(const GenList <T> & l) const
{ // Make a copy of l.
    first = Copy (l.first);
}

// Workhorse
GenListNode <T>* GenList <T>::Copy(GenListNode <T>* p)
{ // Copy the nonrecursive list with no shared sublists pointed at by p.
    GenListNode <T> *q = 0;
    if (p) {
        q = new GenListNode <T>;
        q->tag = p->tag;
        if (p->tag) q->down = Copy (p->down);
        else q->data = p->data;
        q->next = Copy (p->next);
    }
    return q;
}

```

Program 4.35: Copying a list

Program 4.35 reflects exactly the definition of a (generalized) list. We see immediately that *Copy* works correctly for an empty list. A simple proof using induction will verify the correctness of the entire function. Now let us consider the computing time of this function. The empty list takes a constant amount of time. For the list $A = ((a,b),((c,d),e))$, which has the representation of Figure 4.33, p takes on the values given in Figure 4.34. The sequence of values should be read down the columns; b, r, s, t, u, v, w , and x are the addresses of the eight nodes of the list. From this example one should be able to see that nodes with $tag = \text{false}$ will be visited twice, whereas nodes with $tag = \text{true}$ will be visited three times. Thus, if a list has a total of m nodes, no more than $3m$ executions of any statement will occur. Hence, the algorithm is $O(m)$, or linear, which is the best we can hope to achieve. Another factor of interest is the maximum depth of recursion or, equivalently, how many locations are needed for the recursion stack. Again, by carefully following the algorithm on the previous example, we

234 Linked Lists

see that the maximum depth is a combination of the lengths and depths of all sublists. However, a simple upper bound is m , the total number of nodes. Although this bound will be extremely large in many cases, it is achievable, for instance, if $A = (((((a))))))$.

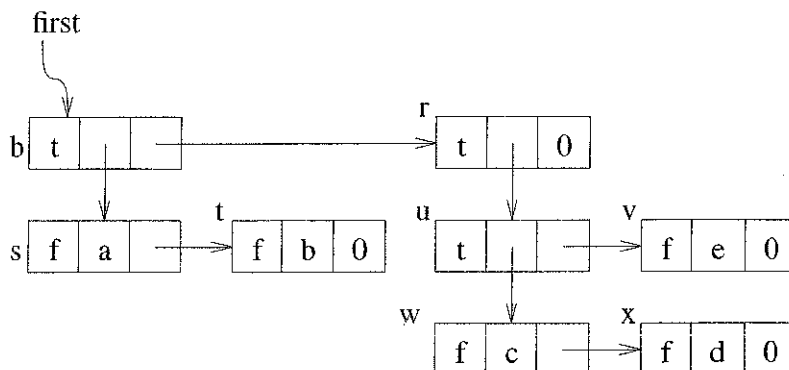


Figure 4.33: Linked representation for A

level of recursion	value of p	continuing level	p	continuing level	p
1	b	2	r	3	u
2	s	3	u	4	v
3	t	4	w	5	0
4	0	5	x	4	v
3	t	6	0	3	u
2	s	5	x	2	r
1	b	4	w	3	0
				2	r
				1	b

Figure 4.34 Values of parameters in execution of $GenList\ <T>::Copy(A)$

4.11.2.2 List Equality

Another useful function determines whether two lists are identical. To be identical, the lists must have the same structure and the same data in corresponding data members. Again, using the recursive definition of a list, we can write a short recursive function (Program 4.36) to accomplish this task.

```
// Driver
template <class T>
bool operator==(const GenList<T>& l) const
{
    // *this and l are non-recursive lists.
    // The function returns true iff the two lists are identical.
    return Equal(first, l.first);
}

// Workhorse
bool Equal(GenListNode<T>* s, GenListNode<T>* t)
{
    if ((!s) && (!t)) return true;
    if (s && t && (s->tag == t->tag))
        if (s->tag)
            return Equal(s->down, t->down) && Equal(s->next, t->next);
        else return (s->data == t->data) && Equal(s->next, t->next);
    return false;
}
```

Program 4.36: Determining if two lists are identical

The computing time of Program 4.36 is clearly no more than linear when no sublists are shared, since it looks at each node of the two lists being compared no more than three times. For unequal lists the program terminates as soon as it discovers that the lists are not identical.

4.11.2.3 List Depth

Another handy operation on nonrecursive lists is the function that computes the depth of a list. The depth of the empty list is defined to be zero and, in general,

$$\text{depth}(s) = \begin{cases} 0 & \text{if } s \text{ is an atom} \\ 1 + \max \{ \text{depth}(x_1), \dots, \text{depth}(x_n) \} & \text{if } s \text{ is the list } (x_1, \dots, x_n), n \geq 1 \end{cases}$$

236 Linked Lists

Function *Depth* (Program 4.37) is a very close transformation of the definition, which is itself recursive.

```
// Driver
template <class T>
int GenList <T>::Depth ()
{ // Compute the depth of a non-recursive list.
    return Depth (first);
}

// Workhorse
template <class T>
int GenList <T>::Depth(GenListNode <T> *s)
{
    if (!s) return 0; // empty list
    GenListNode <T> *current = s;
    int m = 0;
    while (current) {
        if (current->tag) m = max(m, Depth (current->down));
        current = current->next;
    }
    return m + 1;
}
```

Program 4.37: Computing the depth of a list

4.11.3 Reference Counts, Shared and Recursive Lists

In this section we shall consider some of the problems that arise when lists are allowed to be shared by other lists and when recursive lists are permitted. Sharing of sublists can, in some situations, result in great savings in storage used, as identical sublists occupy the same space. To facilitate specifying shared sublists, we extend the definition of a list to allow for naming of sublists. A sublist appearing within a list definition may be named through the use of a list name preceding it. For example, in the list $A = (a, (b, c))$, the sublist (b, c) could be assigned the name Z by writing $A = (a, Z(b, c))$. In fact, to be consistent, we would then write $A(a, Z(b, c))$ which would define the list A as above.

Lists that are shared by other lists, such as list B of Figure 4.32, create problems when you wish to add or delete a node at the front. If the first node of

B is deleted, it is necessary to change the pointers from list C to point to the second node. If a new node is added, pointers from C have to be changed to point to the new first node. However, we normally do not know all the points from which a particular list is being referenced. (Even if you did have this information, addition and deletion of nodes could require a large amount of time.) This problem is easily solved through the use of header nodes. If you expect to perform any additions and deletions at the front of lists, then the use of a header node with each list or named sublist will eliminate the need to retain a list of all pointers to any specific list. If each list is to have a header node, then lists (1) to (4) are represented as in Figure 4.35. The values in the *data/down* fields of the header nodes is the reference count (defined below) of the corresponding list. Even in situations in which you do not wish to add or delete nodes from lists dynamically, as in the case of multivariate polynomials, header nodes prove useful in determining when the nodes of a particular structure may be returned to the storage pool.

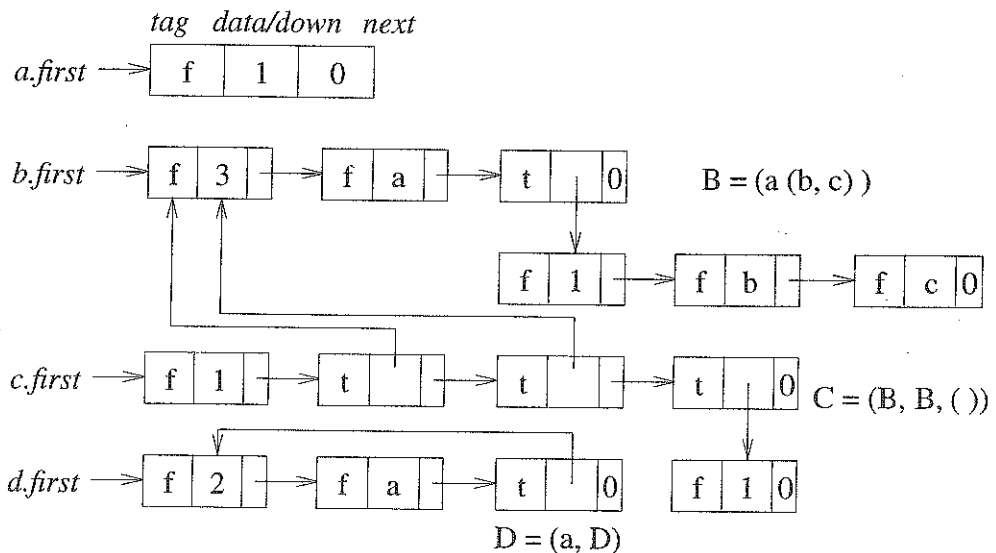


Figure 4.35: Structure with header nodes for lists (1) to (4)

Whenever lists are being shared by other lists, we need a mechanism to help determine whether or not the list nodes may be physically returned to the available-space list. This mechanism is generally provided through the use of a reference count maintained in the header node of each list. Since the *data* field of the header nodes is free, the reference count is maintained in this field.

238 Linked Lists

(Alternatively, a third variant may be introduced, with *tag* having three possible values: 0, 1, and 2.) The reference count of a list is the number of pointers (either program variables or pointers from other lists) to that list. The reference counts for the lists are

- (1) $a.first \rightarrow ref = 1$ accessible only via $a.first$
- (2) $b.first \rightarrow ref = 3$ pointed to by $b.first$ and two pointers from c
- (3) $c.first \rightarrow ref = 1$ accessible only via $c.first$
- (4) $d.first \rightarrow ref = 2$ accessible via $d.first$ and one pointer from itself.

Now a call to $t.\sim GenList<T>()$ (list destructor) should result only in a decrementing by 1 of the reference counter of $t.first$. Only when the reference count becomes zero are the nodes of t deleted. The same is to be done with the sublists of t .

Suppose we change the definition of $GenListNode<T>$ to

```
template <class T>
class GenListNode <T>
{
friend class GenList <T>;
private:
    GenListNode <T> *next;
    int tag; // 0 for data, 1 for down, 2 for ref
    union {
        T data;
        GenListNode <T> *down;
        int ref;
    };
};
```

A recursive function to delete a list is given in Program 4.38. The workhorse proceeds by examining the top-level nodes of a list whose reference count has become zero. Any sublists encountered are deleted recursively, and finally, the top-level nodes are linked into the available-space list.

A call to $b.\sim GenList<T>()$, where b is list (ii) of Figure 4.35, now has only the effect of decreasing the reference count of b to 2. Such a call followed by a call to $c.\sim GenList<T>()$ results in

- (1) the reference count of c becomes zero
- (2) $b.first \rightarrow ref$ becomes 1 when the second top-level node of c is processed
- (3) $b.first \rightarrow ref$ becomes 0 when the third top-level node of c is processed;

```

// Driver
template <class T>
GenList <T>::~~GenList ()
{
    // Each header node has a reference count.
    if (first)
    {
        Delete (first);
        first = 0;
    }
}

// Workhorse
void GenList <T>::Delete(GenListNode <T>* x)
{
    x->ref--; // decrement reference count of header node.
    if (!x->ref)
    {
        GenListNode <T> *y = x; // y traverses top level of x.
        while (y->next) { y = y->next; if (y->tag == 1) Delete (y->down); }
        y->next = av; // attach top-level nodes to av list
        av = x;
    }
}

```

Program 4.38: Deleting a list recursively

now, the five nodes of list $B(a, (b, c))$ are returned to the available-space list

- (4) the top-level nodes of c are linked into the available-space list.

The use of header nodes with reference counts solves the problem of determining when nodes are to be physically freed in the case of shared sublists. However, for recursive lists, the reference count never becomes zero. $d.\sim\text{GenList} <T>()$ results in $d.\text{first} \rightarrow \text{ref}$ becoming one. The reference count does not become zero, even though this list is no longer accessible either through program variables or through other structures. The same is true in the case of indirect recursion (Figure 4.36). After calls to $r.\sim\text{GenList} <T>()$ and $s.\sim\text{GenList} <T>()$, $r.\text{first} \rightarrow \text{ref} = 1$ and $s.\text{first} \rightarrow \text{ref} = 2$ but the structure consisting of r and s is no longer accessible. So, its nodes should have been returned to the available-space list.

Unfortunately, there is no simple way to supplement the list structure of

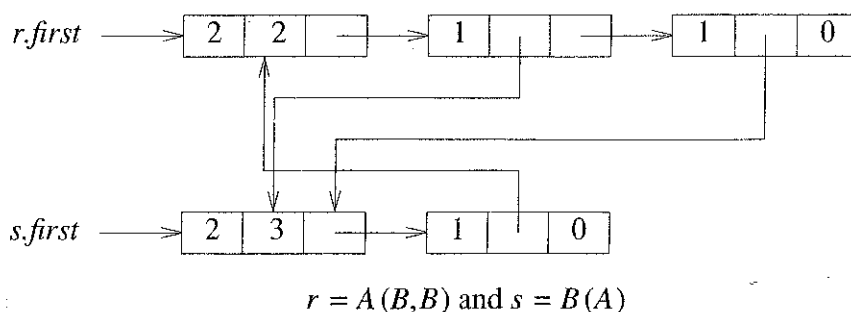


Figure 4.36: Indirect recursion of lists $r = A$ and $s = A$

Figure 4.36 so as to be able to determine when recursive lists may be physically deleted. It is no longer possible to return all free nodes to the available-space list when they become free. When recursive lists are being used, it is possible to run out of available space, even though not all nodes are in use.

EXERCISES

1. Describe how a multivariate polynomial may be represented using the following node structure that can be used to represent any generalized list:

<i>tag</i> = false/true	<i>data/down</i>	<i>next</i>
-------------------------	------------------	-------------

2. Write a nonrecursive version of $GenList<T>::\sim GenList<T>()$ (Program 4.38).
3. Write a nonrecursive version of **operator==** (Program 4.36).
4. Write a nonrecursive version of $GenList<T>::Depth$ (Program 4.37).
5. Write a function that inverts an arbitrary nonrecursive list l with no shared sublists. The sublists of l also are inverted. For example, if $l = (a, (b, c))$, then inverse (l) = $((c, b), a)$.
6. Devise a function that produces the list representation of an arbitrary list, given its linear form as a string of atoms, commas, blanks, and parentheses. For example, for the input $l = (a, (b, c))$, your function should produce the structure of Figure 4.37.

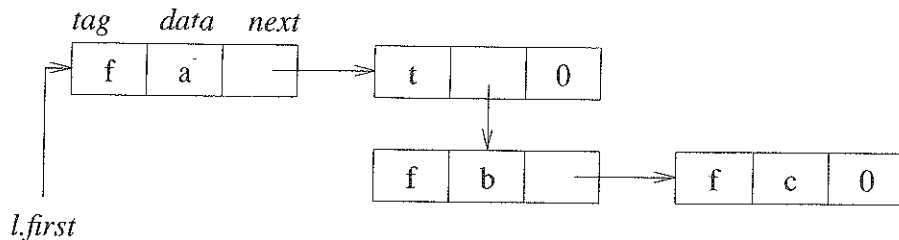


Figure 4.37: Structure for Exercise 6

7. One way to represent generalized lists is use nodes that have two fields plus a table that contains all atoms and list names, together with pointers to these lists. Let the two fields of each node be named *alink* and *blink*. Then *blink* either points to the next node on the same level, if there is one, or it is 0. The *alink* points either to a node at a lower level or, in the case of an atom or list name, to the appropriate entry in the symbol table. For example, the list $B(A, (D, E), (), B)$ would have the representation given in Figure 4.38. The notation d^* means a pointer to the first node of list D .

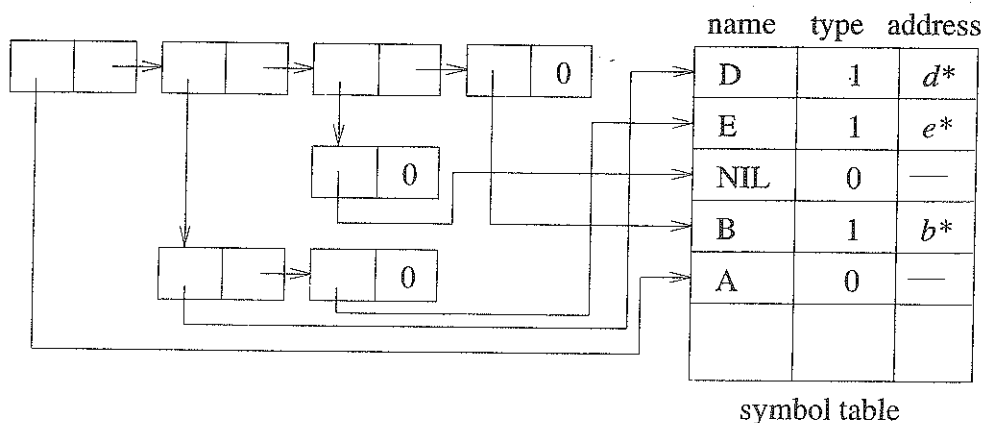


Figure 4.38: Representation for Exercise 7

(The list names D and E were already in the table at the time the list B was input. A was not in the table and is assumed to be an atom.)

242 Linked Lists

The symbol table retains a type bit for each entry. This bit is 1 if the entry is a list name and 0 for an atom. The NIL atom may be in the table, or *alink* can be set to 0 to represent the NIL atom. Write the C++ function **operator>>**, which inputs a list in parenthesis notation and sets up its linked representation as above. Note that no header nodes are in use. The following functions may be used by **operator>>**:

- (a) **int Get(*a*)** searches the symbol table for the name *a*. -1 is returned if *a* is not found in the table; otherwise, the position of *a* in the table is returned.
- (b) **Put(*n,t,a*)** enters the triple (*n,t,a*) into the table. If the name *n* is already in the table, then the type and address fields of the old entry are updated to *t* and *a*, respectively.
- (c) **NextToken()** gets the next token in the input list. (A token may be a list name, atom, '(', ')', or ','. A '#' is returned if there are no more tokens.)

Write code for all functions used by you. You may assume that the input list is syntactically correct. If a sublist is labeled, as in the list *C(D,E(F,G))*, the structure should be set up as in the case *C(D,(F,G))*, and *E* should be entered into the symbol table as a list with the appropriate starting address.

- 8. What goes wrong when Program 4.35 is used to copy lists that have shared sublists?
- 9. Write and test a C++ template class for generalized lists with reference counts. You must include a copy constructor and destructor for your class as well as functions to determine the depth of a list and to determine whether two lists are identical. What is the time complexity of your functions?